

UNIVERSIDAD DE BUENOS AIRES
Facultad de Ciencias Exactas y Naturales
Departamento de Computación

Estrategias de prueba de normalización fuerte en cálculos lambda tipados

Nuevas medidas decrecientes

Tesis presentada para optar al título de
Doctor de la Universidad de Buenos Aires en el área Ciencias de la Computación

Cristian Fabián Sottile

Director de tesis: Dr. Alejandro Díaz-Caro
Codirector de tesis: Dr. Pablo E. Martínez López

Consejero de estudios: Dr. Juan Pablo Galeotti

Lugar de trabajo: Instituto de Investigación en Ciencias de la Computación (UBA-CONICET)
Fecha de defensa: 3 de julio de 2026

Buenos Aires, 2026

Agradecimientos

A Yamila Vazquez, la mujer que cada vez que miro al costado me está acompañando; por la vida juntos en general, por su crucial apoyo durante esta etapa en particular, y por saber mejorar los días con su sonrisa. A mi mamá Silvia Miño, por encarar sola la titánica travesía de criarme, logrando darme siempre toda la atención, cariño y recursos que necesité, y por creer tanto en mí desde siempre. A mis amigos[1], por andar la vida conmigo, por la resistente al tiempo y unida familia que supimos construir, y por ponerse contentos cuando llego. Especialmente a Ulises Rigone, el hermano que me dio la vida, y a Juan Francisco Osacar, el otro que mejor me entiende, por la esencial cercanía en estos años. También a Iván Facal, por vivir mis alegrías como si fueran suyas. A mi abuelo Juan Carlos "Totoy" Miño, por mostrarme con el ejemplo que el ingenio no tiene límites, y que la realidad está ahí para que la torzamos. A mi abuela Yolanda Peroni, por empezar a educarme antes de tiempo, y por su valor para dejar todo atrás cuando hizo falta. A mis tías "Anita" Miño y "Luisita" González, por el fundamental soporte que brindaron durante mi crecimiento. A todos ellos, por esperarme mientras me aventuro en mis búsquedas de propósito.

A mi director Alejandro "Jano" Díaz-Caro, por mostrarme, enseñarme y llevarme por el camino de la ciencia, con tanto afecto, generosidad y compromiso. A mi codirector Pablo E. "Fidel" Martínez López, por señalarme una puerta que busqué durante años. A Pablo Barenbaum y Simona Ronchi Della Rocca, por tratarme como colega antes de que lo fuéramos. A Delia Kesner y Giulio Manzonetto, por interacciones que contribuyeron significativamente a darle forma a mi doctorado. A Mariana Milicich, una amiga que encontré en la distancia. A Malena Ivnisky y Rafael Romero, mis compañeros en esta aventura. A Gilles Dowek, gracias a quien conocí a Jano. A los miembros del jurado, Mariangiola Dezani-Ciancaglini, Giulio Guerrieri y Femke van Raamsdonk, por su cuidadosa lectura de mi tesis, y por la cálida y profunda discusión en la defensa.

A quienes me prestaron su tiempo y me dejaron enseñanzas o ideas valiosas. Entre otros, sin duda: Ariel Arbiser, Gabriel Baum, Pierre-Louis Curien, Roberto Di Cosmo, Joost Joosten, Carlos López Pombo, Alexandre Miquel, Alejandro Pardo, Uriel Pittatore, Claudia Pons, Facundo Quiroga, Francisco Soulignac, Nora Szasz, Éric Tanter, Álvaro Tasistro, Marcos Viera, Beta Ziliani.

Al Instituto de Investigación en Ciencias de la Computación (ICC) de la Universidad de Buenos Aires (UBA) y el Consejo Nacional de Investigaciones Científicas y Técnicas (CONICET), y al Departamento de Computación de la Facultad de Ciencias Exactas y Naturales de la Universidad de Buenos Aires, mi hogar durante el doctorado, por ser el mejor lugar en Argentina para ser un apasionado de la disciplina. A Esteban Mocskos y la Subcomisión de Doctorado, por el inestimable acompañamiento durante la carrera. A la Comunidad de Programación Informática del Departamento de Ciencia y Tecnología de la Universidad Nacional de Quilmes (UNQ), por siempre apoyar mi crecimiento académico. Al Laboratorio de Investigación y Formación en Informática Avanzada (LIFIA) de la Facultad de Informática de la Universidad Nacional de La Plata, donde di mis primeros pasos en investigación. A mis amigos de la licenciatura[2], por tanto compartido allí y ahora. A todos los profesores que tuve durante estos años de formación; en especial a mi primera profesora en la universidad, Gabriela Cerra, que me supo transmitir un entusiasmo permanente por la técnica.

A las instituciones que destinaron recursos a que yo escriba esta tesis. Exactas-UBA, cuyo doctorado es gratuito. El CONICET, con la imprescindible beca doctoral. La UNQ, con el cargo de investigación que obtuve en el tramo final. La Agencia I+D+i, la Unión Europea, el CONICET y la UNQ, con los proyectos[3] que financiaron equipamiento, viajes y publicaciones.

Al pueblo argentino, por la irreverencia y la autenticidad, y por la tranquilidad de saber que pertenezco a algo más grande.

[1] Tenemos un grupo algo extenso: Facundo Acevedo, Federico Alonso, Jonathan da Cruz, Iván Facal, Juan Agustín Ledesma, Guido Mandagarán Durán, Lucía Marchetti, Gastón Mársico, Lara Ángeles Ongaro, Juan Francisco Osacar, Juan Ignacio Papa, Naiara Perera, Ulises Rigone y Luis Sánchez Dávila.

[2] Natalia Aparicio, Ignacio Babbini, Cristian Barbaro, Bruno Cascio y Cirano Eusebi.

[3] PICT-2021-I-A-00090; MSCA SE "QCOMICAL" 101182520; PIP 11220200100368CO; PUNQ 2218/22 y 1342/19.

Acknowledgments

To Yamila Vazquez, the woman who is by my side whenever I look; for our life together in general, for her crucial support throughout this stage in particular, and for knowing how to make days better with her smile. To my mother Silvia Miño, for facing alone the titanic journey of raising me, managing to always provide me with all the attention, affection, and resources I needed, and for always believing so much in me. To my friends[1], for walking through life with me, for the time-proof, close-knit family we managed to build, and for being happy to see me arrive. Especially to Ulises Rigone, the brother life gave me, and to Juan Francisco Osacar, the other one who understands me best, for the essential closeness during these years. Also to Iván Facal, for living my joys as if they were their own. To my grandfather Juan Carlos "Totoy" Miño, for showing me by example that ingenuity has no limits, and that reality is there for us to bend. To my grandmother Yolanda Peroni, for starting my education early, and for her courage to leave everything behind when needed. To my aunts "Anita" Miño and "Luisita" González, for their fundamental support as I grew up. To all of them, for waiting for me while I venture off in my searches for purpose.

To my supervisor Alejandro Díaz-Caro, for showing me, teaching me, and leading me along the path of science, with so much care, generosity, and commitment. To my co-supervisor Pablo E. "Fidel" Martínez López, for pointing me towards a door I had sought for years. To Pablo Barenbaum and Simona Ronchi Della Rocca, for treating me as a colleague before we were such. To Delia Kesner and Giulio Manzonetto, for exchanges that greatly helped shape my PhD. To Mariana Milicich, a friend I found across the distance. To Malena Ivnisky and Rafael Romero, my companions on this journey. To Gilles Dowek, thanks to whom I met Alejandro. To the jury members, Mariangiola Dezani-Ciancaglini, Giulio Guerrieri, and Femke van Raamsdonk, for their careful reading of my thesis, and for the warm and deep discussion at the defense.

To those who gave me their time and left me with valuable ideas or lessons. Among them, without a doubt: Ariel Arbiser, Gabriel Baum, Pierre-Louis Curien, Roberto Di Cosmo, Joost Joosten, Carlos López Pombo, Alexandre Miquel, Alejandro Pardo, Uriel Pittatore, Claudia Pons, Facundo Quiroga, Francisco Soulignac, Nora Szasz, Éric Tanter, Álvaro Tasistro, Marcos Viera, Beta Ziliani.

To the *Instituto de Investigación en Ciencias de la Computación* (ICC) of *Universidad de Buenos Aires* (UBA) and the *Consejo Nacional de Investigaciones Científicas y Técnicas* (CONICET), and the *Departamento de Computación* of the *Facultad de Ciencias Exactas y Naturales* at *Universidad de Buenos Aires*, my home during the PhD, for being the best place in Argentina to be passionate about the field. To Esteban Mocskos and the *Subcomisión de Doctorado*, for the invaluable companionship during the program. To the *Comunidad de Programación Informática* of the *Departamento de Ciencia y Tecnología* at *Universidad Nacional de Quilmes* (UNQ), for always supporting my academic growth. To the *Laboratorio de Investigación y Formación en Informática Avanzada* (LIFIA) of the *Facultad de Informática* at *Universidad Nacional de La Plata*, where I took my first steps in research. To my friends from the *licenciatura*[2], for so much shared then and now. To all the professors I had during these years of education; with a special mention to my first professor at university, Gabriela Cerra, who instilled in me a permanent enthusiasm for technique.

To the institutions that devoted resources to the writing of this thesis. *Exactas* at UBA, whose PhD program is free. CONICET, with the indispensable doctoral fellowship. UNQ, with the research position I obtained during the final stretch. *Agencia I+D+i*, the European Union, CONICET, and UNQ, through the projects[3] that funded equipment, travel, and publications.

To the Argentinian people, for the irreverence and authenticity, and for the peace of knowing I belong to something greater.

[1] We have quite a big group: Facundo Acevedo, Federico Alonso, Jonathan da Cruz, Iván Facal, Juan Agustín Ledesma, Guido Mandagarán Durán, Lucía Marchetti, Gastón Mársico, Lara Ángeles Ongaro, Juan Francisco Osacar, Juan Ignacio Papa, Naiara Perera, Ulises Rigone, and Luis Sánchez Dávila.

[2] Natalia Aparicio, Ignacio Babbini, Cristian Barbaro, Bruno Cascio and Cirano Eusebi.

[3] PICT-2021-I-A-00090; MSCA SE "QCOMICAL" 101182520; PIP 11220200100368CO; PUNQ 2218/22, 1342/19.

Estrategias de prueba de normalización fuerte en λ-cálculos tipados

Nuevas medidas decrecientes

La terminación es una propiedad de seguridad deseable en lenguajes de programación. En lenguajes con reglas de ejecución no deterministas, como el λ–cálculo, tenemos dos variantes. *Weak normalization* (WN): existe un camino de ejecución que llega al resultado. *Strong normalization* (SN): todos los posibles caminos de ejecución llegan al resultado. Los principales λ–cálculos tipados satisfacen SN, estando sus pruebas entre los resultados más complejos en teoría de tipos.

La primera prueba de SN para el λ–cálculo simplemente tipado (STLC) fue introducida por Luis Elpidio Sanchis, miembro de esta Casa de Estudios, en 1965. Se publicó en 1967, el mismo año en el que William Walker Tait presentó una prueba de WN. Tait introdujo una técnica llamada *computabilidad*, que mostró tener gran capacidad de adaptación, siendo extendida por Jean-Yves Girard para demostrar SN en 1971, y para demostrar SN del λ–cálculo polimórfico en 1972. Fue renombrada a *reducibilidad*, y al día de hoy constituye la forma principal de demostrar SN en λ–cálculos. Siendo todo lo potente y concisa que es, también es una técnica abstracta que no aporta mucha intuición sobre *por qué* la propiedad vale. Aunque la validez de la propiedad ya se conocía, la comunidad científica continuó trabajando en pruebas alternativas. Se escribieron muchos papers analizando reducibilidad, introduciendo formas alternativas de demostrar SN, y haciendo comparaciones detalladas reducibilidad con las pruebas alternativas.

La técnica de *medidas decrecientes* es una de las principales alternativas para demostrar SN en λ–cálculos tipados. Consiste en asignar elementos de un orden bien fundado a los programas, de manera tal que un paso de computación lleva a decrecer el elemento asignado. La primera medida decreciente para STLC fue introducida por Gandy en 1980, asignando números naturales a términos mediante una interpretación en el dominio de las funciones hereditaria y estrictamente monótonas. En 1987 de Vrijer simplificó la medida de Gandy, usando un dominio similar de funcionales crecientes, proveyendo un refinamiento donde el número asignado es igual a la cantidad de pasos desde el camino de ejecución más largo desde el programa.

En esta tesis presentamos nuevas medidas decrecientes para dos sistemas. Para STLC, presentamos primero una medida basada en un cálculo *con memoria* que *recuerda* los argumentos consumidos, de forma tal que nos permite simular la ejecución de manera controlada, y contar la cantidad de argumentos recordados en el resultado. Usando el mismo cálculo con memorias, proveemos otra medida para STLC, en este caso mejorando una medida dada por Turing para una noción restringida de ejecución. La de Turing implica WN, y nosotros la extendimos para que implique SN. La suya está basada en multiset de *grados*; la nuestra agrega multiconjuntos anidados a cada grado, con información de simulaciones controladas de ciertos caminos de ejecución.

Extendimos la primera medida a tipos intersección idempotente (IIT), un sistema de asignación de tipos que introduce en STLC una forma finitaria de polimorfismo. Proveemos la primera medida nativa para IIT que decrece para cualquier paso de ejecución, dando una prueba directa de SN. Tiene un dominio más simple que las otras, los números naturales. Como la presentación de IIT es en general con tipado extrínseco, y nuestra técnica requiere tipado intrínseco, definimos una variante intrínseca y demostramos correspondencia y simulación con respecto a la versión extrínseca.

En medidas que dan números, el valor asignado a cada programa es una cota superior para la longitud de su camino de ejecución más largo. Una pregunta que surge es si dicho número se corresponde con esa longitud, que no es el caso de nuestra primera medida. Presentamos su refinamiento a exactitud, donde probamos que el número obtenido es una cota superior exacta para la longitud del camino de ejecución más largo. Para eso adaptamos la creación de memorias en el cálculo con memorias, de manera tal que solo los argumentos *borrados* son recordados. Discutimos el refinamiento a exactitud de la extensión a IIT, que es un trabajo en progreso.

Strong normalization proof strategies for typed λ-calculi

New decreasing measures

Termination is one of the desired safety properties for programming languages. In languages with non-deterministic execution rules, such as the λ-calculus, we have two variants of this property. *Weak normalization* (WN) establishes that there exists one execution path that reaches the result. *Strong normalization* (SN) establishes that all possible execution paths reaches the result. Main typed λ–calculi satisfy SN, with their proofs being among the most complex results in type theory.

The first proof of SN for the simply typed λ–calculus (STLC) was introduced by Luis Elpidio Sanchis, a member of this University, in 1965. It was published in 1967, the same year where a proof of WN was given by William Walker Tait. Tait introduced a technique called *computability* that showed a great deal of extensibility, being extended by Jean-Yves Girard to prove SN in 1971, and to prove SN of the polymorphic λ–calculus in 1972. It was later renamed to *reducibility*, and consists until today of the main technique for proving SN in typed λ–calculi. As powerful and concise as it is, it is also an abstract technique that does not provide much insight on *why* the property holds. Although the property holding was already known, the scientific community kept working on alternative proofs. Several papers were written analyzing reducibility, introducing alternative ways of proving SN, and deeply comparing reducibility and the alternative proofs.

The technique of *decreasing measures* is one of the main alternatives for proving SN in typed λ–calculi. It consists of assigning elements from a well founded order to programs, in such a way that a computation step leads to a decrease in the element assigned, providing a proof by a descent argument. The first one for STLC was introduced by Gandy in 1980, assigning natural numbers to programs through an interpretation on the domain of hereditarily strictly monotonic functions. In 1987 de Vrijer simplified Gandy's measure, relying on the similar domain of increasing functionals, and provided a refined variant where the number assigned was equal to the number of steps of the longest execution path from the program.

In this thesis we present new decreasing measures for two different systems. For STLC, we first present a measure based on a *memory* calculus that *remembers* its consumed arguments, in such a way that we are able to simulate execution in a controlled manner, and count the amount of remembered arguments in the result. Using the same memory calculus, we provide another measure for STLC, this time enhancing a measure given by Turing for a restricted notion of execution. Turing's entailed WN, and we extended it to entail SN. His was based on a multiset of *degrees*; ours adds a nested multiset to each degree, consisting of information on the simulated execution of specific paths, again in a controlled manner.

We extended the first measure to the setting of idempotent intersection types (IIT), a type assignment system that introduces a finitary form of polymorphism to the STLC. We provided the first native measure for IIT that decrease for any execution step, thus giving a direct proof of SN. It has a simpler codomain than the others, yielding a natural number as a result. Since IIT is usually presented with extrinsic typing, and our technique requires intrinsic typing, we define an intrinsic variant and showed correspondence and simulation with respect to the extrinsic one.

For decreasing measures yielding natural numbers, the value assigned to each program consists of an upper bound for the length of the longest execution path from it. A natural question that arises is whether the number corresponds exactly with such length, which is not the case for our first measure. We present then a refinement to exactness of that measure, where we prove that the number obtained consists of an exact upper bound for the longest execution path length. We do so by adapting the creation of *memories* in the memory calculus, in a way such that only *erased* arguments are remembered. A refinement to exactness for the extension to IIT is discussed, and remains a work in progress.

Contents

1 Introduction **8**
- 1.1 Historical overview . . . 8
- 1.2 Typed functional languages . . . 9
- 1.3 Proof techniques for strong normalization . . . 11
  - 1.3.1 Reducibility . . . 11
  - 1.3.2 Decreasing measures . . . 12
  - 1.3.3 Reduction to weak normalization and redex degrees . . . 13
  - 1.3.4 Other approaches . . . 13
- 1.4 Gödel's *koan* . . . 14
- 1.5 Contributions . . . 14
  - 1.5.1 Two decreasing measures for simply typed λ-terms . . . 15
  - 1.5.2 An extension to idempotent intersection types . . . 15
  - 1.5.3 A refinement to exactness . . . 16
  - 1.5.4 Other (non-included) works . . . 16
- 1.6 Organization of this thesis . . . 16
- 1.7 Summary in Spanish . . . 17

2 Two decreasing measures for the simply typed λ–calculus **18**
- 2.1 Introduction . . . 18
  - 2.1.1 Organization of the chapter . . . 19
- 2.2 Redex–degrees . . . 19
- 2.3 STLC with memories . . . 20
- 2.4 $\mathcal{W}$–measure: counting wrappers . . . 25
  - 2.4.1 Simplification . . . 26
  - 2.4.2 Forgetful reduction . . . 31
  - 2.4.3 The measure . . . 35
- 2.5 $\mathcal{T}^{\mathbf{m}}$–measure: nesting developments by redexes and degrees . . . 36
  - 2.5.1 Properties . . . 38
  - 2.5.2 The measure . . . 39
- 2.6 Conclusions . . . 47
- 2.7 Summary in Spanish . . . 49

3 A decreasing measure for idempotent intersection types **50**
- 3.1 Introduction . . . 50
  - 3.1.1 Comparison with related works . . . 52
  - 3.1.2 Organization of the chapter . . . 53
- 3.2 Idempotent intersection types à la Church . . . 54

3.2.1 Extrinsically typed system . . . 54
3.2.2 Intrinsically typed system . . . 55
3.2.3 Correspondence between Curry and Church-style systems . . . 60
3.3 Idempotent intersection types with memories . . . 63
3.3.1 The memory calculus $\Lambda^{\mathbf{m}}_{\cap}$ . . . 63
3.4 $\mathcal{W}_{\cap}$-measure: counting wrappers in $\Lambda^{\mathbf{m}}_{\cap}$ . . . 67
3.4.1 Simplification . . . 67
3.4.2 Forgetful reduction . . . 70
3.4.3 The decreasing measure . . . 72
3.5 Conclusions . . . 73
3.6 Summary in Spanish . . . 74

4 A refinement to exactness **75**
4.1 Introduction . . . 75
4.2 The measure $\hat{\mathcal{W}}$: an exact refinement of $\mathcal{W}$ . . . 76
4.2.1 Refinement of STLC with memories . . . 77
4.2.2 Simplification . . . 78
4.2.3 Forgetful reduction . . . 80
4.2.4 $\hat{\mathcal{W}}$-measure . . . 81
4.2.5 Exact upper bound . . . 82
4.3 Towards the $\hat{\mathcal{W}}_{\cap}$ measure: an exact refinement of $\mathcal{W}_{\cap}$ . . . 85
4.4 Conclusions . . . 91
4.5 Summary in Spanish . . . 92

5 Conclusions and future work **93**
5.1 Summary . . . 93
5.2 Future directions . . . 97
5.3 Summary in Spanish . . . 98

A Proofs of Chapter 2 **108**
A.1 Proofs of Section 2.4 . . . 108
A.2 Proofs of Section 2.5 . . . 112
A.2.1 Reduction by degrees . . . 112
A.2.2 $\mathcal{T}^{\mathbf{m}}$ measure . . . 126

B Proofs of Chapter 3 **140**
B.1 Proofs of Section 3.2 . . . 140
B.2 Proofs of Section 3.3 . . . 142
B.3 Proofs of Section 3.4 . . . 144

C Proofs of Chapter 4 **147**
C.1 Proofs of Section 4.2 . . . 147

Chapter 1

Introduction

Programming sits at the intersection of two disciplines: engineering, which demands that ideas work in the physical world, and mathematical logic, which studies formal systems of reasoning, including the functions that can be effectively computed as well as what properties they satisfy. One of the earliest and most fundamental properties we expect from programs is *termination*: that computation eventually produces a result. In the setting of typed λ-calculi, termination is guaranteed by the type system itself and is known as *strong normalization* (SN). Proofs of SN are among the deepest and most complex results in type theory [81, §12], and the search for new proof techniques remains an active area of research. This thesis introduces new proofs of SN, by means of *decreasing measures*, for the simply typed λ-calculus ($\lambda^{\to}$) and idempotent intersection types (IIT).

1.1 Historical overview

Since the very early days of programming, we have been attempting to show that our programs behave as expected. In 1949, Turing submitted a proof of (a representation of) two nested *for* loops terminating [101]. In 1967, Floyd introduced the idea of assigning "meaning" to programs [40]. In 1969, Hoare developed a formal framework to state properties of imperative programs, and to verify that they are satisfied throughout execution [55]. These works laid the foundation for program verification as we know it[1].

Programming languages are ways to specifically describe to the computer what it should do. Whenever we write a program, we expect it to satisfy several properties. One of them is *termination*. Unless, of course, we are writing a program that is expected to run forever. But even if that is the case, we would probably be able to split the program into finite and infinite parts. There would be terminating pieces of the software that produce specific outputs, and there would be an infinite master process that uses the terminating pieces. Think of a server that is expected to run forever: the loop that listens to requests is the infinite part, while the functions that handle each request are expected to end and produce an output after each request.

We can hence safely state that *we want our programs to terminate*. Specifically, we would never want that a program that was not intended to loop enters an infinite execution path. The problem of determining whether a program will eventually terminate is fundamental in computer science, and it is impossible to solve in general. This was shown by Alan Turing in 1936 [100], when he introduced the Halting Problem, stating that it is impossible to decide whether an arbitrary Turing

[1] Interestingly, as told by Morris and Jones [59, 75], Floyd and Hoare were not aware of Turing's work at the moment, so a direct influence is not documented.

machine will terminate. A direct consequence is that, for arbitrary programs of so-called *Turing-complete* programming languages, it is impossible to decide if they will finish execution. For those worried about termination, not everything is lost though.

Programming languages are called Turing-complete when they are able to express *all* effectively computable functions; this entails encoding unbounded recursion, and therefore non-termination[2]. For most programming tasks, we do not need such powerful tools, and restricted languages can provide us with strong safety guarantees. To understand this, let us proceed to describe the class of programming languages that we are particularly interested in.

On one hand, there are several approaches on how to define programming languages, one of them being the *functional* paradigm. It is based on mathematical functions as the building blocks, with its origins in the field of logic in the 1930s. On the other hand, there are two very general sources of errors: those based on bad combinations of the language, *e.g.* attempting to multiply two letters, and those that produce a valid, yet incorrect, result, *e.g.* multiplying by 1 when the solution required multiplying by 2. We can call the first class *structural errors*, and the second *value errors*. The first way, in both historical and engineering senses, to avoid misbehaviour in programs is to avoid *illegal combinations*. *Type systems* are logical frameworks that we endow our programming languages with, in order to rule out bad combinations and so determining what are the valid programs of the language. A *type* is a logical expression that describes some aspects of the pieces of a program. For instance, a letter could be given type `Char`, so we would know it would be problematic to use it in a sum. A program yielding a letter would also have type `Char`. A program taking a number and producing another number could have type *function* from `Nat` to `Nat`, usually noted with an arrow in between input and output, `Nat` $\rightarrow$ `Nat`. The amount of information we can put in the logical expression corresponding to a type depends on the expressiveness of the underlying logic of the typing systems. In this thesis we will stick with type systems with minimal logics. These will not involve parameterization at the level of types. We will be able to build types in two ways: by primitively, *i.e.*, not through language constructions, grouping values together; and by combining existing types into a function type. Examples of primitives types would be numbers and booleans, but we will actually abstract off these details, and refer to these primitive sets simply as base types, often noted with the greek letter τ. The reason is that we care more about the structure of programs we build, than domain related details such as the specific output values that they might yield. In the next section I will introduce the base language and type systems over which this thesis builds upon.

It is worth noticing, before proceeding, that the expressiveness of type systems can allow for very complex type expressions. For instance, when allowing to quantify over the values of both the logic and the language, it is possible to refer to arbitrary sets of values built from comprehension schemes, such as `Vec 10 Even`, the set of lists of length 10 that contain only even numbers.

1.2 Typed functional languages

The λ-calculus, introduced by Alonzo Church in 1936 [20], is a programming language and the model of computation underneath all functional programming languages, such as Haskell and Caml. It consists of three syntactic constructors: variables, abstractions and applications. Terms of the language are referred to using metavariables t, s, u, r and v. Variables are noted with x, y, z and w, often using mnemonic letters to signal the kind of input expected, such as f for functions and n, m for numbers. Abstractions correspond to anonymous functions and are noted with a λ followed by

[2]For a deep explanation on effectively computable functions, and on the origins of the main models of computation, the reader can check Philip Wadler's paper about proofs as programs [108].

the bound variable and the body of the function: $\lambda x.t$. Applications are written putting two terms together: ts. There are no further constructors; so, for instance, if one wants to compute numbers, one must encode them in some way. Church numerals do that, with numbers being functions that take a function and an arbitrary value, and apply the function to the value as many times as the number being defined. Hence 2 would be written $\lambda f.\lambda x.f(fx)$. Computation is defined, for applications with a λ as the left subterm, by substituting the argument for the bound variable in the abstraction body: $(\lambda x.t)s \to_\beta t[s/x]$.

The untyped version of the λ–calculus is a Turing-complete language, with infinite looping being encoded, for instance, through the term Ω

$$(\lambda x.xx)(\lambda x.xx)$$

and unrestricted recursion being implemented through the term known as Y-combinator

$$\lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$$

As mentioned in the previous section, we define type systems that we later attach to our languages. We do this to guarantee that they satisfy some logical properties that we can express within our system. The type system of the λ–calculus was introduced in 1940 [21], and it is known as the simply typed λ–calculus, often abbreviated STLC and noted with $\lambda^\to$. It consists of a base, primitive type τ, of which we abstract details, and the function type denoted by the arrow $\to$. We use metavariables A, B, C and D to refer to arbitrary types. Note that any combination of types using an arrow is valid. So, for instance, $(\tau \to \tau) \to \tau$ is a valid type: it is allowed to take functions as parameters in the type system. On top of types, type systems consist of typing rules that determine how it is valid to combine typed terms. We will not dive into too much details here; they will be addressed in the correspondings technical sections. We will only analyze the application rule here, which is the most characteristic typing rule of $\lambda^\to$, because it is the one that actually combines two terms. The rule $\to_\mathtt{e}$ states that a term t of type $A \to B$ (the function) can be combined with a term s of type C (the argument), if the argument has the same type as the parameter of the function, *i.e.*, $A = C$. This way, the type system rules out nonsensical programs. If we had, for instance, two different types for natural numbers `Nat` and letters `Char`, then applying with a letter a function that expects a number would be an invalid program under this type system, because $\mathtt{Nat} \neq \mathtt{Char}$.

Variables can be assigned a single type within their scope, so by inspecting Ω and Y one can conclude that they are not valid programs in $\lambda^\to$: they require x to have a function type, *e.g.* $A \to B$, and, within the same scope, to also have the type of its own parameter, *e.g.* A. This is a price to pay for safety: we relegate computational power and keep control.

There are systems that aim to recover some of this power without losing too much control, such as intersection type systems, where variables can have several types and therefore the self-application xx is indeed typable. However, Ω and Y do not have a type there either. The idempotent variant of intersection type systems, which was the first one introduced, is the basis for one the works presented in this thesis, so we will explore more of them later. Other, more expressive type systems are those based on dependent typing, where it is possible to quantify over elements of sets, and possibly over types too. These are the basis of proof assistants like Coq and Agda.

Coming back to simply typed λ–calculus, it is important to note that Turing-completeness is easily recovered if one introduces a term constructor for looping or recursion. Even in this scenario, one would expect that programs that do not use the looping or recursion constructor terminate. So, for $\lambda^\to$, termination is one of the essential safety properties, called *strong normalization*. It is

not a property we will express as a type and then assign to programs to ensure they terminate, it will rather be a metatheoretical property that is met by any program having a type in our system.

Let us describe the property intuitively. A term of the form $(\lambda x.t)s$ is called a *redex* (for reducible expression). We introduced computation in λ–calculus as the rule $(\lambda x.t)s \to_\beta t[s/x]$. The rule is defined closed under arbitrary contexts, hence a given term has as many redexes as occurrences of a λ applied with any term. Each different redex consists of an additional computation path. For instance, the term $I(II)$, where $I = \lambda x.x$ is the identity function, has two possible computation paths: one computing inside the argument II, and another computing the head redex $I(II)$. Other, more complex terms remain having several redexes after any computation, so we can speak about the reduction tree of a term. The property of strong normalization states that all possible reduction paths from a term, *i.e.*, all the branches of its reduction tree, are finite.

An interesting question here would be why to aim for all branches to be finite. Would it not be enough for *one* of them to be finite, in order to achieve the result? That is a property on its own, called *weak normalization*. From a computing perspective, where one expects to reach a result, this would be enough (provided there is only one possible result, which would be a safe assumption at this point). There are several reasons to expect to prove SN instead of WN. From a theoretical computer science perspective, one expects to deliver a language with proofs of the actual safety guarantees, not only with those that are essential for it to be useful. From a programming languages design perspective, one expects that the safety guarantees of the system are independent from implementation choices, such as the strategy chosen to evaluate programs and compute results. This gives the implementers the freedom to choose the computing strategy that best fits their needs. From a logic perspective, where programming languages are understood as logics, it is pragmatically relevant, since the proof of consistency of the logic is often easier to carry out by relying on the strong normalization proof of the language. From the perspective of curious people, a set of which the reader of this thesis is definitely a member, strong normalization is such an interesting question, with such interesting different ways of providing an answer, that it would be impossible to just pass it by. It is therefore the standard, when defining an extension of typed λ–calculi, to attempt for it to satisfy SN, and to prove the theorem.

This thesis is about new variants of one of the techniques for proving strong normalization for simply typed λ–calculus, also extended to IIT, and further refined to provide quantitative information about computing. In the remaining sections of this chapter, I will introduce some of the most known techniques for proving SN, as well as my contributions to the field.

1.3 Proof techniques for strong normalization

As stated by Pierce [81, §12], proofs of strong normalization are among the deepest and most elegant mathematics to be found in type theory. In this section I will introduce the most used technique for proving SN, which we could call the "semantic" one, and I will state what we miss from it. Then I will introduce an alternative technique, which we could call the "syntactic" one, and set the basis to the description of my own work. Then I will introduce a third approach based on reducing the problem of SN to that of WN. Finally, I will comment on some other proofs that can be found in the literature.

1.3.1 Reducibility

The method was introduced by Tait in 1967 [98], originally called *computability*, to prove weak normalization of the STLC. It was extended by Girard to SN [50] and to SN of System F [51] (the polymorphic λ-calculus), becoming the standard way to prove SN. It was later generalized by

abstracting over the target predicate (*i.e.*, SN), leading to *logical relations*, one of the most powerful techniques for proving properties of typed terms [2, 18, 41, 73, 74, 83, 94–96]. Strong normalization has played a crucial role at advancing and shaping proof techniques in type theory. For a broader discussion, the reader can check the historical takes on logical relations by Robert Harper [53] and Jean Gallier [42].

After giving reducibility the glowing presentation it deserves, we will explain in further detail the technique, and state some aspects that could be seen as limitations. It consists of three main parts:

1. Finding a set of properties that terms must satisfy to be provably terminating; according to Girard [52, §14.1.2], these were found originally by trial and error, so there is a notable arbitrary component in the formulation of this technique.

2. Assigning to each type a set of properties that their terms must satisfy in order to satisfy the properties from the previous item.

3. Proving that the set of terms of a type satisfy the properties from the previous item.

Reducibility is a very precise technique that finds exactly what is needed for terms to satisfy termination, cutting out any detail that is not strictly required to conduct the proof. This was noted by de Vrijer [32] and van de Pol [105], who state that one technical reason for reducibility to feel so abstract is that it ignores every detail that is not necessary. Sometimes we require more context to understand complex ideas, so some of this unnecessary information might indeed be of interest to us. Reducibility proofs rely on semantic arguments, which, while effective, lack intuition about *why* execution brings programs closer to termination (for a deeper analysis see Gallier's work [45, §1]). There have been several works analyzing reducibility proofs [43, 44, 46, 57, 66, 105]. Other proof methods were actively sought in the subsequent years. We introduce some of them in the next subsections.

It is worth clarifying that, despite possible appearances, this thesis does not oppose reducibility. Quite the opposite, I find it fascinating. This work is motivated by a strong interest in reducibility, and aims, up to some extent, to provide a deeper understanding of its internal structure.

1.3.2 Decreasing measures

In 1980, a new proof of SN for simply typed λ–calculus was introduced: Gandy defined a *decreasing measure*, providing the first combinatorial approach for proving the property [48]. Decreasing measures are functions assigning ordered values to programs, such that the associated value decreases with each execution step. This implies termination because we cannot provide an infinite chain of decreasing values within a well-founded order. While the technique of decreasing measures was not the first one introduced to the λ–calculus world for proving termination, proving finiteness of sequences by mapping the elements to well founded orders is an idea that accompanies mathematicians since antiquity (*e.g.*, Euclid's proof that every composite integer is divided by some prime number [54, Book VII, Proposition 31], Fermat's infinite descent [29], Turing's large routine check [101]).

Since then, other decreasing measures were defined, both for simply typed λ–calculus [30, 32] and for other extensions [17, 56, 62, 70, 109]. The one by de Vrijer is the most known one of these, since it introduces a simplification of Gandy's and became the standard decreasing measure to work with (*e.g.* , it is the one that TeReSe [99] uses).

While more explicit, this method may require heavy technical work in the proofs, and has resulted in complex functions that are more expensive to compute than running the actual program. It makes sense, however, that these functions are complicated: they take a program, its type derivation, and, implicitly, all its potential execution paths, and project these into a single element of a well-founded order. The fact that they find such an element that was, in a way, part of the program, is what makes them so informative. They can be thought of as condensing a program's complexity into a single piece of information. For reference, Gandy's measure gives a natural number as a result. Defining a decreasing measure involves looking, through the previously mentioned dimensions of a program, for pieces of information that can be proved to strictly shrink through execution. Of course, since the measure aims to prove SN, it is not possible to assume it and use the complete reduction tree as measure.

Decreasing measures have been defined for restricted notions of reductions, such as specific strategies. I will introduce a naming convention to distinguish them. Measures that decrease when a specific redex is contracted (*e.g.*, the rightmost innermost) entail weak normalization, so I will call them "weak" decreasing measures. Measures that decrease when arbitrary redexes are contracted entail strong normalization, so I will call them "strong" decreasing measures. Gandy's and de Vrijer's are strong.

1.3.3 Reduction to weak normalization and redex degrees

Another approach to prove SN, introduced by Nederpelt [76] and Klop [65], is to modify the calculus with non-erasing rules, *i.e.*, avoiding in some way reduction steps from erasing its argument, obtaining a system where SN and WN are equivalent, and therefore being required to prove only WN, which is usually simpler. These ideas are related to the λI-calculus, a variant of λ–calculus where bound variables must occur in the body of abstractions, therefore avoiding erasure without the need for different reduction rules. It is interesting to note here that the original λ–calculus defined by Church had this restriction, later dropped for convenience.

It is common for proofs taking this path [30, 60, 62, 64] to use weak decreasing measures to fulfill the WN proof. Specifically, it is usual for these measures to be based on the one defined by Turing [47], which relied on redex degrees. The degree of a redex corresponds to the height of the type of its abstraction. Turing observed that contracting a redex of a given degree cannot create new redexes of equal or greater degree. Following this idea, one can define a normalizing strategy by always selecting the rightmost innermost redex among those of maximum degree. The strategy can be proven normalizing by a decreasing measure argument, since the number of redexes of the higher degree always shrinks.

A detailed survey on most of the proofs based on these approaches was written by Sørensen as part of a paper that generalizes some of its aspects [91, §3].

1.3.4 Other approaches

Several other approaches, more or less related to those mentioned above, were followed to prove strong normalization for STLC throughout the literature. There has been great interest in syntactic (or combinatorial) proofs, providing *simpler* ways to prove the property. Some of them explicitly state that the avoidance of reducibility arguments is a target, seeking a proof formalizable in Peano or primitive recursive arithmetic. Sanchis [89] relied on an induction on the size of substituted variables, resorting to a notion of reachable terms[3]. Independently, van Daalen [104, pp.507-516] and David [26] used inductions on a lexicographic order consisting of three elements: the type of

[3]Sanchis' was actually the first proof of SN for $\lambda^{\rightarrow}$.

the substituted variable, the length of longest reductions from a term, and the complexity of the term. Loader [71] relied on the notions of applicative and substitutive types, based on polarity. Joachimski and Matthes [58] conducted an induction on types, requiring to simultaneously prove closure of SN under application and under substitution. Detailed surveys on the history of SN proofs can be found in Sørensen's thesis [90, §1.5] and in a work by Blanqui [14, §1].

1.4 Gödel's *koan*

Termination of programs is such a complex and fascinating question, that it remains as an interesting field of research. Although we already know the answer for most of the systems of our interest, it is still valuable to develop new techniques for achieving the proof, as well as new ways of understanding both the problem and the answer, offering insight on what is going on behind the process of computing.

Before Gandy introduced his measure, Gödel wondered whether the construction of an "easy" decreasing measure was possible. Gandy's measure is not considered easy. Neither are the several others defined afterwards. Gödel asked that question to Howard, who made significant advances towards solving it, but without achieving a solution. Due to its imprecise nature, it is not considered a problem, but a mathematical *koan*[4]. The koan remains open today as Problem #26 of the *Typed Lambda Calculi and its Applications* list of open problems [15], and as Problem #19 of the *Rewriting Techniques and Applications* list of open problems [33].

An "easy" decreasing measure might be one that is simple to define, not too complicated to prove, straightforward to understand, and cheap to compute. And it might as well be elusive: we do not know if we can take a program's information and isolate the data that establish termination into a simple, decreasing element. Nevertheless, the koan remains a deep motivation for researchers looking to understand the very fundamentals of λ-calculi programs dynamics. Even if we don't solve the koan, each decreasing measure defined finds a different way of isolating termination information from the term and its reduction tree, shedding light on a new aspect of SN.

While solving the koan sounds ambitious, it serves as a guiding goal to keep looking for easier decreasing measures. A measure with the "easy" attributes mentioned above would be of great use in program analysis, both statically and dynamically. Consider each of them, even separately. Having simple, intuitive, or cheap ways of computing a measure, would serve the purposes of implementation, formalization (in the mechanization sense), didactics, and program assessment. For instance, if we have several programs available to perform different tasks, we can use the measure to pick the most efficient one. If the measure is significantly cheaper to compute than to run the actual program, this could be done even in real time. A concrete example could be that of comparing programs in systems modulo isomorphisms [3, 36–38, 93] where programs are identified by an equivalence relation based on isomorphic types, in such a way that those with the same behaviour belong to the same class.

1.5 Contributions

This thesis contains the results of two published papers, along with unpublished results and a discussion of a work in progress. It is centered around the definition of decreasing measures for

[4] According to Pólya [87], the candidate solution of a problem should be clearly recognisable as such in a well-defined way. The proof of correctness of the candidate solution still may be hard. A "koan" is a non-precise question for which the space of solutions is a priori not clear. This use of the word was reportedly [15, Problem#26] attributed to Klop.

typed λ–calculi, starting with two distinct decreasing measures for simply typed λ–calculus, followed by the extension of one of them to idempotent intersection types, which is a finitary form of polymorphism (where, for instance, auto–application is allowed), and finishing with the basis of a quantitative improvement of that measure.

1.5.1 Two decreasing measures for simply typed λ-terms

In a joint work with Pablo Barenbaum [8], we introduced two new proofs of SN for the simply typed λ–calculus, through the definition of two strong decreasing measures. They are defined constructively, based on a variant of the existing *memory* calculi [30,65,76,77] which avoids erasure of arguments by keeping a syntactic construct in terms. We dubbed these constructs *wrappers*, and handling them allowed us to produce the two different measures. Intuitively speaking, they are based on exploring the reduction tree of a term in controlled ways.

The first measure is based on "simply" counting wrappers in the normal form of terms in the memory calculus. Normal forms are obtained using a function defined by structural recursion that partially simulates reduction. This function is guaranteed to yield a result by Turing's observation about redex contractions not creating redexes of equal or greater degree holding in the calculus[5]. The way of exploring the reduction tree is to enlarge it by adding wrappers, and then to gather all the consumed arguments through a controlled simulated execution that is proved to arrive at a normal form.

The second measure adapts a result by Alan Turing to a stronger one. He defined a weak decreasing measure for simply typed λ–calculus, by noticing that picking the rightmost innermost redex, from those of the greatest degree, leads to the normal form. This holds because, by its own prior observation, contracted redexes cannot create other redexes of equal or greater degree. Therefore, contracting those specific redexes decreases the number of occurrences of those of greater degree in the term. We extended this idea to strong decreasing measures by looking for additional information, related to redex degrees, that shrinks throughout execution. We obtained a complex structure of degree-bounded nested multisets, where the elements of these multisets are pairs containing the degree of a redex and a lower-degree-bounded multiset. The way of analyzing the reduction tree consists of recursively branching, from highest degree to lowest, on all the possible ways to reduce to terms by only contracting redexes of the current single degree. All of these after enlargening the reduction trees with the wrappers.

1.5.2 An extension to idempotent intersection types

The wrappers counting measure showed to be intuitive to grasp. In a joint work with Pablo Barenbaum and Simona Ronchi Della Rocca [7], we adapted it to idempotent intersection types, providing a strong decreasing measure for this system.

Intersection types systems, which we will note as $\Lambda_\cap$, allow variables to have several types. As a result, terms that would not have a type in the simply typed λ–calculus can be assigned a type in $\Lambda_\cap$. For instance, it is possible to give type to terms like $\lambda x.xx$, because the bound x can be assigned the set $\{A \to A, A\}$, and each occurrence of x can be assigned one of these. Since these systems present a finitary form of polymorphism, it is, from the perspective of the technique's extensibility, a relevant adaptation.

It is worth highlighting here, especially for the reader that is familiar with intersection types, that we are dealing with the idempotent variant of them. The non-idempotent one has a more direct relation to decreasing measures, due to derivations handling highly quantitative information.

[5] Actually, a weaker version of Turing's observation is enough for this measure, as will be explained later.

The usual presentation of intersection types has extrinsic typing, *i.e.*, variables are assigned types through the contexts, as opposed to types occurring in the grammar of terms through a label in variables. The memory calculus on which our technique is based has intrinsic typing due to redex degrees (and thus typing information about variables) being a constantly required information. Hence we defined an intrinsic variant of intersection types, that we proved to correspond to and simulate the extrinsic one.

There were two previously defined weak decreasing measures for idempotent intersection types. Kfoury and Wells [62] extended an extrinsic version to compute a measure yielding a multiset of natural numbers. Boudol [17] defined an ad-hoc intrinsic version, *i.e.*, without exploring the proof-theoretical relation with respect to the extrinsic, and extended it to compute a measure yielding a pair of natural numbers. A strong decreasing measure could be defined following the translation of $\Lambda_\cap$ to STLC [19], plus the use of a measure for the simply typed case; such an approach, though, carries disadvantages that will be discussed later. Ours consists thus of the first strong decreasing measure defined natively for IIT, having the simplest codomain of natural number, and with a novel presentation of intrinsic typing for $\Lambda_\cap$, proven to be in correspondence and simulation with respect to it.

1.5.3 A refinement to exactness

Decreasing measures that yield natural numbers have the side contribution of determining an upper bound for the number of computation steps. Indeed, consider a term t whose measure is N. If there was a reduction path with $M > N$ steps, then the normal form would necessarily have a measure lower than 0, since each step subtracts at least 1 from N, which is impossible.

An interesting question about this kind of decreasing measures is thus how tight the number is. We stated earlier that Gandy introduced the first strong decreasing measure for simply typed λ–calculus [48], and that this measure was later simplified by de Vrijer [32]. In that same work, de Vrijer also introduces a refinement of the measure, which consists precisely of a refinement to exactness of the upper bound. That is, the measure yields a number that coincides with the number of steps of the longest reduction path from the term. A natural improvement over our technique would be to refine it to exactness.

We defined a refinement to exactness for the wrappers counting technique in simply typed λ–calculus, and analyzed the challenges that we encountered when trying to apply the same plan to $\Lambda_\cap$, outlining early ideas for future work.

1.5.4 Other (non-included) works

During my PhD, I supervised the Master's thesis of Agustín Séttimo together with Cecilia Manzino, both from Universidad Nacional de Rosario. We formalized in Agda a directed variant of a STLC modulo isomorphisms, including the strong normalization and progress proofs. A publication will appear at the postproceedings of the 37th Symposium on Implementation and Application of Functional Languages (IFL'25). A preliminary version can be found at arXiv [97].

1.6 Organization of this thesis

The thesis is structured mostly following the papers, thematically and chronologically. In Chapter 2, I introduce the auxiliary memory λ–calculus, together with the two decreasing measures. In Chapter 3, I introduce the adaptation of the wrappers counting measure to idempotent intersection types, including the intrinsic presentation of the original system. In Chapter 4, I introduce the

refinement to exactness of the wrappers counting measure for simply typed λ–calculus, together with an analysis of the problems found in adapting it for $\Lambda_\cap$, and an early proposal on how to solve them. In Chapter 5, I conclude, summarizing the work done and discussing possible directions for future work.

1.7 Resumen en español (Summary in Spanish)

La idea de demostrar formalmente que un programa termina está presente en la comunidad desde los inicios de la computación, con el trabajo "Checking a Large Routine" de Alan Turing en 1949 [75, 101] demostrando que dos (representaciones de) *for* anidados terminan, observando cómo sus índices se acercan invariablemente a la condición de corte. Uno de los aspectos característicos del λ-cálculo tipado es su seguridad: la computación preserva el tipo, y los programas tipados terminan. Es usual para sus extensiones que la propiedad de terminación, que llamamos normalización fuerte (SN por sus siglas en inglés) sea una de las primeras importantes que se intenta demostrar. Las pruebas de terminación son de las más complejas en el área de teoría de tipos, y se desarrollaron varias técnicas para demostrarla a lo largo de las décadas. Las más conocidas son las de reducibilidad y medidas decrecientes. En esta tesis definimos dos medidas decrecientes para el λ-cálculo simplemente tipado (STLC), extendemos una de ellas a tipos intersección idempotente (IIT), y luego aplicamos un refinamiento para STLC y lo discutimos para IIT.

Chapter 2

Two decreasing measures for the simply typed λ–calculus

2.1 Introduction

As stated in Chapter 1, there are different approaches for proving SN of typed λ–calculi. The most used one is *reducibility*, based on interpreting types as sets of "well–behaved" terms, and then proving that all typed terms are actually well–behaved, according to the restrictions posed by its type. We argue that reducibility-based proofs are quite abstract; this occurs because they ignore every detail that is not strictly necessary to reach the goal, as stated by de Vrijer and van de Pol [32, 105].

Another approach for proving SN is that of reducing it to WN. This is the path taken by Klop [65] and Nederpelt [76], to whom the technique can be traced back. This reduction is obtained by translating each typable λ-term M to a typable term M' of the *λI-calculus*; for a detailed explanation see Sørensen's book [92, Section 3.5]. The λI-calculus is the restriction of the λ–calculus in which every abstraction $\lambda x.M$ contains at least one free occurrence of x in the body M. One key aspect of the λI-calculus is that it is *non-erasing*. Other approaches to avoid erasure with similar results can be found in the literature [30, 60, 62, 64, 91]. It is usual for this approach [30, 60, 62, 64] to resort to WN proofs based on redex–degrees arguments to complete the picture, *i.e.*, proving SN. With redex–degrees proofs we mean those based on the independent observation of Turing and Prawitz, about contraction of redexes not being able to create redexes of greater or equal degree[1]. Based on it, Turing defined a *weak* decreasing measure, that is, such that decreases along a specific reduction strategy[2].

A third way of proving SN is that of *strong* decreasing measures, that is, those that decrease along any reduction. Gandy and de Vrijer pioneered these for simply typed λ–calculus, introducing different variations of interpretations of terms into the domain of hereditarily strictly monotonic functions and increasing functionals. Gandy [48] first proposed a translation of terms from simply typed λ–calculus into a non-erasing λI extended with arithmetic constructions and rewriting rules, so that computing a result there yielded a natural number that served as a decreasing measure. De Vrijer [32] simplified and later refined Gandy's measure into exactness, *i.e.*, providing a number serving as an exact upper bound for the length of the longest reduction chain from any term. His

[1] Although the observation was made independently by both Turing and Prawitz, due to word economy and historical precedence, we will refer to it as Turing's observation.

[2] As stated in the introduction, classifying decreasing measures as *weak* or *strong* merely follows the convention of the property they entail, and must not be understood as an adjectivation.

measure did not involve WN of the intermediate calculus augmented with arithmetic; rather, it directly computed the number in the interpretation.

In this chapter I present the results of a work with Pablo Barenbaum that was published at FSCD'23 [8]. We combined non-erasure and redex–degrees to produce strong decreasing measures. We constructively defined two strong decreasing measures based on an auxiliary non-erasing calculus and operations that rely fundamentally on Turing's observation. The first measure, $\mathcal{W}$, is based on counting syntactic elements in the normal form of the auxiliary calculus, which is obtained by a structural recursive function whose soundness proof implies a weaker version of Turing's observation. The second measure, $\mathcal{T}^{\mathbf{m}}$, is an adaptation of Turing's weak decreasing measure, and consists of nested multisets of developments strictly separated by redex–degrees. Since the measures greatly differ, we postpone the discussion of their specific roots and related works to their corresponding sections.

2.1.1 Organization of the chapter

In Section 2.2, we formalize the ideas behind redex–degrees. In Section 2.3, we introduce the auxiliary calculus. In Section 2.4, we introduce the first measure, together with its corresponding decreasing proof. In Section 2.5, we dig into some technical properties of the auxiliary calculus that are needed to define the second measure, and introduce it and its decreasing proof. Prototype implementations in Haskell of the measures introduced in this chapter can be found at `https://github.com/cfsottile/measures`.

2.2 Redex–degrees

A *redex* in the STLC is an applied abstraction, *i.e.*, a term of the form $(\lambda x.M)N$. The *degree* of a redex is defined as the *height* of the type of its abstraction. For example, the redex $(\lambda x^{\tau}.x)y^{\tau}$, which has type τ, has degree $\mathsf{h}(\tau \to \tau) = 1$, since its abstraction is $\lambda x^{\tau}.x$ of type $\tau \to \tau$. A crucial observation, that can be attributed to an unpublished note by Turing (as reported by Gandy [47]; for a thorough explanation the reader can consult a work by Barendregt and Manzonetto [12]), is that *contracting a redex cannot create a redex of higher or equal degree*. Recall that a redex S is *created* by the contraction of a redex R if S has no *ancestor* before R. Indeed, as shown by Lévy [68], in the λ-calculus, redexes can be created in exactly one of the three ways below:

$$\begin{array}{lrcl}
1. & (\underline{\lambda}x.x)(\lambda y.M)N & \to_\beta & (\underline{\lambda}y.M)N \\
2. & (\underline{\lambda}x.\lambda y.M)NP & \to_\beta & (\underline{\lambda}y.M[N/x])P \\
3. & (\underline{\lambda}x.\ldots xM\ldots)(\lambda y.N) & \to_\beta & \ldots(\underline{\lambda}y.N)M[\lambda y.N/x]\ldots
\end{array}$$

where we underline the λ of the contracted redex on the left, and the λ of the created redex on the right. In each of these cases, it can be seen that the degree of the created redex is strictly lower than the degree of the contracted redex. For instance, in creation case 1, the type of the contracted redex is of the form $(A \to B) \to (A \to B)$, while the type of the created redex is $A \to B$, so the height strictly decreases.

With this fact in mind, for each term M one can define what we call *Turing's measure*, *i.e.*, the multiset $\mathcal{T}(M)$ of the degrees of all the redexes of M. One may hope that any reduction step $M \to_\beta N$ decreases the measure, *i.e.*, $\mathcal{T}(M) \succ \mathcal{T}(N)$, where "$\succ$" is the usual well-founded multiset ordering induced by the ordering $(\mathbb{N}, >)$ of its elements [34]. Unfortunately, this is not the case: even though contracting a redex can only create redexes of strictly lower degree, it can still make an arbitrary number of *copies* of redexes of arbitrarily large degrees.

In his notes, Turing observed that one can follow a reduction strategy that always selects the *rightmost innermost* redex of highest degree. This strategy ensures that the contracted redex does not copy redexes of higher or equal degree, which makes the $\mathcal{T}(\cdot)$ measure strictly decrease, thus proving that the STLC is WN. An even simpler measure that also decreases, if one follows this strategy, is $\mathcal{T}'(M) = (D, n)$, where D is the maximum degree of the redexes in M and n is the number of redexes of degree D in M. Similar ideas were exploited by Prawitz [86] and Gentzen (as reported by von Plato [82]) to normalize proofs in natural deduction.

Redex degrees had thus been used to define concrete measures such as $\mathcal{T}(M)$ and $\mathcal{T}'(M)$, that are computable in linear time and decrease when following a particular reduction strategy. As stated previously, following the terminology of the property they entail, which is that of weak normalization, I dubbed them *weak decreasing measures*.

2.3 STLC with memories

The $\lambda^{\mathbf{m}}$-calculus is an extension of the STLC with terms of the form $t\langle s\rangle$, in which the β-reduction rule keeps an extra memorized copy of the argument in the "wrapper" $\langle s\rangle$, so that contracting a redex like $(\lambda x.t)s$ does not erase s, even if x does not occur free in t. A wrapper $t\langle s\rangle$ should be understood as essentially the term t in which s is a *memorized term*, that is, leftover garbage that can be reduced but cannot interact with the context in any way. The type of $t\langle s\rangle$ is the same as the type of t, disregarding the type of s. Note that terms of the $\lambda^{\mathbf{m}}$-calculus are ranged over by $t, s, \ldots$ (rather than $M, N, \ldots$); this is in order to quickly make clear to which system a term belongs, as some properties relate terms from STLC and $\lambda^{\mathbf{m}}$.

The β-reduction rule is modified so that contracting a redex $(\lambda x.t)s$, besides substituting the free occurrences of x by s in t, produces a wrapper that contains a copy of the argument s. The reduction rule is $(\lambda x.t)\langle r_1\rangle \ldots \langle r_n\rangle s \to_{\mathbf{m}} t[s/x]\langle s\rangle\langle r_1\rangle \ldots \langle r_n\rangle$. Note that we allow the presence of an arbitrary number of memorized terms mediating between the abstraction and the application. This is to avoid memorized terms *blocking* redexes. For example, if $I = \lambda x.x$:

$$(\lambda x.x(xy))I \to_{\mathbf{m}} (I(Iy))\langle I\rangle \to_{\mathbf{m}} (Iy)\langle Iy\rangle\langle I\rangle \to_{\mathbf{m}} (Iy)\langle y\langle y\rangle\rangle\langle I\rangle \to_{\mathbf{m}} y\langle y\rangle\langle y\langle y\rangle\rangle\langle I\rangle$$

Then we study some syntactic properties of $\lambda^{\mathbf{m}}$. In particular, we define a relation $t \rhd s$ of *forgetful reduction*, meaning that s is obtained from t by erasing one memorized subterm. For example, $x\langle x\langle y\rangle\rangle\langle y\langle z\rangle\rangle \rhd x\langle y\langle z\rangle\rangle$. Forgetful reduction is used as a technical tool to prove that the measures are decreasing in the following sections.

In this section we define the $\lambda^{\mathbf{m}}$-calculus, and we prove some of the properties that are needed in the following sections to prove that the $\mathcal{W}$-measure and the $\mathcal{T}^{\mathbf{m}}$-measure are decreasing. In particular, we discuss *subject reduction* (Proposition 2.3.7) and *confluence* (Proposition 2.3.14); we define, by structural recursion, an operation of *simplification* (Definition 2.4.1) which turns out to calculate the normal form of a term (Proposition 2.4.10); and we define the relation called *forgetful reduction* (Definition 2.4.11), which is shown to commute with reduction (Proposition 2.4.15).

Related works and influence The idea of keeping "leftover garbage" can be traced back to at least the works of Nederpelt [76] and Klop [65], who studied non-erasing variants of (possibly) erasing rewriting systems, in order to relate weak and strong normalization. Many variations of these ideas have been explored in the past, such as in de Groote's notion of β_S reduction [30] or Neergaard and Sørensen calculus with memory [77]. Instead of using the $\lambda^{\mathbf{m}}$-calculus, it is possible that other non-erasing systems may be used. The interesting parts are not the specific non-erasure flavour, but the definitions and properties proved on them, particularly the avoidance of directly

relying on the theorem reducing SN to WN, in favour of an internalization of that idea into the measure computation. The choices behind our memory calculus are simply a matter of convenience towards our objectives.

The definition of reduction in the $\lambda^{\mathbf{m}}$-calculus, which allows arbitrary memory in between the abstraction and the application, is inspired by Accattoli and Kesner's work on calculi with explicit substitutions "at a distance" [4]. This mechanism can be traced back, again, to at least the work of Nederpelt [76].

The property of *increasingness* will be discussed in the section introducing the measure $\mathcal{W}$. The definition of the $\lambda^{\mathbf{m}}$-calculus as a means to obtain an *increasing* measure was inspired by the fact that, in explicit substitution calculi without erasure, labeled reduction (in the sense of Lévy labels [68]) increases the sum of the sizes of all the labels in the term [6].

The $\lambda^{\mathbf{m}}$-calculus

Let us first fix the notation and nomenclature. Types of the simply typed λ–calculus are either a base type (τ) or arrow types ($A \to B$). Terms are either variables ($x^A, y^A, \ldots$), abstractions ($\lambda x^A.M$), or applications (MN), with the usual typing rules. Terms are defined up to α-renaming of bound variables. We adopt an *à la* Church presentation of the simply typed λ–calculus, but we omit type decorations on variables, if it is clear from the context. The β-reduction rule is $(\lambda x.M)N \to_\beta M[N/x]$ where $M[N/x]$ is the capture-avoiding substitution of the free occurrences of x in M by N. We adopt Barendregt's convention for variable names: no name is used for two distinct variables. As stated previously, we use $M, N, P, \ldots$ to range over terms of the simply typed λ–calculus, and $t, s, r, \ldots$ to range over $\lambda^{\mathbf{m}}$-terms.

The set of types of $\lambda^{\mathbf{m}}$ remains unchanged from simply typed λ–calculus. The set of $\lambda^{\mathbf{m}}$-*terms* is given by the following grammar:

$$t \quad ::= x^A \mid \lambda x^A.t \mid tt \mid t\langle t\rangle$$

The four kinds of terms are respectively called *variables*, *abstractions*, *applications*, and *wrappers*. In a wrapper $t\langle s\rangle$, the subterm t is called the *body* and s is called the *memorized term*. As in the simply typed λ–calculus, we usually omit type annotations and terms are regarded up to α-renaming. A *context* is a term $\mathtt{C}$ with a single free occurrence of a distinguished variable $\Box$, and $\mathtt{C}[t]$ is the variable-capturing substitution of the occurrence of $\Box$ in $\mathtt{C}$ by t.

Typing judgments are of the form $\Gamma \vdash t : A$ where Γ is a partial function mapping variables to types. Derivable typing judgments are defined by the following rules:

$$\frac{}{\Gamma, x : A \vdash x^A : A} \qquad \frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x^A.t : A \to B} \qquad \frac{\Gamma \vdash t : A \to B \quad \Gamma \vdash s : A}{\Gamma \vdash ts : B} \qquad \frac{\Gamma \vdash t : A \quad \Gamma \vdash s : B}{\Gamma \vdash t\langle s\rangle : A}$$

A term t is *typable* if $\Gamma \vdash t : A$ holds for some Γ and some A. Unless otherwise specified, when we speak of "terms" we mean "typable terms". It is straightforward to show that a typable term has a unique type. We write $\mathsf{type}(t)$ for the type of t.

A *memory*, written $\mathtt{L}$, is a list of wrappers, given by the grammar $\mathtt{L} ::= \Box \mid \mathtt{L}\langle t\rangle$. If t is a term and $\mathtt{L}$ is a memory, we write $t\mathtt{L}$ for the term that results from appending all the wrappers in $\mathtt{L}$ to t, that is, $(t)(\Box\langle s_1\rangle \ldots \langle s_n\rangle) = t\langle s_1\rangle \ldots \langle s_n\rangle$. We write $t[s/x]$ for the operation of capture-avoiding substitution of the free occurrences of x in t by s. The $\lambda^{\mathbf{m}}$-calculus is the rewriting system whose objects are typable $\lambda^{\mathbf{m}}$-terms, endowed with the following notion of reduction, closed by compatibility under arbitrary contexts:

Definition 2.3.1 (Reduction in the $\lambda^{\mathbf{m}}$-calculus)**.** $(\lambda x.t)\mathtt{L}s \to_{\mathbf{m}} t[s/x]\langle s\rangle\mathtt{L}$

Abstractions followed by a memory, *i.e.*, terms of the form $(\lambda x.t)\mathtt{L}$, are called **m**-*abstractions*. Note that all abstractions are also **m**-abstractions, as $\mathtt{L}$ may be empty.

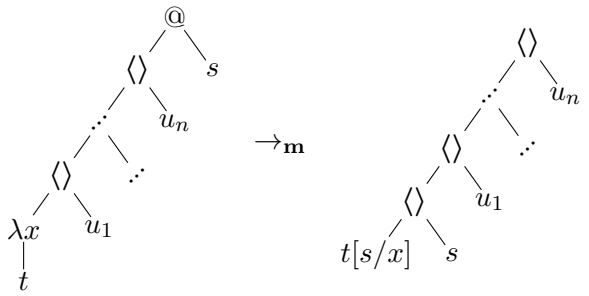
Remark 2.3.2. $t\langle s\rangle$ is an **m**-abstraction if and only if t is an **m**-abstraction.

We highlight that memories $\mathtt{L}$ are meta-level notation and not $\lambda^{\mathbf{m}}$-terms themselves. For instance, $\Box\langle t\rangle$ is not a $\lambda^{\mathbf{m}}$-term.

Example 2.3.3. Let $\mathtt{L} = \Box\langle u_1\rangle \dots \langle u_n\rangle$. Then the reduction step is

$$\begin{array}{rcll} (\lambda x.t)\mathtt{L}s & = & (\lambda x.t)(\Box\langle u_1\rangle \dots \langle u_n\rangle)s & \\ & = & (\lambda x.t)\langle u_1\rangle \dots \langle u_n\rangle s & \\ & \to_{\mathbf{m}} & t[s/x]\langle s\rangle\langle u_1\rangle \dots \langle u_n\rangle & \\ & = & (t[s/x]\langle s\rangle)(\Box\langle u_1\rangle \dots \langle u_n\rangle) & = \quad t[s/x]\langle s\rangle\mathtt{L} \end{array}$$

The syntax trees of the source and target terms are the following. We use $\langle\rangle$ as the node symbol for terms with wrappers, and we put the wrapped term as the right subtree.



Example 2.3.4. Let $\mathtt{L} = \Box\langle w_1\rangle\langle w_2\rangle$. Then

$$\begin{array}{rcll} (\lambda x.y)\mathtt{L}z & = & (\lambda x.y)(\Box\langle w_1\rangle\langle w_2\rangle)z & \\ & = & (\lambda x.y)\langle w_1\rangle\langle w_2\rangle z & \\ & \to_{\mathbf{m}} & y\langle z\rangle\langle w_1\rangle\langle w_2\rangle & \\ & = & (y\langle z\rangle)(\Box\langle w_1\rangle\langle w_2\rangle) & = \quad y\langle z\rangle\mathtt{L} \end{array}$$

A *redex* is an expression matching the left-hand side of the $\to_{\mathbf{m}}$-reduction rule, which must be an *applied* **m**-*abstraction*, *i.e.*, a term of the form $(\lambda x.t)\mathtt{L}s$. The *height* of a type is given by $\mathsf{h}(\tau) := 0$ and $\mathsf{h}(A \to B) := 1 + \max(\mathsf{h}(A), \mathsf{h}(B))$. The *degree* of an **m**-abstraction $(\lambda x.t)\mathtt{L}$ is defined as the height of its type; note that this number is always strictly positive, since the type must be of the form $A \to B$. Moreover, this type is unique, so the operation is well-defined. The *degree* of a redex $(\lambda x.t)\mathtt{L}s$ is defined as the degree of the **m**-abstraction $(\lambda x.t)\mathtt{L}$. The *max-degree* of a term t is written $\delta_{\mathsf{max}}(t)$ and it is defined as the maximum degree of the redexes in t, or 0 if t has no redexes. The *weight* $\mathsf{w}(t)$ of a $\lambda^{\mathbf{m}}$-term t is the number of wrappers in t.

Example 2.3.5. Let $t := (\lambda x^{\tau\to\tau}.\lambda y^{\tau}.y^{\tau}\langle x^{\tau\to\tau}(x^{\tau\to\tau}z^{\tau})\rangle)Iw^{\tau}$, where $I := \lambda x^{\tau}.x^{\tau}$. One possible way to reduce t is:

$$\begin{array}{rcl} (\lambda x.\lambda y.y\langle x(xz)\rangle)Iw & \to_{\mathbf{m}} & (\lambda y.y\langle I(Iz)\rangle)\langle I\rangle w \quad \to_{\mathbf{m}} \quad w\langle I(Iz)\rangle\langle w\rangle\langle I\rangle \\ & \to_{\mathbf{m}} & w\langle I(z\langle z\rangle)\rangle\langle w\rangle\langle I\rangle \quad \to_{\mathbf{m}} \quad w\langle z\langle z\rangle\langle z\langle z\rangle\rangle\rangle\langle w\rangle\langle I\rangle = s \end{array}$$

The degrees of the redexes contracted in each step are 2, 1, 1, and 1, in that order. Note that $\delta_{\mathsf{max}}(t) = 2$ and that the weight of the resulting term is $\mathsf{w}(s) = 6$.

The $\lambda^{\mathbf{m}}$-calculus satisfies the properties of subject reduction and confluence.

Lemma 2.3.6 (Substitution lemma)**.** Let $\Gamma, x : A \vdash t : B$ and $\Gamma \vdash s : A$. Then $\Gamma \vdash t[s/x] : B$.

Proof. By induction on t. We consider the wrappers case, which is the only different from the usual proof. Let $t = t_1\langle t_2\rangle$. Then $\Gamma, x : A \vdash t_1\langle t_2\rangle : B$, with $\Gamma, x : A \vdash t_1 : B$ and $\Gamma, x : A \vdash t_2 : C$. By definition of the substitution function $(t_1\langle t_2\rangle)[s/x] = t_1[s/x]\langle t_2[s/x]\rangle$. By IH $\Gamma \vdash t_1[s/x] : B$ and $\Gamma \vdash t_2[s/x] : C$, therefore by typing rules and substitution definition, we have $\Gamma \vdash (t_1\langle t_2\rangle)[s/x] : B$. □

Proposition 2.3.7 (Subject reduction)**.** Let $\Gamma \vdash t : A$ and $t \to_{\mathbf{m}} s$. Then $\Gamma \vdash s : A$.

Proof. By induction on the reduction $t \to_{\mathbf{m}} s$. We consider only the base case and wrappers cases, which are those that differ from the usual proof.

1. $(\lambda x^B.t_1)\mathtt{L}\, t_2 \to_{\mathbf{m}} t_1[t_2/x]\langle t_2\rangle\mathtt{L}$: Then t must be typed with the following derivation scheme, where n is the number of wrappers in $\mathtt{L}$ and Π_i is the derivation of the i-th term wrapped in $\mathtt{L}$:

$$\dfrac{\dfrac{\dfrac{\Gamma, x : B \vdash t_1 : A}{\Gamma \vdash \lambda x^B.t_1 : B \to A}\ \Pi_1}{\vdots\qquad\ \Pi_n}}{\dfrac{\Gamma \vdash (\lambda x^B.t_1)\mathtt{L} : B \to A \quad \Gamma \vdash t_2 : B}{\Gamma \vdash (\lambda x^B.t_1)\mathtt{L}\, t_2 : A}}$$

   Hence from Lemma 2.3.6, since $\Gamma, x : B \vdash t_1 : A$ and $\Gamma \vdash t_2 : B$, we obtain $\Gamma \vdash t_1[t_2/x] : A$. Then from the corresponding $n + 1$ applications of the wrapper rule we obtain $\Gamma \vdash t_1[t_2/x]\langle t_2\rangle\mathtt{L} : A$.

2. $t_1\langle t_2\rangle \to_{\mathbf{m}} t_1'\langle t_2\rangle$ with $t_1 \to_{\mathbf{m}} t_1'$: Then $\Gamma \vdash t_1\langle t_2\rangle : A$, with $\Gamma \vdash t_1 : A$ and $\Gamma \vdash t_2 : B$. By IH, we have $\Gamma \vdash t_1' : A$. Thus $\Gamma \vdash t_1'\langle t_2\rangle : A$.

3. $t_1\langle t_2\rangle \to_{\mathbf{m}} t_1\langle t_2'\rangle$ with $t_2 \to_{\mathbf{m}} t_2'$: Then $\Gamma \vdash t_1\langle t_2\rangle : A$, with $\Gamma \vdash t_1 : A$ and $\Gamma \vdash t_2 : B$. By IH, we have $\Gamma \vdash t_2' : B$. Thus $\Gamma \vdash t_1\langle t_2'\rangle : A$.

□

In the remainder of this subsection we prove that confluence holds for this calculus. We follow the parallel reduction approach.

Definition 2.3.8.

1. The parallel reduction $\Rightarrow_{\mathbf{m}}$ is defined on terms and memories by the following rules, closed under contexts.

$$\begin{array}{llllll}
x^A & \Rightarrow_{\mathbf{m}} & x^A & & & \\
\lambda x^A.t & \Rightarrow_{\mathbf{m}} & \lambda x^A.t' & \text{if} & t \Rightarrow_{\mathbf{m}} t' & \\
t\, s & \Rightarrow_{\mathbf{m}} & t'\, s' & \text{if} & t \Rightarrow_{\mathbf{m}} t' \quad \text{and} & s \Rightarrow_{\mathbf{m}} s' \\
(\lambda x^A.t)\mathtt{L}\, s & \Rightarrow_{\mathbf{m}} & t'[s'/x]\langle s'\rangle\mathtt{L}' & \text{if} & t \Rightarrow_{\mathbf{m}} t',\ s \Rightarrow_{\mathbf{m}} s',\ \mathtt{L} \Rightarrow_{\mathbf{m}} \mathtt{L}' & \\
t\langle s\rangle & \Rightarrow_{\mathbf{m}} & t'\langle s'\rangle & \text{if} & t \Rightarrow_{\mathbf{m}} t' \quad \text{and} & s \Rightarrow_{\mathbf{m}} s' \\
\Box & \Rightarrow_{\mathbf{m}} & \Box & & & \\
\mathtt{L}\langle t\rangle & \Rightarrow_{\mathbf{m}} & \mathtt{L}'\langle t'\rangle & \text{if} & \mathtt{L} \Rightarrow_{\mathbf{m}} \mathtt{L}' \quad \text{and} & t \Rightarrow_{\mathbf{m}} t'
\end{array}$$

   Note that $\Rightarrow_{\mathbf{m}}$ is non-deterministic, since when the term is a $\to_{\mathbf{m}}$-redex the application rule may be applied instead of contracting the head redex.

2. The complete development of a term t is the following function, which denotes the result of the simultaneous reduction of all visible redexes in t:

$$\begin{array}{rcll} \mathtt{C}(x^A) & := & x^A & \\ \mathtt{C}(\lambda x^A.t) & := & \lambda x^A.\mathtt{C}(t) & \\ \mathtt{C}((\lambda x^A.t)\mathtt{L}\, s) & := & \mathtt{C}(t)[\mathtt{C}(s)/x]\langle\mathtt{C}(s)\rangle\mathtt{C}(\mathtt{L}) & \\ \mathtt{C}(t\, s) & := & \mathtt{C}(t)\,\mathtt{C}(s) & \text{if } t \text{ is not an } m\text{-abstraction} \\ \mathtt{C}(t\langle s\rangle) & := & \mathtt{C}(t)\langle\mathtt{C}(s)\rangle & \\ \mathtt{C}(\Box) & := & \Box & \\ \mathtt{C}(\mathtt{L}\langle t\rangle) & := & \mathtt{C}(\mathtt{L})\langle\mathtt{C}(t)\rangle & \end{array}$$

The $\Rightarrow_{\mathbf{m}}$ reduction trivially satisfies the following properties.

Lemma 2.3.9 (Properties of $\Rightarrow_{\mathbf{m}}$)**.**

1. $t \to_{\mathbf{m}} t'$ implies $t \Rightarrow_{\mathbf{m}} t'$.
2. $t \Rightarrow_{\mathbf{m}} t'$ implies $t \to_{\mathbf{m}}^* t'$.
3. $\to_{\mathbf{m}}^*$ is the transitive closure of $\Rightarrow_{\mathbf{m}}$. □

Lemma 2.3.10 (Substitution)**.** If $x \notin \mathsf{fv}(r)$, then $t[s/x][r/y] = t[r/y][s[r/y]/x]$

Proof. By induction on t. The only interesting case is that of variables, which consists on three subcases, depending on the variable being equal to x, to y, or to neither.

1. $t = x$: Then $t[s/x][r/y] = x[s/x][r/y] = s[r/y]$. Since $x[r/y][s[r/y]/x] = x[s[r/y]/x] = s[r/y]$, we conclude.
2. $t = y$: Then $t[s/x][r/y] = y[s/x][r/y] = y[r/y] = r$. By hypothesis x does not occur free in r, therefore $y[r/y][s[r/y]/x] = r[s[r/y]/x] = r$.
3. $t = z$ with $x \neq y \neq z$: Then $t[s/x][r/y] = z[s/x][r/y] = z[r/y] = z$. Since $z[r/y][s[r/y]/x] = z[s[r/y]/x] = z$, we conclude.

□

Lemma 2.3.11 (Compatibility of substitution and parallel reduction)**.** Let $t \Rightarrow_{\mathbf{m}} t'$ and $s \Rightarrow_{\mathbf{m}} s'$. Then $t[s/x] \Rightarrow_{\mathbf{m}} t'[s'/x]$.

Proof. By induction on t, generalizing the statement to memories, where the interesting cases are variables and contracted redexes.

1. $t = x$: Then $t[s/x] = x[s/x] = s \Rightarrow_{\mathbf{m}} s' = x[s'/x]$.
2. $t = y$ with $y \neq x$: Then $t[s/x] = y[s/x] = y \Rightarrow_{\mathbf{m}} y = y[s'/x]$.
3. $t = (\lambda y.u)\mathtt{L}\, r$ with $y \neq x$: We analyze only the case where the parallel reduction contracts the head redex, since the other application case is straightforward by IH. We have then $t = (\lambda y.u)\mathtt{L}\, r' \Rightarrow_{\mathbf{m}} u'[r'/y]\langle r'\rangle\mathtt{L}' = t'$ with $u \Rightarrow_{\mathbf{m}} u'$, $\mathtt{L} \Rightarrow_{\mathbf{m}} \mathtt{L}'$, and $r \Rightarrow_{\mathbf{m}} r'$. By IH, it holds that $u[s/x] \Rightarrow_{\mathbf{m}} u'[s'/x]$, $\mathtt{L}[s/x] \Rightarrow_{\mathbf{m}} \mathtt{L}'[s'/x]$, and $r[s/x] \Rightarrow_{\mathbf{m}} r'[s'/x]$. Therefore

$$t[s/x] = ((\lambda y.u)\mathtt{L}r)[s/x]$$

$$
\begin{aligned}
&= ((\lambda y.u[s/x])\mathtt{L}[s/x]r[s/x]) \\
&\Rightarrow_{\mathbf{m}} u'[s'/x][r'[s'/x]/y]\langle r'[s'/x]\rangle \mathtt{L}'[s'/x] \\
&=_{\text{Lemma 2.3.10}} u'[r'/y][s'/x]\langle r'[s'/x]\rangle \mathtt{L}'[s'/x] \\
&= (u'[r'/y]\langle r'\rangle \mathtt{L}')[s'/x] \\
&= t'[s'/x]
\end{aligned}
$$

□

Lemma 2.3.12. $t \Rightarrow_{\mathbf{m}} t'$ implies $t' \Rightarrow_{\mathbf{m}} \mathtt{C}(t)$.

Proof. By induction on t. The interesting case is $t = (\lambda x.u)\mathtt{L}\, s$. There t' is either $(\lambda x.u')\mathtt{L}'\, s'$ or $u'[s'/x]\langle s'\rangle \mathtt{L}'$, with $u \Rightarrow_{\mathbf{m}} u'$, $s \Rightarrow_{\mathbf{m}} s'$, and $\mathtt{L} \Rightarrow_{\mathbf{m}} \mathtt{L}'$. We analyze each case, noticing that, by IH, we have $u' \Rightarrow_{\mathbf{m}} \mathtt{C}(u)$, $s' \Rightarrow_{\mathbf{m}} \mathtt{C}(s)$, and $\mathtt{L}' \Rightarrow_{\mathbf{m}} \mathtt{C}(\mathtt{L})$.

1. $t' = (\lambda x.u')\mathtt{L}'\, s'$: Then $t' \Rightarrow_{\mathbf{m}} (\lambda x.\mathtt{C}(u'))\mathtt{C}(\mathtt{L}')\mathtt{C}(s') \to_{\mathbf{m}} \mathtt{C}((\lambda x.u)\mathtt{L}\, s)$ by IH and a head step.
2. $t' = u'[s'/x]\langle s'\rangle \mathtt{L}'$: Then by IH and Lemma 2.3.11 we have $u'[s'/x]\langle s'\rangle \Rightarrow_{\mathbf{m}} \mathtt{C}(u)[\mathtt{C}(s)/x]$, hence $u'[s'/x]\langle s'\rangle \mathtt{L}' \Rightarrow_{\mathbf{m}} \mathtt{C}(u)[\mathtt{C}(s)/x]\mathtt{C}(\mathtt{L}) = \mathtt{C}(t)$.

The other cases follow directly from IH. □

Lemma 2.3.13 (Diamond)**.** If $t \Rightarrow_{\mathbf{m}} t_1$ and $t \Rightarrow_{\mathbf{m}} t_2$, then there exists t_3 such that $t_1 \Rightarrow_{\mathbf{m}} t_3$ and $t_2 \Rightarrow_{\mathbf{m}} t_3$.

Proof. By Lemma 2.3.12, take $t_3 = \mathtt{C}(t)$. □

Proposition 2.3.14 (Confluence)**.** If $t_1 \to_{\mathbf{m}}^* t_2$ and $t_1 \to_{\mathbf{m}}^* t_3$, there exists a term t_4 such that $t_2 \to_{\mathbf{m}}^* t_4$ and $t_3 \to_{\mathbf{m}}^* t_4$.

Proof. By Lemma 2.3.9.3 and Lemma 2.3.13. □

2.4 $\mathcal{W}$–measure: counting wrappers

In this section, we define the $\mathcal{W}$-measure (Definition 2.4.17) and we prove that it is decreasing (Theorem 2.4.20).

To define it, we resort to an operation $\mathtt{S}_d(t)$ that simultaneously contracts all the redexes of degree d in a term of the $\lambda^{\mathbf{m}}$-calculus, that is, the result of the *complete development* of all the redexes of degree d. The degree of a redex $(\lambda x.t)\langle r_1\rangle \dots \langle r_n\rangle s$ is defined similarly as for the STLC, as the height of the type of the abstraction. To calculate the $\mathcal{W}$-measure of a λ-term M, let D be the maximum degree of the redexes in M, and define $\mathcal{W}(M)$ as the number of wrappers in $\mathtt{S}_1(\mathtt{S}_2(\dots \mathtt{S}_D(M)))$. For example, if $M = (\lambda x.x(xy))(\lambda z.w)$, it turns out that $\mathtt{S}_1(\mathtt{S}_2(M)) = w\langle w\langle y\rangle\rangle\langle \lambda z.w\rangle$ which has three wrappers, so $\mathcal{W}(M) = 3$. The $\mathcal{W}$-measure maps each typable λ-term to a natural number. The main result of this section is Theorem 2.4.20, stating that $\mathcal{W}$ is decreasing, *i.e.*, that $M \to_\beta N$ implies $\mathcal{W}(M) > \mathcal{W}(N)$.

We first convey some ideas that led to the definition of the $\mathcal{W}$-measure. Recall that an abstract rewriting system $(A, \to)$ is *weakly Church–Rosser* (WCR) if $\leftarrow\to \subseteq \to^*\leftarrow^*$, *Church–Rosser* (CR) if $\leftarrow^*\to^* \subseteq \to^*\leftarrow^*$, and *increasing* (Inc) if there exists a function $|\cdot| : A \to \mathbb{N}$ such that $a \to b$ implies $|a| < |b|$. Let us also recall *Klop–Nederpelt's lemma* [99, Theorem 1.2.3 (iii)], which states that Inc $\wedge$ WCR $\wedge$ WN $\implies$ SN $\wedge$ CR.

Let $(A, \to)$ be increasing and WCR. Given a reduction $a \to^* b$, where b is a normal form, we can find a *decreasing* measure for the set of objects reachable from a, that is, the set $\{c \in A \mid a \to^* c\}$. In fact, by Klop–Nederpelt's lemma, we know that for every $c \in A$ such that $a \to^* c$ we have that $c \to^* b$, which implies that $|c| \leq |b|$, and hence we can define $\#(c) := |b| - |c|$. It is easy to see that $\#(-)$ is a decreasing measure, since $c \to c'$ implies that $|c| < |c'|$ so $\#(c) := |b| - |c| > |b| - |c'| = \#(c')$. Furthermore, the value of $\#(c)$ does not depend on the choice of a, by uniqueness of normal forms.

The idea behind the $\mathcal{W}$-measure is that the construction of a *decreasing* measure can be based on an *increasing* measure, according to the previous observation. It is not possible to build an increasing measure directly for the STLC; *e.g.* the following infinite sequence of expansions $t \leftarrow It \leftarrow I(It) \leftarrow \dots$ would induce an infinite decreasing chain of natural numbers $|t| > |It| > |I(It)| > \dots$.

One could try to define an increasing measure in a variant of the STLC such as Endrullis *et al.*'s clocked λ-calculus [39], in which the β-rule becomes $(\lambda x.t)s \to \tau(t[s/x])$, that is, contracting a β-redex produces a counter "τ" that keeps track of the number of contracted redexes. One could then count the number of τ's: for example, in the reduction sequence $(\lambda x.x(xy))I \to \tau(I(Iy)) \to \tau\tau(Iy) \to \tau\tau\tau y$ the number of counters strictly increases with each step. Unfortunately, this does not define an increasing measure, due to *erasure*. For example, $(\lambda x.y)t \to \tau y$ erases all the counters in t.

This is the motivation behind the definition of the $\lambda^{\mathbf{m}}$-calculus, which avoids erasure by always keeping an extra copy of the argument in a wrapper. The $\lambda^{\mathbf{m}}$-calculus is indeed increasing: in a step $t \to_{\mathbf{m}} s$ one has that $\mathsf{w}(t) < \mathsf{w}(s)$, where we recall that $\mathsf{w}(t)$ denotes the *weight*, *i.e.*, the number of wrappers in t. For example, the step $(\lambda x.y)(z\langle z\rangle) \to_{\mathbf{m}} y\langle z\langle z\rangle\rangle$ increases the number of wrappers.

Note, however, that the construction given by Klop–Nederpelt's lemma, *i.e.*, $\#(t) := \mathsf{w}(v) - \mathsf{w}(t)$ with v normal form of t, yields a decreasing measure for the increasing calculus $\lambda^{\mathbf{m}}$, not for the STLC. The transfer from $\lambda^{\mathbf{m}}$ to STLC requires additional work, and it is treated in the last two subsections. The decreasing measure $\mathcal{W}(M)$ is defined essentially by reducing M to normal form in the $\lambda^{\mathbf{m}}$-calculus and counting the number of wrappers in the result.

This section is divided as follows. First, in Section 2.4.1, we introduce the *simplification* operation, which is the key to compute the measure. Then, in Section 2.4.2, we introduce the *forgetful* reduction, which is a key part of proving that the measure decreases. Finally, in Section 2.4.3, we define the measure and prove that it decreases.

It is worth noting that the simplification operation, as well as the forgetful reduction, are also a part of the decreasing measure presented in the next chapter. On one hand, I decided to avoid abstracting out these definitions because of how the development proceeded. We first worked on the $\mathcal{W}$ –measure, and, on top of that, we kept working, as a sort of incremental but separated result, on the $\mathcal{T}^{\mathbf{m}}$ –measure. On the other hand, I believe factorizing those definitions away would not help in fully and independently grasping the details about $\mathcal{W}$. These are important for this document, as we later present $\mathcal{W}_{\cap}$, its extension to idempotent intersection types, and $\hat{\mathcal{W}}$, its refinement to exactness.

2.4.1 Simplification

In this subsection, we introduce the operations of *simplification* and *full simplification*, noted by $\mathtt{S}.(\cdot)$ and $\mathtt{S}_*(\cdot)$ respectively, and we prove some properties that they satisfy.

Let $d \geq 1$ be a natural number. The *simplification of degree* d, written $\mathtt{S}_d(t)$, is the result of simultaneously contracting all the redexes of degree d in t, that is, the result of the *complete development* of all redexes of degree d. Formally, for each $\lambda^{\mathbf{m}}$-term t we define $\mathtt{S}_d(t)$, and, for each

memory $\mathtt{L}$, we define $\mathtt{S}_d(\mathtt{L})$ as follows:

Definition 2.4.1 (Simplification)**.**

$$\begin{aligned}
\mathtt{S}_d(x) &:= x \\
\mathtt{S}_d(\lambda x.t) &:= \lambda x.\mathtt{S}_d(t) \\
\mathtt{S}_d(ts) &:= \begin{cases} \mathtt{S}_d(t')[\mathtt{S}_d(s)/x]\langle \mathtt{S}_d(s)\rangle \mathtt{S}_d(\mathtt{L}) & \text{if } t = (\lambda x.t')\mathtt{L} \text{ and it is of degree } d \\ \mathtt{S}_d(t)\mathtt{S}_d(s) & \text{otherwise} \end{cases} \\
\mathtt{S}_d(t\langle s\rangle) &:= \mathtt{S}_d(t)\langle \mathtt{S}_d(s)\rangle
\end{aligned}$$

where if $\mathtt{L}$ is a memory, $\mathtt{S}_d(\mathtt{L})$ is defined by $\mathtt{S}_d(\Box) := \Box$ and $\mathtt{S}_d(\mathtt{L}\langle t\rangle) := \mathtt{S}_d(\mathtt{L})\langle \mathtt{S}_d(t)\rangle$.

Definition 2.4.2 (Full simplification)**.** If t is a $\lambda^{\mathbf{m}}$-term of max-degree D, we define the *full simplification* of t as the term that results from iteratively taking the simplification of degree i from D down to 1. More precisely:

$$\mathtt{S}_*(t) := \mathtt{S}_1(\dots \mathtt{S}_{D-1}(\mathtt{S}_D(t)))$$

Example 2.4.3. Consider the λ-term $M = (\lambda x^{\tau\to\tau}.x^{\tau\to\tau}(x^{\tau\to\tau}y^{\tau}))(\lambda z^{\tau}.w^{\tau})$. It can be regarded also as a $\lambda^{\mathbf{m}}$-term, and we have:

$$\begin{aligned}
\mathtt{S}_2(M) &= ((\lambda z^{\tau}.w^{\tau})((\lambda z^{\tau}.w^{\tau})y^{\tau}))\langle \lambda z^{\tau}.w^{\tau}\rangle \\
\mathtt{S}_*(M) = \mathtt{S}_1(\mathtt{S}_2(M)) &= w^{\tau}\langle w^{\tau}\langle y^{\tau}\rangle\rangle\langle \lambda z^{\tau}.w^{\tau}\rangle
\end{aligned}$$

Note that M has only one redex, whose abstraction is of type $(\tau \to \tau) \to \tau$ and hence of degree 2, and that $\mathtt{S}_2(M)$ has two redexes, whose abstractions are of type $\tau \to \tau$ and hence of degree 1. Moreover, consider the λ-term $N = (\lambda z^{\tau}.w^{\tau})((\lambda z^{\tau}.w^{\tau})y^{\tau})$. Then $\mathtt{S}_*(N) = \mathtt{S}_1(N) = w\langle w\langle y\rangle\rangle$. Note that N has two redexes whose abstraction is of type $\tau \to \tau$ and hence of degree 1. As an additional note, in the λ-calculus there is a reduction step $M \to_\beta N$, and we have that $\mathtt{w}(\mathtt{S}_*(M)) = 3 > 2 = \mathtt{w}(\mathtt{S}_*(N))$. So this example illustrates that the $\mathcal{W}$-measure (as defined in Definition 2.4.17) is decreasing (as we will show in Theorem 2.4.20).

As it turns out, full simplification corresponds to reduction to normal form. We formally state this in Proposition 2.4.10, which entails in particular that the $\lambda^{\mathbf{m}}$-calculus is weakly normalizing. To prove the proposition, we make use of some auxiliary lemmas, with the most relevant ones being those stating soundness of simplification, *i.e.*, terms reduce to its simplification, and that simplifying the max–degree of a term shrinks the max–degree, *i.e.*, produces a term where the max–degree is lower. We state the interesting parts of the proofs, leaving the complete details to Appendix A.1.

Lemma 2.4.4 (Soundness of simplification)**.** For every term t and for all $k \geq 1$, we have that $t \to^*_{\mathbf{m}} \mathtt{S}_k(t)$.

Proof. We generalize the statement to memories, *i.e.*, $\mathtt{L} \to^*_{\mathbf{m}} \mathtt{S}_k(\mathtt{L})$, and proceed by simultaneous induction on t and $\mathtt{L}$. The interesting case is that of $\mathbf{m}$–abstraction of degree d, *i.e.*, $t = (\lambda x.s)\mathtt{L}r$ where $(\lambda x.s)\mathtt{L}$ is an $\mathbf{m}$-abstraction of degree k. We have by IH and definition of simplification:

$$\begin{aligned}
(\lambda x.s)\mathtt{L}r &\to^*_{\mathbf{m}} (\lambda x.\mathtt{S}_k(s))\mathtt{S}_k(\mathtt{L})\mathtt{S}_k(r) \\
&\to_{\mathbf{m}} \mathtt{S}_k(s)[\mathtt{S}_k(r)/x]\langle \mathtt{S}_k(r)\rangle \mathtt{S}_k(\mathtt{L}) \\
&= \mathtt{S}_k((\lambda x.s)\mathtt{L}r) \\
&= \mathtt{S}_k(t)
\end{aligned}$$

The remaining cases are straightforward resorting to the IH. □

The next four lemmas are used for proving that max–degree simplification shrinks max–degree. The first one states that substituting a non–abstraction in a non–abstraction of a greater type does not produce an abstraction. The second one that d–simplifying a non–abstraction of type of height d does not produce an abstraction, when d exceeds the max–degree. Note that the height of the main type and the degrees of the redexes are somehow independent: one could add abstractions and enlarge the type indefinitely, or fully apply functions until getting height 0. A height greater than the max–degree, thus, implies that the term still expects more complex arguments, or simply more arguments, than the redexes being solved internally. The third lemma generalizes max–degree to memories, and the fourth lemma states that substitution does not exceed the max–degree of both terms.

Lemma 2.4.5 (Non–abstraction substitution preserves greater non–abstraction)**.** If $\mathsf{h}(\mathsf{type}(t)) > \mathsf{h}(\mathsf{type}(s))$ and t not an **m**-abstraction, then $t[s/x]$ is not an **m**-abstraction.

Proof. By induction on t, with two interesting cases:

1. $t = y$: It is not possible that $y = x$, since it would imply that the height of t and s are equal, and we know by hypothesis that it is greater. Therefore $y[s/x] = y$, not an **m**-abstraction.

2. $t = t_1\langle t_2\rangle$: Note that t_1 is not an **m**-abstraction by Remark 2.3.2. Furthermore, note that $\mathsf{type}(t_1) = \mathsf{type}(t)$ so in particular $\mathsf{h}(\mathsf{type}(t_1)) = \mathsf{h}(\mathsf{type}(t)) > \mathsf{h}(\mathsf{type}(s))$. We are under the conditions to apply the IH on t_1, hence $t_1[s/x]$ is not an **m**-abstraction. To conclude, note that $t[s/x] = t_1[s/x]\langle t_2[s/x]\rangle$ cannot be an **m**-abstraction by Remark 2.3.2.

□

Lemma 2.4.6 (High simplification preserves non–abstraction)**.** If $\mathsf{h}(\mathsf{type}(t)) \geq k \geq \delta_{\mathsf{max}}(t)$ and t is not an **m**-abstraction, then $\mathtt{S}_k(t)$ is not an **m**-abstraction.

Proof. By induction on t, where the interesting cases are:

1. $t = (\lambda x.s)\mathtt{L}\, r$ where $(\lambda x.s)\mathtt{L}$ is an **m**-abstraction of degree k: the hypothesis states that k is greater or equal than the max–degree of t. If we take, for instance, t of type B, we would have $\lambda x.s$ and (therefore t) of degree $\mathsf{h}(A \to B) > \mathsf{h}(B) \geq \delta_{\mathsf{max}}(t)$. This contradicts the hypothesis: we have a redex greater than $\delta_{\mathsf{max}}(t)$. Hence this case is impossible.

2. $t = s\langle r\rangle$: Note that $\mathsf{type}(s) = \mathsf{type}(t)$, so in particular $\mathsf{h}(\mathsf{type}(s)) = \mathsf{h}(\mathsf{type}(t)) \geq k$. Moreover, $\delta_{\mathsf{max}}(s) \leq \delta_{\mathsf{max}}(t) \leq k$ and s is not an **m**-abstraction by Remark 2.3.2. We are under the conditions to apply the IH on s, hence $\mathtt{S}_k(s)$ is not an **m**-abstraction. To conclude, note that $\mathtt{S}_k(t) = \mathtt{S}_k(s)\langle \mathtt{S}_k(r)\rangle$ cannot be an **m**-abstraction by Remark 2.3.2.

□

Lemma 2.4.7 (Max–degree of memories)**.** $\delta_{\mathsf{max}}(t\mathtt{L}) = \max(\delta_{\mathsf{max}}(t), \delta_{\mathsf{max}}(\mathtt{L}))$

Proof. Straightforward by induction on $\mathtt{L}$, since $\delta_{\mathsf{max}}(t\langle s\rangle) = \max(\delta_{\mathsf{max}}(t), \delta_{\mathsf{max}}(s))$. □

Lemma 2.4.8 (Strict upper bound for max–deg is stable under substitution)**.** If $\delta_{\mathsf{max}}(t) < k$ and $\delta_{\mathsf{max}}(s) < k$ and $\mathsf{h}(\mathsf{type}(s)) < k$ then $\delta_{\mathsf{max}}(t[s/x]) < k$.

Proof. By induction on t:

1. $t = y$: We consider two subcases, depending on whether $y = x$ or not. If $y = x$, then $\delta_{\mathsf{max}}(x[s/x]) = \delta_{\mathsf{max}}(s) < k$. If $y \neq x$, then $\delta_{\mathsf{max}}(y[s/x]) = \delta_{\mathsf{max}}(y) = \delta_{\mathsf{max}}(t) < k$.

2. $t = \lambda y.t'$: By α-conversion we assume that $x \neq y$. Note that $\delta_{\mathsf{max}}(t') = \delta_{\mathsf{max}}(\lambda y.t') = \delta_{\mathsf{max}}(t) < k$ by hypothesis. Then $\delta_{\mathsf{max}}((\lambda y.t')[s/x]) = \delta_{\mathsf{max}}(\lambda y.t'[s/x]) = \delta_{\mathsf{max}}(t'[s/x]) < k$ by IH.

3. $t = t_1\, t_2$: Note that $\delta_{\mathsf{max}}(t_1) \leq \delta_{\mathsf{max}}(t_1\, t_2) = \delta_{\mathsf{max}}(t) < k$ and, similarly, $\delta_{\mathsf{max}}(t_2) < k$. This means that we can apply the IH to obtain that $\delta_{\mathsf{max}}(t_1[s/x]) < k$ and $\delta_{\mathsf{max}}(t_2[s/x]) < k$. We proceed by case analysis, depending on whether $\mathsf{h}(\mathsf{type}(t_1)) < k$ or $\mathsf{h}(\mathsf{type}(t_1)) \geq k$:

   3.1. If $\mathsf{h}(\mathsf{type}(t_1)) < k$ then, by the substitution lemma (Lemma 2.3.6), the terms $t_1[s/x]$ and t_1 have the same type. In particular, $t_1[s/x]\, t_2[s/x]$ cannot be a redex of degree k or greater, since $\mathsf{h}(\mathsf{type}(t_1[s/x])) < k$. As a consequence, if $t_1[s/x]\, t_2[s/x]$ is a redex, its degree is at most $k-1$. Hence $\delta_{\mathsf{max}}((t_1\, t_2)[s/x]) = \delta_{\mathsf{max}}(t_1[s/x]\, t_2[s/x]) \leq \max(k-1, \delta_{\mathsf{max}}(t_1[s/x]), \delta_{\mathsf{max}}(t_2[s/x])) < k$.

   3.2. If $\mathsf{h}(\mathsf{type}(t_1)) \geq k$, note that t_1 cannot be an **m**-abstraction, because then $t = t_1\, t_2$ would be a redex of degree k or greater, but by hypothesis we know that $\delta_{\mathsf{max}}(t) < k$. Note that we are under the conditions of Lemma 2.4.5, so we know that $t_1[s/x]$ is not an **m**-abstraction. In particular, $t_1[s/x]\, t_2[s/x]$ cannot be a redex. Hence $\delta_{\mathsf{max}}((t_1\, t_2)[s/x]) = \delta_{\mathsf{max}}(t_1[s/x]\, t_2[s/x]) = \max(\delta_{\mathsf{max}}(t_1[s/x]), \delta_{\mathsf{max}}(t_2[s/x])) < k$.

4. $t = t_1\langle t_2\rangle$: Note that $\delta_{\mathsf{max}}(t_1) \leq \delta_{\mathsf{max}}(t_1\langle t_2\rangle) = \delta_{\mathsf{max}}(t) < k$ and, similarly, $\delta_{\mathsf{max}}(t_2) < k$. This means that we can apply the IH to obtain that $\delta_{\mathsf{max}}(t_1[s/x]) < k$ and $\delta_{\mathsf{max}}(t_2[s/x]) < k$. Hence $\delta_{\mathsf{max}}((t_1\langle t_2\rangle)[s/x]) = \delta_{\mathsf{max}}(t_1[s/x]\langle t_2[s/x]\rangle) = \max(\delta_{\mathsf{max}}(t_1[s/x]), \delta_{\mathsf{max}}(t_2[s/x])) < k$.

□

Lemma 2.4.9 (Simplification of max-degree decreases the max-degree)**.** Suppose that $k \geq 1$. If $\delta_{\mathsf{max}}(t) \leq k$ then $\delta_{\mathsf{max}}(\mathtt{S}_k(t)) < k$.

Proof. Let $k \geq 1$ be such that $\delta_{\mathsf{max}}(t) \leq k$. We argue that $\delta_{\mathsf{max}}(\mathtt{S}_k(t)) < k$, that is, all the redexes in t have degree less than k. To prove it by induction, we generalize the statement to memories, proving also that $\delta_{\mathsf{max}}(\mathtt{S}_k(\mathtt{L})) < k$. We prove the statement simultaneously by induction on t and $\mathtt{L}$:

1. $t = x$: Then $\mathtt{S}_k(x) = x$ has no redexes, so $\delta_{\mathsf{max}}(x) = 0 < 1 \leq k$.

2. $t = \lambda x.s$: Note that $\delta_{\mathsf{max}}(s) = \delta_{\mathsf{max}}(\lambda x.s) \leq k$ so by IH $\delta_{\mathsf{max}}(\mathtt{S}_k(s)) < k$. Moreover, $\delta_{\mathsf{max}}(\mathtt{S}_k(\lambda x.s)) = \delta_{\mathsf{max}}(\lambda x.\mathtt{S}_k(s)) = \delta_{\mathsf{max}}(\mathtt{S}_k(s)) < k$.

3. $t = (\lambda x.s)\mathtt{L}\, r$ where $(\lambda x.s)\mathtt{L}$ is an **m**-abstraction of degree k: Note that $\delta_{\mathsf{max}}(s) \leq \delta_{\mathsf{max}}(t)$ because any redex in the subterm s is also a redex in the whole term t, so in particular $\delta_{\mathsf{max}}(s) \leq k$ and we may apply the IH on s to conclude that $\delta_{\mathsf{max}}(\mathtt{S}_k(s)) < k$. Similarly, by IH, we have that $\delta_{\mathsf{max}}(\mathtt{S}_k(\mathtt{L})) < k$ and $\delta_{\mathsf{max}}(\mathtt{S}_k(r)) < k$.

   Since t is typable, its type is of the form $\mathsf{type}(t) = B$ with $\mathsf{type}((\lambda x.s)\mathtt{L}) = A \to B$ and $\mathsf{type}(r) = A$. Note that $\mathsf{h}(\mathsf{type}(r)) = \mathsf{h}(A) < \mathsf{h}(A \to B) = k$ since $(\lambda x.s)\mathtt{L}$ is of degree k by hypothesis of this case.

To conclude this case, note that:

$$\begin{array}{rl} & \delta_{\mathsf{max}}(\mathtt{S}_k(t)) \\ = & \delta_{\mathsf{max}}(\mathtt{S}_k(s)[\mathtt{S}_k(r)/x]\langle \mathtt{S}_k(r)\rangle \mathtt{S}_k(\mathtt{L})) \\ & \quad \text{by definition} \\ \leq & \max(\delta_{\mathsf{max}}(\mathtt{S}_k(s)[\mathtt{S}_k(r)/x]), \delta_{\mathsf{max}}(\mathtt{S}_k(r)), \delta_{\mathsf{max}}(\mathtt{S}_k(\mathtt{L}))) \\ & \quad \text{by Lemma 2.4.7} \\ < & \max(k,k,k) \\ & \quad \text{by Lemma 2.4.8 and the IH} \\ = & k \end{array}$$

For the last inequality, we use the fact that $\mathsf{h}(\mathsf{type}(r)) < k$.

4. $t = s\,r$ where s is not an $\mathbf{m}$-abstraction of degree k: Note that $\delta_{\mathsf{max}}(s) \leq \delta_{\mathsf{max}}(t)$ because any redex in the subterm s is also a redex in the whole term t, so in particular $\delta_{\mathsf{max}}(s) \leq k$ and we may apply the IH on s to conclude that $\delta_{\mathsf{max}}(\mathtt{S}_k(s)) < k$. Similarly, by IH, we have that $\delta_{\mathsf{max}}(\mathtt{S}_k(r)) < k$.

   We proceed by case analysis, depending on whether $\mathsf{h}(\mathsf{type}(s)) < k$ or $\mathsf{h}(\mathsf{type}(s)) \geq k$:

   4.1. If $\mathsf{h}(\mathsf{type}(s)) < k$, then by Lemma 2.4.4 we know that $s \to_{\mathbf{m}}^* \mathtt{S}_k(s)$ and by subject reduction (Proposition 2.3.7) we have that $\mathsf{type}(s) = \mathsf{type}(\mathtt{S}_k(s))$. In particular, $\mathtt{S}_k(s)\,\mathtt{S}_k(r)$ cannot be a redex of degree k or greater, because $\mathsf{h}(\mathsf{type}(\mathtt{S}_k(s))) = \mathsf{h}(\mathsf{type}(s)) < k$. That is, if $\mathtt{S}_k(s)\,\mathtt{S}_k(r)$ is a redex, its degree is at most $k-1$. Hence we have that $\delta_{\mathsf{max}}(\mathtt{S}_k(t)) = \delta_{\mathsf{max}}(\mathtt{S}_k(s)\,\mathtt{S}_k(r)) \leq \max(k-1, \delta_{\mathsf{max}}(\mathtt{S}_k(s)), \delta_{\mathsf{max}}(\mathtt{S}_k(r))) < k$.

   4.2. If $\mathsf{h}(\mathsf{type}(s)) \geq k$, note that s cannot be an $\mathbf{m}$-abstraction. Indeed, we know by hypothesis of this case that s is not an abstraction of degree k. Furthermore, s cannot be an abstraction of degree $k' > k$, because then $t = s\,r$ would be a redex of degree $k' > k$, but then we would have that $k < k' \leq \delta_{\mathsf{max}}(t) \leq k$, which is a contradiction. Since s is not an $\mathbf{m}$-abstraction, $\delta_{\mathsf{max}}(s) \leq k$, and $\mathsf{h}(\mathsf{type}(s)) \geq k$, we are under the conditions to apply Lemma 2.4.6 to conclude that $\mathtt{S}_k(s)$ is not an $\mathbf{m}$-abstraction. This means that $\mathtt{S}_k(s)\,\mathtt{S}_k(r)$ cannot be a redex. Hence we have that $\delta_{\mathsf{max}}(\mathtt{S}_k(t)) = \delta_{\mathsf{max}}(\mathtt{S}_k(s)\,\mathtt{S}_k(r)) = \max(\delta_{\mathsf{max}}(\mathtt{S}_k(s)), \delta_{\mathsf{max}}(\mathtt{S}_k(r))) < k$.

5. $t = s\langle r\rangle$: Note that $\delta_{\mathsf{max}}(s) \leq \delta_{\mathsf{max}}(t)$, so in particular $\delta_{\mathsf{max}}(s) \leq k$ and we may apply the IH on s to conclude that $\delta_{\mathsf{max}}(\mathtt{S}_k(s)) < k$. Similarly, by IH, we have that $\delta_{\mathsf{max}}(\mathtt{S}_k(r)) < k$. Hence we have that $\delta_{\mathsf{max}}(\mathtt{S}_k(t)) = \delta_{\mathsf{max}}(\mathtt{S}_k(s)\langle \mathtt{S}_k(r)\rangle) = \max(\delta_{\mathsf{max}}(\mathtt{S}_k(s)), \delta_{\mathsf{max}}(\mathtt{S}_k(r))) < k$.

6. $\mathtt{L} = \Box$: Immediate, as $\delta_{\mathsf{max}}(\Box) = 0 < 1 \leq k$.

7. $\mathtt{L} = \mathtt{L}'\langle t\rangle$: Similar to case 5 of this lemma. □

Proposition 2.4.10 (Full simplification yields the normal form)**.** $t \to_{\mathbf{m}}^* \mathtt{S}_*(t)$, and moreover $\mathtt{S}_*(t)$ is a $\to_{\mathbf{m}}$-normal form.

Proof. By repeated application of Lemma 2.4.4, we have $t \to_{\mathbf{m}}^* \mathtt{S}_D(t) \to_{\mathbf{m}}^* \mathtt{S}_{D-1}(\mathtt{S}_D(t)) \ldots \to_{\mathbf{m}}^* \mathtt{S}_1(\ldots \mathtt{S}_{D-1}(\mathtt{S}_D(t))) = \mathtt{S}_*(t)$, where D is the max-degree of t.

To show that $\mathtt{S}_*(t)$ is a $\to_{\mathbf{m}}$-normal form, the key property is that, after performing a simplification of the max-degree, no redexes of that degree remain. Let $D = \delta_{\mathsf{max}}(t)$. By repeatedly applying Lemma 2.4.9, we can get, starting with $\mathtt{S}_D(t) < D$ and descending downto $\mathtt{S}_1(\ldots \mathtt{S}_{D-1}(\mathtt{S}_D(t))) < 1$, that $\delta_{\mathsf{max}}(\mathtt{S}_*(t)) < 1$. Since there are no redexes of degree 0, we have that $\mathtt{S}_*(t)$ does not contain redexes, and therefore it is a $\to_{\mathbf{m}}$-normal form. See Proposition A.1.7 in the appendix for detailed proofs. □

Note that the previous lemmas partially imply Turing's observation for $\lambda^{\mathbf{m}}$: contracting a redex *of max-degree* that has no occurrences of equal degree redexes does not create redexes of greater or equal degrees. This weaker variant of the observation is enough for the measure defined in this section.

2.4.2 Forgetful reduction

At this point, we have proved that $\lambda^{\mathbf{m}}$ satisfies the conditions of Klop–Nederpelt's lemma [99, Theorem 1.2.3 (iii)]: Inc, WCR, and WN. We can then construct the decreasing measure explained at the beginning of Section 2.4: $\#(t) := \mathtt{w}(\mathtt{S}_*(t)) - \mathtt{w}(t)$. Such a measure, however, works for $\lambda^{\mathbf{m}}$, while we seek one for STLC. Since the set of terms of the $\lambda^{\mathbf{m}}$-calculus is a superset of that of the STLC, we can easily extract from $\#$ a measure for STLC. Indeed, notice that all terms M in STLC are $\lambda^{\mathbf{m}}$-terms with no wrappers, so $\#(M) = \mathtt{w}(\mathtt{S}_*(M)) - \mathtt{w}(M) = \mathtt{w}(\mathtt{S}_*(M)) - 0 = \mathtt{w}(\mathtt{S}_*(M))$. This is precisely how we will define the measure $\mathcal{W}$ in Section 2.4.3. While this transfer of the measure definition is immediate, its decrease proof requires additional work, due to the differences between reductions $\to_\beta$ and $\to_{\mathbf{m}}$. In this subsection we define another notion of reduction that erases wrappers, allowing us to relate the terms and reductions of STLC to those of $\lambda^{\mathbf{m}}$.

We introduce the relation of *forgetful reduction* $t \rhd^+ s$, and we prove that it commutes with reduction.

Definition 2.4.11. A $\lambda^{\mathbf{m}}$-term t *reduces via a forgetful step* to s, written $t \rhd s$, according to the following axiom, closed by compatibility under arbitrary contexts:

$$t\langle s\rangle \rhd t$$

We say that t *reduces via forgetful reduction* to s if and only if $t \rhd^+ s$, where $\rhd^+$ denotes the transitive closure of $\rhd$.

Example 2.4.12. $(\lambda x.x\langle y\langle y\rangle\rangle)\langle z\langle z\rangle\rangle \rhd (\lambda x.x\langle y\langle y\rangle\rangle)\langle z\rangle \rhd (\lambda x.x)\langle z\rangle \rhd \lambda x.x$.

The forgetful reduction relation is generalized to operate on substitution contexts so that, for example, $(\Box\langle x\rangle\langle y\rangle) \rhd (\Box\langle y\rangle)$.

Lemma 2.4.13 (Properties of forgetful reduction)**.**

1. If $t \rhd t'$ then $t\mathtt{L} \rhd t'\mathtt{L}$.
2. If $\mathtt{L} \rhd \mathtt{L}'$ then $t\mathtt{L} \rhd t\mathtt{L}'$.
3. If $t \rhd t'$ then $t[s/x] \rhd t'[s/x]$.
4. If $s \rhd s'$ then $t[s/x] \rhd^* t[s'/x]$ (in zero or more steps).

Proof. Items 1 and 2 are straightforward by induction on $\mathtt{L}$. Items 3 and 4 are straightforward by induction on t. For item 4, note that when $t = y$ with $y \neq x$, we have that $y[s/x] = y \rhd^* y = y[s'/x]$ in exactly zero steps. Note also that more that one step of $\rhd$ may be required when t is an application or a wrapper. □

Lemma 2.4.14 (Local commutation of reduction and forgetful reduction)**.** If $t \rhd s$ and $t \to_{\mathbf{m}} t'$, there exists a term s' such that $t' \rhd^+ s'$ and $s \to_{\mathbf{m}}^{=} s'$, where $\rhd^+$ is the transitive closure of $\rhd$, and $\to_{\mathbf{m}}^{=}$ is the reflexive closure of $\to_{\mathbf{m}}$. Graphically:

$$\begin{array}{ccc} t & \rhd & s \\ \downarrow & & \vdots = \\ t' & \rhd^+ & s' \end{array}$$

Proof. By induction on t:

1. $t = x$: Note that this case is impossible, since there are no steps $x \to_{\mathbf{m}} t'$.

2. $t = \lambda x.t_1$: Since $\lambda x.t_1 \to_{\mathbf{m}} t'$, we know that t' must be of the form $t' = \lambda x.t_1'$ with $t_1 \to_{\mathbf{m}} t_1'$. Note that the $\rhd$ step is internal, that is, $\lambda x.t_1 \rhd \lambda x.s_1 = s$ with $t_1 \rhd s_1$. By IH there exists s_1' such that $t_1' \rhd^+ s_1'$ and $s_1 \to_{\mathbf{m}}^{=} s_1'$. Taking $s' := \lambda x.s_1'$ we have:

$$\begin{array}{ccc} \lambda x.t_1 & \rhd & \lambda x.s_1 \\ \downarrow & & \downarrow{=} \\ \lambda x.t_1' & \rhd^+ & \lambda x.s_1' \end{array}$$

3. $t = t_1\, t_2$: We consider three subcases, depending on whether the step $t_1\, t_2 \to_{\mathbf{m}} t'$ is a $\to_{\mathbf{m}}$ step at the root, internal to t_1, or internal to t_2:

   3.1. If the $\to_{\mathbf{m}}$ step is at the root, then t_1 is an $\mathbf{m}$-abstraction of the form $t_1 = (\lambda x.t_{11})\mathtt{L}$ and the step is of the form $t = (\lambda x.t_{11})\mathtt{L}\, t_2 \to_{\mathbf{m}} t_{11}[t_2/x]\langle t_2\rangle\mathtt{L} = t'$. Moreover, since $t = (\lambda x.t_{11})\mathtt{L}\, t_2 \rhd t'$, we consider three further subcases, depending on whether the step $t \rhd s$ is internal to t_{11}, internal to $\mathtt{L}$, or internal to t_2:

   3.1.1. If the $\rhd$ step is internal to t_{11}, then $s = (\lambda x.s_{11})\mathtt{L}\, t_2$ with $t_{11} \rhd s_{11}$. Taking $s' := s_{11}[t_2/x]\langle t_2\rangle\mathtt{L}$ we have:

$$\begin{array}{ccc} (\lambda x.t_{11})\mathtt{L}\, t_2 & \rhd & (\lambda x.s_{11})\mathtt{L}\, t_2 \\ \downarrow & & \downarrow \\ t_{11}[t_2/x]\langle t_2\rangle\mathtt{L} & \rhd & s_{11}[t_2/x]\langle t_2\rangle\mathtt{L} \end{array}$$

   For the $\rhd$ step at the bottom, by Lemma 2.4.13 (1) it suffices to show that $t_{11}[t_2/x] \rhd s_{11}[t_2/x]$. This is a consequence of Lemma 2.4.13 (3).

   3.1.2. If the $\rhd$ step is internal to $\mathtt{L}$, then $s = (\lambda x.t_{11})\mathtt{L}'\, t_2$ with $\mathtt{L} \rhd \mathtt{L}'$. Taking $s' := t_{11}[t_2/x]\langle t_2\rangle\mathtt{L}'$ we have:

$$\begin{array}{ccc} (\lambda x.t_{11})\mathtt{L}\, t_2 & \rhd & (\lambda x.t_{11})\mathtt{L}'\, t_2 \\ \downarrow & & \downarrow \\ t_{11}[t_2/x]\langle t_2\rangle\mathtt{L} & \rhd & t_{11}[t_2/x]\langle t_2\rangle\mathtt{L}' \end{array}$$

   The $\rhd$ step at the bottom holds by Lemma 2.4.13 (2).

   3.1.3. If the $\rhd$ step is internal to t_2, then $s = (\lambda x.t_{11})\mathtt{L}\, s_2'$ with $t_2 \rhd s_2$. Taking $s' := t_{11}[t_2/x]\langle s_2'\rangle\mathtt{L}$ we have:

$$\begin{array}{ccc} (\lambda x.t_{11})\mathtt{L}\, t_2 & \rhd & (\lambda x.t_{11})\mathtt{L}\, s_2 \\ \downarrow & & \downarrow \\ t_{11}[t_2/x]\langle t_2\rangle\mathtt{L} & \rhd^+ & t_{11}[s_2/x]\langle s_2\rangle\mathtt{L} \end{array}$$

   For the bottom of the diagram, note that $t_{11}[t_2/x] \rhd^* t_{11}[s_2/x]$ by Lemma 2.4.13 (4). Hence $t_{11}[t_2/x]\langle t_2\rangle \rhd^* t_{11}[s_2/x]\langle t_2\rangle \rhd t_{11}[s_2/x]\langle s_2\rangle$. Resorting to Lemma 2.4.13 (1) we conclude.

   3.2. If the $\to_{\mathbf{m}}$ step is internal to t_1, the step is of the form $t_1\, t_2 \to_{\mathbf{m}} t_1'\, t_2$ with $t_1 \to_{\mathbf{m}} t_1'$. We consider two further subcases, depending on whether the $\rhd$ step is internal to t_1 or internal to t_2:

3.2.1. If the $\rhd$ step is internal to t_1, then $s = s_1\, t_2$ with $t_1 \rhd s_1$. By IH there exists s_1' such that $t_1' \rhd^+ s_1'$ and $s_1 \to_{\mathbf{m}}^{=} s_1'$. Taking $s' := s_1'\, t_2$ we have:

$$\begin{array}{ccc} t_1\, t_2 & \rhd & s_1\, t_2 \\ \downarrow & & \downarrow{=} \\ t_1'\, t_2 & \rhd^+ & s_1'\, t_2 \end{array}$$

3.2.2. If the $\rhd$ step is internal to t_2, then $s = t_1\, s_2$ with $t_2 \rhd s_2$. Taking $s' := t_1'\, s_2$ we have:

$$\begin{array}{ccc} t_1\, t_2 & \rhd & t_1\, s_2 \\ \downarrow & & \downarrow \\ t_1'\, t_2 & \rhd & t_1'\, s_2 \end{array}$$

3.3. If the $\to_{\mathbf{m}}$ step is internal to t_2, the proof is similar to the previous case.

4. $t = t_1\langle t_2\rangle$: We consider two subcases, depending on whether the step $t_1\langle t_2\rangle \to_{\mathbf{m}} t'$ is internal to t_1 or internal to t_2:

4.1. If the $\to_{\mathbf{m}}$ step is internal to t_1, then $t_1\langle t_2\rangle \to_{\mathbf{m}} t_1'\langle t_2\rangle = t'$ with $t_1 \to_{\mathbf{m}} t_1'$. We consider three further subcases, depending on whether the step $t_1\langle t_2\rangle \rhd s$ is at the root of the wrapper, internal to t_1, or internal to t_2:

4.1.1. If the $\rhd$ step is at the root of the wrapper, then $t_1\langle t_2\rangle \rhd t_1 = s$. Taking $s' := t_1'$ we have:

$$\begin{array}{ccc} t_1\langle t_2\rangle & \rhd & t_1 \\ \downarrow & & \downarrow \\ t_1'\langle t_2\rangle & \rhd & t_1' \end{array}$$

4.1.2. If the $\rhd$ step is internal to t_1, then $s = s_1\langle t_2\rangle$ with $t_1 \rhd s_1$, and we conclude by IH similarly as for case 3.2.1.

4.1.3. If the $\rhd$ step is internal to t_2, then $s = t_1\langle s_2\rangle$ with $t_2 \rhd s_2$, and we conclude taking $s' := t_1'\langle s_2\rangle$ similarly as for case 3.2.2.

4.2. If the $\to_{\mathbf{m}}$ step is internal to t_2, then $t_1\langle t_2\rangle \to_{\mathbf{m}} t_1\langle t_2'\rangle = t'$ with $t_2 \to_{\mathbf{m}} t_2'$. We consider three further subcases, depending on whether the step $t_1\langle t_2\rangle \rhd s$ is at the root of the wrapper, internal to t_1, or internal to t_2:

4.2.1. If the $\rhd$ step is at the root of the wrapper, then $t_1\langle t_2\rangle \rhd t_1 = s$. Taking $s' := t_1$ we have:

$$\begin{array}{ccc} t_1\langle t_2\rangle & \rhd & t_1 \\ \downarrow & & \| \\ t_1\langle t_2'\rangle & \rhd & t_1 \end{array}$$

4.2.2. If the $\rhd$ step is internal to t_1, then $t_1\langle t_2\rangle \rhd s_1\langle t_2\rangle = s$ with $t_1 \rhd s_1$, and we conclude taking $s' := s_1\langle t_2'\rangle$ similarly as for case 3.2.2.

4.2.3. If the $\rhd$ step is internal to t_2, then $t_1\langle t_2\rangle \rhd t_1\langle s_2\rangle = s$ with $t_2 \rhd s_2$, and we conclude by IH similarly as for case 3.2.1.

□

Proposition 2.4.15 (Forgetful reduction commutes with reduction)**.** If $t \rhd^+ s$ and $t \to_{\mathbf{m}}^* t'$, there exists a term s' such that $t' \rhd^+ s'$ and $s \to_{\mathbf{m}}^* s'$. Graphically:

$$\begin{array}{ccc} t & \rhd^+ & s \\ \downarrow^* & & \vdots^* \\ t' & \rhd^+ & s' \end{array}$$

Furthermore, if $t \rhd^+ s$ and t is a $\to_{\mathbf{m}}$-normal form, then s is also a normal form.

Proof. First we claim that if $t \rhd^+ s$ and $t \to_{\mathbf{m}} t'$, there exists a term s' such that $t' \rhd^+ s'$ and $s \to_{\mathbf{m}}^= s'$. This can be seen by induction on the number of $\rhd$ steps in a reduction sequence $t \rhd^+ s$, resorting to the local commutation lemma (Lemma 2.4.14).

The main statement of the proposition can be seen by induction on the number of steps in a reduction sequence $t \to_{\mathbf{m}}^* t'$, resorting to the claim.

For the "furthermore" part in the statement, it suffices to show that if $t \rhd s$ in one step and t is a $\to_{\mathbf{m}}$-normal form, then s is also a normal form. This is straightforward by induction on t. □

Each step in the STLC has a *corresponding* step in the $\lambda^{\mathbf{m}}$-calculus, that contracts the redex in the same position. For instance the step $(\lambda x.xy)I \to_\beta Iy$ in the STLC has a corresponding step $(\lambda x.xy)I \to_{\mathbf{m}} (Iy)\langle I\rangle$ in the $\lambda^{\mathbf{m}}$-calculus. In this example, $(Iy)\langle I\rangle \rhd Iy$. The following easy lemma confirms that this is a general fact:

Lemma 2.4.16 (Reduce/forget)**.** Let $M \to_\beta N$ be a β-step and let $M \to_{\mathbf{m}} s$ be the corresponding step in $\lambda^{\mathbf{m}}$. Then $s \rhd N$.

Proof. We proceed by induction on M:

1. $M = x$: Impossible, as there are no steps $x \to_\beta N$.
2. $M = \lambda x.M_1$: Then the step must be of the form $M = \lambda x.M_1 \to_\beta \lambda x.N_1 = N$ with $M_1 \to_\beta N_1$, and the corresponding step must be of the form $M = \lambda x.M_1 \to_{\mathbf{m}} \lambda x.s_1 = s$ where $M_1 \to_{\mathbf{m}} s_1$ is the step corresponding to $M_1 \to_\beta N_1$. By IH $s_1 \rhd N_1$, so $s = \lambda x.s_1 \rhd \lambda x.N_1 = N$.
3. $M = M_1M_2$: We consider three subcases, depending on whether the step is at the root, internal to M_1, or internal to M_2:
   - 3.1. If the step is at the root, the step must be of the form $M = (\lambda x.M_{11})M_2 \to_\beta M_{11}[M_2/x]$ with $M_1 = \lambda x.M_{11}$, and the corresponding step is
     $$M = (\lambda x.M_{11})\, M_2 \to_{\mathbf{m}} M_{11}[M_2/x]\langle M_2\rangle = s$$
     Then:
     $$\begin{array}{rcl} s & = & M_{11}[M_2/x]\langle M_2\rangle \\ & \rhd & M_{11}[M_2/x] \\ & = & N \end{array}$$
   - 3.2. If the step is internal to M_1, the step must be of the form $M_1\, M_2 \to_\beta N_1\, M_2 = N$ with $M_1 \to_\beta N_1$, and the corresponding step is $M_1\, M_2 \to_{\mathbf{m}} s_1\, M_2 = s$ where $M_1 \to_{\mathbf{m}} s_1$ is the step corresponding to $M_1 \to_{\mathbf{m}} N_1$. By IH we have that $s_1 \rhd N_1$, so $s = s_1\, M_2 \rhd N_1\, M_2 = N$.
   - 3.3. If the step is internal to M_2, the proof is similar to the previous case.

□

2.4.3 The measure

Definition 2.4.17 (The measure $\mathcal{W}$)**.** For each typable λ-term M, we define $\mathcal{W}(M) := \mathtt{w}(\mathtt{S}_*(M))$.

As shown in Proposition 2.4.10, the full simplification $\mathtt{S}_*(M)$ turns out to be exactly the normal form of M in the $\lambda^{\mathbf{m}}$-calculus. We insist in writing $\mathtt{S}_*(M)$ to emphasize that the *definition* of $\mathcal{W}$ does not require to prove that the $\lambda^{\mathbf{m}}$-calculus is weakly normalizing. Indeed, the simplification $\mathtt{S}_d(t)$ can be defined by structural induction on t, and the full simplification $\mathtt{S}_*(t) = \mathtt{S}_1(\mathtt{S}_2(\dots \mathtt{S}_D(t)))$ can be calculated in exactly D iterations. On the other hand, the *proof* that the $\mathcal{W}$-measure is decreasing does rely on the fact that $\mathtt{S}_*(M)$ is the normal form of M.

In the remainder of this section, we prove that the $\mathcal{W}$-measure is indeed decreasing. The following lemma states that forgetful reduction decreases weight, and it is straightforward to prove:

Lemma 2.4.18. If $t \rhd^+ s$ then $\mathtt{w}(t) > \mathtt{w}(s)$. □

The proof that the $\mathcal{W}$-measure decreases relies on the two following properties that relate full simplification $\mathtt{S}_*(\cdot)$ respectively with reduction ($\to_{\mathbf{m}}$) and forgetful reduction ($\rhd^+$):

Lemma 2.4.19. 1. If $t \to_{\mathbf{m}} s$ then $\mathtt{S}_*(t) = \mathtt{S}_*(s)$. 2. If $t \rhd^+ s$ then $\mathtt{S}_*(t) \rhd^+ \mathtt{S}_*(s)$.

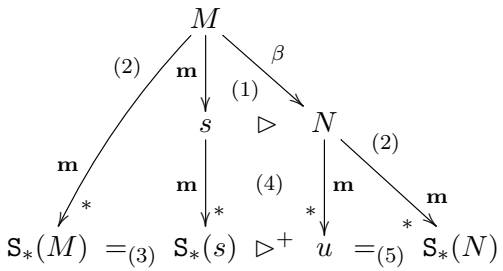
Proof. For the first item, note that by Proposition 2.4.10, we know that $t \to_{\mathbf{m}}^* \mathtt{S}_*(t)$ and that $t \to_{\mathbf{m}} s \to_{\mathbf{m}}^* \mathtt{S}_*(s)$, where moreover $\mathtt{S}_*(t)$ and $\mathtt{S}_*(s)$ are $\to_{\mathbf{m}}$-normal forms. By confluence (Proposition 2.3.14), this means that $\mathtt{S}_*(t) = \mathtt{S}_*(s)$.

For the second item, note that by Proposition 2.4.10, we know that $t \to_{\mathbf{m}}^* \mathtt{S}_*(t)$. Since we also know $t \rhd^+ s$ by hypothesis, and since forgetful reduction commutes with reduction (Proposition 2.4.15), there exists a term r such that $s \to_{\mathbf{m}}^* r$ and $\mathtt{S}_*(t) \rhd^+ r$. By Proposition 2.4.10 we know that $\mathtt{S}_*(t)$ is in normal form, so by Proposition 2.4.15 r must also be a normal form. On the other hand, by Proposition 2.4.10 we know that $s \to_{\mathbf{m}}^* \mathtt{S}_*(s)$, where $\mathtt{S}_*(s)$ must also be a normal form. In summary, we have that $s \to_{\mathbf{m}}^* r$ and $s \to_{\mathbf{m}}^* \mathtt{S}_*(s)$, where both r and $\mathtt{S}_*(s)$ are normal forms. By confluence (Proposition 2.3.14) $r = \mathtt{S}_*(s)$, and from this we obtain that $\mathtt{S}_*(t) \rhd^+ r = \mathtt{S}_*(s)$, as required. □

Theorem 2.4.20. Let M, N be typable λ-terms such that $M \to_\beta N$. Then $\mathcal{W}(M) > \mathcal{W}(N)$.

Proof. Given the step $M \to_\beta N$, consider the corresponding step $M \to_{\mathbf{m}} s$, and note that $s \rhd^+ N$ by the reduce/forget lemma (Lemma 2.4.16). Since $M \to_{\mathbf{m}} s \rhd^+ N$, by Lemma 2.4.19, we have that $\mathtt{S}_*(M) = \mathtt{S}_*(s) \rhd^+ \mathtt{S}_*(N)$. Finally, by Lemma 2.4.18, $\mathcal{W}(M) = \mathtt{w}(\mathtt{S}_*(M)) > \mathtt{w}(\mathtt{S}_*(N)) = \mathcal{W}(N)$. □

To give more intuition on the previous proof, we provide the following diagram showing some of the properties involved.



(1) Reduce/forget (Lemma 2.4.16)

(2) $\mathtt{S}_*$ is normal form (Proposition 2.4.10)

(3) Confluence (Proposition 2.3.14)

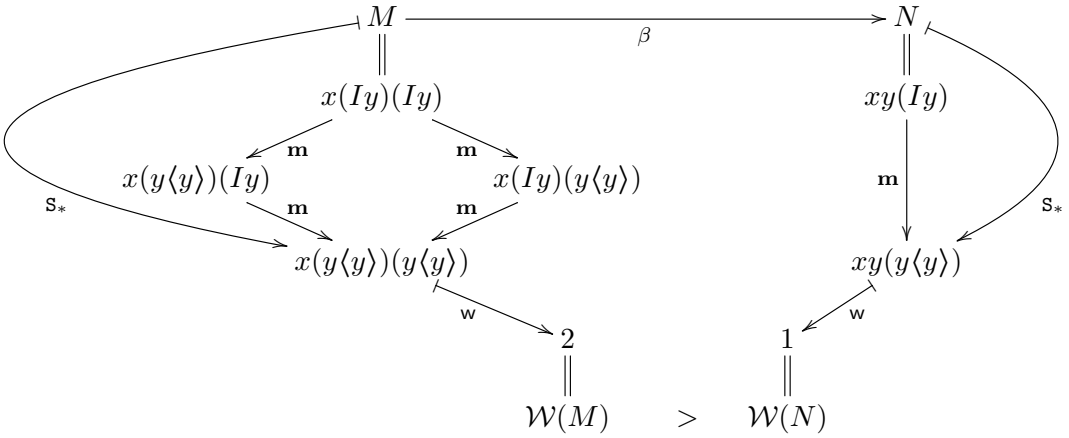
(4) Commutation of $\rhd$ and $\to_{\mathbf{m}}$ (Proposition 2.4.15)

(5) $\rhd$ preserves normal form (Proposition 2.4.15) and confluence (Proposition 2.3.14)

Note that Lemma 2.4.19 is decomposed into its parts in the diagram, in order to convey relevant information into it. Indeed, (2) and (3) are the pillars of the proof of Item 1, whose conclusion

is $\mathtt{S}_*(M) = \mathtt{S}_*(s)$, while (4) and (5) are the pillars of the proof of Item 2, whose conclusion is $\mathtt{S}_*(s) \rhd^+ \mathtt{S}_*(N)$.

Example 2.4.21. Let us introduce a simple example to quickly illustrate the decrease of the measure. Let $M = x(Iy)(Iy)$, which reduces *in the STLC* to $N = xy(Iy)$. Then we have the following measures.



Example 2.4.22. For an example with more wrappers involved, let

$$M = (\lambda x^{\tau}.y^{\tau\to\tau\to\tau}x^{\tau}x^{\tau})((\lambda x^{\tau\to\tau}.x^{\tau\to\tau}z^{\tau})f^{\tau\to\tau})$$

and consider the step

$$M = (\lambda x.yxx)((\lambda x.xz)f) \to_{\beta} (\lambda x.yxx)(fz) = N$$

Then we have that

$$\mathtt{S}_*(M) = (y(fz)\langle f\rangle(fz)\langle f\rangle)\langle(fz)\langle f\rangle\rangle \qquad \mathtt{S}_*(N) = (y(fz)(fz))\langle fz\rangle$$

Therefore

$$\mathcal{W}(M) = \mathtt{w}(\mathtt{S}_*(M)) = 4 > 1 = \mathtt{w}(\mathtt{S}_*(N)) = \mathcal{W}(N)$$

For another example, see 2.4.3.

2.5 $\mathcal{T}^{\mathbf{m}}$–measure: nesting developments by redexes and degrees

In this section we present the decreasing measure $\mathcal{T}^{\mathbf{m}}$. Its main objective is to adapt Turing's weak decreasing measure [47], which works by picking a specific reduction strategy, to a strong decreasing measure, *i.e.*, one that works under no assumed strategy.

As stated previously, Turing's measure yields a multiset of natural numbers, where each number correspond to the degree of an occurrence of a redex. For instance, the measure for the term $(\lambda x^{\tau}.\lambda y^{\tau}.x)((\lambda z^{\tau}.z)w)$ is determined by the types of the abstractions of the two redexes: the head redex has degree $\mathtt{h}(\tau \to \tau \to \tau) = 2$, and the redex in the argument has degree $\mathtt{h}(\tau \to \tau) = 1$. Therefore Turing's measure $\mathcal{T}$ yields

$$\underbrace{\mathcal{T}((\lambda x^{\tau}.\lambda y^{\tau}.x)(\underbrace{(\lambda z^{\tau}.z)w}_{1}))}_{2} = [2,1]$$

This multiset decreases when contracting the *rightmost innermost greatest* redex. The location of that specific redex ensures that no redexes of the same degree (nor of greater, because there is not any) can be copied, while Turing's observation forbids the creation of redexes of greater or equal degree. Hence each step decreases the amount of redexes of the greatest degree, and with it the occurrences of the greatest element in the multiset.

If we allow any redex contraction, we can easily find examples where contracting one redex of degree d decreases the number of d-redexes, but increases the number of D-redexes, with $D > d$. This produces a greater multiset than the one prior to the contraction, so it does not constitute a strong decreasing measure. For instance, the argument in the following term has a redex of higher degree (2) than the head redex (1); since the head redex copies the argument, its contraction subtract a redex of degree 1 while adding another of degree 2, increasing the multiset assigned by the measure.

$$\begin{aligned}
&\underbrace{\mathcal{T}((\lambda x^{\tau}.\underbrace{(\lambda y^{\tau}.\lambda z^{\tau}.y)x}_{2}\, x)(\underbrace{(\lambda w_2^{\tau\to\tau}.w_2)(\lambda w_1^{\tau}.w_1)}_{2}\, w_0))}_{1}\\
&= [2,2,1]\\
&< [2,2,2]\\
&= \underbrace{\mathcal{T}((\lambda y^{\tau}.\lambda z^{\tau}.y)(\underbrace{(\lambda w_2^{\tau\to\tau}.w_2)(\lambda w_1^{\tau}.w_1)}_{2}\, w_0)}_{2}(\underbrace{(\lambda w_2^{\tau\to\tau}.w_2)(\lambda w_1^{\tau}.w_1)}_{2}\, w_0))
\end{aligned}$$

While attempting to make Turing's measure decrease for any redex, we face the need to consider more information than the one in the pair. The result is a structure of nested multisets, built from the degrees of all redexes, together with the *developments* of each degree.

We define two auxiliary measures $\mathcal{T}^{\mathbf{m}}_{\leq D}(t)$ and $\mathcal{R}^{\mathbf{m}}_{D}(t)$, indexed by a natural number $D \in \mathbb{N}_0$, mutually recursively:

- $\mathcal{T}^{\mathbf{m}}_{\leq D}(t)$ is the multiset of pairs $(d, \mathcal{R}^{\mathbf{m}}_{d}(t))$, for each redex occurrence of degree $d \leq D$ in t;
- $\mathcal{R}^{\mathbf{m}}_{D}(t)$ is the multiset of elements $\mathcal{T}^{\mathbf{m}}_{\leq D-1}(t')$, for each reduction sequence $t \xrightarrow{D}{}^{*}_{\mathbf{m}} t'$.

The measure $\mathcal{T}^{\mathbf{m}}_{\leq D}(t)$ is defined for every $D \geq 0$, while $\mathcal{R}^{\mathbf{m}}_{D}(t)$ is defined only for $D \geq 1$. Multisets are ordered according to the usual multiset ordering, and pairs according to the lexicographic ordering. To calculate the $\mathcal{T}^{\mathbf{m}}$-measure of a λ-term M, let D be the maximum degree of the redexes in M, and define $\mathcal{T}^{\mathbf{m}}_{\leq}(M) := \mathcal{T}^{\mathbf{m}}_{\leq D}(M)$. The measure $\mathcal{T}^{\mathbf{m}}_{\leq}(M)$ yields a structure of nested multisets of nesting depth at most $2D$. The main result of this section is Theorem 2.5.17, stating that $\mathcal{T}^{\mathbf{m}}$ is decreasing, *i.e.*, that $M \to_{\beta} N$ implies $\mathcal{T}^{\mathbf{m}}_{\leq}(M) > \mathcal{T}^{\mathbf{m}}_{\leq}(N)$.

As stated in the previous section, note that we reutilize some of the definitions from the $\mathcal{W}$–measure. Specifically, the simplification operation and forgetful reduction are reused here with no modifications. This reflects the original flow of work in the development of this paper, where $\mathcal{W}$ was produced as a whole before $\mathcal{T}^{\mathbf{m}}$. Also this independence for $\mathcal{W}$ seemed to be the clearest process for introducing them, considering that later in this document it gets extended to another system and further refined to match certain output expectations.

In Section 2.5.1 we prove some deeply technical properties that are needed for the decreasing proof of $\mathcal{T}^{\mathbf{m}}$. In Section 2.5.2 we define the decreasing measures, and we prove it decreases.

2.5.1 Properties

In this section we study *reduction by degrees*, a restricted notion of reduction in the $\lambda^{\mathbf{m}}$-calculus, written $t \xrightarrow{d}_{\mathbf{m}} s$, meaning that t reduces to s by contracting a redex of degree d. It is a purely technical section that contains technical commutation, termination, and postponement results. The aim is to develop tools that we use in the following section to reason about the $\mathcal{T}^{\mathbf{m}}$-measure. To do so, we need to introduce witnesses of steps and reduction sequences, treating the $\lambda^{\mathbf{m}}$-calculus as an *abstract rewriting system* [99, Def. 8.2.2] or as a *transition system* [72, Def. 1]. *Objects* are $\lambda^{\mathbf{m}}$-terms, *steps* are 5-uples $R = (\mathtt{C}, x, t, \mathtt{L}, s)$ witnessing the reduction step $\mathtt{C}[(\lambda x.t)\mathtt{L}s] \to_{\mathbf{m}} \mathtt{C}[t[s/x]\langle s\rangle\mathtt{L}]$ under a context $\mathtt{C}$, and *reductions* $(\rho, \sigma, \ldots)$ are sequences of composable steps. Similarly, *forgetful steps* are triples $R = (\mathtt{C}, t, s)$ witnessing the forgetful reduction $\mathtt{C}[t\langle s\rangle] \rhd \mathtt{C}[t]$, and *forgetful reductions* (also written $\rho, \sigma, \ldots$) are sequences of composable forgetful steps. We write ρ^{src} and ρ^{tgt} respectively for the source and target terms of ρ.

For each $d \in \mathbb{N}_0$, we define *reduction of degree* d as follows:

Definition 2.5.1. $t \xrightarrow{d}_{\mathbf{m}} s$ if and only if $t \to_{\mathbf{m}} s$ by contracting a redex of degree d.

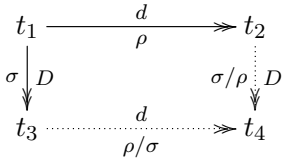
We write $R : t \xrightarrow{d}_{\mathbf{m}} s$ if R is a step witnessing a reduction step of degree d, and $\rho : t \xrightarrow{d}{}^{*}_{\mathbf{m}} s$ if ρ is a reduction witnessing a sequence of reduction steps of degree d.

The following results require to explicitly manipulate steps and reductions. We give sketches of the proofs. See Appendix A.2.1 for detailed proofs.

Proposition 2.5.2 (Commutation of reduction by degrees)**.** For any two reductions $\rho : t_1 \xrightarrow{d}{}^{*}_{\mathbf{m}} t_2$ and $\sigma : t_1 \xrightarrow{D}{}^{*}_{\mathbf{m}} t_3$, there exists a term t_4 such that one can construct reductions $\sigma/\rho : t_2 \xrightarrow{D}{}^{*}_{\mathbf{m}} t_4$ and $\rho/\sigma : t_3 \xrightarrow{d}{}^{*}_{\mathbf{m}} t_4$. Furthermore, if $d \neq D$, then:

1. ρ/σ contains at least as many steps as ρ
2. ρ/σ determines ρ, that is, $\rho_1/\sigma = \rho_2/\sigma$ implies $\rho_1 = \rho_2$

Graphically:



Proof. This is reduced to the fact that the $\lambda^{\mathbf{m}}$-calculus can be understood as an orthogonal higher-order rewriting system in the sense of Nipkow [78]. Indeed, ρ/σ and σ/ρ can be taken to be the standard notion of projection based on residuals for orthogonal HRSs. We note that residuals keep the same type, and hence the same degree, as their ancestor; therefore ρ/σ and σ/ρ are of degree d and D respectively. Furthermore, item 1 holds because the $\lambda^{\mathbf{m}}$-calculus is non-erasing; item 2 is a consequence of the *unique ancestor* property, *i.e.*, each redex *descends* from at most one redex.

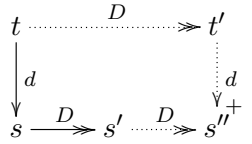
Alternatively, the proof can be conducted by relying on an auxiliary notion of *multi-step* of degree d, written $\mathbf{S} : t \xRightarrow{d}_{\mathbf{m}} s$, that allows contracting many redexes of degree d simultaneously. The result can be reduced to a local commutation result, stating that for each step $R : t_1 \xrightarrow{d}_{\mathbf{m}} t_2$ and each multi-step $\mathbf{S} : t_1 \xRightarrow{D}_{\mathbf{m}} t_3$, there exists a term t_4, a multi-step $\mathbf{S}/R : t_2 \xRightarrow{D}_{\mathbf{m}} t_4$ and a multi-step $R/\mathbf{S} : t_3 \xRightarrow{d}_{\mathbf{m}} t_4$. This can be checked by exhaustive case analysis. See Proposition A.2.7 in the appendix for the detailed proof. □

Corollary 2.5.3 (Termination of reduction by degrees)**.** The relation $\xrightarrow{d}_{\mathbf{m}}$ is strongly normalizing.

Proof. This is a consequence of the fact that HRSs enjoy the Finite Developments property [99, Theorem 11.5.11], observing that reduction of degree d does not create redexes of degree d.

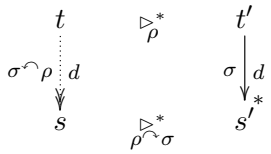
Alternatively, it can be easily shown that $t \xrightarrow{d}{}^{*}_{\mathbf{m}} \mathtt{S}_d(t)$ and $\mathtt{S}_d(t)$ is in $\xrightarrow{d}_{\mathbf{m}}$-normal form, so $\xrightarrow{d}_{\mathbf{m}}$ is WN (see Lemma A.2.8 and Lemma A.2.11 in the Appendix A.2.1). Then one can observe that $\xrightarrow{d}_{\mathbf{m}}$ is *uniformly normalizing* [63], given that there is no erasure, which entails that $\xrightarrow{d}_{\mathbf{m}}$ is SN. The last part could also be proved from $\xrightarrow{d}_{\mathbf{m}}$ being WCR by Proposition 2.5.2, and increasing, because $t \xrightarrow{d}_{\mathbf{m}} t'$, which implies $\mathtt{w}(t) < \mathtt{w}(t')$. As noted at the beginning of Section 2.4, this means that $\#(t) := \mathtt{w}(\mathtt{S}_d(t)) - \mathtt{w}(t)$ is decreasing, entailing SN for $\xrightarrow{d}_{\mathbf{m}}$. □

Proposition 2.5.4 (Lifting property for lower steps)**.** Let $d < D$ and $t \xrightarrow{d}_{\mathbf{m}} s \xrightarrow{D}{}^{*}_{\mathbf{m}} s'$. Then there exist terms t', s'' such that $t \xrightarrow{D}{}^{*}_{\mathbf{m}} t'$ and $s' \xrightarrow{D}{}^{*}_{\mathbf{m}} s''$ and $t' \xrightarrow{d}{}^{+}_{\mathbf{m}} s''$. Graphically:



Proof. Note that $t \xrightarrow{D}{}^{*}_{\mathbf{m}} \mathtt{S}_D(t)$. By Proposition 2.5.2, there exists a term r such that $s \xrightarrow{D}{}^{*}_{\mathbf{m}} r$ and $\mathtt{S}_D(t) \xrightarrow{d}{}^{+}_{\mathbf{m}} r$. Again by Proposition 2.5.2, there exists s'' such that $r \xrightarrow{D}{}^{*}_{\mathbf{m}} s''$ and $s' \xrightarrow{D}{}^{*}_{\mathbf{m}} s''$. Moreover, $\mathtt{S}_D(t)$ is in $\xrightarrow{D}_{\mathbf{m}}$-normal form. Since $\mathtt{S}_D(t) \xrightarrow{d}{}^{*}_{\mathbf{m}} r$ with $d < D$ and reduction does not create redexes of higher degree, r is also in $\xrightarrow{D}_{\mathbf{m}}$-normal form, so $r = s''$, and we are done. □

Proposition 2.5.5 (Postponement of forgetful reduction)**.** For any two reductions $\rho : t \rhd^{*} t'$ and $\sigma : t' \xrightarrow{d}{}^{*}_{\mathbf{m}} s'$, there exists a term s and reductions $\rho^{\frown}\sigma : s \rhd^{*} s'$ and $\sigma^{\curvearrowleft}\rho : t \xrightarrow{d}{}^{*}_{\mathbf{m}} s$. Furthermore, $\sigma^{\curvearrowleft}\rho$ determines σ, that is, $\sigma_1^{\curvearrowleft}\rho = \sigma_2^{\curvearrowleft}\rho$ implies $\sigma_1 = \sigma_2$. Graphically:



Proof. This can be reduced to an analysis of the critical pairs between the rewriting rules defining $\rhd^{-1}$ and $\to_{\mathbf{m}}$. Critical pairs are of the form $(\lambda x.t)\mathtt{L}_1\langle s\rangle\mathtt{L}_2 r \rhd (\lambda x.t)\mathtt{L}_1\mathtt{L}_2 r \to_{\mathbf{m}} t[r/x]\langle r\rangle\mathtt{L}_1\mathtt{L}_2$ and can be closed by $(\lambda x.t)\mathtt{L}_1\langle s\rangle\mathtt{L}_2 r \to_{\mathbf{m}} t[r/x]\langle r\rangle\mathtt{L}_1\langle s\rangle\mathtt{L}_2 \rhd t[r/x]\langle r\rangle\mathtt{L}_1\mathtt{L}_2$. □

2.5.2 The measure

In this section, we define the $\mathcal{T}^{\mathbf{m}}$-measure (Definition 2.5.9) and we prove that it is decreasing (Theorem 2.5.17). We start with some preliminary notions.

A partially ordered set $(X, >)$ is *well-founded* if there are no infinite decreasing chains. $\mathbb{M}(X)$ denotes the set of *finite multisets* over a set X, which are functions $\mathfrak{m} : X \to \mathbb{N}_0$ such that $\mathfrak{m}(x) > 0$ for finitely many values of $x \in X$. We write $\mathfrak{m} + \mathfrak{n}$ for the sum of multisets, and $x \in \mathfrak{m}$ if $\mathfrak{m}(x) > 0$. We write $[x_1, \ldots, x_n]$ for the multiset of elements $x_1, \ldots, x_n$, taking multiplicities into account. If X is a finite set and $f : X \to Y$ is a function, we use the "multiset builder" notation $[f(x) \mid\mid x \in X]$

to denote the multiset $\sum_{x\in X}[f(x)]$. If $(X,>)$ is a partially ordered set, we define a binary relation $\succ^1$ on multisets by declaring that $\mathfrak{m}+[x] \succ^1 \mathfrak{m}+\mathfrak{n}$ if $x > y$ for every $y \in \mathfrak{n}$. The *multiset order* induced by $(X,>)$ is the strict order relation on multisets defined by declaring that $\mathfrak{m} \succ \mathfrak{n}$ if and only if $\mathfrak{m}\ (\succ^1)^+\ \mathfrak{n}$. We recall the following widely known theorem by Dershowitz and Manna [34]:

Theorem 2.5.6. If $(X,>)$ is well-founded, then $(\mathbb{M}(X),\succ)$ is well-founded.

As usual, $\mathfrak{m} \succeq \mathfrak{n}$ stands for $(\mathfrak{m}=\mathfrak{n} \vee \mathfrak{m} \succ \mathfrak{n})$, and $\mathfrak{m} \preceq \mathfrak{n}$ stands for $\mathfrak{n} \succeq \mathfrak{m}$. We define an operation $k \otimes \mathfrak{m}$ by the recursive equations $0 \otimes \mathfrak{m} := [\,]$ and $(1+k)\otimes\mathfrak{m} := \mathfrak{m} + k\otimes\mathfrak{m}$. The relation $\mathfrak{m} :\succ: \mathfrak{n}$, called the *pointwise multiset order*, is defined to hold if $\mathfrak{m}$ and $\mathfrak{n}$ can be written as of the forms $\mathfrak{m}=[x_1,\ldots,x_n]$ and $\mathfrak{n}=[y_1,\ldots,y_n]$ in such a way that $x_i > y_i$ for all $i \in 1..n$. Observe that if $\mathfrak{m} :\succ: \mathfrak{n}$ then for all $k \in \mathbb{N}_0$ we have that $\mathfrak{m} \succeq k \otimes \mathfrak{n}$. Another easy-to-check property is that if $\mathfrak{m} :\succ: \mathfrak{n}$ and $\mathfrak{m}$ is non-empty then $\mathfrak{m} \succ \mathfrak{n}$.

A first frustrated attempt As previously mentioned, Turing's measure, given by $\mathcal{T}(M) := [d \,||\, R \text{ is a redex occurrence of degree } d \text{ in } M]$, decreases when contracting the rightmost innermost redex of highest degree. Our goal is to extend $\mathcal{T}$ in such a way that contracting *any* redex decreases the measure. The difficulty is that a redex of degree d may copy redexes of a higher or equal degree $d' \geq d$. So one can wonder: whenever a redex R of degree d makes n copies of a redex S of degree $d' \geq d$, in what sense can the copies of S be considered "smaller" than S? To address this, we generalize $\mathcal{T}$ to a family of measures $\mathcal{T}_D(M) := [(d, \mathcal{T}_{d-1}(M)) \,||\, R \text{ is a redex occurrence of degree } d \leq D \text{ in } M]$ indexed by a degree $D \in \mathbb{N}_0$. Note that $\mathcal{T}_0(M)$ is the empty multiset because there are no redexes of degree 0.

Let us try to argue that if $d \leq D$ and $M \xrightarrow{d}_\beta N$ then $\mathcal{T}_D(M) \succ \mathcal{T}_D(N)$. Here $M \xrightarrow{d}_\beta N$ means that $M \to_\beta N$ by contracting a redex of degree d. Suppose that the contraction of the redex $R : M \xrightarrow{d}_\beta N$ copies a redex S of degree d', where we assume that $d < d' \leq D$, producing n copies $S_1,\ldots,S_n$. Note that the contribution of S to the multiset is $(d', \mathcal{T}_{d'-1}(M))$, and the contribution of each S_i is $(d', \mathcal{T}_{d'-1}(N))$. By induction on D, we could inductively argue that $\mathcal{T}_{d'-1}(M) \succ \mathcal{T}_{d'-1}(N)$, since $d'-1 < d' \leq D$. So far the property would seem to hold.

The problem with this proposal is that a redex R of degree d may still make copies of redexes of degree *exactly* d, whose contribution does not necessarily decrease. For instance, in

$$M = (\lambda x^\tau.y^{\tau\to\tau\to\tau}x^\tau x^\tau)((\lambda z^\tau.z^\tau)w^\tau) \xrightarrow{1}_\beta y^{\tau\to\tau\to\tau}((\lambda z^\tau.z^\tau)w^\tau)((\lambda z^\tau.z^\tau)w^\tau) = N$$

the measure does not decrease, as $\mathcal{T}_1(M) = [(1,[]),(1,[])] = \mathcal{T}_1(N)$.

A second frustrated attempt The difficulty is to deal with the situation in which a redex R of degree d makes n copies of a redex S of the same degree d. A key observation is that a reduction sequence $M \xrightarrow{d}_\beta^* N$ must be a *development*[3] of the set of redexes of degree d. This is because contracting a redex of degree d can only create redexes of degree strictly less than d, so any redex of degree d that remains after one $\xrightarrow{d}_\beta$-step must be a *residual* of a preexisting redex. This motivates our second attempt to define a measure, consisting of two families of measures $\mathcal{T}^\beta_{\leq D}(\cdot)$ and $\mathcal{R}^\beta_D(\cdot)$, indexed by $D \in \mathbb{N}_0$ and defined mutually recursively:

$$\mathcal{T}^\beta_{\leq D}(M) := [(d, \mathcal{R}^\beta_d(M)) \,||\, R \text{ is a } \beta\text{-redex occurrence of degree } d \leq D \text{ in } M]$$

[3]Recall that a development of a set of redexes X is a reduction sequence $M \to^*_\beta N$ in which each step contracts a *residual* of a redex in X. The residuals of a redex $S : t \to_\beta s$ after the contraction of a redex $R : t \to_\beta t'$ are, informally speaking, the "copies" left of S in t'. For formal definitions see Barendregt's book [9, Section 11.2].

$$\mathcal{R}^{\beta}_{D}(M) := [\mathcal{T}^{\beta}_{\leq D-1}(M') \mid\mid \rho : M \xrightarrow{D}{}^{*}_{\beta} M']$$

Note that there are no redexes of degree 0, so $\mathcal{T}^{\beta}_{\leq D}(M)$ may not depend on $\mathcal{R}^{\beta}_{0}(M)$. In fact, $\mathcal{R}^{\beta}_{D}(M)$ is defined only for $D \geq 1$. The recursive definition is well-founded because $\mathcal{T}^{\beta}_{\leq D}(M)$ may depend on $\mathcal{R}^{\beta}_{1}(M), \ldots, \mathcal{R}^{\beta}_{D}(M)$ which in turn may only depend on $\mathcal{T}^{\beta}_{\leq d}(M')$ for $d < D$. The multiplicity of $\mathcal{T}^{\beta}_{\leq D-1}(M')$ in the multiset $\mathcal{R}^{\beta}_{D}(M)$ is given by the number of reduction sequences that contract only redexes of degree D, that is, the number of different paths $M \xrightarrow{D}{}^{*}_{\mathbf{m}} M'$. One important point is that, for the measure $\mathcal{R}^{\beta}_{D}(t)$ to be well defined, one needs to argue that the number of paths $M \xrightarrow{D}{}^{*}_{\mathbf{m}} M'$ is finite. Since $M \xrightarrow{D}{}^{*}_{\mathbf{m}} M'$ is a development, this is a consequence of the *finite developments* (FD) property for orthogonal HRSs [99, Theorem 11.5.11].[4]

Let us try to argue that if $d \leq D$ and $M \xrightarrow{d}_{\beta} N$ then $\mathcal{T}^{\beta}_{\leq D}(M) \succ \mathcal{T}^{\beta}_{\leq D}(N)$. On the first hand, if a redex $R : M \xrightarrow{d}_{\beta} N$ of degree d copies a redex S of *exactly* the same degree d making n copies $S_1, \ldots, S_n$, the contribution of S to the multiset is $(d, \mathcal{R}^{\beta}_{d}(M))$, whereas each S_i contributes $(d, \mathcal{R}^{\beta}_{d}(N))$, and we can argue that $\mathcal{R}^{\beta}_{d}(M) \succ \mathcal{R}^{\beta}_{d}(N)$, because we can injectively map each reduction sequence $\rho : N \xrightarrow{d}{}^{*}_{\beta} N'$ to the reduction sequence $R\rho : M \xrightarrow{d}_{\beta} N \xrightarrow{d}{}^{*}_{\beta} N'$, where $R\rho$ denotes the composition of R and ρ. Furthermore, there is an empty reduction sequence $M \xrightarrow{d}{}^{*}_{\beta} M$ contributing an element $\mathcal{T}^{\beta}_{\leq d-1}(M)$ to $\mathcal{R}^{\beta}_{d}(M)$ but not to $\mathcal{R}^{\beta}_{d}(N)$.

On the other hand, if the contraction of a redex $R : M \xrightarrow{d}_{\beta} N$ of degree d copies a redex S of *strictly* greater degree $d' > d$ making n copies $S_1, \ldots, S_n$, the weight of S is $(d', \mathcal{R}^{\beta}_{d'}(M))$ and the weight of each S_i is $(d', \mathcal{R}^{\beta}_{d'}(N))$, and we would need to show that $\mathcal{R}^{\beta}_{d'}(M) \succ \mathcal{R}^{\beta}_{d'}(N)$. One way to do so would be to map each reduction sequence $\rho : N \xrightarrow{d}{}^{*}_{\beta} N'$ to a reduction sequence $\sigma : M \xrightarrow{d}{}^{*}_{\beta} M'$ such that $\mathcal{T}^{\beta}_{\leq d'-1}(M') \succ \mathcal{T}^{\beta}_{\leq d'-1}(N')$. However, there does not seem to be a way to rule out the possibility that σ might erase R and that $M' = N'$, which would yield $\mathcal{T}^{\beta}_{\leq d'-1}(M') = \mathcal{T}^{\beta}_{\leq d'-1}(N')$, rather than a strict inequality. The root of the problem seems again to be *erasure*.

Definition of the $\mathcal{T}^{\mathbf{m}}$-measure The $\mathcal{T}^{\mathbf{m}}$-measure is based on the ideas described above, but considering reduction in the $\lambda^{\mathbf{m}}$-calculus rather than in the STLC, to ensure that there is no erasure. Informally, the $\mathcal{T}^{\mathbf{m}}$-measure is defined by means of the two following equations. These equations are exactly as the ones defining $\mathcal{T}^{\beta}_{\leq D}(\cdot)$ and $\mathcal{R}^{\beta}_{D}(\cdot)$ above, with the only difference that they deal with $\lambda^{\mathbf{m}}$-terms and $\to_{\mathbf{m}}$-reduction rather than with pure λ-terms and $\to_{\beta}$-reduction:

$$\mathcal{T}^{\mathbf{m}}_{\leq D}(t) := [(d, \mathcal{R}^{\mathbf{m}}_{d}(t)) \mid\mid R \text{ is an } \mathbf{m}\text{-redex occurrence of degree } d \leq D \text{ in } t]$$

$$\mathcal{R}^{\mathbf{m}}_{D}(t) := [\mathcal{T}^{\mathbf{m}}_{\leq D-1}(t') \mid\mid \rho : t \xrightarrow{D}{}^{*}_{\mathbf{m}} t']$$

To be able to reason about these measures inductively, it will be convenient to define an auxiliary measure $\mathcal{T}^{\mathbf{m}}_{d}(t_0, t)$ as the multiset of elements of the form $(d, \mathcal{R}^{\mathbf{m}}_{d}(t_0))$ for each $\mathbf{m}$-redex occurrence of degree *exactly* d in t. This auxiliary measure takes two arguments t_0 and t, and it is defined by structural recursion on the second argument (t), while the first argument (t_0) is used to keep

[4] Note that FD only ensures that developments are finite. To see that the set $\{\rho \mid M \xrightarrow{D}{}^{*}_{\mathbf{m}} M'\}$ is finite, one should resort to König's lemma, together with the fact that the STLC is finitely branching. For a constructive proof, one can use a computable decreasing measure, such as in de Vrijer's proof of FD [31].

track of the original term. Note that, with this auxiliary definition, we can write $\mathcal{T}^{\mathbf{m}}_{\leq D}(t)$ as a sum, namely $\mathcal{T}^{\mathbf{m}}_{\leq D}(t) = \mathcal{T}^{\mathbf{m}}_1(t,t) + \ldots + \mathcal{T}^{\mathbf{m}}_D(t,t)$.

To define the measure formally, we begin by precisely defining its codomain.

Definition 2.5.7 (Codomain of the $\mathcal{T}^{\mathbf{m}}$-measure)**.** For each $d \geq 0$, we define a set $\mathbb{T}_d$, and for $d \geq 1$ we define a set $\mathbb{R}_d$, mutually recursively:

$$\mathbb{T}_d := \mathbb{M}(\{(i,b) \mid 1 \leq i \leq d,\ b \in \mathbb{R}_i\}) \qquad \mathbb{R}_d := \mathbb{M}(\mathbb{T}_{d-1})$$

The sets $\mathbb{T}_d$ and $\mathbb{R}_d$ are partially ordered by the induced multiset ordering on their elements. Tuples (i,b) are ordered with the lexicographic order, that is, $(i,b) > (i',b')$ if and only if $i > i' \vee (i = i' \wedge b \succ b')$. Note that $\mathbb{T}_0 = \{[\,]\}$ and that if $d \leq d'$ then $\mathbb{T}_d \subseteq \mathbb{T}_{d'}$ and $\mathbb{R}_d \subseteq \mathbb{R}_{d'}$. Moreover, $(\mathbb{T}_d, \succ)$ and $(\mathbb{R}_d, \succ)$ are well-founded partial orders by Theorem 2.5.6.

Given typable $\lambda^{\mathbf{m}}$-terms t_0, t, and $d \in \mathbb{N}_0$, we define $\mathcal{T}^{\mathbf{m}}_d(t_0,t) \in \mathbb{T}_d$ and $\mathcal{T}^{\mathbf{m}}_{\leq d}(t) \in \mathbb{T}_d$, and if $d > 0$ we define $\mathcal{R}^{\mathbf{m}}_d(t) \in \mathbb{R}_d$, by induction on d as follows. Note that $\mathcal{T}^{\mathbf{m}}_d(t_0,t)$ is defined by a nested induction on t, and it is also defined on memories ($\mathcal{T}^{\mathbf{m}}_d(t_0,\mathtt{L})$):

Definition 2.5.8 (The measures $\mathcal{T}^{\mathbf{m}}_d(\cdot,\cdot)$, $\mathcal{T}^{\mathbf{m}}_{\leq d}(\cdot)$, and $\mathcal{R}^{\mathbf{m}}_d(\cdot)$)**.**

$$\begin{aligned}
\mathcal{T}^{\mathbf{m}}_d(t_0,x) &:= [\,] \\
\mathcal{T}^{\mathbf{m}}_d(t_0,\lambda x.s) &:= \mathcal{T}^{\mathbf{m}}_d(t_0,s) \\
\mathcal{T}^{\mathbf{m}}_d(t_0,sr) &:= \begin{cases} \mathcal{T}^{\mathbf{m}}_d(t_0,s') + \mathcal{T}^{\mathbf{m}}_d(t_0,\mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t_0,r) + [(d,\mathcal{R}^{\mathbf{m}}_d(t_0))] \\ \qquad\qquad \text{if } s = (\lambda x.s')\mathtt{L} \text{ and it is of degree } d \\ \mathcal{T}^{\mathbf{m}}_d(t_0,s) + \mathcal{T}^{\mathbf{m}}_d(t_0,r) \qquad\qquad \text{otherwise} \end{cases} \\
\mathcal{T}^{\mathbf{m}}_d(t_0,s\langle r\rangle) &:= \mathcal{T}^{\mathbf{m}}_d(t_0,s) + \mathcal{T}^{\mathbf{m}}_d(t_0,r)
\end{aligned}$$

$$\begin{aligned}
\mathcal{T}^{\mathbf{m}}_d(t_0,\Box) &:= [\,] \\
\mathcal{T}^{\mathbf{m}}_d(t_0,\mathtt{L}\langle t\rangle) &:= \mathcal{T}^{\mathbf{m}}_d(t_0,\mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t_0,t)
\end{aligned}$$

$$\begin{aligned}
\mathcal{T}^{\mathbf{m}}_{\leq d}(t) &:= \textstyle\sum_{i=1}^{d} \mathcal{T}^{\mathbf{m}}_i(t,t) \\
\mathcal{R}^{\mathbf{m}}_d(t) &:= [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(t') \mid\mid \rho : t \xrightarrow{d}{}^{*}_{\mathbf{m}} t']
\end{aligned}$$

Moreover, the **$\mathcal{T}^{\mathbf{m}}$-measure** itself is defined for λ-terms as follows:

Definition 2.5.9. If M is a typable λ-term, $\mathcal{T}^{\mathbf{m}}(M) := \mathcal{T}^{\mathbf{m}}_{\leq D}(M)$ where $D := \delta_{\mathsf{max}}(M)$.

When we write $\mathcal{T}^{\mathbf{m}}_{\leq D}(M)$, we implicitly regard M as a $\lambda^{\mathbf{m}}$-term without any memorized terms.

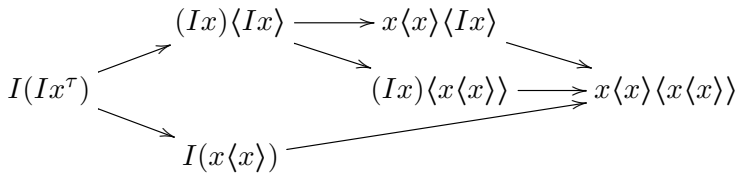
From a higher-level perspective, the $\mathcal{T}^{\mathbf{m}}_d(t_0,t)$ measure defined above is the multiset of pairs of the form $(d,\mathcal{R}^{\mathbf{m}}_d(t_0))$ for each redex of degree d in t. Similarly, $\mathcal{T}^{\mathbf{m}}_{\leq D}(t)$ is the multiset of pairs of the form $(d,\mathcal{R}^{\mathbf{m}}_d(t))$ for each redex of degree $d \leq D$ in t. In particular, $\mathcal{T}^{\mathbf{m}}_0(t_0,t)$ and $\mathcal{T}^{\mathbf{m}}_{\leq 0}(t)$ are empty multisets, because there are no redexes of degree 0. Two easy remarks are that $D \leq D'$ implies $\mathcal{T}^{\mathbf{m}}_{\leq D}(t) \preceq \mathcal{T}^{\mathbf{m}}_{\leq D'}(t)$, and that $\mathcal{T}^{\mathbf{m}}_d(t_0,t\mathtt{L}) = \mathcal{T}^{\mathbf{m}}_d(t_0,t) + \mathcal{T}^{\mathbf{m}}_d(t_0,\mathtt{L})$.

Remark 2.5.10. As mentioned in the preceding discussion, one important point is that, for $\mathcal{R}^{\mathbf{m}}_d(\cdot)$ to be well-defined, we need to argue that the set $\{\rho \mid \exists t'.\rho : t \xrightarrow{d}{}^{*}_{\mathbf{m}} t'\}$ is finite. This is a consequence of Corollary 2.5.3.

Example 2.5.11 (Counting reduction sequences)**.** Let $I = \lambda x^{\tau}.x$, and consider $t_0 = I(Ix^{\tau})$. We have two redexes of degree 1 in t_0, so its $\mathcal{T}^{\mathbf{m}}$ value is

$$\mathcal{T}^{\mathbf{m}}(t_0) = \mathcal{T}^{\mathbf{m}}_{\leq 1}(t_0) = [(1,\mathcal{R}^{\mathbf{m}}_1(t_0)),\ (1,\mathcal{R}^{\mathbf{m}}_1(t_0))]$$

To compute $\mathcal{R}_1^{\mathbf{m}}(t_0)$, we consider all the 1-degree developments from t_0 in the $\lambda^{\mathbf{m}}$-calculus:



We can count eight developments: the empty one, four ending in intermediate terms (one for each), and three ending in the normal form. Since $\mathcal{T}_{\leq 0}^{\mathbf{m}}(\cdot)$ is always the empty multiset, we get

$$\mathcal{R}_1^{\mathbf{m}}(t_0) = 8 \otimes [[\,]]$$

We can observe that $\mathcal{T}_{\leq 1}^{\mathbf{m}}$ decreases for both reducts of t_0, as they contain no redex degree greater than 1, and the nested multiset shrinks.

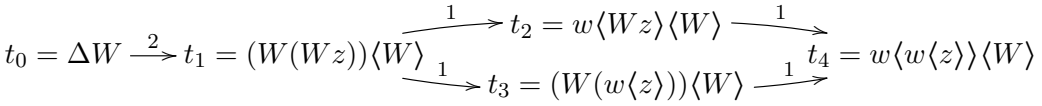
- $t_1 = (Ix)\langle Ix\rangle$: We count here two redexes and five 1-degree developments; the empty one, two intermediate ones, and two for the normal form, obtaining

$$\mathcal{T}_{\leq 1}^{\mathbf{m}}(t_1) = [(1, \mathcal{R}_1^{\mathbf{m}}(t_1)),\ (1, \mathcal{R}_1^{\mathbf{m}}(t_1))] \qquad \text{with } \mathcal{R}_1^{\mathbf{m}}(t_1) = 5 \otimes [[\,]]$$

- $t_2 = I(x\langle x\rangle)$: We count here one redex and two 1-degree developments; the empty one and the normal form one, obtaining

$$\mathcal{T}_{\leq 1}^{\mathbf{m}}(t_2) = [(1, \mathcal{R}_1^{\mathbf{m}}(t_2))] \qquad \text{with } \mathcal{R}_1^{\mathbf{m}}(t_2) = 2 \otimes [[\,]]$$

Example 2.5.12. For a more exhaustive example, let $\Delta := \lambda x^{\tau\to\tau}.x^{\tau\to\tau}(x^{\tau\to\tau}z^{\tau})$ and $W := \lambda y^{\tau}.w^{\tau}$, and consider the diagram:



We have then that

$$\begin{array}{rcccl}
 & & \mathcal{T}_{\leq 2}^{\mathbf{m}}(t_0) & = & [(2, \mathcal{R}_2^{\mathbf{m}}(t_0))] \\
\mathcal{T}_{\leq 2}^{\mathbf{m}}(t_1) & = & \mathcal{T}_{\leq 1}^{\mathbf{m}}(t_1) & = & [(1, \mathcal{R}_1^{\mathbf{m}}(t_1)), (1, \mathcal{R}_1^{\mathbf{m}}(t_1))] \\
\mathcal{T}_{\leq 2}^{\mathbf{m}}(t_2) & = & \mathcal{T}_{\leq 1}^{\mathbf{m}}(t_2) & = & [(1, \mathcal{R}_1^{\mathbf{m}}(t_2))] \\
\mathcal{T}_{\leq 2}^{\mathbf{m}}(t_3) & = & \mathcal{T}_{\leq 1}^{\mathbf{m}}(t_3) & = & [(1, \mathcal{R}_1^{\mathbf{m}}(t_3))] \\
\mathcal{T}_{\leq 2}^{\mathbf{m}}(t_4) & = & \mathcal{T}_{\leq 1}^{\mathbf{m}}(t_4) & = & [\,]
\end{array}$$

Since $\mathcal{T}_{\leq 1}^{\mathbf{m}}(t_0) = \mathcal{T}_{\leq 0}^{\mathbf{m}}(t_1) = \mathcal{T}_{\leq 0}^{\mathbf{m}}(t_2) = \mathcal{T}_{\leq 0}^{\mathbf{m}}(t_3) = \mathcal{T}_{\leq 0}^{\mathbf{m}}(t_4) = [\,]$, we have:

$$\begin{aligned}
\mathcal{R}_1^{\mathbf{m}}(t_1) &= [\mathcal{T}_{\leq 0}^{\mathbf{m}}(t_1), \mathcal{T}_{\leq 0}^{\mathbf{m}}(t_2), \mathcal{T}_{\leq 0}^{\mathbf{m}}(t_3), \mathcal{T}_{\leq 0}^{\mathbf{m}}(t_4), \mathcal{T}_{\leq 0}^{\mathbf{m}}(t_4)] \\
&= 5 \otimes [[\,]] \\
\mathcal{R}_1^{\mathbf{m}}(t_2) &= [\mathcal{T}_{\leq 0}^{\mathbf{m}}(t_2), \mathcal{T}_{\leq 0}^{\mathbf{m}}(t_4)] \\
&= 2 \otimes [[\,]] \\
\mathcal{R}_1^{\mathbf{m}}(t_3) &= [\mathcal{T}_{\leq 0}^{\mathbf{m}}(t_3), \mathcal{T}_{\leq 0}^{\mathbf{m}}(t_4)] \\
&= 2 \otimes [[\,]]
\end{aligned}$$

$$\begin{aligned}\mathcal{R}_2^{\mathbf{m}}(t_0) &= [\mathcal{T}_{\leq 1}^{\mathbf{m}}(t_0), \mathcal{T}_{\leq 1}^{\mathbf{m}}(t_1)] \\ &= [[\,], [(1, \mathcal{R}_1^{\mathbf{m}}(t_1)), (1, \mathcal{R}_1^{\mathbf{m}}(t_1))]] \\ &= [[\,], [(1, 5 \otimes [[\,]]), (1, 5 \otimes [[\,]])]]\end{aligned}$$

Therefore

$$\begin{array}{rcccl} & & \mathcal{T}_{\leq 2}^{\mathbf{m}}(t_0) & = & [(2, [[\,], [(1, 5 \otimes [[\,]]), (1, 5 \otimes [[\,]])]])] \\ \mathcal{T}_{\leq 2}^{\mathbf{m}}(t_1) & = & \mathcal{T}_{\leq 1}^{\mathbf{m}}(t_1) & = & [(1, 5 \otimes [[\,]]), (1, 5 \otimes [[\,]])] \\ \mathcal{T}_{\leq 2}^{\mathbf{m}}(t_2) & = & \mathcal{T}_{\leq 1}^{\mathbf{m}}(t_2) & = & [(1, 2 \otimes [[\,]])] \\ \mathcal{T}_{\leq 2}^{\mathbf{m}}(t_3) & = & \mathcal{T}_{\leq 1}^{\mathbf{m}}(t_3) & = & [(1, 2 \otimes [[\,]])] \\ \mathcal{T}_{\leq 2}^{\mathbf{m}}(t_4) & = & \mathcal{T}_{\leq 1}^{\mathbf{m}}(t_4) & = & [\,] \end{array}$$

Finally we can note that

$$\mathcal{T}_{\leq 2}^{\mathbf{m}}(t_0) \succ \mathcal{T}_{\leq 2}^{\mathbf{m}}(t_1) \succ \mathcal{T}_{\leq 2}^{\mathbf{m}}(t_2) \succ \mathcal{T}_{\leq 2}^{\mathbf{m}}(t_4) \qquad \mathcal{T}_{\leq 2}^{\mathbf{m}}(t_1) \succ \mathcal{T}_{\leq 2}^{\mathbf{m}}(t_3) \succ \mathcal{T}_{\leq 2}^{\mathbf{m}}(t_4)$$

The $\mathcal{T}^{\mathbf{m}}$-measure is decreasing Lastly, we show the main theorem of this section, stating that if $M \to_\beta N$ then $\mathcal{T}^{\mathbf{m}}(M) \succ \mathcal{T}^{\mathbf{m}}(N)$. This theorem is based on three technical results, that we call *high/increase*, *low/decrease*, and *forget/decrease*:

1. **High/increase** (Proposition 2.5.14) establishes that $\mathcal{T}_{\leq d}^{\mathbf{m}}(\cdot)$ non-strictly increases if one contracts a redex of higher degree $D > d$. More precisely, if $0 \leq d < D$ and $t \xrightarrow{D}_{\mathbf{m}} t'$, then $\mathcal{T}_{\leq d}^{\mathbf{m}}(t) \preceq \mathcal{T}_{\leq d}^{\mathbf{m}}(t')$. Note that $\mathcal{T}_{\leq d}^{\mathbf{m}}(t)$ only looks at redexes of degree $i \leq d$, and contracting a redex of degree $D > d$ *cannot erase* a redex of any degree $i \leq d$, because the $\lambda^{\mathbf{m}}$-calculus is non-erasing. This property is needed to prove the low/decrease property, and it relies crucially on the commutation property of the previous section Proposition 2.5.2.

2. **Low/decrease** (Proposition 2.5.15) establishes that $\mathcal{T}_{\leq D}^{\mathbf{m}}(\cdot)$ strictly decreases if one contracts a redex of lower degree $d < D$. More precisely, if $1 \leq d \leq D$ and $t \xrightarrow{d}_{\mathbf{m}} t'$ then $\mathcal{T}_{\leq D}^{\mathbf{m}}(t) \succ \mathcal{T}_{\leq D}^{\mathbf{m}}(t')$. This is the core of the argument, and the most technically difficult part to prove. It relies crucially on the lifting property of the previous section (Proposition 2.5.4).

3. **Forget/decrease** (Proposition 2.5.16) establishes that forgetful reduction (non-strictly) decreases the measure. More precisely, if $t \rhd t'$ then $\mathcal{T}_{\leq d}^{\mathbf{m}}(t) \succeq \mathcal{T}_{\leq d}^{\mathbf{m}}(t')$. This property is used as a final step in the main theorem, and it relies crucially on postponement of forgetful reduction, as studied in the previous section (Proposition 2.5.5). It is easy to grasp intuitively, since forgetting a wrapper may only discard the developments involving its redexes, so the measure cannot grow. Note that it does not necessarily shrink either, since the forgotten wrapper may contain no redexes at all.

Below we sketch the proofs of these three properties. See Proposition A.2.18, Proposition A.2.21, and Proposition A.2.22 in the appendix for detailed proofs. Let us first mention a straightforward lemma.

Lemma 2.5.13 (Measure of a substitution)**.**

1. $\mathcal{T}_d^{\mathbf{m}}(t_0, t) \preceq \mathcal{T}_d^{\mathbf{m}}(t_0, t[s/x])$.
2. If s is not an $\mathbf{m}$-abstraction of degree d, then $\mathcal{T}_d^{\mathbf{m}}(t_0, t[s/x]) = \mathcal{T}_d^{\mathbf{m}}(t_0, t) + k \otimes \mathcal{T}_d^{\mathbf{m}}(t_0, s)$ for some $k \in \mathbb{N}_0$.

Proof. By induction on t. See Lemma A.2.17 and Lemma A.2.20 in the appendix for details. □

Proposition 2.5.14 (High/increase)**.** Let $D \in \mathbb{N}_0$. Then the following hold:

1. If $1 \leq d < D$ and $t \xrightarrow{D}_{\mathbf{m}} t'$ then $\mathcal{R}^{\mathbf{m}}_d(t) \preceq \mathcal{R}^{\mathbf{m}}_d(t')$.
2. If $0 \leq d < D$ and $t_0 \xrightarrow{D}_{\mathbf{m}} t'_0$ then $\mathcal{T}^{\mathbf{m}}_d(t_0, t) \preceq \mathcal{T}^{\mathbf{m}}_d(t'_0, t)$.
3. If $0 \leq d < D$ and $t_0 \xrightarrow{D}_{\mathbf{m}} t'_0$ and $t \xrightarrow{D}_{\mathbf{m}} t'$ then $\mathcal{T}^{\mathbf{m}}_d(t_0, t) \preceq \mathcal{T}^{\mathbf{m}}_d(t'_0, t')$.
4. If $0 \leq d < D$ and $t \xrightarrow{D}_{\mathbf{m}} t'$ then $\mathcal{T}^{\mathbf{m}}_{\leq d}(t) \preceq \mathcal{T}^{\mathbf{m}}_{\leq d}(t')$.

Proof. The four items are proved simultaneously by induction on d, where item 1 resorts to the IH, and the following items may resort to the previous items without decreasing d. Items 2 and 3 proceed by a nested induction on t. Most cases are straightforward.

One interesting situation occurs in item 3 when $t = (\lambda x.s)\mathtt{L}\, r$ is the redex of degree D contracted by the step $t \xrightarrow{D}_{\mathbf{m}} t'$. Then we resort to the first part of Lemma 2.5.13.

Another interesting part of the proof is item 1. Let $1 \leq d < D$ and $t \xrightarrow{D}_{\mathbf{m}} t'$ and let us show that $\mathcal{R}^{\mathbf{m}}_d(t) \preceq \mathcal{R}^{\mathbf{m}}_d(t')$. Indeed, let $X := \{\rho \mid (\exists s)\ \rho : t \xrightarrow{d}{}^*_{\mathbf{m}}\ s\}$ and $Y := \{\sigma \mid (\exists s')\ \sigma : t' \xrightarrow{d}{}^*_{\mathbf{m}}\ s'\}$, and let $R : t \xrightarrow{D}_{\mathbf{m}} t'$. Using Proposition 2.5.2, we can define an injective function $\varphi : X \to Y$ by $\varphi(\rho) := \rho/R$. Note that $\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathtt{tgt}}) \preceq \mathcal{T}^{\mathbf{m}}_{\leq d-1}(\varphi(\rho)^{\mathtt{tgt}})$ holds for every $\rho \in X$ using item 4 of the IH (noting that $1 \leq d - 1 < D$ holds because $1 \leq d < D$), resorting to the IH as many times as the length of the reduction $s \xrightarrow{D}{}^*_{\mathbf{m}}\ s'_\rho$. To conclude the proof, let $Z = Y \setminus \varphi(X)$. Then:

$$\begin{aligned}\mathcal{R}^{\mathbf{m}}_d(t) &= [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in X] \preceq^{(\star)} [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\varphi(\rho)^{\mathtt{tgt}}) \mid\mid \rho \in X] =^{(\star\star)} [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\sigma^{\mathtt{tgt}}) \mid\mid \sigma \in \varphi(X)] \\ &\preceq [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\sigma^{\mathtt{tgt}}) \mid\mid \sigma \in \varphi(X)] + [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\sigma^{\mathtt{tgt}}) \mid\mid \sigma \in Z] = [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\sigma^{\mathtt{tgt}}) \mid\mid \sigma \in Y] = \mathcal{R}^{\mathbf{m}}_d(t')\end{aligned}$$

To justify the step marked with $(\star)$, note that $[\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in X] = \sum_{\rho \in X}[\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathtt{tgt}})] \preceq \sum_{\rho \in X}[\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\varphi(\rho)^{\mathtt{tgt}})] = [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\varphi(\rho)^{\mathtt{tgt}}) \mid\mid \rho \in X]$ because $\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathtt{tgt}}) \preceq \mathcal{T}^{\mathbf{m}}_{\leq d-1}(\varphi(\rho)^{\mathtt{tgt}})$, as we have already claimed. To justify the step marked with $(\star\star)$, note that φ is injective. □

Proposition 2.5.15 (Low/decrease)**.** Let $D \in \mathbb{N}_0$. Then the following hold:

1. The measures for the j–developments are smaller or lesser after a $\leq d$–step:
   If $1 \leq d \leq j \leq D$ and $t \xrightarrow{d}_{\mathbf{m}} t'$ then $\mathcal{R}^{\mathbf{m}}_j(t) \succ \mathcal{R}^{\mathbf{m}}_j(t')$.
2. If $1 \leq d \leq j \leq D$ and $t_0 \xrightarrow{d}_{\mathbf{m}} t'_0$ then $\mathcal{T}^{\mathbf{m}}_j(t_0, t)$ **:≻:** $\mathcal{T}^{\mathbf{m}}_j(t'_0, t)$.
3. If $1 \leq d \leq D$ and $t_0 \xrightarrow{d}_{\mathbf{m}} t'_0$ and $t \xrightarrow{d}_{\mathbf{m}} t'$, then for all $\mathfrak{m} \in \mathbb{T}_{d-1}$ we have $\mathcal{T}^{\mathbf{m}}_d(t_0, t) \succ \mathcal{T}^{\mathbf{m}}_d(t'_0, t') + \mathfrak{m}$.
4. If $1 \leq d < j \leq D$ and $t_0 \xrightarrow{d}_{\mathbf{m}} t'_0$ and $t \xrightarrow{d}_{\mathbf{m}} t'$ then $\mathcal{T}^{\mathbf{m}}_j(t_0, t) \succeq \mathcal{T}^{\mathbf{m}}_j(t'_0, t')$.
5. If $1 \leq d \leq D$ and $t \xrightarrow{d}_{\mathbf{m}} t'$ then $\mathcal{T}^{\mathbf{m}}_{\leq D}(t) \succ \mathcal{T}^{\mathbf{m}}_{\leq D}(t')$.

Proof. The five items are proved simultaneously by induction on D, where item 1 resorts to the IH, and the following items may resort to the previous items without decreasing d. Items 2–4 proceed by a nested induction on t. We mention some of the interesting parts of the proof.

For item 1, let $1 \leq d \leq j \leq D$ and $t \xrightarrow{d}_{\mathbf{m}} t'$ and let us show that $\mathcal{R}^{\mathbf{m}}_j(t) \succ \mathcal{R}^{\mathbf{m}}_j(t')$. Let $X := \{\rho \mid (\exists s)\ \rho : t \xrightarrow{j}{}^*_{\mathbf{m}}\ s\}$ and $Y := \{\sigma \mid (\exists s')\ \sigma : t' \xrightarrow{j}{}^*_{\mathbf{m}}\ s'\}$, and consider two subcases:

- If $d = j$, let $R : t \xrightarrow{d}_{\mathbf{m}} t'$, define an injective function $\varphi : Y \to X$ by $\varphi(\sigma) = R\sigma$, let $Z = X \setminus \varphi(Y)$, and note that:

$$\begin{aligned}\mathcal{R}^{\mathbf{m}}_j(t) &= [\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in \varphi(Y)] + [\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in Z] \\ &= [\mathcal{T}^{\mathbf{m}}_{\leq j-1}(R\sigma^{\mathtt{tgt}}) \mid\mid \sigma \in Y] + [\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in Z] \text{ since } \varphi \text{ is injective} \\ &= [\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\sigma^{\mathtt{tgt}}) \mid\mid \sigma \in Y] + [\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in Z] = \mathcal{R}^{\mathbf{m}}_j(t') + [\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in Z]\end{aligned}$$

  To conclude that $\mathcal{R}^{\mathbf{m}}_j(t) \succ \mathcal{R}^{\mathbf{m}}_j(t')$, note that Z is non-empty because it contains the empty reduction $\epsilon : t \xrightarrow{d}{}^{*}_{\mathbf{m}} t$.

- If $d < j$, we construct a function $\varphi : Y \to X$ as follows. By Proposition 2.5.4, for each reduction $\sigma : t' \xrightarrow{j}{}^{*}_{\mathbf{m}} s'$ there exist s_σ, r_σ, and reductions $\varphi(\sigma) : t \xrightarrow{j}{}^{*}_{\mathbf{m}} s_\sigma$ and $s' \xrightarrow{j}{}^{*}_{\mathbf{m}} r_\sigma$ and $s_\sigma \xrightarrow{d}{}^{+}_{\mathbf{m}} r_\sigma$. For every $\sigma \in Y$, we have $\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\varphi(\sigma)^{\mathtt{tgt}}) = \mathcal{T}^{\mathbf{m}}_{\leq j-1}(s_\sigma) \succ^{\dagger} \mathcal{T}^{\mathbf{m}}_{\leq j-1}(r_\sigma) \succeq^{\ddagger} \mathcal{T}^{\mathbf{m}}_{\leq j-1}(s') = \mathcal{T}^{\mathbf{m}}_{\leq j-1}(\sigma^{\mathtt{tgt}})$ where † holds by item 5 of the IH, observing that $1 \leq d \leq j-1 < D$ because $d < j \leq D$, and ‡ holds by high/increase (Proposition 2.5.14), observing that $0 \leq j-1 < j$. Hence, every element $\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\sigma^{\mathtt{tgt}})$ in $\mathcal{R}^{\mathbf{m}}_j(t')$ is strictly smaller than some element $\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\varphi(\sigma)^{\mathtt{tgt}})$ in $\mathcal{R}^{\mathbf{m}}_j(t)$. Then:

$$\begin{aligned}\mathcal{R}^{\mathbf{m}}_j(t) &= [\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\rho^{\mathtt{tgt}}_1)] + ... + [\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\rho^{\mathtt{tgt}}_n)] \\ &\succ^1 \mathfrak{n}_1 + ... + [\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\rho^{\mathtt{tgt}}_n)] \\ &\succ^1 ... \\ &\succ^1 \mathfrak{n}_1 + ... + \mathfrak{n}_n \\ &= \mathcal{R}^{\mathbf{m}}_j(t')\end{aligned}$$

  where $\mathfrak{n}_i$ is the multiset of elements $\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\sigma^{\mathtt{tgt}})$ from $\mathcal{R}^{\mathbf{m}}_j(t')$ such that $\varphi(\sigma) = \rho_i$, which entails $\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\rho^{\mathtt{tgt}}_i) \succ \mathcal{T}^{\mathbf{m}}_{\leq j-1}(\sigma^{\mathtt{tgt}})$. We conclude the proof by noting that the sequence cannot be empty: Y contains the empty reduction $\epsilon : t' \xrightarrow{j}{}^{*}_{\mathbf{m}} t'$, therefore $\mathcal{R}^{\mathbf{m}}_j(t')$ is non-empty, and so is $\mathcal{R}^{\mathbf{m}}_j(t)$.

Another interesting situation occurs in item 3, when $t = (\lambda x.s)\mathtt{L}\, r$ is the redex of degree d contracted by the step $t \xrightarrow{d}_{\mathbf{m}} t'$. The step is of the form $t = (\lambda x.s)\mathtt{L}r \xrightarrow{d}_{\mathbf{m}} s[r/x]\langle r\rangle\mathtt{L} = t'$. Note that r is not an abstraction of degree d, because it is the argument of an abstraction of degree d. So by Lemma 2.5.13 there exists $k \in \mathbb{N}_0$ such that $\mathcal{T}^{\mathbf{m}}_d(t'_0, s[r/x]) = \mathcal{T}^{\mathbf{m}}_d(t'_0, s) + k \otimes \mathcal{T}^{\mathbf{m}}_d(t'_0, r)$. The crucial observation is that $\mathcal{T}^{\mathbf{m}}_d(t_0, r) \succeq (1+k) \otimes \mathcal{T}^{\mathbf{m}}_d(t'_0, r)$, which is because by item 2 we have that $\mathcal{T}^{\mathbf{m}}_d(t_0, r) \mathbin{:\!\succ\!:} \mathcal{T}^{\mathbf{m}}_d(t'_0, r)$.

Finally, for item 5, let $1 \leq d \leq D$ and $t \xrightarrow{d}_{\mathbf{m}} t'$ and let us show that $\mathcal{T}^{\mathbf{m}}_{\leq D}(t) \succ \mathcal{T}^{\mathbf{m}}_{\leq D}(t')$. Indeed:

$$\begin{aligned}\mathcal{T}^{\mathbf{m}}_{\leq D}(t) &= \textstyle\sum_{i=1}^{D} \mathcal{T}^{\mathbf{m}}_i(t,t) \succeq \mathcal{T}^{\mathbf{m}}_d(t,t) + \sum_{j=d+1}^{D} \mathcal{T}^{\mathbf{m}}_j(t,t) \\ &\succ \mathcal{T}^{\mathbf{m}}_{\leq d-1}(t') + \mathcal{T}^{\mathbf{m}}_d(t',t') + \textstyle\sum_{j=d+1}^{D} \mathcal{T}^{\mathbf{m}}_j(t,t) \text{ by item 3, taking } \mathfrak{m} := \mathcal{T}^{\mathbf{m}}_{\leq d-1}(t') \\ &\succeq \mathcal{T}^{\mathbf{m}}_{\leq d-1}(t') + \mathcal{T}^{\mathbf{m}}_d(t',t') + \textstyle\sum_{j=d+1}^{D} \mathcal{T}^{\mathbf{m}}_j(t',t') = \mathcal{T}^{\mathbf{m}}_{\leq D}(t') \text{ by item 4.}\end{aligned}$$

□

Proposition 2.5.16 (Forget/decrease)**.** Let $d \in \mathbb{N}_0$. Then the following hold:

1. If $t \rhd t'$ then $\mathcal{R}^{\mathbf{m}}_d(t) \succeq \mathcal{R}^{\mathbf{m}}_d(t')$.

2. If $t_0 \rhd t_0'$ then $\mathcal{T}_d^{\mathbf{m}}(t_0, t) \succeq \mathcal{T}_d^{\mathbf{m}}(t_0', t)$.
3. If $t_0 \rhd t_0'$ and $t \rhd t'$ then $\mathcal{T}_d^{\mathbf{m}}(t_0, t) \succeq \mathcal{T}_d^{\mathbf{m}}(t_0', t')$.
4. If $t \rhd t'$ then $\mathcal{T}_{\leq d}^{\mathbf{m}}(t) \succeq \mathcal{T}_{\leq d}^{\mathbf{m}}(t')$.

Proof. The four items are proved simultaneously by induction on D, where item 1 resorts to the IH, and the following items may resort to the previous items without decreasing d. Items 2 and 3 proceed by a nested induction on t.

The interesting part is item 1, so let $t \rhd t'$ and let us show that $\mathcal{R}_d^{\mathbf{m}}(t) \succeq \mathcal{R}_d^{\mathbf{m}}(t')$. Let $X := \{\rho \mid (\exists s)\ \rho : t \xrightarrow{d}{}^*_{\mathbf{m}}\ s\}$ and $Y := \{\sigma \mid (\exists s')\ \sigma : t' \xrightarrow{d}{}^*_{\mathbf{m}}\ s'\}$. Define an injective function $\varphi : Y \to X$ by $\varphi(\sigma) := \sigma^{\frown} R$, resorting to Proposition 2.5.5, where $\sigma^{\frown} R : t \xrightarrow{d}{}^*_{\mathbf{m}} s_\sigma.$ and $s_\sigma \rhd^* s'$. Note that for every $\sigma \in Y$ we have $\mathcal{T}_{\leq d-1}^{\mathbf{m}}(\varphi(\sigma)^{\mathsf{tgt}}) = \mathcal{T}_{\leq d-1}^{\mathbf{m}}(s_\sigma) \succeq^{\dagger} \mathcal{T}_{\leq d-1}^{\mathbf{m}}(s') = \mathcal{T}_{\leq d-1}^{\mathbf{m}}(\sigma^{\mathsf{tgt}})$, where $\dagger$ holds by item 4 of the IH, observing that $d - 1 < d$. To conclude the proof, let $Z = X \setminus \varphi(Y)$, and note that:

$$\mathcal{R}_d^{\mathbf{m}}(t) = [\mathcal{T}_{\leq d-1}^{\mathbf{m}}(\rho^{\mathsf{tgt}}) \mid\mid \rho \in \varphi(Y)] + [\mathcal{T}_{\leq d-1}^{\mathbf{m}}(\rho^{\mathsf{tgt}}) \mid\mid \rho \in Z] \succeq [\mathcal{T}_{\leq d-1}^{\mathbf{m}}(\rho^{\mathsf{tgt}}) \mid\mid \rho \in \varphi(Y)]$$

$$=^{(\star)} [\mathcal{T}_{\leq d-1}^{\mathbf{m}}(\varphi(\sigma)^{\mathsf{tgt}}) \mid\mid \sigma \in Y] \succeq^{(\star\star)} [\mathcal{T}_{\leq d-1}^{\mathbf{m}}(\sigma^{\mathsf{tgt}}) \mid\mid \sigma \in Y] = \mathcal{R}_d^{\mathbf{m}}(t')$$

For the step marked with $(\star)$, note that φ is injective. For the step marked with $(\star\star)$, note that $[\mathcal{T}_{\leq d-1}^{\mathbf{m}}(\varphi(\sigma)^{\mathsf{tgt}}) \mid\mid \sigma \in Y] = \sum_{\sigma \in Y} [\mathcal{T}_{\leq d-1}^{\mathbf{m}}(\varphi(\sigma)^{\mathsf{tgt}})] \succeq \sum_{\sigma \in Y} [\mathcal{T}_{\leq d-1}^{\mathbf{m}}(\sigma^{\mathsf{tgt}})] = [\mathcal{T}_{\leq d-1}^{\mathbf{m}}(\sigma^{\mathsf{tgt}}) \mid\mid \sigma \in Y]$ because $\mathcal{T}_{\leq d-1}^{\mathbf{m}}(\varphi(\sigma)^{\mathsf{tgt}}) \succeq \mathcal{T}_{\leq d-1}^{\mathbf{m}}(\sigma^{\mathsf{tgt}})$, as we have already justified. □

Finally, we prove the main theorem in this section:

Theorem 2.5.17. Let M, N be typable λ-terms such that $M \to_\beta N$. Then $\mathcal{T}^{\mathbf{m}}(M) > \mathcal{T}^{\mathbf{m}}(N)$.

Proof. Let $D = \delta_{\mathsf{max}}(M)$ and $D' = \delta_{\mathsf{max}}(N)$. Let $M \to_{\mathbf{m}} s$ be the step corresponding to $M \to_\beta N$. By Lemma 2.4.16 note that $s \rhd N$. Then:

$$\mathcal{T}^{\mathbf{m}}(M) = \mathcal{T}_{\leq D}^{\mathbf{m}}(M) \succ^{\text{Proposition 2.5.15}} \mathcal{T}_{\leq D}^{\mathbf{m}}(s) \succeq^{\text{Proposition 2.5.16}} \mathcal{T}_{\leq D}^{\mathbf{m}}(N) \succeq \mathcal{T}_{\leq D'}^{\mathbf{m}}(N) = \mathcal{T}^{\mathbf{m}}(N)$$

The last inequality holds because $D \geq D'$ since, as is well-known, contraction of a β-redex in the simply typed λ-calculus cannot create a redex of higher degree. □

2.6 Conclusions

We defined two decreasing measures for the simply typed λ–calculus, the $\mathcal{W}$-measure (Definition 2.4.17) and the $\mathcal{T}^{\mathbf{m}}$-measure (Definition 2.5.9). These measures are decreasing (Theorem 2.4.20 and Theorem 2.5.17 respectively) and, to the best of our knowledge, they provide two new proofs of strong normalization for the STLC. Both measures are defined constructively and by purely syntactic methods, using the $\lambda^{\mathbf{m}}$-calculus as an auxiliary tool. We combined Nederpelt [76] and Klop's [65] idea of of avoiding erasure through syntactic constructions on terms, with Turing's [47] observation about redex degrees, to produce decreasing measures that follows Gandy's [48] approach of directly proving SN; *i.e.*, decreasing for any redex contraction. Other approaches [30, 60, 62, 64] have relied on non-erasure and redex degrees to produce SN proofs, but treating them as separate components to reduce SN to WN, and to prove WN separately; we instead composed both ideas into single constructions (each decreasing measure), providing direct proofs of SN.

The main contribution of the measure $\mathcal{W}$ is the concretization of previously known ideas into a decreasing measure. We built upon Klop–Nederpelt's lemma [99, Theorem 1.2.3 (iii)] (on strong

normalization being deducible from increasingness, weak confluence, and weak normalization) by designing a variant of non-erasing calculi, in which, through a recursive operation based on redex-degrees, we compute a number and obtain a strong decreasing measure that we then transfer to the STLC. While, as for all non-erasing calculi, the conservation theorem reducing SN to WN holds for $\lambda^{\mathbf{m}}$, we do not rely on it to conduct our proof; we instead provide a direct proof of SN for STLC through the measure. One way of understanding this technique is as a lifting of redex-degree based measures from proving WN to proving SN. Another, albeit tentative, is as an internalization into the decreasing measure of the conservation theorem, which is the key property to reduce SN to WN in non-erasing calculi. A formal analysis on the specific relation between the conservation theorem and our decreasing measures would be an interesting work for the future.

The measure $\mathcal{T}^{\mathbf{m}}$ extends a result by Alan Turing, strenghtening it from a weak decreasing measure depending on a strategy to decrease, to a strong one working for arbitrary redex contractions. The proof is based on a delicate analysis of how contracting a redex of degree d may create and copy redexes of degree d', depending on whether $d < d'$, or $d = d'$, or $d > d'$. We relied on complete developments by degree to obtain the set of all the possible single degree reduction paths, computing the measure over the last terms of these paths. Non-erasure was again key to prove that the computed multisets were indeed decreasing.

An ideal decreasing measure should fulfill multiple (partly subjective) requirements: (a) the measure should be easy to calculate, in terms of computational complexity; (b) its codomain (a well-founded ordering) should be simple, in terms of its ordinal type; (c) it should give us insight on why β-reduction terminates; (d) it should be easy to prove that the measure is decreasing. A measure that excels simultaneously at all these requirements is elusive, and perhaps unattainable. The proposed measures have different strengths and weaknesses.

One strength of the $\mathcal{W}$-measure is that its codomain is simple: each term is mapped to a natural number. One weakness is that the definition of the $\mathcal{W}$-measure relies on reduction in the $\lambda^{\mathbf{m}}$-calculus, and computing the $\mathcal{W}$-measure is at least as costly as evaluating the λ-term itself. Measures based on Gandy's [32, 48] have similar characteristics. One question is whether the values of the $\mathcal{W}$-measure and measures based on Gandy's can be related. It is not immediate to establish a precise correspondence. Apart from the specifics of the computed values and the general analysis and constructions being different, we identify a proof-theoretical domain distinction between them. For the measure computation and shrinking proofs, Gandy's and de Vrijer's relied on properties of the semantic domain of hereditarily strictly monotonic functions (or increasing functionals). In contrast, we relied on definitions and properties native to the λ–calculus, such as the simplification operation (a structural recursive function based on redex-degrees that computes a complete development of the term) and the properties of confluence and commutation of reductions. These are applied to a variant of the known non-erasing family of extensions of the λ–calculus, computing the measure and proving its properties while staying semantically close to the calculus. The intent to remain as close as possible to the original calculus can be found in the literature; for instance, Sørensen [91] introduced an extension of Klop's non-erasing approach where the notion of reduction in the target calculus coincides with that of the non-erasing calculus. Keeping a proximity to the λ–calculus might contribute to the simplicity in the measure's understanding.

The $\mathcal{T}^{\mathbf{m}}$-measure, on the other hand, seems to fail under all of our own posed criteria. The codomain of the $\mathcal{T}^{\mathbf{m}}$-measure is not so simple, as the $\mathcal{T}^{\mathbf{m}}$-measure maps each term to a structure of nested multisets. Yet, it is "reasonably simple": the fact that the partial orders $\mathbb{T}_d$ and $\mathbb{R}_d$ are well-founded only relies on the ordinary multiset and lexicographic orderings. The $\mathcal{T}^{\mathbf{m}}$-measure is costly to compute; in particular $\mathcal{R}^{\mathbf{m}}_d(t)$ is defined as a sum over all reductions $\rho : t \xrightarrow{d}{}^{*}_{\mathbf{m}} t'$, which may produce a combinatorial explosion. Another weakness is that our proofs make use of relatively

heavy rewriting machinery, as we have to keep explicit track of witnesses (*e.g.* in Section 2.5.1). In other words: it is extremely expensive to compute, the proof is quite convoluted, and it is very hard to grasp due to its complex codomain. However, we claim that it has two quite positive aspects. The first one is historical (and probably subjective, but I will allow myself the indulgence). Discovering that Alan Turing's assignment on λ-terms can be expanded into a more complex structure, entailing a stronger property, seems worthwhile in its own right, as well as constituting a sort of tribute. The second one is that it puts together a great deal of related information that we can analyze. If one argues against the measure, one could argue that it is too complex to be interesting. My response to my own criticism is that it is because it gathers as much information as it can. From there, we can try to discard information, in an attempt to simplify it. If successful, a simplification of the measure could bring us closer to an actually easy measure, just as Gödel intended in the koan.

2.7 Resumen en español (Summary in Spanish)

En este capítulo presento las medidas decrecientes $\mathcal{W}$ y $\mathcal{T}^{\mathbf{m}}$, que introducimos con Pablo Barenbaum en el trabajo publicado en la conferencia FSCD'23. Se computa sobre el cálculo con memoria $\lambda^{\mathbf{m}}$, que *recuerda* cada argumento consumido en una memoria adjunta a cada término. Los desarrollos se basan en tres ideas centrales. Una es la de los cálculos sin borrado, introducidos originalmente por Alonzo Church en la propia introducción del λ–cálculo, y retomados por la comunidad de reescritura en la década de 1970. Otra la observación de Turing y Prawitz de que la contracción de un redex no puede crear redexes de mayor o igual grado, solo copiarlos. La última son las medidas decrecientes totales, introducidas para λ–calculus por Gandy y refinadas por de Vrijer.

Para calcular $\mathcal{W}$, definimos una función recursiva estructural que reconstruye parcialmente el árbol de ejecución de los programas $\lambda^{\mathbf{m}}$, obteniendo el resultado final en el metalenguaje, sin depender de la ejecución en el lenguaje. Sobre ese resultado contamos la cantidad de memorias. Para demostrar que cada paso de computación en simply typed λ–calculus decrece la medida, observamos que el propio paso que se está observando aporta una medida en el programa que aporta a su reducto.

En el caso de $\mathcal{T}^{\mathbf{m}}$, el principal objetivo es adaptar la medida parcial dada por Turing y publicada por Gandy, que funciona al aplicar la estrategia *rightmost innermost*, para que sea total, *i.e.*, que funcione al contraer cualquier redex. La medida de Turing da un multiconjunto de números que corresponden al grado de ocurrencias de redexes en el programa. Ese multiconjunto decrece al contraer el redex *rightmost innermost*, porque su ubicación asegura que no puede copiar redexes de su mismo grado, y la observación de Turing impide que se creen redexes nuevos de mayor o igual grado. Al buscar que decrezca para cualquier redex, tenemos que contemplar más información. Al habilitar la contracción de cualquier redex, es fácil encontrar ejemplos donde la contracción de un redex intermedio replica redexes mayores, haciendo que el multiconjunto crezca en vez de decrecer.

Buscando extender la medida para que decrezca ante la contracción de cualquier redex, exploramos qué otra información podemos incorporar al multiconjunto para transformar la medida en total. El resultado es una estructura de multiconjuntos anidados construidos a partir de los grados de redexes y de los *developments* de cada grado.

Chapter 3

A decreasing measure for idempotent intersection types

3.1 Introduction

Intersection types were introduced by Coppo and Dezani [22], with the explicit aim of increasing the typability power of simple types. Beyond the arrow connective of simple types ($\to$), these systems incorporate an intersection connective ($\wedge$), which is commutative, associative and idempotent (*i.e.*, $A \wedge A \equiv A$), which means that intersection is essentially a notation for *sets* of types.

From a *pragmatic* perspective, they introduce a finitary form of polymorphism. For instance, in simply typed λ–calculus we cannot give type to auto-application, *i.e.*, we cannot derive the following judgment:

$$x : ? \vdash xx : A$$

This happens because we are not able to pick *one* type for x. Given that it serves both as a function and as its own argument, we need its type to be both an arrow $B \to C$ and the argument B to that same arrow type. Therefore B should be equal to $B \to C$, an unsolvable equation. By admitting x to be assigned a set with the two types we require it to have, $\Lambda_\cap$ allows us to give auto-application a type:

$$x : \{A \to A, A\} \vdash xx : A$$

Intersection type assignment systems come in different versions, and are able to *characterize* semantic properties of terms. For example, Coppo and Dezani's original system characterizes strong normalization, in the sense that a term is typable if and only if it is strongly normalizing.

There are many other intersection type systems characterizing other properties besides strong normalization. One example is the systems characterizing head normalization [23,24,67], where the strategy contracting the head redex always terminates. Furthermore, intersection types are a very powerful tool to reason about the interpretation of terms in various models of λ–calculus [10,11,88].

The existing proofs of the characterization properties are not straightforward. For both strong and head normalization, the "typable implies normalizing" part is usually carried out using reducibility [67,85]. As stated in the introduction, reducibility proofs are standard, but it is not easy to extract intuition from them, so having explicit measures that decrease with β-reduction would be valuable. In the case of intersection systems characterizing strong normalization, there are three syntactical proofs based on a decreasing measure [17,19,62]; we will briefly discuss them and their relation with the present work at the end of this section.

A non-idempotent version of intersection types has been developed and largely studied in the last years [28,49,61], where the intersection connective is *not* idempotent (*i.e.*, $A \wedge A \not\equiv A$), and thus

becomes a notation for *multisets* of types. Non-idempotent systems preserve some of the properties of idempotent ones; for instance, they can characterize strong and head normalization [13]. From a semantical point of view, they can describe relational models of λ-calculus [79]; while this class of models is not very sophisticated with respect to the λ-theories that they induce, they are useful to explore quantitative properties of typable terms. Unlike the case of the idempotent version, the proof of typability implying strong and head termination in non-idempotent intersection systems can be carried out by an induction on the size of the typing derivation, which decreases at every reduction step. This relies on the quantitative nature of non-idempotency, and in particular on the fact that, in a derivation, the cardinality of the multiset of types of a variable x is an upper bound of the number of occurrences of x in the typed term. Since these proofs rely crucially on the fact that intersection is non-idempotent, there is not much hope of extending them to the idempotent case.

We thought that it would be interesting to explore if there is a decreasing measure for idempotent intersection types which could constitute an improvement over the existing ones. In this chapter I present a positive answer to this problem introducing the measure $\mathcal{W}_\cap$, an adaptation of the measure $\mathcal{W}$ presented in Section 2.4 to idempotent intersection types. It is the result of a joint work with Pablo Barenbaum and Simona Ronchi Della Rocca, which was published at MFPS'25 [7]. The measure consists of the definition of a Church-style variant of the system, that we prove corresponds to the original Curry-style, over which we successfully apply the same process as for simply typed λ–calculus: wrappers extension, simplification definition, measure definition, and decrease proof. The obtained measure is the simplest one in terms of codomain, and, to the best of our knowledge, it is the first strong one (*i.e.*, that decreases under the contraction of *any* redex) defined directly for IIT (*i.e.*, based on its definitions and not on a translation to simply typed systems).

In order to adapt the technique presented in Section 2.4 to the setting of idempotent intersection types, the first step is to design an *intrinsically typed* version of intersection types, which are usually presented as extrinsically typed systems. Let us recall that in an *extrinsically typed* (Curry-style) system terms are untyped, and there is a set of typing rules assigning types to them, whereas in an *intrinsically typed* (Church-style) system terms are decorated with types, and typing rules are just a check of syntactical correctness. There are some attempts in the literature in this direction [5,16,69], but they are not suitable for the application of the technique behind $\mathcal{W}$. In fact, the main difference between simple and intersection types, on which the power of the latter relies, is that if M is a term of the λ–calculus typable by intersection types and N is a subterm of M occurring once in it, then a typing derivation of a judgement $\Gamma \vdash M : A$ can contain $n \geq 1$ subderivations with object N. Since N can contain redexes, performing a reduction inside N corresponds to performing n reductions *in parallel* on the derivation. The natural way to achieve this is to equip the Church-style intersection type system with a parallel notion of reduction, which is indeed what all the existing proposals do. In order to apply the desired method, we would need to reduce these subderivations independently of each other.

We start from an extrinsically typed subset of Λ, called $\Lambda_\cap^{\mathsf{Cu}}$, defined through an intersection type assignment system. This system is a variant of well-known systems in the literature that characterize strong normalization. In order to build an intrinsically typed version of $\Lambda_\cap^{\mathsf{Cu}}$, we exploit the idea that idempotent intersection represents a set of types, and we explicitly use sets in the grammar of both types and terms of the typed language, so, for example, if t and s are terms of type A and B respectively, then $\{t, s\}$ is a set of terms which can be assigned the set of types $\{A, B\}$. Following this idea, we formulate an intrinsically typed version of $\Lambda_\cap^{\mathsf{Cu}}$ called $\Lambda_\cap^{\mathsf{Ch}}$, where sets of types are decorations for sets of terms. While these two systems can be dealt with independently of each other, they are shown to correspond to and simulate each other. The new typed language $\Lambda_\cap^{\mathsf{Ch}}$ enjoys all the good properties we expect, such as subject reduction and

confluence, and it is suitable for an adaptation of the method from $\mathcal{W}$. Using the notion of *full simplification* and relying on an auxiliary non-erasing intersection typed language $\Lambda^{\mathbf{m}}_\cap$, we define a measure yielding a natural number that decreases at *every* reduction step in $\Lambda^{\mathsf{Ch}}_\cap$. Then, we prove that, for every term M in $\Lambda^{\mathsf{Cu}}_\cap$, there is a term t in $\Lambda^{\mathsf{Ch}}_\cap$ such that an infinite reduction sequence in M entails an infinite reduction sequence in t, thus obtaining a new proof of strong normalization for $\Lambda^{\mathsf{Cu}}_\cap$. Finally, we complete the picture by proving that $\Lambda^{\mathsf{Ch}}_\cap$ gives type to all the strongly normalizing terms, hence showing that it has the same typability power of the original version of idempotent intersection types [22].

3.1.1 Comparison with related works

There are three other syntactical proofs of strong normalization for idempotent intersection types. Kfoury and Wells [62] and Boudol [17] used non-erasing calculi to reduce the problem of SN to WN, and then defined weak decreasing measures, following the approach discussed in Chapter 1. Bucciarelli, Piperno, and Salvo [19], on the other hand, do not actively introduce a decreasing measure. They define a translation of idempotent intersection types to simply typed λ–calculus, so a strong decreasing measure could be defined by using the translation and relying on one of the existing ones for STLC. We analyze the approaches separately.

Weak decreasing measures through non-erasure

The works of Kfoury and Wells and Boudol differ from ours in the strong normalization proof approach and in the chosen intersection types flavour. They are based on the strategy mentioned on Chapter 1 of reducing the problem of SN to that of WN through non-erasure, and then proving WN through a weak decreasing measure based on redex degrees. Both of them define an intermediate intersection typed language equipped with a suitable non-erasing reduction rule, following the approach of Nederpelt [76] and Klop [65]. They prove weak normalization following Turing's approach of defining a measure that decreases when the rightmost innermost of greatest degree strategy is followed, and obtain strong normalization as a corollary.

Kfoury and Wells use, as intermediate language, an intrinsically typed λ-calculus extended with lists, the reduction rule is the composition of a parallel β-rule and a commutation rule, and the measure is a multiset of natural numbers decreasing under the innermost reduction.

Boudol's intermediate language is a variant of Klop's calculus, extrinsically intersection typed, and the reduction is the β-reduction, but copying its argument when it is going to be erased; the weak normalization proof is obtained through the weak cut-elimination property of the type derivation, and the measure is a pair of natural numbers, decreasing under the strategy choosing the innermost redex of maximum degree.

Our work presents the following novelties with respect to them.

- **The measure is *strong*** The most relevant difference is that our technique provides a strong decreasing measure, *i.e.*, one that decreases for *every* reduction strategy. As stated previously, there are proofs with these characteristics for the λ-calculus, such as Gandy's proof and its derivatives [32, 48], but there are no such proofs for intersection types, as far as we know. A possibility for a strong measure in intersection types is discussed in the comparison with Bucciarelli's *et al.* translation, in the next section.

- **Simplest codomain** The codomain of our measure consists of a natural number, being simpler than Kfoury and Wells' multiset of natural numbers, and Boudol's pair of natural

numbers. This particular feature sets up a basis for considering useful upper bounds for longest reduction chains, as will be discussed later.

- **Proven corresponding intrinsic version** We share with Kfoury and Wells the use of an intermediate intersection language which is intrinsically typed. However, while their language is *ad hoc* and its properties are not explored, we show that $\Lambda_\cap^{\mathsf{Ch}}$ is the Church version of the Curry-style system: $\Lambda_\cap^{\mathsf{Cu}}$ and $\Lambda_\cap^{\mathsf{Ch}}$ simulate each other, and reduction corresponds to cut-elimination in the derivation tree. We consider this to be a relevant result on its own.

Finally, the same distinction made in Chapters 1 and 2 between our approach for $\mathcal{W}$ and those of Nederpelt, Klop, Turing, Gandy and de Vrijer, holds for the comparison against Kfoury and Wells' and Boudol's measures. We combined non-erasing calculus with redex degrees in such a way to produce a strong decreasing measure. A relevant addition for this case is that not only is the approach novel, but so is the kind of decreasing measure. As we already stated, it is the first strong one with the simplest codomain, defined following IIT definitions, *i.e.*, without relying on translation to other, non-intersection-based systems.

Strong decreasing measures through translation

Bucciarelli's *et al.* approach [19] is quite different: SN for intersection types is not the main result, but an intermediate step in proving that computational power of intersection types coincides with that of simple types. The authors define an embedding from intersection type to simple type derivations which commutes with β-reduction. They do not define a decreasing measure. One could, however, define a measure for intersection types by combining the translation with one of the existing measures for simply typed λ–calculus, for instance, $\mathcal{W}$ from Chapter 2, or Gandy's [48]. We argue that our approach, which is native to the calculus, works better for our purpose of purely analyzing the calculus reductions, *i.e.*, not comparing it with others. A clear advantage we could find is the possibility of refining the measure to exactness. To develop this idea, let us note the translation $\mathcal{A}_\cap$, and the combined measure $\mathcal{W} \circ \mathcal{A}_\cap$. Such a measure would provide a single natural number as a result, as ours. But it is worth noting that these are distinct: the one based on the translation counts extra memories whenever an argument with several types is erased, providing a number that is greater or equal than ours. The *tightness* factor in decreasing measures is desirable because it allows us to seek an exact upper bound. The worse tightness of $\mathcal{W} \circ \mathcal{A}_\cap$ is due to the loss of information that occurs through the translation: to handle variables having different types, $\mathcal{A}_\cap$ replicates arguments. For instance, $(\lambda x^{\{A\to A, A\}}.xx)z$ gets translated to $(\lambda x^{A\to A}.\lambda y^{A}.xy)zz$, from which we cannot recover the original term. Since $(\lambda x.xx)z$ has only one step until normal form, while $(\lambda x.\lambda y.xy)zz$ has two, we lose the possibility of computing tighter measures.

Even if considering an exact measure for simple types to combine with $\mathcal{A}_\cap$, such as de Vrijer's [32], one would not get a tighter (nor looser, to be fair) measure than $\mathcal{W}_\cap$. The loss of intersection type information that occurs during the translation is a hard limitation for any measure based on it; they cannot be refined to exactness. Since a tighter measure would provide a more accurate computational analysis, and would therefore be generally preferred, we consider the refinability of our measure an advantage over measures based on the translation. Our more coupled approach, since it operates *within* the intersection system, provides a finer analysis of reduction and thus allows for further improvement, which is indeed a work in progress discussed in Chapter 4.

3.1.2 Organization of the chapter

In Section 3.2 we present the languages $\Lambda_\cap^{\mathsf{Cu}}$ and $\Lambda_\cap^{\mathsf{Ch}}$, their properties and their relation. In Section 3.3 we extend $\Lambda_\cap^{\mathsf{Ch}}$ to the non-erasing calculus $\Lambda_\cap^{\mathbf{m}}$. In Section 3.4 we define a decreasing

measure that entails strong normalization of $\Lambda_\cap^{\mathsf{Ch}}$ by means of the auxiliary $\Lambda_\cap^{\mathbf{m}}$. Finally, in Section 3.5 we show how this can be used to prove strong normalization of $\Lambda_\cap^{\mathsf{Cu}}$, and we conclude and propose future works. A prototype implementation in Haskell of the measure introduced in this chapter can be found at `https://github.com/cfsottile/measures`.

3.2 Idempotent intersection types à la Church

This section starts by presenting $\Lambda_\cap^{\mathsf{Cu}}$ (Section 3.2.1), an intersection type assignment system for λ-terms, closely related to other well-known systems that characterize strong normalization. In Section 3.2.2, we formulate an intrinsically typed version of the system, called $\Lambda_\cap^{\mathsf{Ch}}$, and we study its properties, including subject reduction and confluence. In Section 3.2.3 we establish a formal proof-theoretical connection between $\Lambda_\cap^{\mathsf{Cu}}$ and $\Lambda_\cap^{\mathsf{Ch}}$, and we prove that $\Lambda_\cap^{\mathsf{Cu}}$ gives type to all the strongly normalizing terms.

We define the set of untyped λ-terms of Λ as usual by the grammar

$$M ::= x \mid \lambda x.M \mid M\ M$$

The sets of *types* $(A, B, \ldots)$ and *set-types* $(\vec{A}, \vec{B}, \ldots)$ are given mutually inductively as follows:

$$\begin{aligned} A &::= a \mid \vec{A} \to A && (\vec{A} \neq \varnothing) \\ \vec{A} &::= \{A_j\}_{j\in J} && (\forall h, k \in J.\ h \neq k \implies A_h \neq A_k) \end{aligned}$$

where $a, b, \ldots$ range over a denumerable set of base types, and $\vec{A}$ stands for a finite set of types[1], where J denotes a finite set of indices, and $\{A_j\}_{j\in J}$ is the set $\{A_j \mid j \in J\}$. When we write a set-type, we keep the invariant that there are no repetitions, *i.e.*, that if $h \neq k$ then $A_h \neq A_k$, so that the set is of cardinality $|J|$. Sometimes we will use the explicit notation $\{A_1, ..., A_n\}$. Note that a set-type is a finite set of types that may be empty, but the domain of an arrow type is always a *non-empty* set-type.

Sometimes we write $A \to B$ for $\{A\} \to B$. A typing context $(\Gamma, \Delta, \ldots)$ is a function mapping each variable to a set-type such that $\Gamma(x) \neq \varnothing$ for finitely many variables x. We write $\mathsf{dom}(\Gamma)$ for the set of variables which Γ assigns to a non-empty set-type. We adopt the standard notation for typing contexts, *e.g.* writing $x_1 : \vec{A}_1, \ldots, x_n : \vec{A}_n$ for the context Γ such that $\Gamma(x_i) = \vec{A}_i$ for each $i \in 1..n$ and $\Gamma(y) = \varnothing$ for every $y \notin \{x_1, \ldots, x_n\}$. We write $\Gamma \cup \Delta$ for the typing context such that $(\Gamma \cup \Delta)(x) = \Gamma(x) \cup \Delta(x)$ for every x, and we write $\Gamma \subseteq \Delta$ if $\Gamma(x) \subseteq \Delta(x)$ for every x. Note in particular that $\Gamma \subseteq (\Gamma, x : \vec{A})$ if $x \notin \mathsf{dom}(\Gamma)$. If $\mathcal{J}$ is a judgement, we write $\Pi \rhd \mathcal{J}$ to mean that Π is a particular derivation proving $\mathcal{J}$.

3.2.1 Extrinsically typed system

We define a Curry-style intersection type assignment system, which is a variant of well-known systems in the literature, starting with the one introduced by Coppo and Dezani [22].

Definition 3.2.1 (The $\Lambda_\cap^{\mathsf{Cu}}$ type assignment system)**.** The $\Lambda_\cap^{\mathsf{Cu}}$ calculus is defined by means of two forms of judgement:

1. $\Gamma \vdash_{\mathsf{Cu}} M : A$ meaning that the λ-term M has type A under the context Γ.

[1]Note that $\vec{A} ::= \{A_j\}_{j\in J}$ is not intended to be a grammatical production that determines a concrete syntax for finite sets, but rather as a formation rule meaning that a set-type $\vec{A}$ is given by a finite set of types.

2. $\Gamma \Vdash_{\mathsf{Cu}} M : \vec{A}$ meaning that the λ-term M has set-type $\vec{A}$ under the context Γ.

The typing rules are as follows:

$$\frac{B \in \vec{A}}{\Gamma, x : \vec{A} \vdash_{\mathsf{Cu}} x : B}\ \mathtt{Cu\text{-}var} \qquad \frac{(\Gamma \vdash_{\mathsf{Cu}} N : A_j)_{j\in J} \quad (\forall h,k \in J.\ h \neq k \implies A_h \neq A_k)}{\Gamma \Vdash_{\mathsf{Cu}} N : \{A_j\}_{j\in J}}\ \mathtt{Cu\text{-}many}$$

$$\frac{\Gamma, x : \vec{A} \vdash_{\mathsf{Cu}} M : B}{\Gamma \vdash_{\mathsf{Cu}} \lambda x.M : \vec{A} \to B}\ \mathtt{Cu\text{-}I}{\to} \qquad \frac{\Gamma \vdash_{\mathsf{Cu}} M : \vec{A} \to B \quad \vec{A} \neq \varnothing \quad \Gamma \Vdash_{\mathsf{Cu}} N : \vec{A}}{\Gamma \vdash_{\mathsf{Cu}} M\,N : B}\ \mathtt{Cu\text{-}E}{\to}$$

Note that we restrict the codomain of contexts so that $\Gamma(x) \neq \varnothing$, so rule $\mathtt{Cu\text{-}I}{\to}$ cannot introduce an empty set to the left of the arrow.

Example 3.2.2. Let $\Gamma = x : \{\{A, B\} \to C, A, B\}$. Then:

$$\dfrac{\dfrac{\{A,B\} \to C \in \Gamma(x)}{\Gamma \vdash_{\mathsf{Cu}} x : \{A,B\} \to C}\ \mathtt{Cu\text{-}var} \quad \dfrac{\dfrac{A \in \Gamma(x)}{\Gamma \vdash_{\mathsf{Cu}} x : A}\ \mathtt{Cu\text{-}var} \quad \dfrac{B \in \Gamma(x)}{\Gamma \vdash_{\mathsf{Cu}} x : B}\ \mathtt{Cu\text{-}var}}{\Gamma \Vdash_{\mathsf{Cu}} x : \{A,B\}}\ \mathtt{Cu\text{-}many}}{\dfrac{\Gamma \vdash_{\mathsf{Cu}} x\,x : C}{\varnothing \vdash_{\mathsf{Cu}} \lambda x.x\,x : \{\{A,B\} \to C, A, B\} \to C}\ \mathtt{Cu\text{-}I}{\to}}\ \mathtt{Cu\text{-}E}{\to}$$

Remark 3.2.3. The system $\Lambda_\cap^{\mathsf{Cu}}$ presented above is not exactly the same system as that of [22], for two reasons. First, $\Lambda_\cap^{\mathsf{Cu}}$ uses sets instead of intersections, writing $\{A, B\} \to C$ rather than $(A \wedge B) \to C$. This can be understood only as a difference in notation, given that the intersection type constructor of [22] is assumed to be associative, commutative, and idempotent. Second, $\Lambda_\cap^{\mathsf{Cu}}$ allows sets (*i.e.*, intersections) to appear only to the *left* of the arrow ($\to$) connective. For example, $(A \wedge B) \to (C \wedge D)$ is a well-formed type in the system of [22], whereas $\{A, B\} \to \{C, D\}$ is not a well-formed type in $\Lambda_\cap^{\mathsf{Cu}}$. This means that types in $\Lambda_\cap^{\mathsf{Cu}}$ are *strict types*, in the sense introduced by van Bakel [102]. Despite these two differences, we will prove that the original system [22] and $\Lambda_\cap^{\mathsf{Cu}}$ are equivalent in typability power, as they both characterize strongly normalizable λ-terms.

3.2.2 Intrinsically typed system

We define a Church-style presentation of the system, inspired by the linearization proposed by Kfoury [61].

Definition 3.2.4 (The $\Lambda_\cap^{\mathsf{Ch}}$ type assignment system)**.** The sets of *terms* ($t, s, \ldots$) and *set-terms* ($\vec{t}, \vec{s}, \ldots$) are given by the following grammar:

$$t ::= x^A \mid \lambda x^{\vec{A}}.t \mid t\,\vec{t} \qquad\qquad \vec{t} ::= \{t_j\}_{j\in J}$$

where J stands for a finite set of indices and $\{t_j\}_{j\in J}$ is the set $\{t_j \mid j \in J\}$. Note that set-terms are finite sets of terms. When we write a set-term, we keep the invariant that there are no repetitions, *i.e.*, that if $h \neq k$ then $t_h \neq t_k$. The $\Lambda_\cap^{\mathsf{Ch}}$ calculus is defined by means of two forms of judgement:

1. $\Gamma \vdash_{\mathsf{Ch}} t : A$ meaning that a term t is of type A under the context Γ.

2. $\Gamma \Vdash_{\mathsf{Ch}} \vec{t} : \vec{A}$ meaning that a set-term $\vec{t}$ is of set-type $\vec{A}$ under the context Γ.

The typing rules are as follows:

$$\frac{B \in \vec{A}}{\Gamma, x : \vec{A} \vdash_{\mathsf{Ch}} x^B : B}\ \textsf{Ch-var} \qquad \frac{(\Gamma \vdash_{\mathsf{Ch}} t_j : A_j)_{j\in J} \quad (\forall h,k \in J.\, h \neq k \implies A_h \neq A_k)}{\Gamma \Vdash_{\mathsf{Ch}} \{t_j\}_{j\in J} : \{A_j\}_{j\in J}}\ \textsf{Ch-many}$$

$$\frac{\Gamma, x : \vec{A} \vdash_{\mathsf{Ch}} t : B}{\Gamma \vdash_{\mathsf{Ch}} \lambda x^{\vec{A}}.t : \vec{A} \to B}\ \textsf{Ch-I}{\to} \qquad \frac{\Gamma \vdash_{\mathsf{Ch}} t : \vec{A} \to B \qquad \Gamma \Vdash_{\mathsf{Ch}} \vec{s} : \vec{A} \qquad \vec{A} \neq \varnothing}{\Gamma \vdash_{\mathsf{Ch}} t\,\vec{s} : B}\ \textsf{Ch-E}{\to}$$

A term t is *typable* if $\Gamma \vdash_{\mathsf{Ch}} t : A$ holds for some Γ, A. Similarly, a set-term is typable if $\Gamma \Vdash_{\mathsf{Ch}} \vec{t} : \vec{A}$ holds for some $\Gamma, \vec{A}$. Unless otherwise specified, when we speak of *term* (resp. *set-term*) we mean *typable term* (resp. *typable set-term*). We write $\Lambda_\cap^{\mathsf{Ch}}$ for the set of typable terms, *i.e.*, $\Lambda_\cap^{\mathsf{Ch}} := \{t \mid t \text{ is typable}\}$. The notions of free and bound occurrences of variables are defined as usual. The set of *free variables of type* A of a term t is written $\mathsf{fv}_A(t)$ and defined as the set of variables x such that x^A occurs free in t. For example, $\mathsf{fv}_A(\lambda y^{\{\{A,B\}\to C\}}.y^{\{A,B\}\to C}\{x^A, z^B\}) = \{x\}$ when $A \neq B$. The set of *free variables* of a term t is written $\mathsf{fv}(t)$. Terms are considered up to α-renaming of bound variables.

Notation 3.2.5. Sometimes we write $t\,s$ to stand for $t\,\{s\}$. We may omit type annotations over variables if they are clear from the context.

Remark 3.2.6. Note that, in the rule Ch-many, we explicitly require that the function $j \mapsto A_j$ is injective (*i.e.*, $\forall h,k \in J.\, h \neq k \implies A_h \neq A_k$). For example, let $\Gamma = (x : \{A, B\}, y : \{A\})$. Then the following are valid judgments:

$$\Gamma \Vdash_{\mathsf{Ch}} \{x^A\} : \{A\} \qquad \Gamma \Vdash_{\mathsf{Ch}} \{x^A, x^B\} : \{A, B\} \qquad \Gamma \Vdash_{\mathsf{Ch}} \{x^B, y^A\} : \{A, B\}$$

On the other hand, the following judgments are not valid:

$$\Gamma \Vdash_{\mathsf{Ch}} \{x^A, y^A\} : \{A\} \qquad \Gamma \Vdash_{\mathsf{Ch}} \{x^A, x^A\} : \{A\}$$

Example 3.2.7. The derivation of Example 3.2.2 can be encoded in $\Lambda_\cap^{\mathsf{Ch}}$ as the following term, which has type $\{\{A, B\} \to C, A, B\} \to C$ in the empty context:

$$\lambda x^{\{\{A,B\}\to C, A, B\}}.x^{\{A,B\}\to C}\,\{x^A, x^B\}$$

Remark 3.2.8. Note that, in general, a set-term may be an *empty* set of terms, but the argument of an application is always a *non-empty* set-term. We let set-types and set-terms to be empty to be able to write expressions such as $\vec{t} = \{t\} \cup \vec{s}$ to mean that $\vec{t}$ is a set containing at least one element t, leaving the possibility that $\vec{s}$ may be empty.

Remark 3.2.9 (Subterm property). If t is typable, all of its subterms are typable.

Lemma 3.2.10 (Weakening and strengthening)**.**

1. If $\Gamma \vdash_{\mathsf{Ch}} t : A$ (resp. $\Gamma \Vdash_{\mathsf{Ch}} \vec{t} : \vec{A}$) and $\Gamma \subseteq \Delta$ then $\Delta \vdash_{\mathsf{Ch}} t : A$ (resp. $\Delta \Vdash_{\mathsf{Ch}} \vec{t} : \vec{A}$).
2. If $\Gamma, x : \{A\} \cup \vec{B} \vdash_{\mathsf{Ch}} t : C$ (resp. $\Gamma, x : \{A\} \cup \vec{B} \Vdash_{\mathsf{Ch}} \vec{t} : \vec{C}$) and $x \notin \mathsf{fv}_A(t)$, then $\Gamma' \vdash_{\mathsf{Ch}} t : C$ (resp. $\Gamma' \Vdash_{\mathsf{Ch}} \vec{t} : \vec{C}$), where Γ' is $(\Gamma, x : \vec{B})$ if $\vec{B} \neq \varnothing$, and Γ otherwise.

Proof.

1. By induction on the derivation of $\Gamma \vdash_{\mathsf{Ch}} t : A$.

1.1. $\dfrac{}{\Gamma \vdash_{\mathsf{Ch}} x : A}\ \mathsf{Ch\text{-}var}$: Then $x : \vec{A} \in \Gamma$ with $A \in \vec{A}$. Since $\Gamma \subseteq \Delta$, we have $x : \vec{A} \in \Delta$ too, therefore $\Delta \vdash_{\mathsf{Ch}} x : A$.

1.2. $\dfrac{\Gamma, x : \vec{A} \vdash_{\mathsf{Ch}} s : B}{\Gamma \vdash_{\mathsf{Ch}} \lambda x^{\vec{A}}.s : \vec{A} \to B}\ \mathsf{Ch\text{-}I}\to$: Then by IH we know that $\Delta, x : \vec{A} \vdash_{\mathsf{Ch}} s : B$, since $\Gamma \subseteq \Delta$ and thus $\Gamma, x : \vec{A} \subseteq \Delta, x : \vec{A}$, therefore by $\mathsf{Ch\text{-}I}\to$ we conclude $\Delta \vdash_{\mathsf{Ch}} \lambda x^{\vec{A}}.s : \vec{A} \to B$.

1.3. $\dfrac{\Gamma \vdash_{\mathsf{Ch}} s : \vec{A} \to B \quad \Gamma \Vdash_{\mathsf{Ch}} \vec{r} : \vec{A}}{\Gamma \vdash_{\mathsf{Ch}} s\vec{r} : B}\ \mathsf{Ch\text{-}E}\to$: Then by IH we know that $\Delta \vdash_{\mathsf{Ch}} s : \vec{A} \to B$ and $\Delta \Vdash_{\mathsf{Ch}} \vec{r} : \vec{A}$, therefore by $\mathsf{Ch\text{-}E}\to$ we conclude $\Delta \vdash_{\mathsf{Ch}} s\vec{r} : B$.

1.4. $\dfrac{\Gamma \vdash_{\mathsf{Ch}} t_1 : A_1 \ldots \Gamma \vdash_{\mathsf{Ch}} t_n : A_n}{\Gamma \Vdash_{\mathsf{Ch}} \{t_1, \ldots, t_n\} : \vec{A}}\ \mathsf{Ch\text{-}many}$: Then by IH we know that, for all $i \in \{1..n\}$, it holds that $\Delta \vdash_{\mathsf{Ch}} t_i : A_i$, therefore by $\mathsf{Ch\text{-}many}$ we conclude $\Delta \Vdash_{\mathsf{Ch}} \{t_1, \ldots, t_n\} : \vec{A}$.

2. By induction on the derivation of $\Gamma, x : \{A\} \cup \vec{B} \vdash_{\mathsf{Ch}} t : C$.

2.1. $\dfrac{}{\Gamma, x : \{A\} \cup \vec{B} \vdash_{\mathsf{Ch}} x : C}\ \mathsf{Ch\text{-}var}$: By hypothesis we know that x does not belong to the set of free variables of type A of $t = x^C$, therefore $A \neq C$, so, for the typing judgement to be valid, it must be the case that $C \in \vec{B}$. Hence $\Gamma, x : \vec{B} \vdash_{\mathsf{Ch}} x : C$.

2.2. $\dfrac{}{\Gamma, x : \{A\} \cup \vec{B} \vdash_{\mathsf{Ch}} y : C}\ \mathsf{Ch\text{-}var}$ with $y \neq x$: Then $\Gamma(y) = \vec{C}$ where $C \in \vec{C}$. Therefore $\Gamma' \vdash_{\mathsf{Ch}} y : C$.

2.3. $\dfrac{\Gamma, x : \{A\} \cup \vec{B}, y : \vec{C} \vdash_{\mathsf{Ch}} s : D}{\Gamma, x : \{A\} \cup \vec{B} \vdash_{\mathsf{Ch}} \lambda y^{\vec{C}}.s : \vec{C} \to D}\ \mathsf{Ch\text{-}I}\to$: Then by IH we know that $\Gamma', y : \vec{C} \vdash_{\mathsf{Ch}} s : D$, since $\lambda y^{\vec{C}}$ does not bind x of type A and therefore the hypothesis holds for the premise. By $\mathsf{Ch\text{-}I}\to$ we conclude $\Gamma' \vdash_{\mathsf{Ch}} \lambda y^{\vec{C}}.s : \vec{C} \to D$.

2.4. $\dfrac{\Gamma, x : \{A\} \cup \vec{B} \vdash_{\mathsf{Ch}} s : \vec{C} \to D \quad \Gamma, x : \{A\} \cup \vec{B} \Vdash_{\mathsf{Ch}} \vec{r} : \vec{C}}{\Gamma, x : \{A\} \cup \vec{B} \vdash_{\mathsf{Ch}} s\vec{r} : D}\ \mathsf{Ch\text{-}E}\to$: Then by IH we know that $\Gamma' \vdash_{\mathsf{Ch}} s : \vec{C} \to D$ and $\Gamma' \Vdash_{\mathsf{Ch}} \vec{r} : \vec{C}$, therefore by $\mathsf{Ch\text{-}E}\to$ we conclude $\Gamma' \vdash_{\mathsf{Ch}} s\vec{r} : D$.

2.5. $\dfrac{\Gamma, x : \{A\} \cup \vec{B} \vdash_{\mathsf{Ch}} t_1 : C_1 \ldots \Gamma, x : \{A\} \cup \vec{B} \vdash_{\mathsf{Ch}} t_n : C_n}{\Gamma, x : \{A\} \cup \vec{B} \Vdash_{\mathsf{Ch}} \{t_1, \ldots, t_n\} : \vec{C}}\ \mathsf{Ch\text{-}many}$: Then by IH we know that, for all $i \in \{1..n\}$, it holds that $\Gamma' \vdash_{\mathsf{Ch}} t_i : C_i$, therefore by $\mathsf{Ch\text{-}many}$ we conclude $\Gamma' \Vdash_{\mathsf{Ch}} \{t_1, \ldots, t_n\} : \vec{C}$.

□

Definition 3.2.11 (Minimal typing context)**.** For each term t (resp. set-term $\vec{t}$), its *minimal typing context* is written $\gamma(t)$ (resp. $\gamma(\vec{t})$) and defined as follows:

$$\begin{array}{lcllcl} \gamma(x^A) & := & x : \{A\} & \gamma(\lambda x^{\vec{A}}.t) & := & \gamma(t) \setminus x \\ \gamma(t\,\vec{s}) & := & \gamma(t) \cup \gamma(\vec{s}) & \gamma(\{t_1, \ldots, t_n\}) & := & \cup_{i=1}^{n} \gamma(t_i) \end{array}$$

where $\Gamma \setminus x$ is defined in such a way that $(\Gamma \setminus x)(x) = \varnothing$ and $(\Gamma \setminus x)(y) = \Gamma(y)$ for $x \neq y$.

Lemma 3.2.12 (Type uniqueness)**.** If $\Gamma_1 \vdash_{\mathsf{Ch}} t : A_1$ and $\Gamma_2 \vdash_{\mathsf{Ch}} t : A_2$ then $A_1 = A_2$. Furthermore, $\gamma(t) \subseteq \Gamma_1$, $\gamma(t) \subseteq \Gamma_2$, and $\gamma(t) \vdash_{\mathsf{Ch}} t : A_1$.

Proof. By induction on t.

1. x^A: The type of x^A is determined by its label A, therefore $A = A_1 = A_2$, so we have $\dfrac{}{\Gamma_1 \vdash_{\mathsf{Ch}} x^A : A}\ \mathsf{Ch\text{-}var}$ and $\dfrac{}{\Gamma_2 \vdash_{\mathsf{Ch}} x^A : A}\ \mathsf{Ch\text{-}var}$, where $x : \vec{A}$ with $A \in \vec{A}$ must be an assignment in both contexts. Furthermore, $\gamma(x^A) = x : \{A\}$, so $\gamma(x^A) \subseteq \Gamma_1$, $\gamma(x^A) \subseteq \Gamma_2$, and $\dfrac{}{x : \{A\} \vdash_{\mathsf{Ch}} x^A : A}\ \mathsf{Ch\text{-}var}$.

2. $\lambda x^{\vec{A}}.s$: Then we have $\Gamma_1 \vdash_{\mathsf{Ch}} \lambda x^{\vec{A}}.s : \vec{A} \to B_1$ and $\Gamma_2 \vdash_{\mathsf{Ch}} \lambda x^{\vec{A}}.s : \vec{A} \to B_2$, with $A_1 = \vec{A} \to B_1$ and $A_2 = \vec{A} \to B_2$. By inversion of rule $\mathsf{Ch\text{-}I}{\to}$ we have $\Gamma_1, x : \vec{A} \vdash_{\mathsf{Ch}} s : B_1$ and $\Gamma_2, x : \vec{A} \vdash_{\mathsf{Ch}} s : B_2$. By IH we know that $B_1 = B_2$ so we conclude $A = \vec{A} \to B_1 = \vec{A} \to B_2 = A_2$. Furthermore, also by IH, we know that $\gamma(s) \subseteq \Gamma_1, x : \vec{A}$, $\gamma(s) \subseteq \Gamma_2, x : \vec{A}$, so by definition of minimal typing context (Definition 3.2.11) we have $\gamma(\lambda x^{\vec{A}}.s) = \gamma(s)\backslash x \subseteq (\Gamma_1, x : \vec{A})\backslash x = \Gamma_1$ and $\gamma(\lambda x^{\vec{A}}.s) = \gamma(s)\backslash x \subseteq (\Gamma_2, x : \vec{A})\backslash x = \Gamma_2$. Finally, again by IH, we have that $\gamma(s) \vdash_{\mathsf{Ch}} s : B_1$. If $x : \vec{A} \in \gamma(s)$, then by $\mathsf{Ch\text{-}I}{\to}$ we conclude $\gamma(s)\backslash x = \gamma(\lambda x^{\vec{A}}.s) \vdash_{\mathsf{Ch}} \lambda x^{\vec{A}}.s : \vec{A} \to B_1$. If $x : \vec{A} \notin \gamma(s)$, then x does not occur free in s, so we resort to weakening (Lemma 3.2.10) to get $\gamma(s), x : \vec{A} \vdash_{\mathsf{Ch}} s : B_1$, from which by rule $\mathsf{Ch\text{-}I}{\to}$ we conclude $\gamma(s)\backslash x = \gamma(\lambda x^{\vec{A}}.s) \vdash_{\mathsf{Ch}} \lambda x^{\vec{A}}.s : \vec{A} \to B_1$.

3. $s\vec{r}$: Then we have $\Gamma_1 \vdash_{\mathsf{Ch}} s\vec{r} : A_1$ and $\Gamma_2 \vdash_{\mathsf{Ch}} s\vec{r} : A_2$. By inversion of the rule $\mathsf{Ch\text{-}E}{\to}$, we have $\Gamma_1 \vdash_{\mathsf{Ch}} s : \vec{B_1} \to A_1$ and $\Gamma_1 \Vdash_{\mathsf{Ch}} \vec{r} : \vec{B_1}$ on one side, and $\Gamma_2 \vdash_{\mathsf{Ch}} s : \vec{B_2} \to A_2$ and $\Gamma_2 \Vdash_{\mathsf{Ch}} \vec{r} : \vec{B_2}$ on the other. By IH we have $\vec{B_1} \to A_1 = \vec{B_2} \to A_2$, hence $A_1 = A_2$. Furthermore, also by IH, we know that $\gamma(s) \subseteq \Gamma_1$, $\gamma(s) \subseteq \Gamma_2$, $\gamma(\vec{r}) \subseteq \Gamma_1$, and $\gamma(\vec{r}) \subseteq \Gamma_2$. So, by definition of minimal typing contexts (Definition 3.2.11), $\gamma(s\vec{r}) = \gamma(s) \cup \gamma(\vec{r}) \subseteq \Gamma_1 \cup \Gamma_1 = \Gamma_1$ and $\gamma(s\vec{r}) = \gamma(s) \cup \gamma(\vec{r}) \subseteq \Gamma_2 \cup \Gamma_2 = \Gamma_2$. Finally, again by IH, we have that $\gamma(s) \vdash_{\mathsf{Ch}} s : \vec{B} \to A_1$ and $\gamma(\vec{r}) \vdash_{\mathsf{Ch}} \vec{r} : \vec{B_1}$. By repeated application of weakening (Lemma 3.2.10), we obtain $\gamma(s) \cup \gamma(\vec{r}) \vdash_{\mathsf{Ch}} s : \vec{B} \to A_1$ and $\gamma(s) \cup \gamma(\vec{r}) \vdash_{\mathsf{Ch}} \vec{r} : \vec{B_1}$. Therefore, by rule $\mathsf{Ch\text{-}E}{\to}$ we conclude $\gamma(s) \cup \gamma(\vec{r}) = \gamma(s\vec{r}) \vdash_{\mathsf{Ch}} s\vec{r} : A_1$.

4. $\{t_1, \ldots, t_n\}$: Then we have $\Gamma_1 \vdash_{\mathsf{Ch}} \{t_1, \ldots, t_n\} : \vec{B}$ and $\Gamma_2 \vdash_{\mathsf{Ch}} \{t_1, \ldots, t_n\} : \vec{C}$, with $A_1 = \vec{B}$ and $A_2 = \vec{C}$. By inversion of the rule $\mathsf{Ch\text{-}many}$, we have $\Gamma_1 \vdash_{\mathsf{Ch}} t_i : B_i$ for all $i \in \{1..n\}$ on one side, and $\Gamma_2 \vdash_{\mathsf{Ch}} t_i : C_i$ for all $i \in \{1..n\}$ on the other. By IH we have $B_i = C_i$ for all $i \in \{1..n\}$, hence $\vec{B} = \vec{C}$. Furthermore, also by IH, we know that $\gamma(t_i) \subseteq \Gamma_1$ and $\gamma(t_i) \subseteq \Gamma_2$ for all $i \in \{1..n\}$. So, by definition of minimal typing contexts (Definition 3.2.11), $\gamma(\{t_1, \ldots, t_n\}) = \cup_{i=1}^n \gamma(t_i) \subseteq \cup_{i=1}^n \Gamma_1 = \Gamma_1$ and $\gamma(\{t_1, \ldots, t_n\}) = \cup_{i=1}^n \gamma(t_i) \subseteq \cup_{i=1}^n \Gamma_2 = \Gamma_2$ Finally, again by IH, we have that $\gamma(t_i) \vdash_{\mathsf{Ch}} t_i : B_i$ for all $i \in \{1..n\}$. By repeated application of weakening (Lemma 3.2.10), we obtain $\cup_{i=1}^n \gamma(t_i) \vdash_{\mathsf{Ch}} t_i : B_i$. Therefore, by rule $\mathsf{Ch\text{-}E}{\to}$ we conclude $\cup_{i=1}^n \gamma(t_i) = \gamma(\{t_1, \ldots, t_n\}) \vdash_{\mathsf{Ch}} \{t_1, \ldots, t_n\} : \vec{B}$.

□

Remark 3.2.13. Sometimes we write t^A to mean that $\gamma(t) \vdash_{\mathsf{Ch}} t : A$. Note that, by the previous lemma, there is a unique A such that t^A.

Observation 3.2.14 (Bijection between a set-term and its set-type)**.** Let $\Gamma \Vdash_{\mathsf{Ch}} \vec{s} : \vec{A}$. Given $A' \in \vec{A}$, we know by rule $\mathsf{Ch\text{-}many}$ that there exists a derivation $\Gamma \vdash_{\mathsf{Ch}} s' : A'$. This is the only derivation with type A' in the premises of the rule: otherwise, we would contradict the rule's side condition by having different derivations with the same type. Therefore, s' is unique. Given $s' \in \vec{s}$, we know by rule $\mathsf{Ch\text{-}many}$ that it has a type A'; by Lemma 3.2.12, this type is unique.

Definition 3.2.15 (Substitution in $\Lambda_\cap^{\mathsf{Ch}}$)**.** We define operations of capture-avoiding substitution for terms and set-terms by mutual recursion. Let $\Gamma \Vdash_{\mathsf{Ch}} \vec{s} : \vec{A}$ with $\vec{s} = \{s_i\}_{i\in I}$, $\vec{A} = \{A_i\}_{i\in I}$, and $\Gamma \vdash_{\mathsf{Ch}} s_i : A_i$ for all $i \in I$. we define $t[\vec{s}/x^{\vec{A}}]$ recursively as follows.

$$\begin{array}{rcl} x^{A_i}[\vec{s}/x^{\vec{A}}] & := & s_i \\ y^B[\vec{s}/x^{\vec{A}}] & := & y^B \\ (\lambda y^{\vec{B}}.t)[\vec{s}/x^{\vec{A}}] & := & \lambda y^{\vec{B}}.t[\vec{s}/x^{\vec{A}}] \\ (t\,\vec{r})[\vec{s}/x^{\vec{A}}] & := & t[\vec{s}/x^{\vec{A}}]\,\vec{r}[\vec{s}/x^{\vec{A}}] \\ \{t_1,\dots,t_n\}[\vec{s}/x^{\vec{A}}] & := & \{t_1[\vec{s}/x^{\vec{A}}],\dots,t_n[\vec{s}/x^{\vec{A}}]\} \end{array}$$

Note that, in the first case, s_i exists and is unique due to Observation 3.2.14.

Lemma 3.2.16 (Substitution Lemma)**.** Let $\Gamma, x : \vec{A} \vdash_{\mathsf{Ch}} t : B$ and $\Gamma \Vdash_{\mathsf{Ch}} \vec{s} : \vec{A}$. Then $\Gamma \vdash_{\mathsf{Ch}} t[\vec{s}/x^{\vec{A}}] : B$.

Proof. By induction on t, generalizing the statement to set-terms, where the interesting cases are:

1. $t = x^B$: Then $\Gamma, x : \vec{A} \vdash_{\mathsf{Ch}} x^B : B$ with $B \in \vec{A}$. By Observation 3.2.14, there exists a unique $s_i \in \vec{s}$ such that $\Gamma \vdash_{\mathsf{Ch}} s_i : B$. Therefore, $x^B[\vec{s}/x^{\vec{A}}] = s_i$ of type B.
2. $\vec{t} = \{t_1,\dots,t_n\}$: Then B is indeed a set-type $\vec{B} = \{B_1,\dots,B_n\}$, and $\Gamma, x : \vec{A} \Vdash_{\mathsf{Ch}} \{t_1,\dots,t_n\} : \{B_1,\dots,B_n\}$. By IH, $\Gamma \vdash_{\mathsf{Ch}} t_i[\vec{s}/x^{\vec{A}}] : B_i$ for all i such that $1 \le i \le n$, so by `Ch-many` we have $\Gamma \Vdash_{\mathsf{Ch}} \{t_1[\vec{s}/x^{\vec{A}}],\dots,t_n[\vec{s}/x^{\vec{A}}]\} : \{B_1,\dots,B_n\}$. Note that substitution preserves cardinality in typed set-term derivations. If we had $t_i[\vec{s}/x^{\vec{A}}] = t_j[\vec{s}/x^{\vec{A}}]$ with $1 \le i, j \le n$ and $i \neq j$, they would have the same type by Lemma 3.2.12. This would imply that t_i and t_j have the same type, which is impossible by rule `Ch-many`.

□

Definition 3.2.17 (Reduction in $\Lambda_\cap^{\mathsf{Ch}}$)**.** A notion of reduction called `Ch`-reduction is defined over terms and set-terms by the following rule, closed by congruence under arbitrary contexts:

$$(\lambda x^{\vec{A}}.t)\,\vec{s} \to_{\mathsf{Ch}} t[\vec{s}/x^{\vec{A}}]$$

Note in particular that congruence rules allow reducing inside the argument of an application, *i.e.*, $\vec{s} \to_{\mathsf{Ch}} \vec{r}$ implies $t\,\vec{s} \to_{\mathsf{Ch}} t\,\vec{r}$, and inside any of the elements of a set-term, *i.e.*, $t_i \to_{\mathsf{Ch}} t_i'$ implies $\{t_1,\dots,t_i,\dots,t_n\} \to_{\mathsf{Ch}} \{t_1,\dots,t_i',\dots,t_n\}$. We write $\to_{\mathsf{Ch}}^*$ for the reflexive–transitive closure of $\to_{\mathsf{Ch}}$.

Proposition 3.2.18 (Subject reduction)**.** If $t \to_{\mathsf{Ch}} t'$ and $\Gamma \vdash_{\mathsf{Ch}} t : A$ then $\Gamma \vdash_{\mathsf{Ch}} t' : A$.

Proof. We generalize the statement, proving also that if $\vec{t} \to_{\mathsf{Ch}} \vec{t'}$ and $\Gamma \Vdash_{\mathsf{Ch}} \vec{t} : \vec{A}$ then $\Gamma \Vdash_{\mathsf{Ch}} \vec{t'} : \vec{A}$. The proof proceeds by induction on $\to_{\mathsf{Ch}}$, where the interesting cases are β-reduction and set-terms. The remaining cases are straightforward by IH.

1. If t is a redex, i.e, $t = (\lambda x^{\vec{B}}.s)\,\vec{u}$, then the derivation proving $\Gamma \vdash_{\mathsf{Ch}} t : A$ is:

$$\dfrac{\dfrac{\Gamma, x : \vec{B} \vdash_{\mathsf{Ch}} s : A}{\Gamma \vdash_{\mathsf{Ch}} \lambda x^{\vec{B}}.s : \vec{B} \to A} \quad \dfrac{}{\Gamma \Vdash_{\mathsf{Ch}} \vec{u} : \vec{B}}}{\Gamma \vdash_{\mathsf{Ch}} (\lambda x^{\vec{B}}.s)\,\vec{u} : A}$$

By Substitution Lemma 3.2.16, $\Gamma, x : \vec{B} \vdash_{\mathsf{Ch}} s : A$ implies $\Gamma \vdash_{\mathsf{Ch}} s[\vec{u}/x^{\vec{B}}] : A$.
Otherwise, it can be $t = \lambda x^A.s$, where $s \to_{\mathsf{Ch}} s'$, or $t = s\,\vec{u}$, where either $s \to_{\mathsf{Ch}} s'$ or $\vec{u} \to_{\mathsf{Ch}} \vec{u}'$; in all these cases the proof follows by IH.

2. $\vec{t} \to_{\mathsf{Ch}} \vec{t}'$ means $\vec{t} = \{t_1, .., t_n\}$, and $\vec{t}' = \{t'_1, ..., t'_n\}$, and there is i such that $t_i \to_{\mathsf{Ch}} t'_i$, and $t'_j = t_j$, for $j \neq i$. Then the proof follows by IH. □

Proposition 3.2.19 (Confluence)**.** If $t \to^*_{\mathsf{Ch}} s$ and $t \to^*_{\mathsf{Ch}} r$ then there is u such that $s \to^*_{\mathsf{Ch}} u$ and $r \to^*_{\mathsf{Ch}} u$.

Proof. The proof follows the parallel reduction technique. The usual definitions are adapted to account for set-terms and set-types, with the interesting observation that the substitution lemma includes a typing condition, following the same requirement present in Definition 3.2.15. For the full development, see Appendix B.1. □

3.2.3 Correspondence between Curry and Church-style systems

A typable term in the Church-style system $\Lambda^{\mathsf{Ch}}_\cap$ does not necessarily represent a derivation tree in the Curry-style system $\Lambda^{\mathsf{Cu}}_\cap$. To relate these systems, we consider the following notion:

Definition 3.2.20 (Refinement)**.** We define a binary relation $t \sqsubset M$ between typable terms $t \in \Lambda^{\mathsf{Ch}}_\cap$ and untyped λ-terms $M \in \Lambda$, as well as a binary relation $\vec{t} \sqsubset M$ between typable set-terms $\vec{t} \subset \Lambda^{\mathsf{Ch}}_\cap$ and untyped λ-terms $M \in \Lambda$, according to the inductive rules below. In that case, we say that t is a *refinement* of M.

$$\frac{}{x^A \sqsubset x} \qquad \frac{t \sqsubset M}{\lambda x^A.t \sqsubset \lambda x.M} \qquad \frac{t \sqsubset M \quad \vec{s} \sqsubset N}{t\,\vec{s} \sqsubset M\,N} \qquad \frac{(t_i \sqsubset M)_{i \in 1..n} \quad n > 0}{\{t_1, \ldots, t_n\} \sqsubset M}$$

Observation 3.2.21.

1. If $t \sqsubset M_1$ and $t \sqsubset M_2$ then $M_1 = M_2$.
2. If $t \sqsubset M$ and $\vec{s} \sqsubset N$ then $t[\vec{s}/x^{\vec{A}}] \sqsubset M[N/x]$.

Definition 3.2.22 (Uniformity and type erasure)**.** A typable term $t \in \Lambda^{\mathsf{Ch}}_\cap$ is *uniform* if there exists $M \in \Lambda$ such that $t \sqsubset M$. Similarly, a typable set-term $\vec{t} \subseteq \Lambda^{\mathsf{Ch}}_\cap$ is uniform if there exists $M \in \Lambda$ such that $\vec{t} \sqsubset M$.

If $t \in \Lambda^{\mathsf{Ch}}_\cap$ is a uniform term, its *type erasure* $t^{\mathsf{e}} \in \Lambda$ is defined as the unique λ-term such that $t \sqsubset t^{\mathsf{e}}$. Similarly, if $\vec{t} \subset \Lambda^{\mathsf{Ch}}_\cap$ is a uniform set-term, $\vec{t}^{\mathsf{e}} \in \Lambda$ is defined as the unique λ-term such that $\vec{t} \sqsubset \vec{t}^{\mathsf{e}}$. This can be written as a recursive definition:

$$(x^A)^{\mathsf{e}} := x \qquad (\lambda x^{\vec{A}}.t)^{\mathsf{e}} := \lambda x.t^{\mathsf{e}} \qquad (t\,\vec{s})^{\mathsf{e}} := t^{\mathsf{e}}\,\vec{s}^{\mathsf{e}} \qquad \{s_1, \ldots, s_n\}^{\mathsf{e}} := s_1^{\mathsf{e}}$$

If $t \in \Lambda^{\mathsf{Ch}}_\cap$ is uniform, for each derivation $\Pi \rhd \Gamma \vdash_{\mathsf{Ch}} t : A$, we construct a derivation $\Pi^{\mathsf{e}} \rhd \Gamma \vdash_{\mathsf{Cu}} t^{\mathsf{e}} : A$ by erasing all the type annotations in terms. Similarly, each derivation $\Pi \rhd \Gamma \Vdash_{\mathsf{Ch}} \vec{t} : \vec{A}$ is mapped to a derivation $\Pi^{\mathsf{e}} \rhd \Gamma \Vdash_{\mathsf{Cu}} \vec{t}^{\mathsf{e}} : \vec{A}$.

Definition 3.2.23 (Type decoration)**.** For each derivation $\Pi \rhd \Gamma \vdash_{\mathsf{Cu}} M : A$, we construct a term $t \in \Lambda^{\mathsf{Ch}}_\cap$ and a derivation $\Pi^{\mathsf{i}} \rhd \Gamma \vdash_{\mathsf{Ch}} t : A$ inductively as below. Similarly, for each derivation

$\Pi \rhd \Gamma \Vdash_{\mathsf{Cu}} M : \vec{A}$ we construct a set-term $\vec{t} \in \Lambda_\cap^{\mathsf{Ch}}$ and a derivation $\Pi^{\mathtt{i}} \rhd \Gamma \Vdash_{\mathsf{Ch}} \vec{t} : \vec{A}$.

$$\frac{B \in \vec{A}}{\Gamma, x : \vec{A} \vdash_{\mathsf{Cu}} x : B}\ \mathsf{Cu\text{-}var} \qquad \rightsquigarrow \qquad \frac{B \in \vec{A}}{\Gamma, x : \vec{A} \vdash_{\mathsf{Ch}} x^B : B}\ \mathsf{Ch\text{-}var}$$

$$\frac{\Gamma, x : \vec{A} \vdash_{\mathsf{Cu}} M : B}{\Gamma \vdash_{\mathsf{Cu}} \lambda x.M : \vec{A} \to B}\ \mathsf{Cu\text{-}I}{\to} \qquad \rightsquigarrow \qquad \frac{\Gamma, x : \vec{A} \vdash_{\mathsf{Ch}} t : B}{\Gamma \vdash_{\mathsf{Ch}} \lambda x^{\vec{A}}.t : \vec{A} \to B}\ \mathsf{Ch\text{-}I}{\to}$$

$$\frac{\Gamma \vdash_{\mathsf{Cu}} M : \vec{A} \to B \quad \Gamma \Vdash_{\mathsf{Cu}} N : \vec{A}}{\Gamma \vdash_{\mathsf{Cu}} M\, N : B}\ \mathsf{Cu\text{-}E}{\to} \qquad \rightsquigarrow \qquad \frac{\Gamma \vdash_{\mathsf{Ch}} t : \vec{A} \to B \quad \Gamma \Vdash_{\mathsf{Ch}} \vec{s} : \vec{A}}{\Gamma \vdash_{\mathsf{Ch}} t\, \vec{s} : B}\ \mathsf{Ch\text{-}E}{\to}$$

$$\frac{(\Gamma \vdash_{\mathsf{Cu}} N : A_i)_{i \in 1..n}}{\Gamma \Vdash_{\mathsf{Cu}} N : \{A_1, \dots, A_n\}}\ \mathsf{Cu\text{-}many} \qquad \rightsquigarrow \qquad \frac{(\Gamma \vdash_{\mathsf{Ch}} s_i : A_i)_{i \in 1..n}}{\Gamma \Vdash_{\mathsf{Ch}} \{s_1, \dots, s_n\} : \{A_1, \dots, A_n\}}\ \mathsf{Ch\text{-}many}$$

Theorem 3.2.24 (Correspondence)**.**

1. If $\Pi \rhd \Gamma \vdash_{\mathsf{Cu}} M : A$, then there exists a uniform $t \in \Lambda_\cap^{\mathsf{Ch}}$ such that $\Pi^{\mathtt{i}} \rhd \Gamma \vdash_{\mathsf{Ch}} t : A$ and $t \sqsubset M$. Moreover, $(\Pi^{\mathtt{i}})^{\mathtt{e}} = \Pi$.
2. If $\Pi \rhd \Gamma \vdash_{\mathsf{Ch}} t : A$ and t is uniform then $\Pi^{\mathtt{e}} \rhd \Gamma \vdash_{\mathsf{Cu}} t^{\mathtt{e}} : A$. Moreover, $(\Pi^{\mathtt{e}})^{\mathtt{i}} = \Pi$.

Proof.

1. We generalize the statement to set-terms, *i.e.*, if $\Pi \rhd \Gamma \Vdash_{\mathsf{Cu}} M : \vec{A}$ then there exists a uniform $\vec{t} \in \Lambda_\cap^{\mathsf{Ch}}$ such that $\Pi^{\mathtt{i}} \rhd \Gamma \Vdash_{\mathsf{Ch}} \vec{t} : \vec{A}$ and $\vec{t} \sqsubset M$. We proceed by induction on the derivation Π. The only not immediate case is when $\Pi \rhd \Gamma \Vdash_{\mathsf{Cu}} M : \vec{A}$. Then this judgement has been obtained by rule $\mathsf{Cu\text{-}many}$ with premises $(\Gamma \vdash_{\mathsf{Cu}} M : A_i)_{i \in 1..n}$; by IH there are derivations proving $\Gamma \Vdash_{\mathsf{Cu}} t_i : A_i$, where $t_i \sqsubset M$. Then, by definition, $\{t_1, \dots, t_n\} \sqsubset M$, and the proof follows by rule $\mathsf{Ch\text{-}many}$. The other cases come directly from the definition of type decoration.
2. We generalize the statement to set-terms, *i.e.*, if $\Pi \rhd \Gamma \Vdash_{\mathsf{Ch}} \vec{t} : \vec{A}$ and $\vec{t}$ is uniform then $\Pi^{\mathtt{e}} \rhd \Gamma \Vdash_{\mathsf{Cu}} \vec{t^{\mathtt{e}}} : \vec{A}$. We proceed by induction on the derivation Π. The only not immediate case is when $\Pi \rhd \Gamma \Vdash_{\mathsf{Ch}} \vec{t} : \vec{A}$. Let $\vec{t} = \{t_1, \dots, t_n\}$, then $t \sqsubset M$ implies, by definition of uniformity, $t_i \sqsubset M$. The judgement has been obtained by rule $\mathsf{Ch\text{-}many}$, with premises $\Gamma \vdash_{\mathsf{Ch}} t_i : A_i$, so by IH, $\Gamma \vdash_{\mathsf{Cu}} t_i^{\mathtt{e}} : A_i$ where $t_i \sqsubset t_i^{\mathtt{e}}$. By Observation 3.2.21.1, $t_i \sqsubset t_i^{\mathtt{e}}$ and $t_i \sqsubset M$ imply $t_i^{\mathtt{e}} = M$, so $\Gamma \vdash_{\mathsf{Cu}} t : A_i$ and the result follows by applying rule $\mathsf{Cu\text{-}many}$. The other cases come directly from the definition of type erasure. □

Corollary 3.2.25. Given a term $M \in \Lambda$, the following are equivalent:

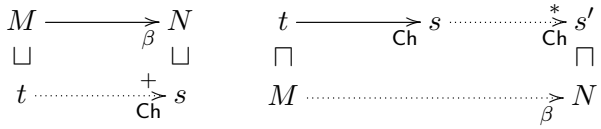
1. There exist Γ, A such that $\Gamma \vdash_{\mathsf{Cu}} M : A$ holds.
2. There exists a term $t \in \Lambda_\cap^{\mathsf{Ch}}$ such that $t \sqsubset M$.

Furthermore, $\to_\beta$ and $\to_{\mathsf{Ch}}$ simulate each other:

Theorem 3.2.26 (Simulation)**.**

1. If $M \to_\beta N$ and $t \sqsubset M$, there exists $s \in \Lambda_\cap^{\mathsf{Ch}}$ such that $t \to_{\mathsf{Ch}}^+ s$ and $s \sqsubset N$.
2. If $t \to_{\mathsf{Ch}} s$ and $t \sqsubset M$, there exist $N \in \Lambda$ and $s' \in \Lambda_\cap^{\mathsf{Ch}}$ such that $M \to_\beta N$ and $s \to_{\mathsf{Ch}}^* s'$ and $s' \sqsubset N$.

Graphically:



Proof.

1. We proceed by induction on M. The case $M = x$ cannot happen, in case $M = \lambda x.N$ the proof follows by IH. Let $M = P\,Q$, so $t = r\vec{u}$, where $r \sqsubset P$ and $\vec{u} \sqsubset Q$. The case $P \to_\beta P'$ is easy, by IH. Let $Q \to_\beta Q'$, and let $\vec{u} = \{u_1, ..., u_n\}$, where, by definition, $u_i \sqsubset Q$, for all $1 \leq i \leq n$. By IH, there are u_i' such that $u_i \to^+_{\mathsf{Ch}} u_i'$, such that $u_i' \sqsubset Q'$. So $\{u_1', ..., u_n'\} \sqsubset Q'$ and $s = r\{u_1', ..., u_n'\}$. Let $P = \lambda x.P'$, so $t = (\lambda x^A.t')\vec{u}$, where $t' \sqsubset P'$, and let $M \to_\beta P'[Q/x]$. By Observation 3.2.21.2, $t'[\vec{u}/x^A] \sqsubset P'[Q/x]$, and since $t \to^+_{\mathsf{Ch}} t'[\vec{u}/x^A]$, $s = t'[\vec{u}/x^A]$.

2. We generalize the statement to set-terms, *i.e.*, if $\vec{t} \to_{\mathsf{Ch}} \vec{s}$ and $\vec{t} \sqsubset M$, there exist N and $\vec{s}'$ such that $M \to_\beta N$ and $\vec{s} \to^*_{\mathsf{Ch}} \vec{s}'$ and $\vec{s}' \sqsubset N$. The proof is by induction on terms. Let $\vec{t} = \{t_1, \ldots, t_n, r\}$, and $\vec{s} = \{t_1, \ldots, t_n, r'\}$, where $r \to_{\mathsf{Ch}} r'$. By IH, there are N and r'' such that $M \to_\beta N$, $r' \to^*_{\mathsf{Ch}} r''$ and $r'' \sqsubset N$. Note that $t_i \sqsubset M$ for all $i \in 1..n$. Now let $i \in 1..n$. By item 1, $M \to_\beta N$ and $t_i \sqsubset M$ imply there is t_i' such that $t_i \to^*_{\mathsf{Ch}} t_i'$ and $t_i' \sqsubset N$; then it suffices to take $\vec{s}' = \{t_1', ..., t_n', r''\}$. The other cases follow by IH, using Observation 3.2.21.2 in case $t = (\lambda x^A.t')\,\vec{r}$, and $s = t'[\vec{r}/x^A]$. □

Note that, if N is a redex subterm of M, then a derivation tree for M in $\Lambda^{\mathsf{Cu}}_\cap$ may contain more than one subderivation with subject N. Contracting N with usual β-reduction corresponds to reducing in parallel *all* the occurrences of N in all the subderivations. In $\Lambda^{\mathsf{Ch}}_\cap$, this corresponds instead to performing the reductions one at a time.

Example 3.2.27. Let $M = (\lambda x.x\,x)\,(I\,I)$, $A = B \to B$, and $I = \lambda x.x$. Consider the derivation Π:

$$\dfrac{\dfrac{\dfrac{\dfrac{\vdots}{x : \{A \to A, A\} \vdash_{\mathsf{Cu}} x\,x : A}}{\vdash_{\mathsf{Cu}} \lambda x.x\,x : \{A \to A, A\} \to A}\ \mathsf{Cu\text{-}E}\to \quad \dfrac{\dfrac{\vdots}{\vdash_{\mathsf{Cu}} I\,I : A \to A} \quad \dfrac{\vdots}{\vdash_{\mathsf{Cu}} I\,I : A}}{\Vdash_{\mathsf{Cu}} I\,I : \{A \to A, A\}}\ \mathsf{Cu\text{-}many}}{\vdash_{\mathsf{Cu}} M : A}\ \mathsf{Cu\text{-}E}\to$$

Then we have that $\Pi^{\mathtt{i}} \rhd \vdash_{\mathsf{Ch}} t : A$, where $t = (\lambda x^{\{A\to A, A\}}.x\,x)\,\{(I\,I)^{A\to A}, (I\,I)^A\}$. Note that:

1. t is a uniform term, and in particular $t \sqsubset M$.
2. $t \to_{\mathsf{Ch}} t_1 = (\lambda x^{\{A\to A, A\}}.x\,x)\,\{I^{A\to A}, (I\,I)^A\}$, where t_1 is not a uniform term.
3. $t \to_{\mathsf{Ch}} t_3 = (I\,I)^{A\to A}\,(I\,I)^A$ where $t_3 \sqsubset I\,I(I\,I)$ and $M \to_\beta I\,I(I\,I)$.
4. $t_1 \to^*_{\mathsf{Ch}} t_2 = (\lambda x^{\{A\to A, A\}}.x\,x)\,\{I^{A\to A}, I^A\}$, where t_2 is a uniform term; in particular $t_2 \sqsubset (\lambda x.x\,x)\,I$ and $M \to_\beta (\lambda x.x\,x)\,I$.

Lemma 3.2.28 (Head subject expansion)**.** If $\Gamma \vdash_{\mathsf{Ch}} t[\vec{s}/x^{\vec{A}}]\vec{s}_1...\vec{s}_n : B$ and $\Gamma \Vdash_{\mathsf{Ch}} \vec{s} : \vec{A}$, then $\Gamma \vdash_{\mathsf{Ch}} (\lambda x^{\vec{A}}.t)\,\vec{s}\,\vec{s}_1...\vec{s}_n : B$.

Proof. By induction on n. Let $n = 0$, so $\Gamma \vdash_{\mathsf{Ch}} t[\vec{s}/x^{\vec{A}}] : B$. We continue by induction on t. The interesting case is that of $t = y^B$, where $B \in \vec{A}$. Then $\Gamma, x^{\vec{A}} \vdash_{\mathsf{Ch}} x^B : B$, by rule (Ch-var), and

$\Gamma \vdash_{\mathsf{Ch}} (\lambda x^{\vec{A}}.x^B)\,\vec{s} : B$ by rules (Ch-$\mathtt{I}\to$) and (Ch-$\mathtt{E}\to$). The remaining cases are straightforward, by IH and substitution definition. Let $n > 0$. Let $\Gamma \vdash_{\mathsf{Ch}} t[\vec{s}/x^{\vec{A}}]\vec{s}_1...\vec{s}_n\,\vec{r} : B$ and $\Gamma \Vdash_{\mathsf{Ch}} \vec{s} : \vec{A}$. Then, by the rules of the system, $\Gamma \vdash_{\mathsf{Ch}} t[\vec{s}/x^{\vec{A}}]\vec{s}_1...\vec{s}_n\,\vec{r} : B$ comes from $\Gamma \vdash_{\mathsf{Ch}} t[\vec{s}/x^{\vec{A}}]\vec{s}_1...\vec{s}_n : \vec{C} \to B$ and $\Gamma \Vdash_{\mathsf{Ch}} \vec{r} : \vec{C}$, for some $\vec{C}$. By IH, $\Gamma \vdash_{\mathsf{Ch}} (\lambda x^A.t)\,\vec{s}\,\vec{s}_1...\vec{s}_n : \vec{C} \to B$, and the result follows by rule (Ch-$\mathtt{E}\to$). □

Lemma 3.2.29 (Strong Normalization typability)**.** Let $M \in \Lambda$ be strongly normalizing. Then there is $t \in \Lambda_\cap^{\mathsf{Ch}}$ such that $t \sqsubset M$ and $\Gamma \vdash_{\mathsf{Ch}} t : A$, for some Γ and A.

Proof. The proof relies on the following inductive definition of the strongly normalizing terms (ISN) [107, §3].

$$\frac{N_1 \in \mathsf{ISN} \;\ldots\; N_n \in \mathsf{ISN}}{xN_1 \ldots N_n \in \mathsf{ISN}}\ \mathsf{ISN}_x \qquad \frac{M \in \mathsf{ISN}}{\lambda x.M \in \mathsf{ISN}}\ \mathsf{ISN}_\lambda \qquad \frac{M[N/x]\,P_1 \ldots P_n \in \mathsf{ISN} \quad N \in \mathsf{ISN}}{(\lambda x.M)\,N\,P_1 \ldots P_n \in \mathsf{ISN}}\ \mathsf{ISN}_@$$

1. ISN_x: By IH there are t_i such that $t_i \sqsubset N_i$ and $\Gamma_i \vdash_{\mathsf{Ch}} t_i : A_i$, so $\Gamma_i \Vdash_{\mathsf{Ch}} \vec{t_i} : \vec{A_i}$. So, by Lemma 3.2.10, and rule (Ch-$\mathtt{E}\to$), $(\bigcup_{1\le i\le n}\Gamma_i)\cup x : \{\vec{A}_1 \to ... \to \vec{A}_n \to B\} \vdash_{\mathsf{Ch}} x^{\{\vec{A}_1\to...\to\vec{A}_n\to B\}}\vec{t}_1\,...\vec{t}_n : B$.

2. ISN_λ: Follows directly from IH.

3. $\mathsf{ISN}_@$: Let $M[N/x]\,P_1...P_n$ and $N \in \mathsf{ISN}$. By IH, there exist $t, s, \Gamma, \Gamma', A, B$ such that $t \sqsubset M[N/x]\,P_1...P_n$, $s \sqsubset N$ and $\Gamma \vdash_{\mathsf{Ch}} t : B$, $\Gamma' \vdash_{\mathsf{Ch}} s : A$. Note that $\Gamma' \vdash_{\mathsf{Ch}} s : A$ implies $\Gamma' \Vdash_{\mathsf{Ch}} \vec{s} : \vec{A}$, where $\vec{s} = \{s\}$ and $\vec{A} = \{A\}$. $\Gamma \vdash_{\mathsf{Ch}} t : B$ and $t \sqsubset M[N/x]\,P_1...P_n$ imply $t = t'\,t_1...t_n$, where $t_i \sqsubset P_i$ and t' can be written as $t''[\vec{s}/x^{\vec{A}}]$. By Lemma 3.2.10, $\Gamma \cup \Gamma' \vdash_{\mathsf{Ch}} t''[\vec{s}/x^{\vec{A}}]\,t_1...t_n : B$ and $\Gamma \cup \Gamma' \Vdash_{\mathsf{Ch}} \vec{s} : \vec{A}$, and the result follows by Lemma 3.2.28.

□

3.3 Idempotent intersection types with memories

In this section, we extend the intrinsically typed system $\Lambda_\cap^{\mathsf{Ch}}$ with memories, obtaning a non-erasing variant of IIT. Concretely, we define $\Lambda_\cap^{\mathbf{m}}$, which adds *wrappers* $\langle\vec{t}\rangle$ to $\Lambda_\cap^{\mathsf{Ch}}$, and we study some technical properties that are needed later.

3.3.1 The memory calculus $\Lambda_\cap^{\mathbf{m}}$

The sets of terms ($t, s, \ldots$) and set-terms ($\vec{t}, \vec{s}, \ldots$) are given by the following grammar:

$$t ::= x^A \mid \lambda x^{\vec{A}}.t \mid t\,\vec{t} \mid t\langle\vec{t}\rangle \qquad\qquad \vec{t} ::= \{t_j\}_{j\in J}$$

where J stands for a finite set of indices. We write $\mathtt{L}$ for a list of wrappers (which we also refer to as *memories*), defined by the grammar $\mathtt{L} ::= \Box \mid \mathtt{L}\langle\vec{t}\rangle$, and $t\mathtt{L}$ for the term that results from extending t with all the wrappers in the list, *i.e.*, $t(\Box\langle\vec{s}_1\rangle \ldots \langle\vec{s}_n\rangle) = t\langle\vec{s}_1\rangle \ldots \langle\vec{s}_n\rangle$. Typing judgements are of the forms $\Gamma \vdash_{\mathbf{m}} t : A$ and $\Gamma \Vdash_{\mathbf{m}} \vec{t} : \vec{A}$, where types and typing contexts are defined as before. The typing rules extend the $\Lambda_\cap^{\mathsf{Ch}}$ type assignment system of Definition 3.2.4 with a typing rule **m**-`wrap`:

$$\frac{B \in \vec{A}}{\Gamma, x : \vec{A} \vdash_{\mathbf{m}} x^B : B}\ \mathbf{m}\text{-}\mathtt{var} \qquad \frac{\Gamma, x : \vec{A} \vdash_{\mathbf{m}} t : B}{\Gamma \vdash_{\mathbf{m}} \lambda x^{\vec{A}}.t : \vec{A} \to B}\ \mathbf{m}\text{-}\mathtt{I}\to$$

$$\frac{\Gamma \vdash_{\mathbf{m}} t : \vec{A} \to B \qquad \Gamma \Vdash_{\mathbf{m}} \vec{s} : \vec{A} \qquad \vec{A} \neq \varnothing}{\Gamma \vdash_{\mathbf{m}} t\,\vec{s} : B}\ \mathbf{m}\text{-E}\to \qquad \frac{\Gamma \vdash_{\mathbf{m}} t : A \quad \Gamma \Vdash_{\mathbf{m}} \vec{s} : \vec{B}}{\Gamma \vdash_{\mathbf{m}} t\langle \vec{s} \rangle : A}\ \mathbf{m}\texttt{-wrap}$$

$$\frac{(\Gamma \vdash_{\mathbf{m}} t_j : A_j)_{j\in J} \quad (\forall h,k \in J.\ h \neq k \implies A_h \neq A_k)}{\Gamma \Vdash_{\mathbf{m}} \{t_j\}_{j\in J} : \{A_j\}_{j\in J}}\ \mathbf{m}\texttt{-many}$$

We extend from $\Lambda_\cap^{\mathsf{Ch}}$ the properties and definitions of weakening, strengthening, minimal typing context, and type uniqueness.

Lemma 3.3.1 (Weakening and strengthening)**.**

1. If $\Gamma \vdash_{\mathbf{m}} t : A$ (resp. $\Gamma \Vdash_{\mathbf{m}} \vec{t} : \vec{A}$) and $\Gamma \subseteq \Delta$ then $\Delta \vdash_{\mathbf{m}} t : A$ (resp. $\Delta \Vdash_{\mathbf{m}} \vec{t} : \vec{A}$).
2. If $\Gamma, x : \{A\} \cup \vec{B} \vdash_{\mathbf{m}} t : C$ (resp. $\Gamma, x : \{A\} \cup \vec{B} \Vdash_{\mathbf{m}} \vec{t} : \vec{C}$) and $x \notin \mathsf{fv}_A(t)$, then $\Gamma' \vdash_{\mathbf{m}} t : C$ (resp. $\Gamma' \Vdash_{\mathbf{m}} \vec{t} : \vec{C}$), where Γ' is $(\Gamma, x : \vec{B})$ if $\vec{B} \neq \varnothing$, and Γ otherwise.

Proof. By induction on the derivations. We extend the proof of Lemma 3.2.10 by adding the wrappers cases.

1. $\dfrac{\Gamma \vdash_{\mathbf{m}} s : A \quad \Gamma \Vdash_{\mathbf{m}} \vec{r} : \vec{B}}{\Gamma \vdash_{\mathbf{m}} s\langle \vec{r} \rangle : A}\ \mathbf{m}\texttt{-wrap}$: Then by IH we know that $\Delta \vdash_{\mathbf{m}} s : A$ and $\Delta \Vdash_{\mathbf{m}} \vec{r} : \vec{B}$, therefore by **m**-`wrap` we conclude $\Delta \vdash_{\mathbf{m}} s\langle \vec{r} \rangle : A$.
2. $\dfrac{\Gamma, x : \{A\} \cup \vec{B} \vdash_{\mathbf{m}} s : C \quad \Gamma, x : \{A\} \cup \vec{B} \Vdash_{\mathbf{m}} \vec{r} : \vec{D}}{\Gamma, x : \{A\} \cup \vec{B} \vdash_{\mathbf{m}} s\langle \vec{r} \rangle : C}\ \mathbf{m}\texttt{-wrap}$: Then by IH we know that $\Gamma' \vdash_{\mathbf{m}} s : C$ and $\Gamma' \Vdash_{\mathbf{m}} \vec{r} : \vec{D}$, therefore by **m**-`wrap` we conclude $\Gamma' \vdash_{\mathbf{m}} s\langle \vec{r} \rangle : C$.

□

Definition 3.3.2 (Minimal typing context)**.** For each term t (resp. set-term $\vec{t}$), its *minimal typing context* is written $\gamma(t)$ (resp. $\gamma(\vec{t})$) and defined as follows:

$$\begin{array}{lcllcl} \gamma(x^A) & := & x : \{A\} & \gamma(\lambda x^{\vec{A}}.t) & := & \gamma(t) \setminus x \\ \gamma(t\vec{s}) & := & \gamma(t) \cup \gamma(\vec{s}) & \gamma(\{t_1, \ldots, t_n\}) & := & \cup_{i=1}^n \gamma(t_i) \\ \gamma(t\langle \vec{s} \rangle) & := & \gamma(t) \cup \gamma(\vec{s}) & & & \end{array}$$

where $\Gamma \setminus x$ is defined in such a way that $(\Gamma \setminus x)(x) = \varnothing$ and $(\Gamma \setminus x)(y) = \Gamma(y)$ for $x \neq y$.

Lemma 3.3.3 (Type uniqueness)**.** If $\Gamma_1 \vdash_{\mathbf{m}} t : A_1$ and $\Gamma_2 \vdash_{\mathbf{m}} t : A_2$ then $A_1 = A_2$. Furthermore, $\gamma(t) \subseteq \Gamma_1$, $\gamma(t) \subseteq \Gamma_2$, and $\gamma(t) \vdash_{\mathbf{m}} t : A_1$.

Proof. By induction on t. We extend the proof of Lemma 3.2.12 by adding the wrappers case. Let $t = s\langle \vec{r} \rangle$. Then we have $\Gamma_1 \vdash_{\mathbf{m}} s\langle \vec{r} \rangle : A_1$ and $\Gamma_2 \vdash_{\mathbf{m}} s\langle \vec{r} \rangle : A_2$. By inversion of the rule **m**-`wrap`, we have $\Gamma_1 \vdash_{\mathbf{m}} s : A_1$ and $\Gamma_1 \Vdash_{\mathbf{m}} \vec{r} : \vec{B_1}$ on one side, and $\Gamma_2 \vdash_{\mathbf{m}} s : A_2$ and $\Gamma_2 \Vdash_{\mathbf{m}} \vec{r} : \vec{B_2}$ on the other. By IH we have $A_1 = A_2$ and $\vec{B_1} = \vec{B_2}$. Furthermore, also by IH, we know that $\gamma(s) \subseteq \Gamma_1$, $\gamma(s) \subseteq \Gamma_2$, $\gamma(\vec{r}) \subseteq \Gamma_1$, and $\gamma(\vec{r}) \subseteq \Gamma_2$. So, by definition of minimal typing contexts (Definition 3.3.2), $\gamma(s\langle \vec{r} \rangle) = \gamma(s) \cup \gamma(\vec{r}) \subseteq \Gamma_1 \cup \Gamma_1 = \Gamma_1$ and $\gamma(s\langle \vec{r} \rangle) = \gamma(s) \cup \gamma(\vec{r}) \subseteq \Gamma_2 \cup \Gamma_2 = \Gamma_2$. Finally, again by IH, we have that $\gamma(s) \vdash_{\mathbf{m}} s : A_1$ and $\gamma(\vec{r}) \vdash_{\mathbf{m}} \vec{r} : \vec{B_1}$. By repeated application of weakening (Lemma 3.3.1), we obtain $\gamma(s) \cup \gamma(\vec{r}) \vdash_{\mathbf{m}} s : A_1$ and $\gamma(s) \cup \gamma(\vec{r}) \vdash_{\mathbf{m}} \vec{r} : \vec{B_1}$. Therefore, by rule **m**-`wrap` we conclude $\gamma(s) \cup \gamma(\vec{r}) = \gamma(s\langle \vec{r} \rangle) \vdash_{\mathbf{m}} s\langle \vec{r} \rangle : A_1$. □

Observation 3.3.4 (Bijection between a set-term and its set-type). Let $\Gamma \Vdash_{\mathbf{m}} \vec{s} : \vec{A}$. Given $A' \in \vec{A}$, we know by rule **m-many** that there exists a derivation $\Gamma \vdash_{\mathbf{m}} s' : A'$. This is the only derivation with type A' in the premises of the rule: otherwise, we would contradict the rule's side condition by having different derivations with the same type. Therefore, s' is unique. Given $s' \in \vec{s}$, we know by rule **m-many** that it has a type A'; by Lemma 3.3.3, this type is unique.

We extend the operation of capture-avoiding substitution $t[\vec{t}/x^{\vec{A}}]$ from Definition 3.2.15 by adding the wrappers case.

Definition 3.3.5 (Substitution in $\Lambda^{\mathbf{m}}_\cap$)**.** We define operations of capture-avoiding substitution for terms and set-terms by mutual recursion. Let $\Gamma \Vdash_{\mathbf{m}} \vec{s} : \vec{A}$ with $\vec{s} = \{s_i\}_{i\in I}$, $\vec{A} = \{A_i\}_{i\in I}$, and $\Gamma \vdash_{\mathbf{m}} s_i : A_i$ for all $i \in I$. we define $t[\vec{s}/x^{\vec{A}}]$ recursively as follows.

$$\begin{array}{rcl}
x^{A_i}[\vec{s}/x^{\vec{A}}] & := & s_i \\
y^B[\vec{s}/x^{\vec{A}}] & := & y^B \\
(\lambda y^{\vec{B}}.t)[\vec{s}/x^{\vec{A}}] & := & \lambda y^{\vec{B}}.t[\vec{s}/x^{\vec{A}}] \\
(t\,\vec{r})[\vec{s}/x^{\vec{A}}] & := & t[\vec{s}/x^{\vec{A}}]\,\vec{r}[\vec{s}/x^{\vec{A}}] \\
\{t_1,\dots,t_n\}[\vec{s}/x^{\vec{A}}] & := & \{t_1[\vec{s}/x^{\vec{A}}],\dots,t_n[\vec{s}/x^{\vec{A}}]\} \\
(t\langle\vec{r}\rangle)[\vec{s}/x^{\vec{A}}] & := & t[\vec{s}/x^{\vec{A}}]\langle\vec{r}[\vec{s}/x^{\vec{A}}]\rangle
\end{array}$$

Note that, in the first case, s_i exists and is unique due to Observation 3.3.4.

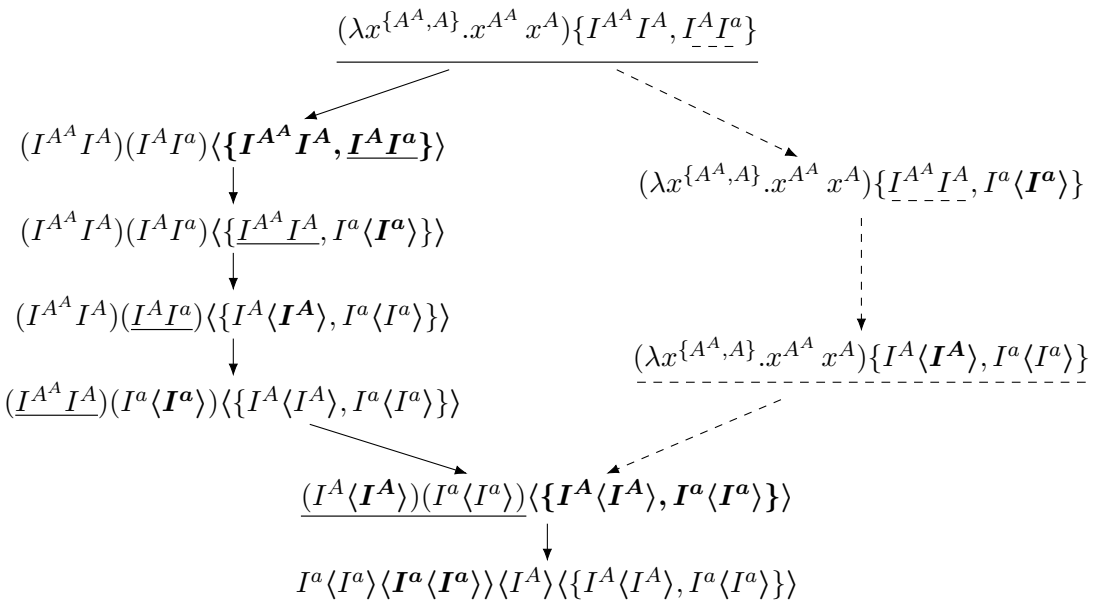
Definition 3.3.6 (Reduction in $\Lambda^{\mathbf{m}}_\cap$)**.** The relation of **m**-reduction is defined over terms and set-terms by the following rule, closed by congruence under arbitrary contexts:

$$(\lambda x^{\vec{A}}.t)\mathtt{L}\,\vec{s} \to_{\mathbf{m}} t[\vec{s}/x]\langle\vec{s}\rangle\mathtt{L}$$

Abstractions followed by memories, *i.e.*, terms of the form $(\lambda x.t)\mathtt{L}$, are called **m**-*abstractions*.

Remark 3.3.7. $t\langle\vec{s}\rangle$ is an **m**-abstraction if and only if t is an **m**-abstraction.

Example 3.3.8. In order to illustrate how the system works, and, especially, how wrappers are handled, we show two possible reductions for a term involving self-application and reductions inside the argument. We underline the contracted redex and highlight the new introduced memory at each step for clarity. Let $A = a \to a$; $A^A = A \to A$; $I^a = \lambda x^{\{a\}}.x$; $I^A = \lambda x^{\{A\}}.x$; $I^{A^A} = \lambda x^{\{A^A\}}.x$.



Example 3.3.9. The following reductions show how erased subterms are memorized by wrappers.

1. $\underline{(\lambda x^{\{A\}}.y^B)((\lambda x^{\{B\}}.z^A)w^B)} \quad \to_{\mathbf{m}} \quad y^B\langle \underline{(\boldsymbol{\lambda x^{\{B\}}.z^A})\boldsymbol{w^B}}\rangle \quad \to_{\mathbf{m}} \quad y^B\langle z^A\langle \boldsymbol{w^B}\rangle\rangle$
2. $\underline{(\lambda x^{\{A\}}.\lambda y^{\{B\}}.x)z^A}w^B \quad \to_{\mathbf{m}} \quad \underline{(\lambda y^{\{B\}}.z^A)\langle \boldsymbol{z^A}\rangle w^B} \quad \to_{\mathbf{m}} \quad z^A\langle \boldsymbol{w^B}\rangle\langle z^A\rangle$

Remark 3.3.10. Each step $t \to_{\mathsf{Ch}} s$ in the $\Lambda_\cap^{\mathsf{Ch}}$-calculus (without wrappers) has a *corresponding* step $t \to_{\mathbf{m}} s'$ in the $\Lambda_\cap^{\mathbf{m}}$-calculus (with wrappers), contracting the same redex but creating one wrapper. For example, the step $(\lambda x.z\,x\,x)\,I \to_{\mathsf{Ch}} z\,I\,I$ has a corresponding step $(\lambda x.z\,x\,x)\,I \to_{\mathbf{m}} (z\,I\,I)\langle I\rangle$. In particular an erasing reduction step in $\Lambda_\cap^{\mathsf{Ch}}$ is mapped a non-erasing one in $\Lambda_\cap^{\mathbf{m}}$. For example, $(\lambda x^{\vec{A}}.z^B)\,y^{\vec{A}} \to_{\mathsf{Ch}} z^B$, while $(\lambda x^{\vec{A}}.z^B)\,y^{\vec{A}} \to_{\mathbf{m}} z^B\langle y^{\vec{A}}\rangle$.

A *redex* is an expression matching the left-hand side of the **m** rule, which must be an applied **m**-abstraction. The *height* of a type A (resp. set-type $\vec{A}$) is written $\mathsf{h}(A)$ (resp. $\mathsf{h}(\vec{A})$) and defined as follows:

$$\begin{array}{rcl} \mathsf{h}(a) & := & 0 \\ \mathsf{h}(\vec{A} \to B) & := & 1 + \max\{\mathsf{h}(\vec{A}), \mathsf{h}(B)\} \\ \mathsf{h}(\{A_1, \dots, A_n\}) & := & \max\{\mathsf{h}(A_1), \dots, \mathsf{h}(A_n)\} \end{array}$$

We write $\mathsf{type}(t)$ for the type of t, which is unique for typable terms by Lemma 3.3.3. The *degree* of an **m**-abstraction is the height of its type. The degree of a redex is the degree of its **m**-abstraction. The *max-degree* of a term $t \in \Lambda_\cap^{\mathbf{m}}$ is written $\delta_{\mathsf{max}}(t)$ and defined as the maximum degree of the redexes in t, or 0 if t has no redexes. This notion is also defined for set-terms ($\delta_{\mathsf{max}}(\vec{t})$) and lists of wrappers ($\delta_{\mathsf{max}}(\mathtt{L})$) in a similar way. The *weight* of a term $t \in \Lambda_\cap^{\mathbf{m}}$ is written $\mathsf{w}(t)$ and defined as the number of wrappers in t.

Lemma 3.3.11 (Substitution)**.** Let $\Gamma, x : \vec{A} \vdash_{\mathbf{m}} t : B$ and $\Gamma \Vdash_{\mathbf{m}} \vec{s} : \vec{A}$. Then $\Gamma \vdash_{\mathbf{m}} t[\vec{s}/x^{\vec{A}}] : B$.

Proof. By induction on t, generalizing the statement to set-terms. We consider only the wrapper case, since the remaining cases are exactly like those in Lemma 3.2.16. Let $t = t_1\langle \vec{t_2}\rangle$. Then $\Gamma, x : \vec{A} \vdash_{\mathbf{m}} t_1\langle \vec{t_2}\rangle : B$ concludes with **m-wrap**, so by inversion we have that $\Gamma, x : \vec{A} \vdash_{\mathbf{m}} t_1 : B$ and $\Gamma, x : \vec{A} \Vdash_{\mathbf{m}} \vec{t_2} : \vec{C}$. By IH, $\Gamma \vdash_{\mathbf{m}} t_1[\vec{s}/x^{\vec{A}}] : B$ and $\Gamma \Vdash_{\mathbf{m}} \vec{t_2}[\vec{s}/x^{\vec{A}}] : \vec{C}$. By **m-wrap** it follows that $\Gamma \vdash_{\mathbf{m}} t_1[\vec{s}/x^{\vec{A}}]\langle \vec{t_2}[\vec{s}/x^{\vec{A}}]\rangle : B$. By definition of substitution $(t_1\langle \vec{t_2}\rangle)[\vec{s}/x^{\vec{A}}] = t_1[\vec{s}/x^{\vec{A}}]\langle \vec{t_2}[\vec{s}/x^{\vec{A}}]\rangle$, hence we conclude that $\Gamma \vdash_{\mathbf{m}} (t_1\langle \vec{t_2}\rangle)[\vec{s}/x^{\vec{A}}] : B$. □

Lemma 3.3.12 (Subject reduction)**.** If $t \to_{\mathbf{m}} t'$ and $\Gamma \vdash_{\mathbf{m}} t : A$, then $\Gamma \vdash_{\mathbf{m}} t' : A$.

Proof. Simultaneously by induction on the reduction relation $\to_{\mathbf{m}}$. We consider only the base case and the congruence on wrappers cases. The remaining ones are straightforward by IH.

1. In the case $t = (\lambda x^{\vec{B}}.s)\mathtt{L}\,\vec{u} \to_{\mathbf{m}} s[\vec{u}/x^{\vec{B}}]\langle \vec{u}\rangle\mathtt{L} = t'$ we have that t must be typed with the following derivation scheme, where n is the amount of wrappers in $\mathtt{L}$:

$$\dfrac{\dfrac{\dfrac{\dfrac{\Gamma, x : \vec{B} \vdash_{\mathbf{m}} s : C}{\Gamma \vdash_{\mathbf{m}} \lambda x^{\vec{B}}.s : \vec{B} \to C}\,\mathbf{m}\text{-}\mathtt{I}\to \qquad \Pi_1}{\vdots \qquad \Pi_n}\,\mathbf{m}\text{-}\mathtt{wrap}}{\Gamma \vdash_{\mathbf{m}} \lambda x^{\vec{B}}.s\mathtt{L} : \vec{B} \to C}\,\mathbf{m}\text{-}\mathtt{wrap} \qquad \Gamma \Vdash_{\mathbf{m}} \vec{u} : \vec{B}}{\Gamma \vdash_{\mathbf{m}} (\lambda x^{\vec{B}}.s)\mathtt{L}\,\vec{u} : C}\,\mathbf{m}\text{-}\mathtt{E}\to$$

   Hence from Lemma 3.3.11, since $\Gamma, x : \vec{B} \vdash_{\mathbf{m}} s : C$ and $\Gamma \Vdash_{\mathbf{m}} \vec{u} : \vec{B}$, we obtain $\Gamma \vdash_{\mathbf{m}} s[\vec{u}/x^{\vec{B}}] : C$. Then from the corresponding $n + 1$ applications of the **m-wrap** we obtain $\Gamma \vdash_{\mathbf{m}} s[\vec{u}/x^{\vec{B}}]\langle \vec{u}\rangle\mathtt{L} : C$.

2. If $t = s\langle\vec{u}\rangle \to_{\mathbf{m}} s'\langle\vec{u}\rangle = t'$ with $s \to_{\mathbf{m}} s'$, then we have that $\Gamma \vdash_{\mathbf{m}} s\langle\vec{u}\rangle : A$ and, by inversion of **m-wrap**, it follows that $\Gamma \vdash_{\mathbf{m}} s : A$ and $\Gamma \Vdash_{\mathbf{m}} \vec{u} : \vec{B}$. Then by IH we have that $\Gamma \vdash_{\mathbf{m}} s' : A$, and by **m-wrap** we conclude $\Gamma \vdash_{\mathbf{m}} s'\langle\vec{u}\rangle : A$.

3. If $t = s\langle\vec{u}\rangle \to_{\mathbf{m}} s\langle\vec{u}'\rangle = t'$ with $\vec{u} \to_{\mathbf{m}} \vec{u}'$, then we have that $\Gamma \vdash_{\mathbf{m}} s\langle\vec{u}\rangle : A$ and, by inversion of **m-wrap**, it follows that $\Gamma \vdash_{\mathbf{m}} s : A$ and $\Gamma \Vdash_{\mathbf{m}} \vec{u} : \vec{B}$. Then by IH we have that $\Gamma \Vdash_{\mathbf{m}} \vec{u}' : \vec{B}$, and by **m-wrap** we conclude $\Gamma \vdash_{\mathbf{m}} s\langle\vec{u}'\rangle : A$. □

Proposition 3.3.13 (Confluence)**.** If $t_1 \to^*_{\mathbf{m}} t_2$ and $t_1 \to^*_{\mathbf{m}} t_3$, there exists a term t_4 such that $t_2 \to^*_{\mathbf{m}} t_4$ and $t_3 \to^*_{\mathbf{m}} t_4$.

Proof. The proof follows the parallel reduction technique, combining the inclusion of wrappers from $\lambda^{\mathbf{m}}$ in Section 2.3 and set-terms, set-types and typed substitution from $\Lambda^{\mathsf{Ch}}_\cap$ in Section 3.2. For the full development, see Appendix B.2. □

3.4 $\mathcal{W}_\cap$-measure: counting wrappers in $\Lambda^{\mathbf{m}}_\cap$

In the previous sections, we made adaptations to the usual extrinsic presentation of idempotent intersection types, $\Lambda^{\mathsf{Cu}}_\cap$, introducing an intrinsic variant, $\Lambda^{\mathsf{Ch}}_\cap$. These adaptations are necessary in order to apply the technique of simplifying and counting wrappers, due to redex degrees depending on the type of functions, and type inference not being decidable in $\Lambda^{\mathsf{Cu}}_\cap$. We then extended the calculus with wrappers, obtaining $\Lambda^{\mathbf{m}}_\cap$.

In this section, we prove strong normalization of the $\Lambda^{\mathsf{Ch}}_\cap$ calculus by providing an explicit decreasing measure, adapting the ideas behind the $\mathcal{W}$ measure from Chapter 2 to the setting of idempotent intersection type systems. We name the new measure $\mathcal{W}_\cap$.

For better readability, we replicate the proof structure of Section 2.4. Proofs were adapted mostly in two places: terms or relations involving substitution, since substitution is now a more complex function than in simply typed λ–calculus, and terms involving the new syntactic construction, *i.e.*, applications and proper set-terms. For most proofs, we avoid repeating details already present in Section 2.4, focusing on the changed parts.

This section is divided in three subsections. In Section 3.4.1, we introduce the simplification operation, which the key of the measure computation. In Section 3.4.2, we introduce forgetful reduction, the key of the measure's decreasement proof. In Section 3.4.3, we define the measure and we prove that it decreases.

3.4.1 Simplification

We now introduce the operations of *simplification* and *full simplification* of a term. They are defined analogously to those of $\lambda^{\mathbf{m}}$ from Section 2.4. We recall that simplification is a function defined by structural recursion on terms and parameterized on a redex degree, that computes the result of contracting all the redexes of the given degree. Note that the result coincides with that of the complete development of the set of redexes of a degree. The *full simplification* iteratively applies the simplification of a term, starting with the maximum degree and decreasing down to degree 1. Both operations are well-defined by recursion: simplification on the structure of terms, and full simplification on the maximum degree of the term.

Definition 3.4.1 (Simplification)**.** For each integer $d \geq 1$ we define an operation written $\mathtt{S}^{\cap}_d(-)$, called *simplification of degree d*, that can be applied on terms ($\mathtt{S}^{\cap}_d(t)$), set-terms ($\mathtt{S}^{\cap}_d(\vec{t})$), and lists

of wrappers ($\mathtt{S}^\cap_d(\mathtt{L})$), mutually recursively as follows:

$$\begin{array}{rcll}
\mathtt{S}^\cap_d(x^A) & := & x^A & \\
\mathtt{S}^\cap_d(\lambda x^{\vec{A}}.t) & := & \lambda x^{\vec{A}}.\mathtt{S}^\cap_d(t) & \\
\mathtt{S}^\cap_d(t\,\vec{s}) & := & \begin{cases} \mathtt{S}^\cap_d(t')[\mathtt{S}^\cap_d(\vec{s})/x^{\vec{A}}]\langle \mathtt{S}^\cap_d(\vec{s})\rangle \mathtt{S}^\cap_d(\mathtt{L}) & \text{if } t = (\lambda x^{\vec{A}}.t')\mathtt{L} \text{ of degree } d \\ \mathtt{S}^\cap_d(t)\,\mathtt{S}^\cap_d(\vec{s}) & \text{otherwise} \end{cases} & \\
\mathtt{S}^\cap_d(t\langle\vec{s}\rangle) & := & \mathtt{S}^\cap_d(t)\langle \mathtt{S}^\cap_d(\vec{s})\rangle & \\
\mathtt{S}^\cap_d(\{t_1,\dots,t_n\}) & := & \{\mathtt{S}^\cap_d(t_1),\dots,\mathtt{S}^\cap_d(t_n)\} & \\
\mathtt{S}^\cap_d(\Box\langle\vec{t_1}\rangle\dots\langle\vec{t_n}\rangle) & := & \Box\langle \mathtt{S}^\cap_d(\vec{t_1})\rangle\dots\langle \mathtt{S}^\cap_d(\vec{t_n})\rangle &
\end{array}$$

If $t \in \Lambda^{\mathbf{m}}_\cap$ is a term and $D = \delta_{\mathsf{max}}(t)$, the *full simplification of* t is written $\mathtt{S}^\cap_*(t)$ and defined as $\mathtt{S}^\cap_*(t) := \mathtt{S}^\cap_1(\dots \mathtt{S}^\cap_{D-1}(\mathtt{S}^\cap_D(t))\dots)$.

Lemma 3.4.2 (Soundness of simplification)**.** If $t \in \Lambda^{\mathbf{m}}_\cap$ and $d \geq 1$ then $t \to^*_{\mathbf{m}} \mathtt{S}^\cap_d(t)$.

Proof. Straightforward by mutual induction on the term t, set-term $\vec{t}$ and list of wrappers $\mathtt{L}$, generalizing the statement to set-terms and lists of wrappers, *i.e.*, $\vec{t} \to^*_{\mathbf{m}} \mathtt{S}^\cap_d(\vec{t})$ and $\mathtt{L} \to^*_{\mathbf{m}} \mathtt{S}^\cap_d(\mathtt{L})$. □

Lemma 3.4.3 (Non-abstraction substitution preserves greater non-abstraction)**.** If $\mathsf{h}(\mathsf{type}(t)) > \mathsf{h}(\mathsf{type}(\vec{s}))$, $\Gamma \Vdash_{\mathbf{m}} \vec{s} : \vec{A}$, $\Gamma, x : \vec{A} \vdash_{\mathbf{m}} t : B$, and t is not an **m**-abstraction, then $t[\vec{s}/x]$ is not an **m**-abstraction.

Proof. By induction on t, analogous to Lemma 2.4.5. We show the interesting cases, which are those of t being the substituted variable, due to the changes in the definition of substitution, and of memories. Notice that we do not generalize the lemma to set-terms, so that case is not considered.

1. $t = y = x$: Note that variables are decorated with its type, with y being x^{A_0}, and x being $x^{\vec{A}}$. We know by Observation 3.2.14 that there exists $s_0 \in \vec{s}$ such that $\Gamma \vdash_{\mathbf{m}} s_0 : A_0$, therefore $x^{A_0}[\vec{s}/x^{\vec{A}}] = s_0$. But, since variables have the same type of the terms they are substituted for, we would have that $\mathsf{h}(\mathsf{type}(t)) = \mathsf{h}(\mathsf{type}(x^{A_0})) = \mathsf{h}(tA_0) = \mathsf{h}(\mathsf{type}(s_0)) \leq \mathsf{h}(\mathsf{type}(\vec{s}))$, which contradicts the hypothesis. Hence this case is impossible.

2. $t = t_1\langle\vec{t_2}\rangle$: Note that, by hypothesis and Remark 3.3.7, t_1 is not an **m**-abstraction. Furthermore, we have $\mathsf{type}(t_1) = \mathsf{type}(t)$, s $\mathsf{h}(\mathsf{type}(t_1)) = \mathsf{h}(\mathsf{type}(t)) > \mathsf{h}(\mathsf{type}(\vec{s}))$. Therefore we can apply the IH on t_1, and we get that $t_1[\vec{s}/x]$ is not an **m**-abstraction. Finally, we have that $(t_1\vec{t_2})[\vec{s}/x] = t_1[\vec{s}/x]\langle t_2[\vec{s}/x]\rangle$ is not an **m**-abstraction by Remark 3.3.7.

□

Lemma 3.4.4 (High simplification preserves non-abstraction)**.** If $t \in \Lambda^{\mathbf{m}}_\cap$ is not an **m**-abstraction and $\mathsf{h}(\mathsf{type}(t)) \geq d \geq \delta_{\mathsf{max}}(t)$, then $\mathtt{S}^\cap_d(t)$ is not an **m**-abstraction.

Proof. By induction on t, where the interesting case is that of t being a redex of degree d. As in $\lambda^{\mathbf{m}}$, this case is impossible. Let $t = (\lambda x^{\vec{A}}.s)\mathtt{L}\,\vec{r}$, where $\lambda x^{\vec{A}}.s$ is of type $\vec{A} \to B$ so that t is of type B. The degree of the redex is $\mathsf{h}(\vec{A} \to B) > \mathsf{h}(B) = \mathsf{h}(\mathsf{type}(t))$, contradicting the hypothesis $\mathsf{h}(\mathsf{type}(t)) \geq d$. The remaining cases are straightforward resorting to the IH. □

Lemma 3.4.5 (Strict upper bound for max-deg is stable under substitution)**.** Let $\Gamma \Vdash_{\mathbf{m}} \vec{s} : \vec{A}$, where $\delta_{\mathsf{max}}(\vec{s}) < d$ and $\mathsf{h}(\vec{A}) < d$. If $\Gamma, x : \vec{A} \vdash_{\mathbf{m}} t : B$ and $\delta_{\mathsf{max}}(t) < d$ then $\delta_{\mathsf{max}}(t[\vec{s}/x^{\vec{A}}]) < d$.

Proof. We generalize the statement for set-terms, claiming that $\Gamma, x : \vec{A} \Vdash_{\mathbf{m}} \vec{t} : \vec{B}$ and $\delta_{\mathsf{max}}(\vec{t}) < d$ imply $\delta_{\mathsf{max}}(\vec{t}[\vec{s}/x^{\vec{A}}]) < d$. The proof proceeds by simultaneous induction on the structure of the term t or set-term $\vec{t}$. The interesting cases are when the term is a variable, an application, or a set-term.

1. $t = y^C$: we consider two subcases, depending on whether $y = x$ or not:

   1.1. $x = y$: Then $C = B$ and since $\Gamma, x : \vec{A} \vdash_{\mathbf{m}} x : B$ holds, we have that $B \in \vec{A}$. Hence there is a term $s_0 \in \vec{s}$ such that $\Gamma \vdash_{\mathbf{m}} s_0 : B$, and $x^B[\vec{s}/x^{\vec{A}}] = s_0$. In particular, by definition we have that $\delta_{\mathsf{max}}(s_0) \leq \delta_{\mathsf{max}}(\vec{s})$, and by hypothesis we have that $\delta_{\mathsf{max}}(\vec{s}) < d$, so $\delta_{\mathsf{max}}(t[\vec{s}/x^{\vec{A}}]) = \delta_{\mathsf{max}}(x^B[\vec{s}/x^{\vec{A}}]) = \delta_{\mathsf{max}}(s_0) < d$, as required.

   1.2. $x \neq y$: Then $\delta_{\mathsf{max}}(t[\vec{s}/x^{\vec{A}}]) = \delta_{\mathsf{max}}(y^C[\vec{s}/x^{\vec{A}}]) = \delta_{\mathsf{max}}(y^C) = \delta_{\mathsf{max}}(t) < d$.

2. $t = t_1\, \vec{t_2}$: Note that t_1 is a subterm of t so $\delta_{\mathsf{max}}(t_1) \leq \delta_{\mathsf{max}}(t) < d$, and similarly $\delta_{\mathsf{max}}(\vec{t_2}) \leq \delta_{\mathsf{max}}(t) < d$. Thus applying the IH on each subterm we have that $\delta_{\mathsf{max}}(t_1[\vec{s}/x^{\vec{A}}]) < d$ and that $\delta_{\mathsf{max}}(\vec{t_2}[\vec{s}/x^{\vec{A}}]) < d$. We proceed by case analysis, depending on whether $\mathsf{h}(\mathsf{type}(t_1)) < d$ or $\mathsf{h}(\mathsf{type}(t_1)) \geq d$:

   2.1. $\mathsf{h}(\mathsf{type}(t_1)) < d$: Then, if $t_1[\vec{s}/x]$ is an abstraction, we have that $t_1[\vec{s}/x]\vec{t_2}[\vec{s}/x]$ is a redex of degree lower than d. Without loss of generality, then, we can assume it is. Hence

   $$\begin{array}{rcl} \delta_{\mathsf{max}}((t_1\, \vec{t_2})[\vec{s}/x]) & = & \delta_{\mathsf{max}}(d-1, t_1[\vec{s}/x]\, \vec{t_2}[\vec{s}/x]) \\ & \leq & \max\big(d-1, \delta_{\mathsf{max}}(t_1[\vec{s}/x]),\ \delta_{\mathsf{max}}(\vec{t_2}[\vec{s}/x])\big) \\ & < & d \end{array}$$

   2.2. If $\mathsf{h}(\mathsf{type}(t_1)) \geq k$, note that t_1 cannot be an $\mathbf{m}$-abstraction, because then $t = t_1\, \vec{t_2}$ would be a redex of degree d or greater, and by hypothesis we know that $\delta_{\mathsf{max}}(t) < d$. Note that we are under the conditions of Lemma 3.4.3, *i.e.*, $\mathsf{h}(\mathsf{type}(t_1)) > \mathsf{h}(\mathsf{type}(\vec{s}))$, $\Gamma \Vdash_{\mathbf{m}} \vec{s} : \vec{A}$, $\Gamma, x : \vec{A} \vdash_{\mathbf{m}} t_1 : B$, and t_1 is not an $\mathbf{m}$-abstraction, so we can conclude that $t_1[\vec{s}/x]$ is not an $\mathbf{m}$-abstraction. Hence $t_1[s/x]\, \vec{t_2}[s/x]$ cannot be a redex. Finally we have

   $$\begin{array}{rcl} \delta_{\mathsf{max}}((t_1\, \vec{t_2})[\vec{s}/x]) & = & \delta_{\mathsf{max}}(t_1[\vec{s}/x]\, \vec{t_2}[\vec{s}/x]) \\ & \leq & \max\big(\delta_{\mathsf{max}}(t_1[\vec{s}/x]),\ \delta_{\mathsf{max}}(\vec{t_2}[\vec{s}/x])\big) \\ & < & d \end{array}$$

3. $\vec{t} = \{t_1, \ldots, t_n\}$: For all $t' \in t$ we have $\delta_{\mathsf{max}}(t') \leq \delta_{\mathsf{max}}(t) < d$, and for all $B' \in \vec{B}$ we have $\mathsf{h}(B') \leq \mathsf{h}(\vec{B}) < d$. We can thus apply the IH on each t', obtaining $t'[\vec{s}/x^{\vec{A}}] < d$, and therefore $\vec{t}[\vec{s}/x^{\vec{A}}] < d$.

The remaining cases are straightforward by resorting to the IH. □

Lemma 3.4.6 (Simplification of max-degree decreases the max-degree)**.** If $d \geq 1$ and $\delta_{\mathsf{max}}(t) \leq d$ then $\delta_{\mathsf{max}}(\mathsf{S}^{\cap}_d(t)) < d$.

Proof. This proof is analogous to its correspondent in the original measure (Lemma 2.4.9). It goes by simultaneous induction on t, generalizing the statement to set-terms ($\delta_{\mathsf{max}}(\vec{t}) \leq d$ implies $\delta_{\mathsf{max}}(\mathsf{S}^{\cap}_d(\vec{t})) < d$). The case of $t = \{t_1, \ldots, t_n\}$ follows from IH, noting that for all $t' \in t$ we have $\delta_{\mathsf{max}}(t') \leq \delta_{\mathsf{max}}(t) \leq d$, so by IH it holds that $\delta_{\mathsf{max}}(\mathsf{S}^{\cap}_d(t')) < d$, therefore $\delta_{\mathsf{max}}(\mathsf{S}^{\cap}_d(\{t_1, \ldots, t_n\})) = \max\{\delta_{\mathsf{max}}(\mathsf{S}^{\cap}_d(t_1)), \ldots, \delta_{\mathsf{max}}(\mathsf{S}^{\cap}_d(t_n))\} < d$. □

Proposition 3.4.7 (Full simplification yields the normal form)**.** Let $t \in \Lambda_\cap^{\mathbf{m}}$. Then $t \to_{\mathbf{m}}^* \mathtt{S}_*^\cap(t)$ and $\mathtt{S}_*^\cap(t)$ is in $\to_{\mathbf{m}}$-normal form.

Proof. The proof is analogous to Proposition 2.4.10. The first part is proven by repeated application of Lemma 3.4.2, which gives us $t \to_{\mathbf{m}}^* \mathtt{S}_D^\cap(t) \to_{\mathbf{m}}^* \mathtt{S}_{D-1}^\cap(\mathtt{S}_D^\cap(t)) \dots \to_{\mathbf{m}}^* \mathtt{S}_1^\cap(\dots \mathtt{S}_{D-1}^\cap(\mathtt{S}_D^\cap(t))) = \mathtt{S}_*^\cap(t)$, where D is the max-degree of t. The second part is proven by repeatedly applying Lemma 3.4.6, allowing us to obtain first $\delta_{\mathsf{max}}(\mathtt{S}_D^\cap(t)) < D$ and finally $\delta_{\mathsf{max}}(\mathtt{S}_1^\cap(\dots \mathtt{S}_{D-1}^\cap(\mathtt{S}_D^\cap(t)))) = \delta_{\mathsf{max}}(\mathtt{S}_*^\cap(t)) < 1$, ensuring $\to_{\mathbf{m}}$-normal form of $\mathtt{S}_*^\cap(t)$. □

Note that the previous lemmas partially imply Turing's observation for $\Lambda_\cap^{\mathbf{m}}$: contracting a redex of max-degree that has no occurrences of equal degree redexes does not create redexes of greater or equal degrees. This weaker variant of the observation is enough for the measure defined in this chapter.

3.4.2 Forgetful reduction

Definition 3.4.8 (Forgetful reduction)**.** We define a binary relation $t \rhd s$ between typable terms as the closure by congruence under arbitrary contexts of the axiom $t\langle \vec{s} \rangle \rhd t$.

Remark 3.4.9. If t is in $\to_{\mathbf{m}}$-normal form and $t \rhd s$ then s is also in $\to_{\mathbf{m}}$-normal form.

Remark 3.4.10. If $t \rhd s$ then $\mathsf{w}(t) > \mathsf{w}(s)$.

Lemma 3.4.11 (Forgetful reduction commutes with reduction)**.** If $t_1 \to_{\mathbf{m}}^* t_2$ and $t_1 \rhd^+ t_3$ there exists t_4 such that $t_2 \rhd^+ t_4$ and $t_3 \to_{\mathbf{m}}^* t_4$.

Proof. It suffices to prove the following local commutation result:

$$\begin{array}{ccc} t_1 & \to_{\mathbf{m}} & t_2 \\ \triangledown & & \triangledown^+ \\ t_3 & \to_{\mathbf{m}}^= & t_4 \end{array}$$

We proceed by induction on t_1. The interesting cases are those involving set-terms, *i.e.*, applications and proper set-terms. See Appendix B for the detailed proof.

1. $t_1 = t_{11}\,\vec{r}$. We consider two subcases here, depending on t_{11} being an **m**-abstraction and on which redex is contracted.

   1.1. $t_{11} = (\lambda x^{\vec{A}}.s)\mathtt{L}$ and the contracted redex is at head position. Then

   $$\begin{array}{ccc} (\lambda x^{\vec{A}}.s)\mathtt{L}\,\vec{r} & \to_{\mathbf{m}} & s[\vec{r}/x^{\vec{A}}]\langle \vec{r} \rangle \mathtt{L} \\ \triangledown & & \\ (\lambda x^{\vec{A}}.s')\mathtt{L}'\,\vec{r}' & & \end{array}$$

   where since there is a single $\rhd$-step, only one of s', $\mathtt{L}'$, or $\vec{r}'$, can be different from s, $\mathtt{L}$, or $\vec{r}$, respectively. If the $\rhd$-step occurs in s or $\mathtt{L}$, then the same step can be reproduced from t_2. Else, considering the terms within $\vec{r}$ can appear more than once in t_2, the $\rhd$-step may have to be reproduced several times, hence the $\rhd^+$. Then we have that

   $$\begin{array}{ccc} (\lambda x^{\vec{A}}.s)\mathtt{L}\,\vec{r} & \to_{\mathbf{m}} & s[\vec{r}/x^{\vec{A}}]\langle \vec{r} \rangle \mathtt{L} \\ \triangledown & & \triangledown^+ \\ (\lambda x^{\vec{A}}.s')\mathtt{L}'\,\vec{r}' & \to_{\mathbf{m}} & s'[\vec{r}'/x^{\vec{A}}]\langle \vec{r}' \rangle \mathtt{L}' \end{array}$$

1.2. t_{11} is not an $\mathbf{m}$-abstraction or the contracted redex is not the one at head position. We consider two subcases.

1.2.1. Both steps occur in the same subterm. It suffices to resort to the IH:

$$\begin{array}{ccc} s\vec{r} & \to_{\mathbf{m}} & s'\vec{r} \\ \triangledown & & \triangledown^{+} \\ s''\vec{r} & \to_{\mathbf{m}}^{=} & s'''\vec{r} \end{array} \qquad \begin{array}{ccc} s\vec{r} & \to_{\mathbf{m}} & s\vec{r}' \\ \triangledown & & \triangledown^{+} \\ s\vec{r}'' & \to_{\mathbf{m}}^{=} & s\vec{r}''' \end{array}$$

1.2.2. The steps occur in different subterms. Then the diagram can be closed immediately:

$$\begin{array}{ccc} s\vec{r} & \to_{\mathbf{m}} & s'\vec{r} \\ \triangledown & & \triangledown \\ s\vec{r}' & \to_{\mathbf{m}} & s'\vec{r}' \end{array} \qquad \begin{array}{ccc} s\vec{r} & \to_{\mathbf{m}} & s\vec{r}' \\ \triangledown & & \triangledown \\ s'\vec{r} & \to_{\mathbf{m}} & s'\vec{r}' \end{array}$$

2. $t_1 = \{s_1, \dots, s_n\}$. We consider two subcases.

2.1. The steps occur in different subterms. Then the diagram can be closed immediately. Indeed, let $i, j \in 1..n$:

$$\begin{array}{ccc} \{s_1, \dots, s_i, \dots, s_j, \dots, s_n\} & \to_{\mathbf{m}} & \{s_1, \dots, s'_i, \dots, s_j, \dots, s_n\} \\ \triangledown & & \triangledown \\ \{s_1, \dots, s_i, \dots, s'_j, \dots, s_n\} & \to_{\mathbf{m}} & \{s_1, \dots, s'_i, \dots, s'_j, \dots, s_n\} \end{array}$$

2.2. Both steps occur in the same subterm. Then we resort to the IH:

$$\begin{array}{ccc} \{s_1, \dots, s_i, \dots, s_n\} & \to_{\mathbf{m}} & \{s_1, \dots, s'_i, \dots, s_n\} \\ \triangledown & & \triangledown^{+} \\ \{s_1, \dots, s''_i, \dots, s_n\} & \to_{\mathbf{m}}^{=} & \{s_1, \dots, s'''_i, \dots, s_n\} \end{array}$$

□

Lemma 3.4.12 (Reduce/forget lemma)**.** Let $t \to_{\mathsf{Ch}} s$ be a step in the $\Lambda_\cap^{\mathsf{Ch}}$-calculus (without wrappers) and consider its corresponding step $t \to_{\mathbf{m}} s'$. Then $s' \rhd s$.

Proof. By induction on t. We show the interesting cases, which are those involving set-terms. The rest are straightforward by IH.

1. $t = t'\,\vec{u}$: We consider three subcases, depending on whether the step is at the root, internal to t', or internal to $\vec{u}$:

1.1. If the step is at the root, then we have that $t' = (\lambda x.r)$, and the Ch-step consists of $t = (\lambda x.r)\,\vec{u} \to_{\mathsf{Ch}} r[\vec{u}/x] = s$. Hence the corresponding $\mathbf{m}$-step is $t = (\lambda x.r)\,\vec{u} \to_{\mathbf{m}} r[\vec{u}/x]\langle\vec{u}\rangle = s'$. Finally $s' = r[\vec{u}/x]\langle\vec{u}\rangle \rhd r[\vec{u}/x] = s$.

1.2. If the step is internal to t', then the Ch-step consists of $t = t'\,\vec{u} \to_{\mathsf{Ch}} r\,\vec{u} = s$ with $t' \to_{\mathsf{Ch}} r$, and the corresponding $\mathbf{m}$-step is $t'\,\vec{u} \to_{\mathbf{m}} r'\,\vec{u} = s'$ where $t' \to_{\mathbf{m}} r'$ is the $\mathbf{m}$-step corresponding to $t' \to_{\mathsf{Ch}} r$. By IH we have that $r' \rhd r$, so $s' = r'\,\vec{u} \rhd r\,\vec{u} = s$.

1.3. If the step is internal to $\vec{u}$, the proof is similar to the previous case.

2. $t = \{t_1, \dots, t_n\}$: Then the Ch-step occurs in one of the terms t_i within t, *i.e.*,

$$t = \{t_1, \dots, t_i, \dots, t_n\} \to_{\mathsf{Ch}} \{t_1, \dots, s_i, \dots, t_n\} = s$$

with $t_i \to_{\mathsf{Ch}} s_i$. The corresponding $\mathbf{m}$-step is $t_i \to_{\mathbf{m}} s_i'$, so by IH we have $s_i' \rhd s_i$. Finally $s' = \{t_1, \dots, s_i', \dots, t_n\} \rhd \{t_1, \dots, s_i, \dots, t_n\} = s$.

□

3.4.3 The decreasing measure

The $\mathcal{W}_\cap$-measure for $\Lambda_\cap^{\mathsf{Ch}}$ is defined in two steps: computing the normal form in $\Lambda_\cap^{\mathbf{m}}$, and counting the number of wrappers in it. The first step was taken care of in the previous section. The second one is simple, it just requires to analyze the resulting term of the full simplification.

The core of this section is proving that the measure decreases, *i.e.*, $t \to_{\mathsf{Ch}} s$ implies $\mathcal{W}_\cap(t) > \mathcal{W}_\cap(s)$. The proof is carried out resorting to the previously proven properties on how $\to_{\mathsf{Ch}}$ and $\to_{\mathbf{m}}$ differ, and how wrappers in $\Lambda_\cap^{\mathbf{m}}$ are handled. Confluence holds in $\Lambda_\cap^{\mathbf{m}}$, so terms share normal form, and therefore full simplification, with all their reducts. The intuition behind the proof is that, if we have $t \to_{\mathsf{Ch}} s$, then the corresponding step $t \to_{\mathbf{m}} s'$ is responsible for the presence of at least one wrapper in $\mathtt{S}_*^\cap(t)$ that is absent from $\mathtt{S}_*^\cap(s)$.

Lemma 3.4.13 (Properties of the full simplification)**.**
1. If $t \to_{\mathbf{m}} s$ then $\mathtt{S}_*^\cap(t) = \mathtt{S}_*^\cap(s)$. 2. If $t \rhd s$ then $\mathtt{S}_*^\cap(t) \rhd^+ \mathtt{S}_*^\cap(s)$.

Proof. We prove each item separately. First, suppose that $t \to_{\mathbf{m}} s$. By the fact that full simplification yields the normal form of a term (Proposition 3.4.7), we know that $t \to_{\mathbf{m}}^* \mathtt{S}_*^\cap(t)$ and $s \to_{\mathbf{m}}^* \mathtt{S}_*^\cap(s)$, where both $\mathtt{S}_*^\cap(t)$ and $\mathtt{S}_*^\cap(s)$ are in $\to_{\mathbf{m}}$-normal form. By confluence (Proposition 3.3.13) we conclude that $\mathtt{S}_*^\cap(t) = \mathtt{S}_*^\cap(s)$.

Second, suppose that $t \rhd s$. By the fact that full simplification yields the normal form of a term (Proposition 3.4.7), we know that $t \to_{\mathbf{m}}^* \mathtt{S}_*^\cap(t)$, where $\mathtt{S}_*^\cap(t)$ is in $\to_{\mathbf{m}}$-normal form. Since forgetful reduction commutes with reduction (Lemma 3.4.11), we have that there exists a term $r \in \Lambda_\cap^{\mathbf{m}}$ such that $s \to_{\mathbf{m}}^* r$ and $\mathtt{S}_*^\cap(t) \rhd^+ r$. By Remark 3.4.9, since $\mathtt{S}_*^\cap(t)$ is in $\to_{\mathbf{m}}$-normal form, r is also in $\to_{\mathbf{m}}$-normal form. Moreover, $s \to_{\mathbf{m}}^* \mathtt{S}_*^\cap(s)$ where $\mathtt{S}_*^\cap(s)$ is in $\to_{\mathbf{m}}$-normal form by Proposition 3.4.7. So by confluence (Proposition 3.3.13) we have that $\mathtt{S}_*^\cap(s) = r$. Then we have that $\mathtt{S}_*^\cap(t) \rhd^+ r = \mathtt{S}_*^\cap(s)$, as required. □

Definition 3.4.14 (The $\mathcal{W}_\cap$-measure)**.** The measure of a term $t \in \Lambda_\cap^{\mathsf{Ch}}$ is defined as the weight of its full simplification, where we recall that the weight is the number of wrappers in a term:

$$\mathcal{W}_\cap(t) := \mathsf{w}(\mathtt{S}_*^\cap(t))$$

Theorem 3.4.15 (The $\mathcal{W}_\cap$-measure is decreasing)**.** Let $t, s \in \Lambda_\cap^{\mathsf{Ch}}$ be typable terms of the $\Lambda_\cap^{\mathsf{Ch}}$-calculus (without wrappers). If $t \to_{\mathsf{Ch}} s$ then $\mathcal{W}_\cap(t) > \mathcal{W}_\cap(s)$.

Proof. Let $t \to_{\mathsf{Ch}} s$ and consider the corresponding step $t \to_{\mathbf{m}} s'$. By the reduce/forget lemma (3.4.12) we have that $s' \rhd s$. By Lemma 3.4.13 (**1**) we have that $\mathtt{S}_*^\cap(t) = \mathtt{S}_*^\cap(s')$. By Lemma 3.4.13 (**2**) we have that $\mathtt{S}_*^\cap(s') \rhd^+ \mathtt{S}_*^\cap(s)$. Hence $\mathtt{S}_*^\cap(t) \rhd^+ \mathtt{S}_*^\cap(s)$. By Remark 3.4.10 we conclude that $\mathcal{W}_\cap(t) = \mathsf{w}(\mathtt{S}_*^\cap(t)) > \mathsf{w}(\mathtt{S}_*^\cap(s)) = \mathcal{W}_\cap(s)$. □

Corollary 3.4.16. $\Lambda_\cap^{\mathsf{Ch}}$ is strongly normalizing.

3.5 Conclusions

In this section we provide a proof of the fact that $\Lambda_\cap^{\mathsf{Cu}}$ characterizes strong normalization, based on the correspondence between it and $\Lambda_\cap^{\mathsf{Ch}}$. This proof justifies the design of $\Lambda_\cap^{\mathsf{Cu}}$, showing that it is a variant of the original system in [22], w.r.t. the typability power. Note that we make use of $\Lambda_\cap^{\mathsf{Ch}}$ to prove the second item for uniformity purposes; it can also be proved entirely within $\Lambda_\cap^{\mathsf{Cu}}$.

Corollary 3.5.1. $\Lambda_\cap^{\mathsf{Cu}}$ characterizes strong normalization.

Proof. Let $M \in \Lambda$ be a λ-term typable in the $\Lambda_\cap^{\mathsf{Cu}}$ system, *i.e.*, such that $\Gamma \vdash_{\mathsf{Cu}} M : A$. By correspondence (Theorem 3.2.24.1), there is a term $t \in \Lambda_\cap^{\mathsf{Ch}}$ typable in the $\Lambda_\cap^{\mathsf{Ch}}$ system such that $\Gamma \vdash_{\mathsf{Ch}} t : A$ and $t \sqsubset M$. By Corollary 3.4.16, t is strongly normalizing. From simulation (Theorem 3.2.26), we can observe that an infinite reduction sequence $M \to_\beta M_1 \to_\beta M_2 \to_\beta \dots$ would induce an infinite reduction sequence $t \to_{\mathsf{Ch}}^+ t_1 \to_{\mathsf{Ch}}^+ t_2 \to_{\mathsf{Ch}}^+ \dots$, and this would contradict strong normalization of $\Lambda_\cap^{\mathsf{Ch}}$. Hence M is strongly normalizing.

Let $M \in \Lambda$ be strongly normalizing. By Lemma 3.2.29, there is $t \in \Lambda_\cap^{\mathsf{Ch}}$ such that $t \sqsubset M$ and $\Gamma \vdash_{\mathsf{Ch}} t : A$, for some Γ and A. Then t is uniform, and, by Theorem 3.2.24.2, $\Gamma \vdash_{\mathsf{Ch}} t : A$ implies $\Gamma \vdash_{\mathsf{Cu}} t^{\mathsf{e}} : A$. Since $t^{\mathsf{e}} = M$, we conclude. □

We defined the first direct strong decreasing measure for idempotent intersection types. Although a strong measure could be defined through the translation of IIT to STLC, we highlight that such an indirect approach comes not without flaws, since no analysis related to the intersection typing information can be conducted, due to that information being lost in the translation. For instance, the indirect approach rules out the possibility for refining the technique to exactness, while ours remains usable, since it operates right within the intersection system. The other direct decreasing measures for IIT are not strong: they shrink under a particular strategy. Moreover, compared with them, ours provides a simpler codomain, yielding a natural number where the others yield multisets and pairs.

The measure defined comes from an adaptation of the technique introduced in Section 2.4, where we combined non-erasure and redex-degrees ideas to produce a new strong decreasing measure. The fact that it was successfully adapted to the setting of a more complex system like IIT, which consists of a finitary form of polymorphism, constitutes a good sign of extensibility for the technique.

Since our approach for defining the measure requires working with typing derivations throughout reduction, while IIT is usually presented as an extrinsic typing system, where only terms and not derivations are considered through reduction, we defined an intrinsic typing version of IIT. Our version is proved to be a faithful typed presentation of the original system, with correspondence and simulation theorems. This system constitutes a novelty on its own.

The work presented opens some future work paths that we intend to explore. One is currently a work in progress, and consists of the previously mentioned idea of refining the measure to exactness; this subject is discussed in the next chapter. Others are related to adapting the technique to different systems. Within the family of idempotent intersection types, we aim to adapt it to the one characterizing solvable terms [24], for which there are no syntactic proofs, as far as we know. Adaptations to more complex systems such as System F, testing the extensibility of the technique, would be a major challenge for its power. Due to the complexity of the problem, we remain cautious, although the existence of a decreasing measure for the polymorphic λ–calculus [70] encourages further exploration.

3.6 Resumen en español (Summary in Spanish)

Los sistemas de tipos intersección introducen una forma restringida de polimorfismo. En STLC no podemos, por ejemplo, dar tipo a la autoaplicación, es decir, derivar el juicio

$$x : ? \vdash xx : A$$

Esto ocurre porque no podemos elegir **un** tipo para x: necesitamos que sea una función que tome argumentos de algún tipo B, y, al tomarse a sí misma como argumento, también necesitamos que tenga tipo B. Tenemos entonces que B debe ser igual a $B \to C$ para algún C, una ecuación que no tiene solución.

Los tipos intersección proponen dar varios tipos a las variables. De esta forma podemos asignarle a x los dos tipos que requerimos, *e.g.* $A \to A$ y A, y dar tipo a la autoaplicación:

$$x : \{A \to A, A\} \vdash xx : A$$

Al igual que en el λ–calculus tradicional, la técnica usada comúnmente para demostrar la propiedad de normalización fuerte es reducibilidad. Curiosamente, al igual que en STLC, existen dos medidas decrecientes previamente definidas por Kfoury y Wells y por Boudol. Pero no son totales, son parciales: solo decrecen para una estrategia de ejecución específica. Una medida total podría definirse a partir de una traducción a tipos simples, aunque perdiendo información valiosa.

En este capítulo presento la medida decreciente $\mathcal{W}_\cap$, una adaptación de $\mathcal{W}$ a tipos intersección idempotente que introdujimos con Pablo Barenbaum y Simona Ronchi Della Rocca en un trabajo publicado en la conferencia MFPS'25. Consiste en la definición de una presentación à la Church, *i.e.*, intrínsicamente tipada o con variables tipadas explícitamente, que demostramos equivalente a la presentación original, sobre la que aplicamos con éxito el mismo proceso que para STLC: extensión con memorias, demostraciones de propiedades del cálculo auxiliar, definición de simplificación, y demostración de decrecimiento. La medida obtenida tiene un codominio más simple que las existentes y es la primera directa y total, es decir, definida nativamente para tipos intersección y decreciente para cualquier paso de reducción.

Chapter 4

A refinement to exactness

4.1 Introduction

The previously defined decreasing measures $\mathcal{W}$ and $\mathcal{W}_\cap$ have as codomain the natural numbers, so the value assigned to a program constitutes an upper bound for the longest reduction chain starting from it. Indeed, consider the following example: let $t \to t_1 \to ... \to t_n$ be the longest reduction path beginning with t; if $\mathcal{W}(t) < n$, then it would be impossible to have $\mathcal{W}(t) > \mathcal{W}(t_1) > ... > \mathcal{W}(t_n)$.

From a quantitative perspective, it would be useful to have the measure bounding the longest chain *exactly*, *i.e.*, $\mathcal{W}(t) = n$, following the example. This is not the case for $\mathcal{W}$ and $\mathcal{W}_\cap$. We provide examples where the measure is greater than the longest reduction chain for both systems.

To take the longest reduction path in STLC, one must analyze redexes to check if the variable occurs in the body of the abstraction. If it occurs, then one must contract the redex. If it does not occur, then contracting the redex would result in the loss of reduction paths, so one must reduce the argument until normal form (see maximal strategies for an in-depth analysis [106]). So, for instance, we would have

$$\begin{array}{ll} & (\lambda x^A.x)((\lambda y^A.y)z^A) \\ \to & (\lambda x^A.x)z^A \\ \to & z^A \end{array}$$

The maximum amount of steps to normal form in this case is *two*. For $\lambda^{\mathbf{m}}$, instead, this same term ends up with *three* wrappers in its memory. This shows how, even with simple terms, the number calculated by $\mathcal{W}$ detaches from the number of steps of the longest reduction chain.

$$\begin{array}{ll} & (\lambda x^A.x)((\lambda y^A.y)z^A) \\ \to_{\mathbf{m}} & ((\lambda y^A.y)z^A)\langle(\lambda y^A.y)z^A\rangle \\ \to_{\mathbf{m}} & ((\lambda y^A.y)z^A)\langle z^A\langle z^A\rangle\rangle \\ \to_{\mathbf{m}} & z^A\langle z^A\rangle\langle z^A\langle z^A\rangle\rangle \end{array}$$

We call a measure $\#(\cdot)$ *exact* when it satisfies $\#(t) = n$, with n the length of the longest reduction chain starting from t. The reason for $\mathcal{W}$ and $\mathcal{W}_\cap$ not being exact is the way wrappers are added. Note that, in the examples, the cases where extra steps are added in $\lambda^{\mathbf{m}}$ are related to reductions performed within the memory. We can refine the definition of the memory calculi to avoid adding these extra memories: instead of always adding a memory with the consumed argument, we add an empty memory if the argument is not being erased. We note it $\to_{\hat{\mathbf{m}}}$-reduction. The

previous example would become

$$\begin{array}{ll} & (\lambda x^A.x)((\lambda y^A.y)z^A) \\ \to_{\hat{\mathbf{m}}} & (\lambda x^A.x)(z^A\langle\rangle) \\ \to_{\hat{\mathbf{m}}} & z^A\langle\rangle\,\langle\rangle \end{array}$$

In this chapter I introduce a refinement of the *wrappers counting* technique presented in Chapter 2, obtaining a new strong decreasing measure for STLC that yields an exact upper bound for the length of the longest reduction chain. A refinement of the measure presented in Chapter 3 remains a work in progress, and we discuss the problems we found.

The idea of using different reduction rules to separately handle redexes depending on whether they erase their argument or not is not new. Variants of Nederpelt [76] and Klop's [65] non-erasing calculi, like those by de Groote [30] and Karr [60], already modified the general non-erasing beta reduction so that it would only *save* arguments that would be erased. However, to the best of our knowledge, these have not been used to provide exact strong decreasing measures. For the case of the simply typed λ–calculus, there exists one introduced by de Vrijer [32] together with its refinement of Gandy's measure [48]. As well as $\mathcal{W}$ for STLC, the measure introduced in this chapter puts together previously known ideas into a concrete, now exact, strong decreasing measure. A more precise correspondence with the value obtained by de Vrijer's measure can now be established, although a precise comparison between their computational processes remains not immediate. Compared with Gandy's, both aspects remain unimmediate. The domain difference where the measures are computed and proved remains: theirs relying on definitions and properties within the semantic domain of hereditarily strictly monotonic functions (or increasing functionals), ours relying on definitions and properties intrinsic to the λ–calculus setting. For the case of IIT, $\mathcal{W}_\cap$ is the only strong decreasing measure yielding a natural number. Although one could be defined by following the translation of IIT into simply typed λ–calculus, that technique would not be suitable for refinement to exactness due to the information loss that occurs through the translation. The measure $\mathcal{W}_\cap$ is not exact, and a refinement would provide the first decreasing measure consisting of an exact upper bound for the length of reductions in the setting of idempotent intersection types.

Organization of the chapter. In Section 4.2 we define a refinement of $\lambda^{\mathbf{m}}$, together with an exact measure $\hat{\mathcal{W}}$ for simply typed λ–calculus, proving its exactness. In Section 4.3 we discuss the challenges found for applying such a refinement for $\Lambda_\cap$, establishing the basis for a solution that is ongoing work.

4.2 The measure $\hat{\mathcal{W}}$: an exact refinement of $\mathcal{W}$

As stated in the introduction, the origin of the *over-measurement* for the value assigned by $\mathcal{W}$ lies in "saving" *every* argument, regardless of its erasure condition. The refinement to exactness of $\mathcal{W}$ consists of three main steps: the definition of the exact non-erasure auxiliary calculus $\lambda^{\hat{\mathbf{m}}}$; the revision and adaptation of the intermediate properties and definitions needed to define the measure and prove its soundness; and the proof of exactness of the obtained number.

The first two steps follow the same pattern of $\mathcal{W}$. They are described in Sections 4.2.1 to 4.2.4. The last step consisted of taking van Raamsdonk maximal strategies [106], and observing that, for any given term, the computed measure is bounded by the length of the maximal strategy. It is addressed in Section 4.2.5.

4.2.1 Refinement of STLC with memories

In this section we define the exact refinement of the memory calculus $\lambda^{\mathbf{m}}$ from Chapter 2, which we dubbed $\lambda^{\hat{\mathbf{m}}}$. We recall that the idea behind using this auxiliary calculus is to avoid erasure through a syntactic term constructor called *wrapper*, preventing arguments from being erased by "saving" them to a wrapper in the reduction step. The original $\lambda^{\mathbf{m}}$ saved all the arguments, and this resulted in an unnecessary replication of terms that led to redundant information in the measure. To avoid this, $\lambda^{\hat{\mathbf{m}}}$ allows for empty wrappers, being able to record that an argument was consumed, while keeping information relevant. To do so, reduction rules in $\lambda^{\hat{\mathbf{m}}}$ inspect if bound variables occur in the body of abstractions, choosing to save the argument in a wrapper only if it would be erased, and adding an empty wrapper otherwise.

Definition 4.2.1 (Grammar of types)**.**

$$A ::= \tau \mid A \to A$$

Definition 4.2.2 (Grammar of terms)**.**

$$t ::= x^A \mid \lambda x^A.t \mid tt \mid t\langle\rangle \mid t\langle t\rangle$$

Definition 4.2.3 (Typing rules)**.**

$$\frac{}{\Gamma, x : A \vdash x^A : A} \qquad \frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x^A.t : A \to B} \qquad \frac{\Gamma \vdash t : A \to B \quad \Gamma \vdash s : A}{\Gamma \vdash t\, s : B}$$

$$\frac{\Gamma \vdash t : A}{\Gamma \vdash t\langle\rangle : A} \qquad \frac{\Gamma \vdash t : A \quad \Gamma \vdash s : B}{\Gamma \vdash t\langle s\rangle : A}$$

Definition 4.2.4 (Reduction in the $\lambda^{\hat{\mathbf{m}}}$-calculus)**.** Reduction is defined over terms by the following rules, closed by congruence under arbitrary contexts:

1. $(\lambda x.t)\mathtt{L}s \;\to_{\hat{\mathbf{m}}}\; t\langle s\rangle\mathtt{L}$ if $x \notin FV(t)$
2. $(\lambda x.t)\mathtt{L}s \;\to_{\hat{\mathbf{m}}}\; t[s/x]\langle\rangle\mathtt{L}$ if $x \in FV(t)$

We recall that we write $\mathtt{L}$ for a list of wrappers. Since wrappers in this calculus can be empty, we adjust the grammar to $\mathtt{L} ::= \Box \mid \mathtt{L}\langle\rangle \mid \mathtt{L}\langle t\rangle$. Abstractions followed by a list of wrappers are called $\hat{\mathbf{m}}$-abstractions.

Proposition 4.2.5 (Subject reduction)**.** Let $\Gamma \vdash t : A$ and $t \to_{\hat{\mathbf{m}}} s$. Then $\Gamma \vdash s : A$.

Proof. By induction on the reduction relation, extending the proof from $\lambda^{\mathbf{m}}$ (Proposition 2.3.7) with the two new cases: non-erasing β-reduction and the empty bin congruence. Both are straightforward. □

Proposition 4.2.6 (Confluence)**.** If $t_1 \to^*_{\hat{\mathbf{m}}} t_2$ and $t_1 \to^*_{\hat{\mathbf{m}}} t_3$, there exists a term t_4 such that $t_2 \to^*_{\hat{\mathbf{m}}} t_4$ and $t_3 \to^*_{\hat{\mathbf{m}}} t_4$.

Proof. The proof follows the same pattern as $\lambda^{\mathbf{m}}$ from Section 2.3. The main change is the split of cases according to whether bound variables occur in the body or not, following the definition of reduction. Also an additional lemma (Lemma C.1.4), stating that the set of free variables of a term is stable through parallel reductions, is required for proving that parallel reducts do reduce to the complete developoment of the antireduct. For the detailed proof see Appendix C.1. □

4.2.2 Simplification

In this subsection we adapt the notion of simplification and the proofs of its properties, to account for the changes in wrappers and reduction rules. We begin by modifying the definition of the simplification operation, which we recall that computes, through structural recursion, the contraction of all the redexes of a certain degree (and its residuals).

Definition 4.2.7 (Simplification)**.**

$$
\begin{aligned}
\hat{\mathtt{S}}_d(x) &:= x \\
\hat{\mathtt{S}}_d(\lambda x.t) &:= \lambda x.\hat{\mathtt{S}}_d(t) \\
\hat{\mathtt{S}}_d(t\,s) &:= \begin{cases} \hat{\mathtt{S}}_d(t')\langle\hat{\mathtt{S}}_d(s)\rangle\hat{\mathtt{S}}_d(\mathtt{L}) & \text{if } t = (\lambda x.t')\mathtt{L}, t \text{ is of degree } d \text{ and } x \notin FV(t') \\ \hat{\mathtt{S}}_d(t')[\hat{\mathtt{S}}_d(s)/x]\langle\rangle\hat{\mathtt{S}}_d(\mathtt{L}) & \text{if } t = (\lambda x.t')\mathtt{L}, t \text{ is of degree } d \text{ and } x \in FV(t') \\ \hat{\mathtt{S}}_d(t)\,\hat{\mathtt{S}}_d(s) & \text{otherwise} \end{cases} \\
\hat{\mathtt{S}}_d(t\langle\rangle) &:= \hat{\mathtt{S}}_d(t)\langle\rangle \\
\hat{\mathtt{S}}_d(t\langle s\rangle) &:= \hat{\mathtt{S}}_d(t)\langle\hat{\mathtt{S}}_d(s)\rangle \\
\hat{\mathtt{S}}_d(\Box) &:= \Box \\
\hat{\mathtt{S}}_d(\mathtt{L}\langle\rangle) &:= \hat{\mathtt{S}}_d(\mathtt{L})\langle\rangle \\
\hat{\mathtt{S}}_d(\mathtt{L}\langle t\rangle) &:= \hat{\mathtt{S}}_d(\mathtt{L})\langle\hat{\mathtt{S}}_d(t)\rangle
\end{aligned}
$$

Definition 4.2.8 (Full simplification)**.** Let D be the max-degree of a term t. Then we define the full simplification $\hat{\mathtt{S}}_*(t)$ as the iterative application of $\hat{\mathtt{S}}_d(t)$ from D to 1:

$$\hat{\mathtt{S}}_*(t) := \hat{\mathtt{S}}_1(\ldots\hat{\mathtt{S}}_{D-1}(\hat{\mathtt{S}}_D(t)))$$

Lemma 4.2.9 (Soundness of simplification)**.** If $d \geq 1$, then $t \to^*_{\hat{\mathbf{m}}} \hat{\mathtt{S}}_d(t)$.

Proof. By induction on t, generalizing the statement to memories. The interesting case is that of t being $(\lambda x.t')\mathtt{L}s$, with $(\lambda x.t')\mathtt{L}$ of degree d:

1. $x \notin \mathsf{fv}(t')$: Then we have, by definition of simplification, that

   $$\hat{\mathtt{S}}_d((\lambda x.t')\mathtt{L}s) = \hat{\mathtt{S}}_d(t')\langle\hat{\mathtt{S}}_d(s)\rangle\hat{\mathtt{S}}_d(\mathtt{L})$$

   By definition of reduction, we have $(\lambda x.t')\mathtt{L}s \to_{\hat{\mathbf{m}}} t'\langle s\rangle\mathtt{L}$. And, by IH, we have $t' \to^*_{\hat{\mathbf{m}}} \hat{\mathtt{S}}_d(t')$, $s \to^*_{\hat{\mathbf{m}}} \hat{\mathtt{S}}_d(s)$, and $\mathtt{L} \to^*_{\hat{\mathbf{m}}} \hat{\mathtt{S}}_d(\mathtt{L})$. Therefore $(\lambda x.t')\mathtt{L}s \to^*_{\hat{\mathbf{m}}} \hat{\mathtt{S}}_d(t')\langle\hat{\mathtt{S}}_d(s)\rangle\hat{\mathtt{S}}_d(\mathtt{L})$.

2. $x \in \mathsf{fv}(t')$: Then we have, by definition of simplification, that

   $$\hat{\mathtt{S}}_d((\lambda x.t')\mathtt{L}s) = \hat{\mathtt{S}}_d(t')[\hat{\mathtt{S}}_d(s)/x]\langle\rangle\hat{\mathtt{S}}_d(\mathtt{L})$$

   By definition of reduction, we have $(\lambda x.t')\mathtt{L}s \to_{\hat{\mathbf{m}}} t'[s/x]\langle\rangle\mathtt{L}$. And, by IH, we have $t' \to^*_{\hat{\mathbf{m}}} \hat{\mathtt{S}}_d(t')$, $s \to^*_{\hat{\mathbf{m}}} \hat{\mathtt{S}}_d(s)$, and $\mathtt{L} \to^*_{\hat{\mathbf{m}}} \hat{\mathtt{S}}_d(\mathtt{L})$. To conclude

   $$(\lambda x.t')\mathtt{L}s \to^*_{\hat{\mathbf{m}}} (\lambda x.\hat{\mathtt{S}}_d(t'))\hat{\mathtt{S}}_d(\mathtt{L})\hat{\mathtt{S}}_d(s) \to_{\hat{\mathbf{m}}} \hat{\mathtt{S}}_d(t')[\hat{\mathtt{S}}_d(s)/x]\langle\rangle\hat{\mathtt{S}}_d(\mathtt{L})$$

   we must check that x still occurs free in $\hat{\mathtt{S}}_d(t')$, in order to be able to apply the same reduction rule. By a straightforward induction on terms, we can see that $FV(t') = FV(\hat{\mathtt{S}}_d(t'))$ for all t', so the reduction holds. □

The proofs of the following three lemmas can be conducted by reusing those from Lemmas 2.4.5, 2.4.6 and 2.4.8. This is doable because reduction does not take place, and hypotheses avoid the simplification redex case (which would have required changes) from happening. The only detail to note is that we have another term constructor, $t\langle\rangle$, but this case is straightforward by IH.

Lemma 4.2.10 (Non–abstraction substitution preserves greater non–abstraction)**.** If $\mathsf{h}(\mathsf{type}(t)) > \mathsf{h}(\mathsf{type}(s))$ and t not an $\hat{\mathbf{m}}$-abstraction, then $t[s/x]$ is not an $\hat{\mathbf{m}}$-abstraction. □

Lemma 4.2.11 (High simplification preserves non–abstraction)**.** If $\delta_{\mathsf{max}}(t) \leq D \leq \mathsf{h}(\mathsf{type}(t))$ and t is not an $\hat{\mathbf{m}}$-abstraction, then $\hat{\mathsf{S}}_D(t)$ is not an $\hat{\mathbf{m}}$-abstraction. □

Lemma 4.2.12 (Strict upper bound for max–deg is stable under substitution)**.** If $\delta_{\mathsf{max}}(t) < D$, $\delta_{\mathsf{max}}(s) < D$, and $\mathsf{h}(\mathsf{type}(s)) < D$, then $\delta_{\mathsf{max}}(t[s/x]) < D$. □

Lemma 4.2.13 (Simplification of max-degree decreases the max-degree)**.** Let $D \geq 1$. If $\delta_{\mathsf{max}}(t) \leq D$, then $\delta_{\mathsf{max}}(\hat{\mathsf{S}}_D(t)) < D$.

Proof. By induction on t. We adapt the proof of Lemma 2.4.9 by modifying the case where t is an abstraction of degree D, and by adding the case of the empty bin.

1. $((\lambda x.t)Ls)$ and $x \notin FV$: Then, by simplification definition, we have that
$$\hat{\mathsf{S}}_D((\lambda x.t)Ls) = \hat{\mathsf{S}}_D(t)\langle\hat{\mathsf{S}}_D(s)\rangle\hat{\mathsf{S}}_D(L)$$
By definition of max-degree
$$\delta_{\mathsf{max}}((\lambda x.t)Ls) = max(\mathsf{h}(A \to B), \delta_{\mathsf{max}}(t), \delta_{\mathsf{max}}(L), \delta_{\mathsf{max}}(s)) \leq D$$
Since for all of them it holds that they are lower or equal than D, by IH we conclude
$$max(\delta_{\mathsf{max}}(\hat{\mathsf{S}}_D(t)), \delta_{\mathsf{max}}(\hat{\mathsf{S}}_D(L)), \delta_{\mathsf{max}}(\hat{\mathsf{S}}_D(s))) < D$$
Finally, by definition of max-degree, we have that
$$\delta_{\mathsf{max}}(\hat{\mathsf{S}}_D(t)\langle\hat{\mathsf{S}}_D(s)\rangle\hat{\mathsf{S}}_D(L)) = max(\delta_{\mathsf{max}}(\hat{\mathsf{S}}_D(t)), \delta_{\mathsf{max}}(\hat{\mathsf{S}}_D(s)), \delta_{\mathsf{max}}(\hat{\mathsf{S}}_D(L)))$$
and therefore
$$\delta_{\mathsf{max}}(\hat{\mathsf{S}}_D(t)\langle\hat{\mathsf{S}}_D(s)\rangle\hat{\mathsf{S}}_D(L)) < D$$
2. $((\lambda x.t)Ls)$ and $x \in FV$: Then, by simplification definition, we have that
$$\hat{\mathsf{S}}_D((\lambda x.t)Ls) = \hat{\mathsf{S}}_D(t)[\hat{\mathsf{S}}_D(s)/x]\langle\rangle\hat{\mathsf{S}}_D(L)$$
By definition of max-degree
$$\delta_{\mathsf{max}}((\lambda x.t)Ls) = max(\mathsf{h}(A \to B), \delta_{\mathsf{max}}(t), \delta_{\mathsf{max}}(L), \delta_{\mathsf{max}}(s)) \leq D$$
Since for all of them it holds that they are lower or equal than D, by IH we conclude
$$max(\delta_{\mathsf{max}}(\hat{\mathsf{S}}_D(t)), \delta_{\mathsf{max}}(\hat{\mathsf{S}}_D(L)), \delta_{\mathsf{max}}(\hat{\mathsf{S}}_D(s))) < D$$
By case hypothesis, the height of the type of the abstraction is D, *i.e.*, $\mathsf{h}(A \to B) = D$. By definition of max-degree, we have
$$\delta_{\mathsf{max}}(\hat{\mathsf{S}}_D(t)[\hat{\mathsf{S}}_D(s)/x]\langle\rangle\hat{\mathsf{S}}_D(L)) = max(\delta_{\mathsf{max}}(\hat{\mathsf{S}}_D(t)[\hat{\mathsf{S}}_D(s)/x]), \delta_{\mathsf{max}}(\hat{\mathsf{S}}_D(L)))$$
Since $\mathsf{h}(A) < \mathsf{h}(A \to B) = D$, by Lemma 4.2.12 we conclude $\delta_{\mathsf{max}}(\hat{\mathsf{S}}_D(t)[\hat{\mathsf{S}}_D(s)/x]) < D$.
3. $t\langle\rangle$: Then $\delta_{\mathsf{max}}(t\langle\rangle) = \delta_{\mathsf{max}}(t) \leq D$, so we resort to the IH. □

Proposition 4.2.14 (Full simplification yields the normal form)**.** $t \to_{\hat{\mathbf{m}}}^{*} \hat{\mathsf{S}}_*(t)$ and $\hat{\mathsf{S}}_*(t)$ is a $\to_{\hat{\mathbf{m}}}$-normal form.

Proof. By induction on $D = \delta_{\mathsf{max}}(t)$:

1. 0: then t has no redexes, thus $t \to_{\hat{\mathbf{m}}}^{0} \hat{\mathsf{S}}_*(t)$ and t normal form.
2. $d+1$: since, by Lemma 4.2.13, $\delta_{\mathsf{max}}(\hat{\mathsf{S}}_{d+1}(t)) \leq d$, we can apply IH, obtaining $\hat{\mathsf{S}}_{d+1}(t) \to_{\hat{\mathbf{m}}}^{*} \hat{\mathsf{S}}_*(t)$, with $\hat{\mathsf{S}}_*(t)$ normal form. By Lemma 4.2.9, we have $t \to_{\hat{\mathbf{m}}}^{*} \hat{\mathsf{S}}_{d+1}(t)$, so we conclude. □

4.2.3 Forgetful reduction

Again, we address the changes in wrappers and reduction rules, this time in the definitions and proofs of *forgetful reduction*, the additional rewriting relation that commutes with $\hat{\mathbf{m}}$-reduction and that we use for proving that the measure we define is decreasing.

Definition 4.2.15 (Forgetful reduction)**.** We define a binary relation $t \rhd t'$ between typable terms as the closure by congruence under arbitrary contexts of the rules $t\langle\rangle \rhd t$ and $t\langle s\rangle \rhd t$.

Lemma 4.2.16 (Local commutation of reduction and forgetful reduction)**.** If $t \rhd s$ and $t \to_{\hat{\mathbf{m}}} t'$, there exists a term s' such that $t' \rhd^+ s'$ and $s \to_{\hat{\mathbf{m}}}^{=} s'$, where $\rhd^+$ is the transitive closure of $\rhd$, and $\to_{\hat{\mathbf{m}}}^{=}$ is the reflexive closure of $\to_{\hat{\mathbf{m}}}$. Graphically:

$$\begin{array}{ccc} t & \rhd & s \\ \downarrow & & \vdots = \\ t' & \rhd^+ & s' \end{array}$$

Proof. By induction on $t \to_{\hat{\mathbf{m}}} t'$. We consider only the cases that are different from $\lambda^{\mathbf{m}}$: head redex contraction and congruence on the new empty wrapper.

1. $(\lambda x.s)\mathtt{L}r \to_{\hat{\mathbf{m}}} s[r/x]\langle\rangle\mathtt{L}$: then $\rhd$ occurs within s, r or $\mathtt{L}$. Since no memory is erased in $\to_{\hat{\mathbf{m}}}$, the same $\rhd$-step can be reproduced from $t[r/x]\langle\rangle\mathtt{L}$ (several times if x occurs several times and $\rhd$ was in r).

2. $(\lambda x.s)\mathtt{L}r \to_{\hat{\mathbf{m}}} s\langle r\rangle\mathtt{L}$: then $\rhd$ occurs within s, r or $\mathtt{L}$. The same $\rhd$-step can be reproduced from $s\langle r\rangle\mathtt{L}$.

3. $s\langle\rangle \to_{\hat{\mathbf{m}}} s'\langle\rangle$: Then we consider two cases, according to $\rhd$ occurring in head position or in s. If it occurs in head position, we simply perform the original $\to_{\hat{\mathbf{m}}}$-step; if it occurs within s, we proceed by IH.

$$\begin{array}{ccc} s\langle\rangle & \to_{\mathbf{m}} & s'\langle\rangle \\ \triangledown & & \triangledown \\ s & = & s' \end{array} \qquad \begin{array}{ccc} s\langle\rangle & \to_{\mathbf{m}} & s'\langle\rangle \\ \triangledown & & \triangledown \\ s''\langle\rangle & \to_{\mathbf{m}} & s'''\langle\rangle \end{array}$$

□

Proposition 4.2.17 (Forgetful reduction commutes with reduction)**.** If $t \rhd^+ s$ and $t \to_{\hat{\mathbf{m}}}^* t'$, there exists a term s' such that $t' \rhd^+ s'$ and $s \to_{\hat{\mathbf{m}}}^* s'$. Graphically:

$$\begin{array}{ccc} t & \rhd^+ & s \\ \downarrow^* & & \vdots^* \\ t' & \rhd^+ & s' \end{array}$$

Furthermore, if $t \rhd^+ s$ and t is a $\to_{\hat{\mathbf{m}}}$-normal form, then s is also a normal form.

Proof. The only differences from $\lambda^{\mathbf{m}}$ are that reduction changes and that wrappers are allowed to be empty. Since these cases are treated in the local commutation lemma, we can reuse the proof of Proposition 2.4.15. □

Lemma 4.2.18 (Reduce/forget lemma)**.** Let $M \to_\beta N$ be a β-step and let $M \to_{\hat{\mathbf{m}}} s$ be the corresponding step in $\lambda^{\hat{\mathbf{m}}}$. Then $s \rhd^+ N$.

Proof. By induction on $M \to_\beta N$. Again, we only consider the interesting case that changed from the proof of Lemma 2.4.16, *i.e.*, $M = (\lambda x.P)Q \to_\beta P[Q/x] = N$. We have two possibilities:

1. $x \notin P$: then $M = (\lambda x.P)Q \to_{\hat{\mathbf{m}}} P\langle Q\rangle \rhd P = P[Q/x]$.
2. $x \in P$: then $M = (\lambda x.P)Q \to_{\hat{\mathbf{m}}} P[Q/x]\langle\rangle \rhd P[Q/x]$. □

4.2.4 $\hat{\mathcal{W}}$-measure

We define the exact decreasing measure $\hat{\mathcal{W}}$, a refinement of $\mathcal{W}$ computing a value that coincides with that of the length of the longest reduction path from the term. The changes with respect to $\mathcal{W}$ were done in the auxiliary calculus $\lambda^{\hat{\mathbf{m}}}$, the operation and proofs of simplification and forgetful reduction, thus the following definitions for $\hat{\mathcal{W}}$ are roughly the same as those for $\mathcal{W}$. We write them again here for completeness, considering they are quite short.

Definition 4.2.19 (Weight of a term)**.** The weight of a term t is the number of wrappers it contains.

Definition 4.2.20 ($\hat{\mathcal{W}}$-measure)**.** $\hat{\mathcal{W}}(t) = \mathsf{w}(\hat{\mathsf{S}}_*(t))$

Lemma 4.2.21. If $t \rhd^+ s$ then $\mathsf{w}(t) > \mathsf{w}(s)$.

Proof. Trivial by definition of weight. □

Lemma 4.2.22. If $t \to_{\hat{\mathbf{m}}} s$ then $\hat{\mathsf{S}}_*(t) = \hat{\mathsf{S}}_*(s)$.

Proof. By Lemma 4.2.9, $t \to^*_{\hat{\mathbf{m}}} \hat{\mathsf{S}}_*(t)$ and $s \to^*_{\hat{\mathbf{m}}} \hat{\mathsf{S}}_*(s)$. Since $t \to_{\hat{\mathbf{m}}} s$, $t \to^*_{\hat{\mathbf{m}}} \hat{\mathsf{S}}_*(t)$ and $t \to^*_{\hat{\mathbf{m}}} \hat{\mathsf{S}}_*(s)$, and, since both $\hat{\mathsf{S}}_*(t)$ and $\hat{\mathsf{S}}_*(s)$ are normal forms, we have by confluence (Proposition 4.2.6) that $\hat{\mathsf{S}}_*(t) = \hat{\mathsf{S}}_*(s)$. □

Lemma 4.2.23. If $t \rhd^+ s$ then $\hat{\mathsf{S}}_*(t) \rhd^+ \hat{\mathsf{S}}_*(s)$.

Proof. By Lemma 4.2.9, $t \to^*_{\hat{\mathbf{m}}} \hat{\mathsf{S}}_*(t)$. By Proposition 4.2.17, there exists r such that $\hat{\mathsf{S}}_*(t) \rhd^+ r$ and $s \to^*_{\hat{\mathbf{m}}} r$. Since $\hat{\mathsf{S}}_*(t)$ is a normal form by Proposition 4.2.14 and $\rhd$ preserves normal forms (Proposition 4.2.17), we have that r is a normal form. Finally $s \to^*_{\hat{\mathbf{m}}} \hat{\mathsf{S}}_*(s)$ and $\hat{\mathsf{S}}_*(s)$ is a normal form, so by confluence (Proposition 4.2.6) $r = \hat{\mathsf{S}}_*(s)$. □

Theorem 4.2.24. Let M, N be typable λ-terms such that $M \to_\beta N$. Then $\hat{\mathcal{W}}(M) > \hat{\mathcal{W}}(N)$.

Proof. Given the step $M \to_\beta N$, consider the corresponding step $M \to_{\hat{\mathbf{m}}} s$, and note that $s \rhd^+ N$ by reduce/forget (Lemma 4.2.18). Since $M \to_{\hat{\mathbf{m}}} s \rhd^+ N$, by Lemmas 4.2.22 and 4.2.23, we have that $\hat{\mathsf{S}}_*(M) = \hat{\mathsf{S}}_*(s) \rhd^+ \hat{\mathsf{S}}_*(N)$. Finally, by Lemma 4.2.21, $\hat{\mathcal{W}}(M) = \mathsf{w}(\hat{\mathsf{S}}_*(M)) > \mathsf{w}(\hat{\mathsf{S}}_*(N)) = \hat{\mathcal{W}}(N)$. □

We revisit the examples from Section 2.4 to note how they change.

Example 4.2.25. Let $M = x(Iy)(Iy)$, which reduces *in the STLC* to $N = xy(Iy)$. Then we have

$$\begin{array}{rcl} \hat{\mathcal{W}}(x(Iy)(Iy)) & = & \mathsf{w}(\hat{\mathsf{S}}_*(x(Iy)(Iy))) \\ & = & \mathsf{w}(x(y\langle\rangle)(y\langle\rangle)) \quad = \quad 2 \;>\; 1 \quad = \quad \mathsf{w}(xy(y\langle\rangle)) \\ & & \qquad\qquad\qquad\qquad\quad = \quad \mathsf{w}(\hat{\mathsf{S}}_*(xy(Iy))) \quad = \quad \hat{\mathcal{W}}(xy(Iy)) \end{array}$$

Example 4.2.26. Let

$$M = (\lambda x^{\tau}.y^{\tau\to\tau\to\tau}x^{\tau}x^{\tau})((\lambda x^{\tau\to\tau}.x^{\tau\to\tau}z^{\tau})f^{\tau\to\tau})$$

and consider the step

$$M = (\lambda x.yxx)((\lambda x.xz)f) \to_{\beta} (\lambda x.yxx)(fz) = N$$

Then we have that

$$\hat{\mathsf{S}}_*(M) = (y(fz)\langle\rangle(fz)\langle\rangle)\langle\rangle \qquad \hat{\mathsf{S}}_*(N) = (y(fz)(fz))\langle\rangle$$

Therefore

$$\hat{\mathcal{W}}(M) = \mathsf{w}(\hat{\mathsf{S}}_*(M)) = 3 > 1 = \mathsf{w}(\hat{\mathsf{S}}_*(N)) = \hat{\mathcal{W}}(N)$$

Example 4.2.27. Let

$$M = (\lambda x^{\tau\to\tau}.x^{\tau\to\tau}(x^{\tau\to\tau}y^{\tau}))(\lambda z^{\tau}.w^{\tau}) \to_{\beta} (\lambda z^{\tau}.w^{\tau})((\lambda z^{\tau}.w^{\tau})y^{\tau}) = N$$

Then

$$\begin{aligned} \hat{\mathsf{S}}_2(M) &= ((\lambda z.w)((\lambda z.w)y))\langle\rangle \\ \hat{\mathsf{S}}_*(M) = \hat{\mathsf{S}}_1(\hat{\mathsf{S}}_2(M)) &= w\langle w\langle y\rangle\rangle\langle\rangle \end{aligned}$$

$$\hat{\mathsf{S}}_*(N) = \hat{\mathsf{S}}_1(N) = w\langle w\langle y\rangle\rangle$$

Therefore

$$\hat{\mathcal{W}}(M) = \mathsf{w}(\hat{\mathsf{S}}_*(M)) = 3 > 2 = \mathsf{w}(\hat{\mathsf{S}}_*(N)) = \hat{\mathcal{W}}(N)$$

4.2.5 Exact upper bound

In this subsection we prove that the measure we define is indeed exact, *i.e.*, the number it computes is an exact upper bound for the length of reductions in the simply typed λ–calculus. To do so, we analyze the value obtained against the longest possible reduction chains from terms. We resort to two works by Femke van Raamsdonk one on maximal strategies [106], which are those that always find the longest execution path of terms, and the other on the inductive characterization of strongly normalizing terms [107].

Notation 4.2.28. Let $D = \delta_{\mathsf{max}}(t)$. We will write $\hat{\mathsf{S}}_{\leq d}(t)$ and $\hat{\mathsf{S}}_{>d}(t)$ to refer to $\hat{\mathsf{S}}_1(\ldots\hat{\mathsf{S}}_{d-1}(\hat{\mathsf{S}}_d(t)))$ and $\hat{\mathsf{S}}_{d+1}(\ldots\hat{\mathsf{S}}_{D-1}(\hat{\mathsf{S}}_D(t)))$, respectively.

The measure $\hat{\mathcal{W}}$ was defined over terms of simply typed λ–calculus, but, since w and $\hat{\mathsf{S}}_*$ are defined for terms of $\lambda^{\hat{\mathbf{m}}}$, $\hat{\mathcal{W}}$ is also defined for terms of $\lambda^{\hat{\mathbf{m}}}$. In the next lemma we prove some properties of the measure over terms involving wrappers.

Lemma 4.2.29.

1. $\hat{\mathcal{W}}(t\langle s\rangle) = \hat{\mathcal{W}}(t) + \hat{\mathcal{W}}(s) + 1$
2. $\hat{\mathcal{W}}(t\langle s\rangle u) = \hat{\mathcal{W}}((tu)\langle s\rangle)$
3. $\hat{\mathcal{W}}(t\langle\rangle) = \hat{\mathcal{W}}(t) + 1$
4. $\hat{\mathcal{W}}(t\langle\rangle u) = \hat{\mathcal{W}}((tu)\langle\rangle)$

Proof.

1. $\hat{\mathcal{W}}(t\langle s\rangle) = \mathsf{w}(\hat{\mathtt{S}}_*(t\langle s\rangle)) = \mathsf{w}(\hat{\mathtt{S}}_*(t)) + \mathsf{w}(\hat{\mathtt{S}}_*(\langle s\rangle)) = \mathsf{w}(\hat{\mathtt{S}}_*(t)) + \mathsf{w}(\hat{\mathtt{S}}_*(s)) + 1 = \hat{\mathcal{W}}(t) + \hat{\mathcal{W}}(s) + 1$. Second step is proved by unfolding $\hat{\mathtt{S}}_*(t\langle s\rangle)$, getting $\hat{\mathtt{S}}_1(\ldots\hat{\mathtt{S}}_{D-1}(\hat{\mathtt{S}}_D(t\langle s\rangle)))$, which, by repeated application of $\hat{\mathtt{S}}_d$ definition, yields $\hat{\mathtt{S}}_1(\ldots\hat{\mathtt{S}}_{D-1}(\hat{\mathtt{S}}_D(t)))\langle\hat{\mathtt{S}}_1(\ldots\hat{\mathtt{S}}_{D-1}(\hat{\mathtt{S}}_D(\langle s\rangle)))\rangle$, which is equal to $\hat{\mathtt{S}}_*(t)\hat{\mathtt{S}}_*(\langle s\rangle)$. Third step is proved using the definition of weight.

2. $\hat{\mathcal{W}}(t\langle s\rangle u) = \mathsf{w}(\hat{\mathtt{S}}_*(t\langle s\rangle u)) = \mathsf{w}(\hat{\mathtt{S}}_{\leq D}(t\langle s\rangle u))$, with $D = \delta_{\mathsf{max}}(t\langle s\rangle u)$. We consider two cases, depending on $\hat{\mathtt{S}}_*(t\langle s\rangle u)$ becoming a redex at some point of the chain of simplifications. Note that $t\langle s\rangle u$ being a redex from the beginning is handled by the second case.

   2.1. If it does not become a redex, then each part of the application is treated in an isolated way, so on one side we have

$$\begin{aligned}\mathsf{w}(\hat{\mathtt{S}}_*(t\langle s\rangle u)) &= \mathsf{w}(\hat{\mathtt{S}}_*(t)\langle\hat{\mathtt{S}}_*(s)\rangle\hat{\mathtt{S}}_*(u))\\ &= \mathsf{w}(\hat{\mathtt{S}}_*(t)) + 1 + \mathsf{w}(\hat{\mathtt{S}}_*(s)) + \mathsf{w}(\hat{\mathtt{S}}_*(u))\end{aligned}$$

   and on the other side we have

$$\begin{aligned}\mathsf{w}(\hat{\mathtt{S}}_*((tu)\langle s\rangle)) &= \mathsf{w}((\hat{\mathtt{S}}_*(t)\hat{\mathtt{S}}_*(u))\langle\hat{\mathtt{S}}_*(s)\rangle)\\ &= \mathsf{w}(\hat{\mathtt{S}}_*(t)) + \mathsf{w}(\hat{\mathtt{S}}_*(u)) + 1 + \mathsf{w}(\hat{\mathtt{S}}_*(s))\end{aligned}$$

   so we conclude.

   2.2. If it does become a redex, then each part of the application is treated in an isolated way until the redex appears, where we would have the intermediate simplification

$$\hat{\mathtt{S}}_{\leq d}((\lambda x.t')\mathtt{L}\langle\hat{\mathtt{S}}_{>d}(s)\rangle\hat{\mathtt{S}}_{>d}(u))$$

   with $1 \leq d \leq D$ and $\hat{\mathtt{S}}_{>d}(t) = (\lambda x.t')\mathtt{L}$. Without loss of generality, we assume that the created redex is of degree d, since simplification would not alter the structure of the redex until reaching the actual degree. We consider then the two possibilities for a d-redex simplification:

   2.2.1. $x \notin FV(t')$:

$$\begin{aligned}\hat{\mathtt{S}}_*(t\langle s\rangle u) &= \hat{\mathtt{S}}_{\leq d}(\hat{\mathtt{S}}_{>d}(t)\langle\hat{\mathtt{S}}_{>d}(s)\rangle\hat{\mathtt{S}}_{>d}(u))\\ &= \hat{\mathtt{S}}_{\leq d}((\lambda x.t')\mathtt{L}\langle\hat{\mathtt{S}}_{>d}(s)\rangle\hat{\mathtt{S}}_{>d}(u))\\ &= \hat{\mathtt{S}}_{<d}(\hat{\mathtt{S}}_d(t')\langle\hat{\mathtt{S}}_{\geq d}(u)\rangle\hat{\mathtt{S}}_d(\mathtt{L})\langle\hat{\mathtt{S}}_{\geq d}(s)\rangle)\\ &= \hat{\mathtt{S}}_{\leq d}((\lambda x.t')\mathtt{L}\hat{\mathtt{S}}_{>d}(u)\langle\hat{\mathtt{S}}_{>d}(s)\rangle)\\ &= \hat{\mathtt{S}}_{\leq d}(\hat{\mathtt{S}}_{>d}(t)\hat{\mathtt{S}}_{>d}(u)\langle\hat{\mathtt{S}}_{>d}(s)\rangle)\\ &= \hat{\mathtt{S}}_*((tu)\langle s\rangle)\end{aligned}$$

   2.2.2. $x \in FV(t')$:

$$\begin{aligned}\hat{\mathtt{S}}_*(t\langle s\rangle u) &= \hat{\mathtt{S}}_{\leq d}(\hat{\mathtt{S}}_{>d}(t)\langle\hat{\mathtt{S}}_{>d}(s)\rangle\hat{\mathtt{S}}_{>d}(u))\\ &= \hat{\mathtt{S}}_{\leq d}((\lambda x.t')\mathtt{L}\langle\hat{\mathtt{S}}_{>d}(s)\rangle\hat{\mathtt{S}}_{>d}(u))\\ &= \hat{\mathtt{S}}_{<d}(\hat{\mathtt{S}}_d(t')[\hat{\mathtt{S}}_{\geq d}(u)/x]\langle\rangle\hat{\mathtt{S}}_d(\mathtt{L})\langle\hat{\mathtt{S}}_{\geq d}(s)\rangle)\\ &= \hat{\mathtt{S}}_{\leq d}((\lambda x.t')\mathtt{L}\hat{\mathtt{S}}_{>d}(u)\langle\hat{\mathtt{S}}_{>d}(s)\rangle)\\ &= \hat{\mathtt{S}}_{\leq d}(\hat{\mathtt{S}}_{>d}(t)\hat{\mathtt{S}}_{>d}(u)\langle\hat{\mathtt{S}}_{>d}(s)\rangle)\\ &= \hat{\mathtt{S}}_*((tu)\langle s\rangle)\end{aligned}$$

3. $\hat{\mathcal{W}}(t\langle\rangle) = \hat{\mathcal{W}}(t) + 1$: trivial.
4. $\hat{\mathcal{W}}(t\langle\rangle u) = \hat{\mathcal{W}}((tu)\langle\rangle)$: analogous to item 2.

□

Definition 4.2.30 (F_∞ [106, Definition 3.21])**.** Let M be a term of simply typed λ–calculus. We define the strategy F_∞ by recursion on M. If M is in normal form, *i.e.*, it does not contain any redex, then $F_\infty(M) = M$. Otherwise:

1. $F_\infty(x\vec{P}N\vec{Q}) = x\vec{P}F_\infty(N)\vec{Q}$ if $\vec{P} \in NF$ and $N \notin NF$
2. $F_\infty(\lambda x.N) = \lambda x.F_\infty(N)$
3. $F_\infty((\lambda x.N)P\vec{Q}) = (\lambda x.N)F_\infty(P)\vec{Q}$ if $x \notin \mathsf{fv}(N)$ and $P \notin NF$
4. $F_\infty((\lambda x.N)P\vec{Q}) = N[P/x]\vec{Q}$ if $x \in \mathsf{fv}(N)$ or $P \in NF$

Lemma 4.2.31 (Maximal strategy [106, Theorem 3.22])**.** The strategy F_∞ is maximal for simply typed λ–calculus. □

We recall that a maximal strategy is one that always finds the longest reduction path. F_∞ is a maximal strategy for simply typed λ–calculus due to Barendregt [9, §13.4]. Several proofs can be found. Van Raamsdonk, Severi, Sørensen and Xi give a simplified one [106, §3.5]. The strategy being maximal implies that, for all simply-typed term M, normalizing using F_∞ requires $|M|$ steps.

Lemma 4.2.32 (Reductions length [106, Proposition 3.26])**.** The size of the longest reduction chain from a (strongly normalizing) term is given by the following equations.

1. $|xN_1 \dots N_n| = |N_1| + \dots + |N_n|$
2. $|\lambda x.M| = |M|$
3. $|(\lambda x.M)N\vec{P}| = |M\vec{P}| + |N| + 1$ if $x \notin FV(M)$
4. $|(\lambda x.M)N\vec{P}| = |M[N/x]\vec{P}| + 1$ if $x \in FV(M)$ □

In the statement of the following theorem, we use $\hat{\mathcal{W}}$ over terms of simply typed λ–calculus. In the proof, we need to apply $\lambda^{\hat{\mathbf{m}}}$ reduction steps, obtaining a term that does not belong to simply typed λ–calculus. However, note that we decompose these terms using Lemma 4.2.29 so that, when we apply the IH, we have terms of the simply typed λ–calculus again. To make this transparent, we will keep simply typed λ–calculus metavariables for the subterms of the $\lambda^{\hat{\mathbf{m}}}$-terms, to stress that they are actually λ-terms.

Theorem 4.2.33 ($\hat{\mathcal{W}}$ is an exact upper bound for reductions length)**.** Let M be a term of the simply typed λ–calculus. Then $\hat{\mathcal{W}}(M) = |M|$.

Proof. We know that $\hat{\mathcal{W}}(M) \geq |M|$, otherwise it would be possible to construct a descending chain in the natural numbers that goes below 0. Hence it suffices to prove that $\hat{\mathcal{W}}(M) \leq |M|$. We proceed by induction on $M \in \mathsf{ISN}$. We recall that ISN is a set of terms given by the following rules, and that all terms in simply typed λ–calculus belong to ISN [107, §3].

$$\frac{N_1 \in \mathsf{ISN} \ \dots \ N_n \in \mathsf{ISN}}{xN_1 \dots N_n \in \mathsf{ISN}}\ \mathsf{ISN}_x \qquad \frac{M \in \mathsf{ISN}}{\lambda x.M \in \mathsf{ISN}}\ \mathsf{ISN}_\lambda \qquad \frac{M[N/x]\, P_1 \dots P_n \in \mathsf{ISN} \quad N \in \mathsf{ISN}}{(\lambda x.M)\, N\, P_1 \dots P_n \in \mathsf{ISN}}\ \mathsf{ISN}_@$$

1. ISN_x: We know $xN_1 \dots N_n$, with $N_1 \in \mathsf{ISN} \dots N_n \in \mathsf{ISN}$. By Definition 4.2.20, Definition 4.2.19 and Definition 4.2.8, we have $\hat{\mathcal{W}}(xN_1 \dots N_n) = \hat{\mathcal{W}}(N_1) + \dots + \hat{\mathcal{W}}(N_n)$. By IH, $\hat{\mathcal{W}}(N_i) \leq |N_i|$. By Lemma 4.2.32, we have $|xN_1 \dots N_n| = |N_1| + \dots + |N_n|$. We conclude then $\hat{\mathcal{W}}(xN_1 \dots N_n) = \hat{\mathcal{W}}(N_1) + \dots + \hat{\mathcal{W}}(N_n) \leq |N_1| + \dots + |N_n| = |xN_1 \dots N_n|$.

2. ISN_λ: We know $\lambda x.M \in \mathsf{ISN}$, with $M \in \mathsf{ISN}$. By Definition 4.2.20, Definition 4.2.19 and Definition 4.2.8, we have $\hat{\mathcal{W}}(\lambda x.M) = \hat{\mathcal{W}}(M)$. By IH, $\hat{\mathcal{W}}(M) \leq |M|$. By Lemma 4.2.32, we have $|\lambda x.M| = |M|$. We conclude then $\hat{\mathcal{W}}(\lambda x.M) = \hat{\mathcal{W}}(M) \leq |M| = |\lambda x.M|$.

3. $\mathsf{ISN}_@$: We know $(\lambda x.M)NP_1 \dots P_n \in \mathsf{ISN}$, with $M[N/x]P_1 \dots P_n \in \mathsf{ISN}$ and $N \in \mathsf{ISN}$. We know by IH that $\hat{\mathcal{W}}(M[N/x]P_1 \dots P_n) \leq |M[N/x]P_1 \dots P_n|$ and $\hat{\mathcal{W}}(N) \leq |N|$. To prove that $\hat{\mathcal{W}}((\lambda x.M)NP_1 \dots P_n) \leq |(\lambda x.M)NP_1 \dots P_n|$, we must consider two cases.

   3.1. $x \notin FV(M)$: Since $(\lambda x.M)N\vec{P} \to_{\hat{\mathbf{m}}} M\langle N\rangle\vec{P}$, by Lemma 4.2.22 we know that their $\hat{\mathtt{S}}_*$, and therefore $\hat{\mathcal{W}}$, are equal. Therefore we have that

$$\begin{aligned}
\hat{\mathcal{W}}((\lambda x.M)N\vec{P}) &= \hat{\mathcal{W}}(M\langle N\rangle\vec{P}) \\
&= \hat{\mathcal{W}}(M\vec{P}\langle N\rangle) && \text{(Lemma 4.2.29.2)} \\
&= \hat{\mathcal{W}}(M\vec{P}) + \hat{\mathcal{W}}(N) + 1 && \text{(Lemma 4.2.29.1)} \\
&\leq |M\vec{P}| + |N| + 1 && \text{(IH)} \\
&= |(\lambda x.M)N\vec{P}| && \text{(Lemma 4.2.32)}
\end{aligned}$$

   3.2. $x \in FV(M)$: Since $(\lambda x.M)N\vec{P} \to_{\hat{\mathbf{m}}} M[N/x]\langle\rangle\vec{P}$, by Lemma 4.2.22 we know that their $\hat{\mathtt{S}}_*$, and therefore $\hat{\mathcal{W}}$, are equal. Therefore we have that

$$\begin{aligned}
\hat{\mathcal{W}}((\lambda x.M)N\vec{P}) &= \hat{\mathcal{W}}(M[N/x]\langle\rangle\vec{P}) \\
&= \hat{\mathcal{W}}(M[N/x]\vec{P}\langle\rangle) && \text{(Lemma 4.2.29.4)} \\
&= \hat{\mathcal{W}}(M[N/x]\vec{P}) + 1 && \text{(Lemma 4.2.29.3)} \\
&\leq |M[N/x]\vec{P}| + 1 && \text{(IH)} \\
&= |(\lambda x.M)N\vec{P}| && \text{(Lemma 4.2.32)}
\end{aligned}$$

□

We have shown that the refined measure $\hat{\mathcal{W}}$ computes the exact length of the longest reduction chain from any simply typed term. The key modification with respect to $\mathcal{W}$ was to avoid saving copies of arguments that are not erased, thus closing the gap between the computed value and the actual execution length. In the next section, we analyze the challenges of extending this refinement to intersection types.

4.3 Towards the $\hat{\mathcal{W}}_\cap$ measure: an exact refinement of $\mathcal{W}_\cap$

The case of intersection types presents a new challenge. We recall that the measure $\mathcal{W}_\cap$ is defined over the intrinsic calculus $\Lambda_\cap^{\mathsf{Ch}}$, resorting to the auxiliary non-erasing $\Lambda_\cap^{\mathbf{m}}$, and then lifted to the extrinsic calculus $\Lambda_\cap^{\mathsf{Cu}}$ through correspondence and simulation. Since we are now interested not only in the existence of a measure, but also in the tightness of its computed value, we must further analyze the differences in reduction between $\Lambda_\cap^{\mathsf{Cu}}$ and $\Lambda_\cap^{\mathsf{Ch}}$.

Let us first briefly compare how the approach for $\hat{\mathcal{W}}$ would be insufficient here. In the simply typed λ–calculus, both Curry and Church style versions share the same reduction paths, so we worked straight with an intrinsically typed calculus, and then the same result applies to the extrinsically typed one. It sufficed to avoid, when reducing in the memory calculus, the addition of unnecessary information in the wrappers. We accomplished this by adapting the intrinsic non-erasing calculus $\lambda^{\mathbf{m}}$ into the exact one $\lambda^{\hat{\mathbf{m}}}$. Recreating this idea for IIT would mean to adapt the intrinsic non-erasing calculus $\Lambda_{\cap}^{\mathbf{m}}$ to the exact one $\Lambda_{\cap}^{\hat{\mathbf{m}}}$. We need this step, but it only partially solves the problem: it allows us to prove exactness of the measure against reduction in $\Lambda_{\cap}^{\mathsf{Ch}}$, not against reduction in $\Lambda_{\cap}^{\mathsf{Cu}}$, which is our main target.

For idempotent intersection types, in contrast with simply typed λ–calculus, the transformation between extrinsic and intrinsic presentations is more complex, and there exists a gap in their reduction relations. This is due to arguments in $\Lambda_{\cap}^{\mathsf{Cu}}$ corresponding to possibly several arguments in set-terms from $\Lambda_{\cap}^{\mathsf{Ch}}$. To clearly see this, let us recall that our translation of the Curry system into the Church one is characterized by the transformation of arguments N of type $\vec{A}$ into set-terms $\vec{s}$, where $\vec{s}$ consists of $\{s_1, \ldots, s_n\}$, and s_i is the intrinsically typed term corresponding to N with type $A_i \in \vec{A}$. This seems to be a must for our redex-degrees based approach; other approaches with parallel reduction do not appear to be of use. Since, to proceed with the measure computation, we rely on redex degrees and therefore on types, the information of each sub-derivation in the derivation of an argument must be tracked separately. Congruence rules allow to reduce inside the argument of an application, so the replication occurred during translation may produce additional reduction paths. Consider, for instance, the term from $\Lambda_{\cap}^{\mathsf{Cu}}$

$$(\lambda x.x)((\lambda y.y)(\lambda z.z))$$

with type derivation

$$\star = \frac{\vdash_{\mathsf{Cu}} \lambda y.y : ((A\to A)\to A\to A)\to(A\to A)\to A\to A \quad \Vdash_{\mathsf{Cu}} \lambda z.z : (A\to A)\to A\to A}{\vdash_{\mathsf{Cu}}(\lambda y.y)(\lambda z.z) : (A\to A)\to A\to A}$$

$$\square = \frac{\vdash_{\mathsf{Cu}} \lambda y.y : (A\to A)\to A\to A \quad \Vdash_{\mathsf{Cu}} \lambda z.z : A\to A}{\vdash_{\mathsf{Cu}}(\lambda y.y)(\lambda z.z) : A\to A}$$

$$\frac{\dfrac{x : \{(A\to A)\to A\to A, A\to A\} \vdash_{\mathsf{Cu}} x : A\to A}{\vdash_{\mathsf{Cu}} \lambda x.x : \{(A\to A)\to A\to A, A\to A\}\to A\to A} \quad \dfrac{\star \qquad \square}{\Vdash_{\mathsf{Cu}} (\lambda y.y)(\lambda z.z) : \{(A\to A)\to A\to A, A\to A\}}}{\vdash_{\mathsf{Cu}} (\lambda x.x)((\lambda y.y)(\lambda z.z)) : A\to A}$$

Its translation to $\Lambda_{\cap}^{\mathsf{Ch}}$ is:

$$(\lambda x^{\{(A\to A)\to(A\to A),A\to A\}}.x^{A\to A})\{(\lambda y^{(A\to A)\to(A\to A)}.y)(\lambda z^{A\to A}.z), (\lambda y^{A\to A}.y)(\lambda z^{A}.z)\}$$

Its longest reduction chain in $\Lambda_{\cap}^{\mathsf{Cu}}$ is

$$\begin{aligned}&(\lambda x.x)((\lambda y.y)(\lambda z.z))\\ \to_{\beta}&(\lambda y.y)(\lambda z.z)\\ \to_{\beta}&\lambda z.z\end{aligned}$$

and its longest reduction chain in $\Lambda_{\cap}^{\mathsf{Ch}}$ is

$$\begin{aligned}&(\lambda x^{\{(A\to A)\to(A\to A),A\to A\}}.x^{A\to A})\{(\lambda y^{(A\to A)\to(A\to A)}.y)(\lambda z^{A\to A}.z), (\lambda y^{A\to A}.y)(\lambda z^{A}.z)\}\\ \to_{\mathsf{Ch}}&(\lambda x^{\{(A\to A)\to(A\to A),A\to A\}}.x^{A\to A})\{\lambda z^{A\to A}.z, (\lambda y^{A\to A}.y)(\lambda z^{A}.z)\}\end{aligned}$$

$\to_{\mathsf{Ch}}(\lambda x^{\{(A\to A)\to(A\to A),A\to A\}}.x^{A\to A})\{\lambda z^{A\to A}.z, \lambda z^{A}.z\}$
$\to_{\mathsf{Ch}}\lambda z^{A}.z$

We can clearly see that the intrinsic version has more steps.

The objective of keeping the measure from counting extra steps must be thus extended to the relation between the two typing variants. The main problem we face is that our types might have more information than needed. This is not a problem for reduction in $\Lambda_{\cap}^{\mathsf{Cu}}$, but it is for $\Lambda_{\cap}^{\mathsf{Ch}}$, because extra type information translates into extra multiplication of arguments. This is reflected in our transformation from terms N into set-terms $\vec{s}$, where the size of $\vec{s}$ is determined by the number of types in A, with A being the type of N. Since reduction is allowed within set-terms, we multiply the number of redexes: if A has k types, $\vec{s}$ has k times the number of redexes of N. The number of occurrences of the variable that N is meant to substitute affects the length of the longest reduction chain, so we might end up with more redexes in the translation, and therefore with a greater than needed measured value. After observing that the problem arises from the irrelevant information that a derivation might have, we consider an approach of working with a subset of $\Lambda_{\cap}^{\mathsf{Cu}}$, let us call it $\Lambda_{\cap}^{\mathsf{Cu}'}$, where derivations only have the minimum information required. Note that, within $\Lambda_{\cap}^{\mathsf{Cu}}$, derivations with extra information have the same reduction paths as minimal derivations; reduction is inherent to the term, not to the derivation.

We foresee two possible paths from here. One would be to rely on presentations with more granular control over contexts, such as a multiplicative variant of intersection types [27], recreating the work from Chapter 3 for them. The second one would be to *filter* out redundant derivations, showing that, leaving unchanged the systems and translation that we already defined and used, it is possible to *shrink* redundant derivations and to compute the measure in their "pure" versions, achieving exactness. Since the second approach aims to reutilize most of the work in Chapter 3, and considering that we found an interesting challenge in shrinking redundant derivations, we briefly explore how it could be conducted. We would define a canonicalization function for derivations, that converts derivations with irrelevant information into pure ones. We would leave $\Lambda_{\cap}^{\mathsf{Ch}}$ unchanged, reusing all its results, and allowing to replicate the $\lambda^{\mathbf{m}}$ to $\lambda^{\hat{\mathbf{m}}}$ process. Derivations in $\Lambda_{\cap}^{\mathsf{Cu}'}$, *i.e.*, pure derivations in $\Lambda_{\cap}^{\mathsf{Cu}}$, would be related through a one-to-one simulation theorem to derivations in $\Lambda_{\cap}^{\mathsf{Ch}}$, which would allow us to obtain an exact result for $\Lambda_{\cap}^{\mathsf{Cu}'}$, extensible to $\Lambda_{\cap}^{\mathsf{Cu}}$, from that of $\Lambda_{\cap}^{\mathsf{Ch}}$. In the remainder of this section we explain in more detail the canonicalization idea.

Canonical typing derivations

This section does not introduce a solution to the problem. It rather explores a direction for a solution, noticing the challenges that we encountered and found relevant.

Redundancy in derivations comes from weakening in general. In IIT, in particular, weakening is twofold: it can occur as unnecessary variables in the context, and as unnecessary types within the set-type of a needed variable in the context. Canonical typing derivations are those that avoid these sources of redundancy, which in turn allows to keep the upper bound exact.

Definition 4.3.1 (Canonical derivation)**.** A derivation is canonical if and only if:

1. for each variable x in the context, and for each type A of its set-type, there exists a free occurrence of x with type A in the term;
2. if a bound variable x appears in the body of its lambda, then for each type A in its set-type, there exists at least one occurrence of x with type A in the body;
3. if a bound variable does not appear in the body of its lambda, then its set-type is a singleton.

Every derivation has a corresponding canonical one that preserves the term while allowing the context and type to shrink. Under some circumstances, there might be more than one canonical derivation corresponding to a given derivation. We show some examples.

Example 4.3.2. Let $\Gamma = y : \{\tau, \tau \to \tau\}$.

$$\dfrac{\dfrac{\overline{y : \{\tau,\tau\to\tau\}, x : \{\tau,\tau\to\tau\} \vdash_{\mathsf{Cu}} x : \tau}}{y : \{\tau,\tau\to\tau\} \vdash_{\mathsf{Cu}} \lambda x.x : \{\tau,\tau\to\tau\}\to\tau} \qquad \dfrac{\overline{y : \{\tau,\tau\to\tau\} \vdash_{\mathsf{Cu}} y : \tau} \quad \overline{y : \{\tau,\tau\to\tau\} \vdash_{\mathsf{Cu}} y : \tau\to\tau}}{y : \{\tau,\tau\to\tau\} \Vdash_{\mathsf{Cu}} y : \{\tau,\tau\to\tau\}}}{y : \{\tau,\tau\to\tau\} \vdash_{\mathsf{Cu}} (\lambda x.x)y : \tau}$$

There is no occurrence of x with type $\tau \to \tau$, so it must be erased from the types of x. Furthermore, since we discarded types from a function, we want them to be reflected in its argument too. The canonical derivation is then:

$$\dfrac{\dfrac{\overline{y : \{\tau\}, x : \{\tau\} \vdash_{\mathsf{Cu}} x : \tau}}{y : \{\tau\} \vdash_{\mathsf{Cu}} \lambda x.x : \{\tau\} \to \tau} \qquad \dfrac{\overline{y : \{\tau\} \vdash_{\mathsf{Cu}} y : \tau}}{y : \{\tau\} \Vdash_{\mathsf{Cu}} y : \{\tau\}}}{y : \{\tau\} \vdash_{\mathsf{Cu}} (\lambda x.x)y : \tau}$$

Example 4.3.3. Let $\Gamma = z : \{A, B\}, w : \{C\}$.

$$\square = \dfrac{\dfrac{\dfrac{}{\Gamma, x : \{A,B\} \to C \vdash_{\mathsf{Cu}} x : \{A,B\} \to C} \qquad \dfrac{\overline{\Gamma \vdash_{\mathsf{Cu}} z : A} \quad \overline{\Gamma \vdash_{\mathsf{Cu}} z : B}}{\Gamma \Vdash_{\mathsf{Cu}} z : \{A,B\}}}{\Gamma, x : \{A,B\} \to C \vdash_{\mathsf{Cu}} xz : C}}{\Gamma \vdash_{\mathsf{Cu}} \lambda x.xz : (\{A,B\} \to C) \to C}$$

$$\star = \dfrac{\dfrac{\dfrac{\overline{\Gamma, y : \{A,B\}, v : \{A,B\} \vdash_{\mathsf{Cu}} w : C}}{\Gamma, y : \{A,B\} \vdash_{\mathsf{Cu}} \lambda v.w : \{A,B\} \to C} \qquad \dfrac{\Gamma, y : \{A,B\} \vdash_{\mathsf{Cu}} Iy : A \quad \Gamma, y : \{A,B\} \vdash_{\mathsf{Cu}} Iy : B}{\Gamma, y : \{A,B\} \Vdash_{\mathsf{Cu}} Iy : \{A,B\}}}{\Gamma, y : \{A,B\} \vdash_{\mathsf{Cu}} (\lambda v.w)(Iy) : C}}{\Gamma \vdash_{\mathsf{Cu}} \lambda y.(\lambda v.w)(Iy) : \{A,B\} \to C}$$

$$\dfrac{\square \qquad \star}{\Gamma \vdash_{\mathsf{Cu}} (\lambda x.xz)(\lambda y.(\lambda v.w)(Iy)) : C}$$

Since the variable v is discarded, keeping more than one type for it would result in an over-replication of its arguments when translating to the Church system. The longest path in this erasing case would be to reduce the argument first, so having more than one term in the intrinsic set-term would break exactness. For instance, we could discard B. But then we must discard B from its argument Iy. This implies that there is no occurrence of $y : B$, hence neither B in $\lambda y.(\lambda v.w)(Iy)$. Finally, since $\lambda y.(\lambda v.w)(Iy) : \{A\} \to C$ is a latent argument of x in $\lambda x.xz$, now x must have type $\{A\} \to C$. We get the following derivation, where $\Gamma' = z : \{A\}, w : \{C\}$

$$\square' = \dfrac{\dfrac{\dfrac{}{\Gamma', x : \{A\} \to C \vdash_{\mathsf{Cu}} x : \{A\} \to C} \qquad \dfrac{\overline{\Gamma' \vdash_{\mathsf{Cu}} z : A}}{\Gamma' \Vdash_{\mathsf{Cu}} z : \{A\}}}{\Gamma', x : \{A\} \to C \vdash_{\mathsf{Cu}} xz : C}}{\Gamma' \vdash_{\mathsf{Cu}} \lambda x.xz : (\{A\} \to C) \to C}$$

$$\star' = \dfrac{\dfrac{\dfrac{\overline{\Gamma', y : \{A\}, v : \{A\} \vdash_{\mathsf{Cu}} w : C}}{\Gamma', y : \{A\} \vdash_{\mathsf{Cu}} \lambda v.w : \{A\} \to C} \qquad \dfrac{\Gamma', y : \{A\} \vdash_{\mathsf{Cu}} Iy : A}{\Gamma', y : \{A\} \Vdash_{\mathsf{Cu}} Iy : \{A\}}}{\Gamma', y : \{A\} \vdash_{\mathsf{Cu}} (\lambda v.w)(Iy) : C}}{\Gamma' \vdash_{\mathsf{Cu}} \lambda y.(\lambda v.w)(Iy) : \{A\} \to C}$$

$$\dfrac{\square' \qquad\qquad \star'}{\Gamma' \vdash_{\mathsf{Cu}} (\lambda x.xz)(\lambda y.(\lambda v.w)(Iy)) : C}$$

We identify two specific problems to address when considering the canonicalization of a derivation.

Unused types in bound variables Bound variables are allowed to have more types than those needed by their occurrences. The translation transforms arguments into set-terms by producing several Church variants of the same Curry term. Since the number of variants produced is determined by the number of types of the bound variable, those extra types pose a problem for the measure's exactness.

Example 4.3.4. Consider $(\lambda x.x)(Iy)$, where $\lambda x.x$ has type $\{A, B\} \to A$ in $\Lambda^{\mathsf{Cu}}_{\cap}$. By correspondence (Theorem 3.2.24), it gets translated into $(\lambda x^{\{A,B\}}.x^A)\{Iy^A, Iy^B\}$. The longest reduction chain in the Curry version has 2 steps, while the Church version ends up with four wrappers in its memory, thus placing the upper bound for the longest reduction chain at 4 steps.

$$\begin{array}{lll} & (\lambda x.x)(Iy) & \\ \to & (\lambda x.x)y & \\ \to & y & \end{array} \qquad \begin{array}{ll} & (\lambda x^{\{A,B\}}.x^A)\{Iy^A, Iy^B\} \\ \to_{\mathbf{m}} & (\lambda x^{\{A,B\}}.x^A)\{y^A\langle y^A\rangle, Iy^B\} \\ \to_{\mathbf{m}} & (\lambda x^{\{A,B\}}.x^A)\{y^A\langle y^A\rangle, y^B\langle y^B\rangle\} \\ \to_{\mathbf{m}} & y^A\langle y^A\rangle\langle\{y^A\langle y^A\rangle, y^B\langle y^B\rangle\}\rangle \end{array}$$

Function types in variables Variables with function types, *e.g.* x of type $\{A, B\} \to C$, can be substituted by functions using less types than indicated, *e.g.* only A.

Example 4.3.5. Consider the expression $x(Iz)$, with $x : \{A, B\} \to C$. Its derivation is

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\Gamma, y : \{A\} \vdash y : A}}{\Gamma \vdash \lambda y.y : A \to A} \quad \dfrac{}{\Gamma \vdash z : A}}{\Gamma \vdash (\lambda y.y)z : A} \quad \dfrac{\dfrac{\dfrac{}{\Gamma, y : \{B\} \vdash y : B}}{\Gamma \vdash \lambda y.y : B \to B} \quad \dfrac{}{\Gamma \vdash z : B}}{\Gamma \vdash (\lambda y.y)z : B}}{\Gamma \vdash (\lambda y.y)z : \{A, B\}} \qquad \dfrac{}{\Gamma \vdash x : \{A, B\} \to C}}{\underbrace{x : \{\{A, B\} \to C\}, z : \{A, B\}}_{\Gamma} \vdash x((\lambda y.y)z) : C}$$

Its reduction is

$$\begin{array}{ll} & x(Iz) \\ \to & xz \end{array}$$

And it would be translated into $x^{\{A,B\}\to C}\{Iz^A, Iz^B\}$, which counts an extra step:

$$\begin{array}{ll} & x^{\{A,B\}\to C}\{Iz^A, Iz^B\} \\ \to_{\hat{\mathbf{m}}} & x^{\{A,B\}\to C}\{z^A\langle z^A\rangle, Iz^B\} \\ \to_{\hat{\mathbf{m}}} & x^{\{A,B\}\to C}\{z^A\langle z^A\rangle, z^B\langle z^B\rangle\} \end{array}$$

However, this specific case does not faithfully illustrate the problem, since we could simply discard those extra types for free variables. The problem arises when the x is under a λ that is applied, because then there are redundant weakenings for its types taking place in the argument derivation. Therefore we must trace them.

Example 4.3.6. Consider $(\lambda x.x(Iz))(\lambda y.yyw)$, where:

- x has type $\{A \to B \to C,\ A \to B,\ A\} \to C$,
- y has occurrences only for types $A \to B \to C$ and A,
- z has type $\{A \to B \to C,\ A \to B,\ A\}$, and
- w has type B.

The derivation would be:

$$\mathcal{I}_1 = \dfrac{\dfrac{\dfrac{}{\Gamma', y : A \to B \to C \vdash y : A \to B \to C}}{\Gamma' \vdash \lambda y.y : (A \to B \to C) \to (A \to B \to C)} \quad \dfrac{}{\Gamma' \vdash z : A \to B \to C}}{\Gamma' \vdash (\lambda y.y)z : A \to B \to C}$$

$$\mathcal{I}_2 = \dfrac{\dfrac{\dfrac{}{\Gamma', y : A \to B \vdash y : A \to B}}{\Gamma' \vdash \lambda y.y : (A \to B) \to (A \to B)} \quad \dfrac{}{\Gamma' \vdash z : A \to B}}{\Gamma' \vdash (\lambda y.y)z : A \to B}$$

$$\mathcal{I}_3 = \dfrac{\dfrac{\dfrac{}{\Gamma', y : A \vdash y : A}}{\Gamma' \vdash \lambda y.y : A \to A} \quad \dfrac{}{\Gamma' \vdash z : A}}{\Gamma' \vdash (\lambda y.y)z : A}$$

$$\mathcal{I} = \dfrac{\mathcal{I}_1 \qquad \mathcal{I}_2 \qquad \mathcal{I}_3}{\Gamma' \vdash (\lambda y.y)z : \{A \to B \to C, A \to B, A\}}$$

$$\mathcal{F} = \dfrac{\dfrac{\dfrac{}{\Gamma' \vdash x : \{A \to B \to C, A \to B, A\} \to C} \quad \mathcal{I}}{\underbrace{\Gamma, x : \{A \to B \to C, A \to B, A\} \to C}_{\Gamma'} \vdash x((\lambda y.y)z) : C}}{\Gamma \vdash \lambda x.x((\lambda y.y)z) : (\{A \to B \to C, A \to B, A\} \to C) \to C}$$

$$\mathcal{A} = \dfrac{\dfrac{\dfrac{\dfrac{}{\Gamma'' \vdash y : A \to B \to C} \quad \dfrac{}{\Gamma'' \vdash y : A}}{\Gamma'' \vdash yy : B \to C} \quad \dfrac{}{\Gamma'' \vdash w : B}}{\Gamma'' = \Gamma, y : \{A \to B \to C, A \to B, A\} \vdash yyw : C}}{\Gamma \vdash \lambda y.yyw : \{A \to B \to C, A \to B, A\} \to C}$$

$$\dfrac{\mathcal{F} \qquad \mathcal{A}}{\underbrace{z : \{A \to B \to C, A \to B, A\}, w : B}_{\Gamma} \vdash (\lambda x.x((\lambda y.y)z))(\lambda y.yyw) : C}$$

Weakening would take place in $\lambda y.\, yyw$, since there is no occurrence of type $A \to B$ for y, so we should exclude it there. That is not enough though: we must also change $\lambda x.\, x(Iz)$, in order to avoid the extra duplication of Iz. We show how we expect the different parts of the derivation to

change. We start with the argument that *under-instantiates* the type of its parameter, because it is there where we would find the mismatch.

$$\mathcal{A}' = \dfrac{\dfrac{\dfrac{\overline{\Gamma'' \vdash y : A \to B \to C} \quad \overline{\Gamma'' \vdash y : A}}{\Gamma'' \vdash yy : B \to C} \quad \overline{\Gamma'' \vdash w : B}}{\Gamma'' = \Gamma, y : \{A \to B \to C, \cancel{A \to B}, A\} \vdash yyw : C}}{\Gamma \vdash \lambda y.yyw : \{A \to B \to C, \cancel{A \to B}, A\} \to C}$$

We continue with the function that binds the under-instantiated variable.

$$\mathcal{F}' = \dfrac{\dfrac{\overline{\Gamma' \vdash x : \{A \to B \to C, \cancel{A \to B}, A\} \to C} \quad \mathcal{I}'_0}{\underbrace{\Gamma, x : \{A \to B \to C, \cancel{A \to B}, A\} \to C}_{\Gamma'} \vdash x((\lambda y.y)z) : C}}{\Gamma \vdash \lambda x.x((\lambda y.y)z) : (\{A \to B \to C, \cancel{A \to B}, A\} \to C) \to C}$$

We have now to *update* the types of the arguments of the function variable.

$$\mathcal{I}'_0 = \dfrac{\mathcal{I}_1 \qquad \mathcal{I}_3}{\Gamma' \vdash (\lambda y.y)z : \{A \to B \to C, \cancel{A \to B}, A\}}$$

Finally, we have to update the types of the arguments related to those we just changed. In this case, we must change the types of z, which, since it is free, will involve modifying the whole context.

$$\dfrac{\mathcal{F}' \qquad \mathcal{A}'}{\underbrace{z : \{A \to B \to C, \cancel{A \to B}, A\}, w : B}_{\Gamma} \vdash (\lambda x.x((\lambda y.y)z))(\lambda y.yyw) : C}$$

Summarizing, we must consider changes in the types of *future* arguments in order to determine what is the *pure* type of a function within a given context (by context here we mean the rest of the term, not the typing context). We introduce the concept of *latent substitute*, which is a term that constitutes a potential substitute for a bound variable through β-reduction. For instance, $\lambda y.yyw$ is latent of x in the previous example. We need to look through all the latent substitutes of every variable with a function type, discarding the unused types from all of them, and propagating the changes back to the original variable. This *specialization* process would constitute the core of our proposal to shrink redundant derivations into pure ones. The ideas in this section remain a work in progress.

4.4 Conclusions

In this chapter we further analyzed the number obtained by the wrappers counting measure $\mathcal{W}$ from Section 2.4. We approached it from a quantitative perspective, treating it as an upper bound for the length of the longest reduction chain from the given term. We wondered if the upper bound was exact, finding out it was not. We identified the root of the overmeasurement, and we adapted our auxiliary non-erasing calculus $\lambda^{\mathbf{m}}$ from Section 2.3 to address it. By relying on the new exact non-erasing calculus $\lambda^{\hat{\mathbf{m}}}$, we defined $\hat{\mathcal{W}}$, the refinement to exactness of $\mathcal{W}$, proving that the number it yields coincides with the number of steps of the longest reduction chain (Theorem 4.2.33).

As stated in the introduction, one prior exact decreasing measure exists for simply typed λ–calculus. The novelty comes from adapting our wrappers counting technique, which has the advantages discussed in Chapter 2, obtaining an exact upper bound through a strong decreasing

measure based on non-erasure and redex degrees. Moreover, in Chapter 3 we saw that $\mathcal{W}$ showed enough flexibility to be extended to other systems, so the improvements made to $\mathcal{W}$ have the additional interest of being transferred to the extensions. In the particular case of $\mathcal{W}_\cap$, obtaining a decreasing measure yielding an exact upper bound would consist of a new result. We discussed the challenges of adapting it to exactness, and proposed some ideas for a solution, which are ongoing work. Interestingly, these challenges are not related to the specifics of the technique, but to the gap in reduction paths between the extrinsically and intrinsically typed variants of IIT.

4.5 Resumen en español (Summary in Spanish)

Las medidas decrecientes $\mathcal{W}$ y $\mathcal{W}_\cap$ definidas previamente tienen como codominio los números naturales, por lo tanto el valor asociado a un programa constituye una cota superior para su cadena de reducción más larga. Por ejemplo: sea $t \to t_1 \to ... \to t_n$ la cadena de reducción más larga que puede partir de t; si $\mathcal{W}(t) < n$, sería imposible tener $\mathcal{W}(t) > \mathcal{W}(t_1) > ... > \mathcal{W}(t_n)$.

Desde una perspectiva cuantitativa, sería útil tener una medida que constituya una cota *exacta* para la cadena más larga. Es decir, siguiendo el ejemplo, tal que $\mathcal{W}(t) = n$. No es el caso para $\mathcal{W}$ y $\mathcal{W}_\cap$, que producen un número mayor a la cantidad máxima de pasos de reducción.

Llamamos entonces *exactas* a las medidas que satisfacen $\#(t) = n$, siendo n la longitud de la cadena de reducción más larga desde t, para todo t. La razón por la que $\mathcal{W}$ y $\mathcal{W}_\cap$ no son exactas es la forma en que se agregan las memorias. Los casos donde se agregan pasos extra en $\lambda^{\mathbf{m}}$ están relacionados con reducciones realizadas dentro de las memorias. Podemos refinar la definición de reducción en los cálculos con memoria, para evitar agregar las extra: en vez de que $\to_{\mathbf{m\hat{n}}}$ siempre agregue una memoria con el argumento consumido, consideramos dos casos: si el argumento no se borra, agregamos una memoria vacía; si se borra, agregamos la memoria con el argumento borrado.

En este capítulo presentamos un refinamiento exacto para $\mathcal{W}$, y discutimos el camino hacia un refinamiento exacto para $\mathcal{W}_\cap$.

Chapter 5

Conclusions and future work

We introduced two new decreasing measures for proving SN in the simply typed λ–calculus, and further developed one of them by extending it to idempotent intersection types and by refining it to exactness. The first measure combines previously known concepts into a new, purely syntactic proof. The second one enhances a measure given by Turing, from proving WN to proving SN. The extension of the first one to IIT provides the first strong measure, *i.e.*, that decreases for any redex and therefore directly proves SN, defined natively for that system. Moreover, it has natural numbers as codomain, which is the simplest one compared with the other native measures. It also includes the definition of an intrinsically typed variant of IIT which was shown to correspond to the extrinsic one, a result on its own. The refinement to exactness for the STLC measure consists of modifications such that the resulting value is an exact upper bound of the length of the longest reduction chain. While another exact measure already existed for STLC, this refinement constitutes an improvement of the first result of this thesis, and it provides the foundation for refining the extension to IIT to exactness, where there is no decreasing measure computing an exact upper bound.

5.1 Summary

Termination of programs in typed λ–calculi is an intricate question that we have been exploring for many decades now. While the earliest proofs appeared around 1967 [89], finding new ways to prove the property has remained an active research field. The best-known technique for conducting the proof is *reducibility*. It was introduced by Tait [98] to prove WN originally, and later adapted to SN [50] and extended to System F [51], in both cases by Girard. Reducibility based proofs handle a minimum amount of information, as put by de Vrijer [32]:

> *The computability predicate should then be analyzed as abstracting from all the information–still explicit in the labeled functionals–that is not strictly needed in an induction on terms which successfully establishes strong normalization.*

Hence they end up consisting of very concise arguments with respect to the strength of the result. In addition to conciseness, another key factor in reducibility's popularity is extensibility, as it is the only widely known[1] way of proving SN for the polymorphic λ–calculus, *i.e.*, System F. The technique later evolved to become *logical relations* [2,18,41,42,53,73,74,83,94–96], a generalization of reducibility to abstract out of the specific property, *i.e.*, SN. It is used precisely for proving

[1] There exists an unpublished note by Loader [70] where he presents an alternative proof, but it is remarkably underknown.

complex properties for programming languages. The mechanization of the logical relations SN proof for STLC has been proposed as a benchmark for the power of proof assistants [1].

Grasping the intuitions underpinning reducibility proofs can be difficult. Several works were introduced with the aim of better understanding the nature of the technique [43, 44, 46, 57, 66, 105]. For instance, in a work that introduces a categorical analysis of reducibility, Gallier [45] wrote as motivation:

> *Reducibility proofs are seductive and thrilling, but also elusive. Following these proofs step-by-step, we see that they "work" (when they are not wrong!), but I claim that most of us would still admit that they are not sure* why *these proofs work!*

So, although such a powerful technique was available, the scientific community kept working on alternative approaches for proving SN [8, 26, 32, 48, 58, 65, 70, 71, 76, 84, 89, 91, 103, 105]. One of these is that of *decreasing measures*, consisting of defining functions from terms to well-founded orders such that the result decreases along reduction. Gandy pioneered this approach, with de Vrijer continuing his work. In the same paper where de Vrijer describes reducibility as abstracting out all the concrete information that is not needed, he compares it with his and Gandy's decreasing measures approach:

> *But the proof of strong normalization that corresponds to it–here called the "quick proof"–is simple and transparent. Presumably more transparent than the ones involving a computability predicate. As a matter of fact the quick proof can be viewed as providing some extra insight into the nature of the computability proofs.*

Later, in a work that formally compares reducibility and decreasing measures, van de Pol states something similar:

> *The proof using functionals seems to be more transparent and economizes on proof theoretical complexity.*

Decreasing measures thus proved to be a good alternative way of demonstrating SN.

There exists a long-standing open *koan*, which can be thought of as a not precisely stated problem[2], which was posed by Gödel[3] and proposes to find an "easy" assignment from terms to ordinals such that it decreases along reduction; *i.e.*, an "easy" decreasing measure. Although there is not a direct link from papers by Gandy and de Vrijer to the koan, it has been a main motivation for us. The "easy" part of the problem description is what makes it a koan and not a problem, since it is not clear what we should look for[4]. However, once an answer is found, it should be clearly recognizable as such. For me, an easy measure would be one that successfully and efficiently condenses the patterns underlying reduction into a structure that allows for a simple and intuitive reasoning about the non-deterministic branching of computation in λ–calculus. I emphasize that this is not what I achieved throughout this thesis. It is what the works presented here had as motivation. In pursuing it, we were able to provide new insights and tools to continue identifying what these patterns are, building understanding and intuitions around the nature of computation, gradually and steadily advancing towards a solution of the koan.

[2] A "koan" is a non-precise question for which the space of solutions is a priori not clear. This use of the word was reportedly [15, Problem#26] attributed to Klop.

[3] It was submitted by Barendregt to TLCA's list of open problems [15], now listed as Problem#26. It also appears as Problem #19 of the *Rewriting Techniques and Applications* list of open problems [33].

[4] According to Pólya [87], the candidate solution of a problem should be clearly recognisable as such in a well-defined way. The proof of correctness of the candidate solution still may be hard.

The first contribution of this thesis consists of the measure $\mathcal{W}$, which I informally dubbed here the *wrappers counting* measure, due to the final step of its calculation. Its main contribution is the concretization of previously known ideas into a strong decreasing measure, providing a purely syntactic proof of SN for STLC. We relied on the definition of a non-erasing extension of the STLC based on Klop's syntactic recording of terms, which allowed us to combine Klop–Nederpelt's deduction of a decreasing measure from an increasing one (together with WCR and WN) with Turing's observation on contraction of redexes not being able to create redexes of greater or equal degree. Since Klop–Nederpelt's construction yields a measure for the non-erasing system, we then transferred the measure and its decrease proof to the STLC by defining a new notion of reduction relating both systems. While, as for standard non-erasing calculi, the conservation theorem reducing SN to WN holds for ours, we do not rely on it to conduct our proof; we instead provide a direct proof of SN for STLC through the measure. Prior strong decreasing measures by Gandy and de Vrijer share with ours the use of some flavour of non-erasure (in their case λI), and differ in the measure construction in general. A specific difference that is worth pointing out is in the decrease arguments: theirs rely on the domain of hereditarily strictly monotonic functions (or increasing functionals); ours on properties of our memory calculus that are natural to λ–calculus settings, keeping the development close to it. The measure has been positively received as easy to grasp, which leaves us satisfied since it was the main target for defining it. I suspect that this ease is related to remaining as much as possible within the λ–calculus domain.

Reusing part of the tools defined for the previously described measure, we introduced another one that gathers a greater amount of information. It consists of an extension of a weak[5] decreasing measure proving WN defined by Alan Turing [47] to a strong version proving SN. The original measure's codomain was multisets of redex degrees, while ours is nested multisets of pairs of redex degrees and targets of reduction paths divided by degrees. The complexity of the measure poses an objective flaw from a pragmatic perspective, since we go against efficiency from all points of view. However, we believe that it could be "distilled" into other analysis of reduction information, which would leave it as a complexity increasing step that could evolve into a complexity decreasing one (much like leaving a local minimum in order to get to the global one). Apart from that, I must admit that improving a result by Turing, making a small tribute to one of the fathers of the discipline, was enough motivation.

Later, we replicated the *wrappers counting* approach in the setting of idempotent intersection types. Since the technique involves dealing with redex degrees, and therefore with function types, we require Church-style versions of the target systems. This is not trivial for IIT: Curry terms with several types carry many derivations, and Church terms have single types, so a Curry-term becomes many Church-terms; this generates a gap in the reduction relation between the two systems. We defined an intrinsically typed (Church-style) presentation of intersection types, and we proved that it formally corresponds to the usual extrinsically typed (Curry-style) presentation of IIT. We extended it to memories, obtaining the non-erasing version that allowed us to construct the decreasing measure. We obtained a strong measure (shrinking with the contraction of any redex) designed natively for IIT (not based on translations to non-intersection type systems). Compared with the other two native measures for IIT, it has the simplest codomain of natural numbers. Furthermore, since both are weak, (in the sense that they decrease for a given strategy, entailing WN instead of SN), ours is the first strong and native one. Compared with translation based measures, which can rely on strong measures from STLC, a first—perhaps subjective—advantage is the elegance of native constructions. Objective advantages can also be found, in support of which

[5]We recall that we follow the convention of classifying decreasing measures as *weak* or *strong*, according to the property (WN or SN) they entail.

we analyzed the numbers computed by them. While the result was that neither is tighter than the other, we found that the translation loses information that our direct approach retains, showing signs of improvability. Specifically, a refinement to exactness is ongoing work, as will be discussed in the next paragraph and in the future works section. In that case, the native construction became a requirement beyond elegance.

Finally, we refined the first technique, paying attention not only to achieving the SN result, but also to the information that the obtained value gives us. Both measures based on the technique produce natural numbers as the assigned value. A decreasing natural number is thus associated with each term in a reduction chain, bounding the amount of next steps until normal form. The question of whether our measures produced an exact upper bound arose, and we found that the answer was negative. We refined to exactness the simply typed λ–calculus measure, and we analyzed challenges and possible ways to do so for the intersection types one. The key for STLC lay in stopping non-erasure mechanisms from *saving* terms that were not going to be erased, and comparing the numbers we obtain with those obtained following maximal reductions [106]. The key challenge for IIT lies in the reduction paths gap between the intrinsic and extrinsic variants. The process of translating a Curry term into many Church terms replicates redexes following typing information. Since typing information may be redundant, we may obtain redundant copies in the set of arguments, creating reduction paths in the Church system that do not exist in the Curry one. Early ideas for refining IIT were presented.

The contributions in this thesis advance syntactic proofs of SN in typed λ-calculi, seeking conceptual clarity and practical utility through simple and intuitive methods. Although the ultimate goal of finding an "easy" decreasing measure, solving the longstanding open koan posed by Gödel, remains ambitious, each incremental step yields valuable advances. By continuing this line of research, we expect to uncover new ways of capturing strong normalization through concrete syntactic properties, providing new combinatorial insights into program complexity and termination. We foresee two main directions to go from here: *deepening* and *expanding*.

The deepening direction is about continuing to seek insight within the defined measures, further exploring the *insides* of the given techniques and proofs, with the broad motivation of solving the koan. Since nobody knows what it is that we must look for in order to define an easy decreasing measure, next steps are not so clear. However, the works in this thesis left some ideas to continue working with. We can identify three paths in this direction. The first one is defining new measures. For instance, refining the IIT measure to exactness, and distilling the nested multisets measure in order to simplify its codomain. The second one is to further analyze the introduced results against the other existing techniques. For instance, providing a detailed analysis of the relation between the mechanisms behind our *wrappers counting* measure and Gandy's and de Vrijer's hereditarily strictly monotonic functions/increasing functionals, and formally exploring whether our measure has actual trackable trails of internalizing the conservation theorem from non-erasing calculi. The third one is mechanizing them in proof assistants.

The expanding direction is about continuing to test the limits of the technique, by adapting it to other, more complex, calculi. There are several systems for which decreasing measures have not been defined yet, and which would benefit from having alternative proof techniques. Again, we identify three paths within this direction, depending on the main feature of the system to expand to. The first one is intersection types: continuing to work with them, in particular defining a measure for the system characterizing head normalization. The second one is polymorphism: adapting the *wrappers counting* technique for System F. The third one is modulo isomorphism: in these systems, types and terms are quotiented by an equivalence relation; the classic reducibility technique does not work there due to neutrality instability, so alternative techniques must be considered, resulting in quite complex and extensive proofs.

The future works that were partially explored are briefly described in the next subsection.

5.2 Future directions

Refinement of $\mathcal{W}_\cap$ to exactness

As explained in Chapter 4, decreasing measures with a codomain of natural numbers yield an upper bound for the length of reduction chains from the given term. If this number coincides with the length of the longest reduction chain, it consists of an exact upper bound, so we call the measure *exact*. The measure $\mathcal{W}_\cap$ computes a value that represents the number of consumed arguments that can exist through the execution of the term. The way it counts consumed arguments within non-erased arguments leads to a gap between the count and the actual number: even when the argument is not erased, we save a copy. For this reason, the number that $\mathcal{W}_\cap$ computes serves as an upper bound for execution length, and not as an exact upper bound. In Section 4.2 we modified the measure $\mathcal{W}$, for STLC, obtaining a refinement to exactness. We did this by changing the way of counting consumed arguments within non-erased arguments, avoiding the extra copy. A natural next step is to apply these changes to $\mathcal{W}_\cap$, since there are no decreasing measures yielding an exact upper bound for $\Lambda_\cap$. However, the refinement for IIT poses other problems. The translation from the extrinsic system (à la Curry) to the intrinsic one (à la Church) involves the syntactic multiplication of arguments to match the amount of types of the variable. Since the size of set-term-arguments we get is determined by types, and types are not an indicator of occurrence, it is not clear a priori which are the multiplied arguments that get erased. The adaptation made for $\mathcal{W}$ has thus to be reworked to deal with some occurrences of *weakening*. We consider two possibilities of tackling the problem. One is to completely avoid the problematic form of weakening, using multiplicative presentations of both extrinsic and intrinsic systems. Another is to keep the same systems and define a function to discard problematic forms of weakening from derivations, restricting us to work with these derivations to compute the measure.

Extension to simply typed λ–calculus modulo isomorphisms

The simply typed λ–calculus modulo isomorphisms, formally dubbed System I [36], was introduced by Díaz-Caro and Dowek, proposing to equate isomorphic types as presented by Di Cosmo [35]. There have been several adaptations and extensions of System I. For instance, it was implemented in Haskell by Díaz-Caro and Martínez López [38]; adapted to untyped λ–calculus by Accattoli and Díaz-Caro [3]; adapted to polymorphism by Díaz-Caro, Martínez López, and me [93]; and formalized in Agda by Manzino, Séttimo, and me [97]. These systems involve congruence relations between types, which induce equivalence relations within terms. Term equivalence relates introductory and eliminatory terms, so neutrality is not stable; this rules out CR3, one of the main properties used in reducibility proofs. This problem was solved for System I by using Parigot's alternative reducibility technique [80], which does not rely on CR3. However, the proof is complicated due to β-reduction admitting an arbitrary number of equivalences applied prior to it, which produces a combinatorial explosion of reduction cases. Finding a decreasing measure would supply a great alternative for proving termination in this family of unstable systems. Moreover, since these systems are quite complex, adapting $\mathcal{W}$ to them would be a great sign of extensibility.

Extension of $\mathcal{W}$ to System F

System F extends simply typed λ–calculus with polymorphism, allowing term-level functions to depend on types. Contracting the redex of a type abstraction can enlarge the type of existing redexes, so Turing's observation does not hold here, and the *wrappers counting* technique cannot be applied as is. One of the most powerful aspects of reducibility is that it can be extended to System F, thanks to the refinement made by Girard when he introduced *reducibility candidates*. This refinement consists, very briefly, of a parameterization of reducibility sets for types, following the system's parameterization on types. Combinatorial proofs that are formalizable in Peano arithmetic are not extensible to polymorphic settings. The reason is that the expressive power of such systems corresponds to second order arithmetic, and, since SN entails consistency, such proofs would go against Gödel's incompleteness theorems. However, there exists a decreasing measure for System F by Loader, which can be found in an unpublished note [70]. We plan to explore if it is possible to adapt our decreasing measure in such a way that it can handle impredicativity.

Extension to System F modulo isomorphisms

System F modulo isomorphisms, dubbed Polymorphic System I (PSI) [93] was introduced by Díaz-Caro, Martínez López, and me. It adds the usual polymorphism constructions as well as their associated isomorphisms. Defining a decreasing measure there seems quite difficult, since we are dealing with the joint complexity and instability problems exhibited for both System F and System I. However, we do not expect this combination of problems to be more troublesome than their parts, so if the extensions of $\mathcal{W}$ to System F and System I are solved, this would be the natural next step.

Extension to IIT characterizing head normalization

Our first extension to IIT is based on the original intersection types system, which characterizes strongly normalizing terms. In this work we propose to tackle the variant characterizing *head normalization*, where terms are typable if and only if the strategy that contracts the redex at head position terminates. The system was introduced by Coppo, Dezani and Venneri [25], and it allows untyped subterms in typed terms. Specifically, there is an axiom that allows to assign the empty type $\{\}$ to any term. For instance, the judgement $x : \{\} \to A \vdash x\Omega : A$ is typable with $\vdash \Omega : \{\}$. Our syntactic approach requires the typing information, so we rely on intrinsically typed presentations of the systems. Specifically, we rely on the type of variables for the simplification function to assess if a given redex must be transformed. Relaxing the typing rules to allow derivations with no typing information poses a new challenge: it is not clear how to type-decorate a term without it. Extending the measure to accommodate these mixed-typed constructs is non-trivial, and it remains open whether our approach can be naturally extended. We consider two possible paths to solve this problem. One is to work with the intersection type system characterizing solvable terms where subterms are mandatorily typable; and the other is to work with an approximants calculus, introducing Ω as an explicit constant with no reducts and replacing all forms of divergence.

5.3 Resumen en español (Summary in Spanish)

Introducimos dos medidas decrecientes para demostrar SN en STLC, y continuamos desarrollando una de ellas extendiéndola a tipos intersección idempotente y refinándola a exactitud. La primera medida combina conceptos conocidos produciendo una nueva prueba puramente sintáctica. La

segunda mejora una medida definida por Turing, pasando de probar WN a probar SN. La extensión de la primera a IIT provee la primera medida fuerte, es decir que decrece ante la contracción de cualquier redex y por lo tanto demuestra SN directamente, definida de manera nativa para este sistema. Su codominio son los números naturales, el más sencillo entre las medidas nativas. Incluyó además la definición de una variante con tipado intrínseco de IIT, que demostramos que se corresponde con el extrínseco, un resultado en sí mismo. El refinado de la medida a exactitud en STLC consistió en modificaciones para que el valor resultante sea una cota superior exacta de la longitud de las cadenas de reducción. Aunque existe otra medida exacta para STLC, este refinado constituye una mejora del primer resultado de esta tesis, y provee la base para el refinado a exactitud de la medida para IIT, donde no hay medidas decrecientes computando cotas exactas.

Bibliography

[1] Andreas Abel, Guillaume Allais, Aliya Hameer, Brigitte Pientka, Alberto Momigliano, Steven Schäfer, and Kathrin Stark. POPLMark reloaded: Mechanizing proofs by logical relations. *Journal of Functional Programming*, 29:e19, 2019.

[2] Samson Abramsky. Domain theory in logical form. *Annals of Pure and Applied Logic*, 51:1–77, 1991.

[3] Beniamino Accattoli and Alejandro Díaz-Caro. Functional pearl: The distributive λ-calculus. In Keisuke Nakano and Konstantinos Sagonas, editors, *Functional and Logic Programming (FLOPS 2020)*, volume 12073 of *Lecture Notes in Computer Science*, pages 34–49. Springer, Cham, 2020.

[4] Beniamino Accattoli and Delia Kesner. The structural *lambda*-calculus. In *Computer Science Logic, 24th International Workshop, CSL 2010, 19th Annual Conference of the EACSL, Brno, Czech Republic, August 23-27, 2010. Proceedings*, pages 381–395, 2010.

[5] Franco Barbanera, Mariangiola Dezani-Ciancaglini, Ugo de'Liguoro, and Betti Venneri. YACC: yet another Church calculus - A birthday present for Herman inspired by his supervisor activity. In Venanzio Capretta, Robbert Krebbers, and Freek Wiedijk, editors, *Logics and Type Systems in Theory and Practice - Essays Dedicated to Herman Geuvers on The Occasion of His 60th Birthday*, volume 14560 of *Lecture Notes in Computer Science*, pages 17–35. Springer, 2024.

[6] Pablo Barenbaum and Eduardo Bonelli. Optimality and the linear substitution calculus. In *2nd International Conference on Formal Structures for Computation and Deduction, FSCD 2017, September 3-9, 2017, Oxford, UK*, pages 9:1–9:16, 2017.

[7] Pablo Barenbaum, Simona Ronchi Della Rocca, and Cristian Sottile. Strong normalization through idempotent intersection types: a new syntactical approach. *Electronic Notes in Theoretical Informatics and Computer Science*, Volume 5 - Proceedings of MFPS XLI, Dec 2025.

[8] Pablo Barenbaum and Cristian Sottile. Two decreasing measures for simply typed λ-terms. In Marco Gaboardi and Femke van Raamsdonk, editors, *8th International Conference on Formal Structures for Computation and Deduction, FSCD 2023, July 3-6, 2023, Rome, Italy*, volume 260 of *LIPIcs*, pages 11:1–11:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023.

[9] H. P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*, volume 103 of *Studies in Logic and the Foundations of Mathematics*. North-Holland, Amsterdam, 1984.

[10] Henk Barendregt, Mario Coppo, and Mariangiola Dezani-Ciancaglini. A filter lambda model and the completeness of type assignment. *J. Symb. Log.*, 48(4):931–940, 1983.

[11] Henk P. Barendregt, Wil Dekkers, and Richard Statman. *Lambda Calculus with Types.* Lambda Calculus with Types. Cambridge University Press, 2013.

[12] Henk P. Barendregt and Giulio Manzonetto. Turing's contributions to lambda calculus. In B. Cooper and J. van Leeuwen, editors, *Alan Turing - His Work and Impact*, pages 139–143. Elsevier, 2013.

[13] Alexis Bernadet and Stéphane Lengrand. Complexity of strongly normalising *lambda*-terms via non-idempotent intersection types. In *FOSSACS 2011*, pages 88–107, 2011.

[14] Frédéric Blanqui. Termination of rewrite relations on λ-terms based on Girard's notion of reducibility. *Theoretical Computer Science*, 611:50–86, 2016.

[15] TCLA Editorial Board. TLCA list of open problems. `http://tlca.di.unito.it/opltlca/`, 2006.

[16] Viviana Bono, Betti Venneri, and Lorenzo Bettini. A typed lambda calculus with intersection types. *Theoretical Computer Science*, 398(1-3):95–113, 2008.

[17] Gérard Boudol. On strong normalization in the intersection type discipline. In Martin Hofmann, editor, *Typed Lambda Calculi and Applications, 6th International Conference, TLCA 2003, Valencia, Spain, June 10-12, 2003, Proceedings*, volume 2701 of *Lecture Notes in Computer Science*, pages 60–74. Springer, 2003.

[18] Val Breazu-Tannen and Thierry Coquand. Extensional models for polymorphism. *Theoretical Computer Science*, 59:85–114, 1988.

[19] Antonio Bucciarelli, Adolfo Piperno, and Ivano Salvo. Intersection types and lambda-definability. *Math. Struct. Comput. Sci.*, 13(1):15–53, 2003.

[20] Alonzo Church. A note on the Entscheidungsproblem. *J. Symb. Log.*, 1(1):40–41, 1936.

[21] Alonzo Church. A formulation of the simple theory of types. *J. Symb. Log.*, 5(2):56–68, 1940.

[22] Mario Coppo and Mariangiola Dezani-Ciancaglini. An extension of the basic functionality theory for the λ-calculus. *Notre Dame J. Formal Log.*, 21(4):685–693, 1980.

[23] Mario Coppo, Mariangiola Dezani-Ciancaglini, and Patrick Sallé. Functional characterization of some semantic equalities inside lambda-calculus. In Hermann A. Maurer, editor, *Automata, Languages and Programming, 6th Colloquium, Graz, Austria, July 16-20, 1979, Proceedings*, volume 71 of *Lecture Notes in Computer Science*, pages 133–146. Springer, 1979.

[24] Mario Coppo, Mariangiola Dezani-Ciancaglini, and Betti Venneri. Functional characters of solvable terms. *Math. Log. Q.*, 27(2-6):45–58, 1981.

[25] Mario Coppo, Mariangiola Dezani-Ciancaglini, and Betti Venneri. Functional characters of solvable terms. *Math. Log. Q.*, 27(2-6):45–58, 1981.

[26] René David. Normalization without reducibility. *Ann. Pure Appl. Log.*, 107(1-3):121–130, 2001.

[27] Erika De Benedetti and Simona Ronchi Della Rocca. A type assignment for λ-calculus complete both for FPTIME and strong normalization. *Information and Computation*, 248:195–214, 2016. Development on Implicit Computational Complexity (DICE 2013).

[28] Daniel de Carvalho. Execution time of λ-terms via denotational semantics and intersection types. *Math. Struct. Comput. Sci.*, 28(7):1169–1203, 2018.

[29] Pierre de Fermat. Relation des nouvelles découvertes en la science des nombres. In Paul Tannery and Charles Henry, editors, *Oeuvres de Fermat*, volume 2, pages 431–436. Gauthier-Villars, Paris, 1894. Letter of August 1659 to Carcavi.

[30] Philippe de Groote. The conservation theorem revisited. In Marc Bezem and Jan Friso Groote, editors, *Typed Lambda Calculi and Applications, International Conference on Typed Lambda Calculi and Applications, TLCA '93, Utrecht, The Netherlands, March 16-18, 1993, Proceedings*, volume 664 of *Lecture Notes in Computer Science*, pages 163–178. Springer, 1993.

[31] Roel de Vrijer. A direct proof of the finite developments theorem. *The Journal of symbolic logic*, 50(2):339–343, 1985.

[32] Roel de Vrijer. Exactly estimating functionals and strong normalization. In *Indagationes Mathematicae (Proceedings)*, volume 90, pages 479–493. North-Holland, 1987.

[33] Nachum Dershowitz, Jean-Pierre Jouannaud, and Jan Willem Klop. Open problems in rewriting. In *International Conference on Rewriting Techniques and Applications*, pages 445–456. Springer, 1991.

[34] Nachum Dershowitz and Zohar Manna. Proving termination with multiset orderings. *Communications of the ACM*, 22(8):465–476, 1979.

[35] Roberto Di Cosmo. *Isomorphisms of Types: From λ-Calculus to Information Retrieval and Language Design*. Progress in Theoretical Computer Science. Birkhäuser Boston, 1995.

[36] Alejandro Díaz-Caro and Gilles Dowek. Proof normalisation in a logic identifying isomorphic propositions. In Herman Geuvers, editor, *4th International Conference on Formal Structures for Computation and Deduction (FSCD 2019)*, volume 131 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 14:1–14:23. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2019.

[37] Alejandro Díaz-Caro and Gilles Dowek. Extensional proofs in a propositional logic modulo isomorphisms. *Theoretical Computer Science*, 977:114172, 2023.

[38] Alejandro Díaz-Caro and Pablo E. Martínez López. Isomorphisms considered as equalities: Projecting functions and enhancing partial application through an implementation of λ^+. *ACM IFL*, 2015(9):1–11, 2015.

[39] Jörg Endrullis, Dimitri Hendriks, Jan Willem Klop, and Andrew Polonsky. Clocked lambda calculus. *Math. Struct. Comput. Sci.*, 27(5):782–806, 2017.

[40] Robert W. Floyd. Assigning meanings to programs. In J. T. Schwartz, editor, *Mathematical Aspects of Computer Science*, volume 19 of *Proceedings of Symposia in Applied Mathematics*, pages 19–32. American Mathematical Society, Providence, RI, 1967.

[41] Harvey Friedman. Equality between functionals. In Rohit Parikh, editor, *Logic Colloquium*, volume 453 of *Lecture Notes in Mathematics*, pages 22–37. Springer-Verlag, 1975.

[42] Jean Gallier. A Note on Logical PERs and Reducibility. Logical Relations Strike Again! In Antoine Amarilli and Alin Deutsch, editors, *The Provenance of Elegance in Computation - Essays Dedicated to Val Tannen*, volume 119 of *Open Access Series in Informatics (OASIcs)*, pages 7:1–7:12, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[43] Jean H. Gallier. On Girard's "candidats de reductibilité". In P. Odifreddi, editor, *Logic and Computer Science*, pages 123–203. Academic Press, London, New York, May 1990.

[44] Jean H. Gallier. On the correspondence between proofs and λ-terms. In P. DeGroote, editor, *The Curry-Howard Isomorphism, Cahiers du Centre de Logique, No. 8*, pages 55–138. Université Catholique de Louvain, 1995.

[45] Jean H. Gallier. Proving properties of typed lambda-terms using realizability, covers, and sheaves. *Theor. Comput. Sci.*, 142(2):299–368, 1995.

[46] Jean H. Gallier. Typing untyped lambda terms, or reducibility strikes again! *Annals of Pure and Applied Logic*, 91:231–270, 1998.

[47] Robin O. Gandy. An early proof of normalization by A.M. Turing. In J.P. Seldin and J.R. Hindley, editors, *To H.B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pages 453–455. Academic Press, 1980.

[48] Robin O. Gandy. Proofs of strong normalization. In J.P. Seldin and J.R. Hindley, editors, *To H.B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pages 457–477. Academic Press, 1980.

[49] Philippa Gardner. Discovering needed reductions using type theory. In Masami Hagiya and John C. Mitchell, editors, *Theoretical Aspects of Computer Software, International Conference TACS '94, Sendai, Japan, April 19-22, 1994, Proceedings*, volume 789 of *Lecture Notes in Computer Science*, pages 555–574. Springer, 1994.

[50] Jean-Yves Girard. Une extension de l'interprétation de Gödel à l'analyse, et son application à l'élimination des coupures dans l'analyse et la théorie des types. In J.E. Fenstad, editor, *Proceedings of the Second Scandinavian Logic Symposium*, volume 63 of *Studies in Logic and the Foundations of Mathematics*, pages 63–92. Elsevier, 1971.

[51] Jean-Yves Girard. *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur*. PhD thesis, Université Paris 7, 1972.

[52] Jean-Yves Girard, Paul Taylor, and Yves Lafont. *Proofs and types*. Cambridge U.P., UK, 1989.

[53] Robert Harper. PFPL supplement: How to (re)invent Tait's method. `https://www.cs.cmu.edu/~rwh/courses/chtt/pdfs/tait.pdf`, 2022.

[54] Thomas L. Heath. *The Thirteen Books of Euclid's Elements*. Cambridge University Press, 1908.

[55] C. A. R. Hoare. An axiomatic basis for computer programming. *Commun. ACM*, 12(10):576–580, October 1969.

[56] W.A. Howard. Assignment of ordinals to terms for primitive recursive functionals of finite type. In A. Kino, J. Myhill, and R.E. Vesley, editors, *Intuitionism and Proof Theory: Proceedings of the Summer Conference at Buffalo N.Y. 1968*, volume 60 of *Studies in Logic and the Foundations of Mathematics*, pages 443–458. Elsevier, 1970.

[57] Gérard Huet. *Initiation au λ-Calcul*. Université Paris VII, 1991. Lecture Notes.

[58] Felix Joachimski and Ralph Matthes. Short proofs of normalization for the simply-typed lambda-calculus, permutative conversions and Gödel's T. *Arch. Math. Log.*, 42(1):59–87, 2003.

[59] Cliff B. Jones. Turing's 1949 paper in context. In Jarkko Kari, Florin Manea, and Ion Petre, editors, *Unveiling Dynamics and Complexity*, pages 32–41, Cham, 2017. Springer International Publishing.

[60] Michael Karr. "delayability" in proofs of strong normalizability in the typed lambda calculus. In Hartmut Ehrig, Christiane Floyd, Maurice Nivat, and James Thatcher, editors, *Mathematical Foundations of Software Development*, pages 208–222, Berlin, Heidelberg, 1985. Springer Berlin Heidelberg.

[61] A. J. Kfoury. A linearization of the lambda-calculus and consequences. *J. Log. Comput.*, 10(3):411–436, 2000.

[62] A. J. Kfoury and J. B. Wells. New notions of reduction and non-semantic proofs of beta-strong normalization in typed lambda-calculi. In *Proceedings, 10th Annual IEEE Symposium on Logic in Computer Science, San Diego, California, USA, June 26-29, 1995*, pages 311–321. IEEE Computer Society, 1995.

[63] Zurab Khasidashvili, Mizuhito Ogawa, and Vincent van Oostrom. Uniform normalisation beyond orthogonality. In Aart Middeldorp, editor, *Rewriting Techniques and Applications, 12th International Conference, RTA 2001, Utrecht, The Netherlands, May 22-24, 2001, Proceedings*, volume 2051 of *Lecture Notes in Computer Science*, pages 122–136. Springer, 2001.

[64] Zurab O. Khasidashvili. β-reductions and β-developments of λ-terms with the least number of steps. In Per Martin-Löf and Grigori Mints, editors, *COLOG-88*, pages 105–111, Berlin, Heidelberg, 1990. Springer Berlin Heidelberg.

[65] Jan Willem Klop. *Combinatory Reduction Systems*. PhD thesis, Utrecht University, 1980.

[66] Jean-Louis Krivine. *Lambda-Calcul, Types et Modèles*. Etudes et Recherches en Informatique. Masson, 1990.

[67] Jean-Louis Krivine. *Lambda-calculus, types and models*. Ellis Horwood series in computers and their applications. Masson, 1993.

[68] Jean-Jacques Lévy. *Réductions correctes et optimales dans le lambda-calcul*. PhD thesis, Université de Paris 7, 1978.

[69] Luigi Liquori and Simona Ronchi Della Rocca. Intersection-types à la Church. *Inf. Comput.*, 205(9):1371–1386, 2007.

[70] Ralph Loader. Normalisation by calculation. Unpublished manuscript announced on the "Types" mailing list, 1995.

[71] Ralph Loader. Notes on simply typed lambda calculus. Technical Report ECS-LFCS-98-381, University of Edinburgh, 1998.

[72] Paul-André Melliès. Axiomatic rewriting theory I: A diagrammatic standardization theorem. In Aart Middeldorp, Vincent van Oostrom, Femke van Raamsdonk, and Roel C. de Vrijer, editors, *Processes, Terms and Cycles: Steps on the Road to Infinity, Essays Dedicated to Jan Willem Klop, on the Occasion of His 60th Birthday*, volume 3838 of *Lecture Notes in Computer Science*, pages 554–638. Springer, 2005.

[73] John C. Mitchell. A type-inference approach to reduction properties and semantics of polymorphic expressions. In *ACM Conference on LISP and Functional Programming*, pages 308–319, 1986.

[74] John C. Mitchell and Eugenio Moggi. Kripke-style models for typed lambda calculus. *Annals of Pure and Applied Logic*, 51:99–124, 1991.

[75] F. Lockwood Morris and Cliff B. Jones. An early program proof by Alan Turing. *Annals of the History of Computing*, 6:139–143, 1984.

[76] Robert Pieter Nederpelt. *Strong normalization in a typed lambda calculus with lambda structured types.* Phd thesis, TU Eindhoven, 1973.

[77] Peter Møller Neergaard and Morten Heine Sørensen. Conservation and uniform normalization in lambda calculi with erasing reductions. *Inf. Comput.*, 178(1):149–179, 2002.

[78] Tobias Nipkow. Higher-order critical pairs. In *Proceedings 1991 Sixth Annual IEEE Symposium on Logic in Computer Science*, pages 342–343. IEEE Computer Society, 1991.

[79] Luca Paolini, Mauro Piccolo, and Simona Ronchi Della Rocca. Essential and relational models. *Math. Struct. Comput. Sci.*, 27(5):626–650, 2017.

[80] Michel Parigot. Proofs of strong normalisation for second order classical natural deduction. *J. Symb. Log.*, 62(4):1461–1479, 1997.

[81] Benjamin C. Pierce. *Types and Programming Languages.* MIT Press, Cambridge, MA, 2002.

[82] Jan von Plato. Gentzen's proof of normalization for natural deduction. *Bulletin of Symbolic Logic*, 14(2):240–257, 2008.

[83] Gordon D. Plotkin. Lambda definability in the full type hierarchy. In J. P. Seldin and J. R. Hindley, editors, *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pages 363–373. Academic Press, London, 1980.

[84] Jaco van de Pol and Helmut Schwichtenberg. Strict functionals for termination proofs. In *Proceedings of the Second International Conference on Typed Lambda Calculi and Applications*, TLCA '95, page 350–364, Berlin, Heidelberg, 1995. Springer-Verlag.

[85] G. Pottinger. A type assignment for the strong normalizable lambda-terms. In J.R. Hindley and J.P. Seldin, editors, *To H.B. Curry: Essays on combinatory logic, lambda calculus and formalisms*, pages 561–577. Academic Press, 1980.

[86] Dag Prawitz. *Natural deduction: a proof-theoretical study.* PhD thesis, Almqvist & Wiksell, 1965.

[87] George Pólya and John Horton Conway. *How to solve it.* Princeton science library. Princeton University Press, Princeton [N.J.], expanded princeton science library ed. edition, 2004. Includes bibliographical references.

[88] Simona Ronchi Della Rocca and Luca Paolini. *The Parametric Lambda Calculus - A Metamodel for Computation.* Texts in Theoretical Computer Science. An EATCS Series. Springer, 2004.

[89] Luis Elpidio Sanchis. Functionals defined by recursion. *Notre Dame Journal of Formal Logic*, 8(3):161–174, July 1967.

[90] Morten Heine Sørensen. *Normalization in λ-calculus and type theory.* PhD thesis, University of Copenhagen, Denmark, 1997.

[91] Morten Heine Sørensen. Strong normalization from weak normalization in typed λ-calculi. *Information and Computation*, 133(1):35–71, 1997.

[92] Morten Heine Sørensen and Pawel Urzyczyn. *Lectures on the Curry-Howard isomorphism*, volume 149. Elsevier, 2006.

[93] Cristian Sottile, Alejandro Díaz-Caro, and Pablo E. Martínez López. Polymorphic System I. In *Proceedings of the 32nd Symposium on Implementation and Application of Functional Languages*, IFL '20, page 127–137, New York, NY, USA, 2021. Association for Computing Machinery.

[94] Richard Statman. Completeness, invariance, and λ-definability. *Journal of Symbolic Logic*, 47(1):17–26, 1982.

[95] Richard Statman. Equality between functionals, revisited. In L. A. Harrington, M. D. Morley, A. Scedrov, and S. G. Simpson, editors, *Harvey Friedman's Research on the Foundations of Mathematics*, pages 331–338. North-Holland, 1985.

[96] Richard Statman. Logical relations and the typed λ-calculus. *Information and Control*, 65(2–3):85–97, 1985.

[97] Agustín Séttimo, Cristian Sottile, and Cecilia Manzino. A formalization of System I with type Top in Agda, 2026. To appear in the postproceedings of IFL'25.

[98] William W. Tait. Intensional interpretations of functionals of finite type I. *J. Symb. Log.*, 32(2):198–212, 1967.

[99] Terese. *Term Rewriting Systems*, volume 55 of *Cambridge Tracts in Theoretical Computer Science.* Cambridge University Press, 2003.

[100] Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proc. London Math. Soc.*, s2-42(1):230–265, 1937.

[101] Alan M. Turing. Checking a large routine. In *Report of a Conference on High Speed Automatic Calculating Machines*, pages 67–69, Cambridge, UK, June 1949. University Mathematical Laboratory, University of Cambridge. Paper presented at the inaugural EDSAC conference (24 June 1949).

[102] Steffen van Bakel. Complete restrictions of the intersection type discipline. *Theor. Comput. Sci.*, 102:135–163, 1992.

[103] D. T. van Daalen. *The language theory of Automath.* PhD thesis, Eindhoven University of Technology, Eindhoven, The Netherlands, 1980.

[104] D.T. van Daalen. The language theory of Automath: Parts of chapters II, IV, V–VIII. In R.P. Nederpelt, J.H. Geuvers, and R.C. de Vrijer, editors, *Selected Papers on Automath*, volume 133 of *Studies in Logic and the Foundations of Mathematics*, pages 493–653. Elsevier, 1994.

[105] Jaco van de Pol. Two *Different* strong normalization proofs? In Gilles Dowek, Jan Heering, Karl Meinke, and Bernhard Möller, editors, *Higher-Order Algebra, Logic, and Term Rewriting, Second International Workshop, HOA '95, Paderborn, Germany, September 21-22, 1995, Selected Papers*, volume 1074 of *Lecture Notes in Computer Science*, pages 201–220. Springer, 1995.

[106] Femke van Raamsdonk, Paula Severi, Morten Heine Sørensen, and Hongwei Xi. Perpetual reductions in lambda-calculus. *Inf. Comput.*, 149(2):173–225, 1999.

[107] Femke van Raamsdonk and Paula G. Severi. On normalisation. Computing Science Reports 95-20, Technische Universiteit Eindhoven, Eindhoven, 1995.

[108] Philip Wadler. Propositions as types. *Commun. ACM*, 58(12):75–84, November 2015.

[109] Gunnar Wilken and Andreas Weiermann. Derivation lengths classification of Gödel's T extending Howard's assignment. *Log. Methods Comput. Sci.*, 8(1), 2012.

Appendix A

Proofs of Chapter 2

A.1 Proofs of Section 2.4

Definition A.1.1 (Generalization of notions to memories)**.** We generalize some of the notions to memories as follows:

1. The reduction relation $\to_{\mathbf{m}}$ is extended to operate on memories with the two following inductively defined rules:
   1.1. If $t \to_{\mathbf{m}} t'$ then $\mathtt{L}\langle t\rangle \to_{\mathbf{m}} \mathtt{L}\langle t'\rangle$.
   1.2. If $\mathtt{L} \to_{\mathbf{m}} \mathtt{L}'$ then $\mathtt{L}\langle t\rangle \to_{\mathbf{m}} \mathtt{L}'\langle t\rangle$.
2. The max-degree is extended to memories as follows:

$$\delta_{\mathsf{max}}(\Box) = 0 \qquad \delta_{\mathsf{max}}(\mathtt{L}\langle t\rangle) = \max(\delta_{\mathsf{max}}(\mathtt{L}), \delta_{\mathsf{max}}(t))$$

Lemma A.1.2 (Terms reduce to its simplification)**.** For every term t and for all $k \geq 1$ we have that $t \to^*_{\mathbf{m}} \mathtt{S}_k(t)$.

Proof. To prove it by induction, we generalize the statement to memories, *i.e.*, $\mathtt{L} \to^*_{\mathbf{m}} \mathtt{S}_k(\mathtt{L})$. We proceed by simultaneous induction on t and $\mathtt{L}$:

1. $t = x$: Immediate, as $x \to^*_{\mathbf{m}} x = \mathtt{S}_k(x)$ in zero steps.
2. $t = \lambda x.s$: Then $\lambda s. \to^*_{\mathbf{m}} \lambda x.\mathtt{S}_k(s) = \mathtt{S}_k(\lambda x.s)$ by IH.
3. $t = (\lambda x.s)\mathtt{L}\, r$ where $(\lambda x.s)\mathtt{L}$ is an $\mathbf{m}$-abstraction of degree k: By IH
$$(\lambda x.s)\mathtt{L}\, r \to^*_{\mathbf{m}} (\lambda x.\mathtt{S}_k(s))\mathtt{S}_k(\mathtt{L})\, \mathtt{S}_k(r) \to_{\mathbf{m}} \mathtt{S}_k(s)[\mathtt{S}_k(r)/x]\langle \mathtt{S}_k(r)\rangle \mathtt{S}_k(\mathtt{L}) = \mathtt{S}_k((\lambda x.s)\mathtt{L}\, r) = \mathtt{S}_k(t)$$
4. $t = s\, r$ where s is not an $\mathbf{m}$-abstraction of degree k: By IH
$$s\, r \to^*_{\mathbf{m}} \mathtt{S}_k(s)\, \mathtt{S}_k(r) = \mathtt{S}_k(s\, r) = \mathtt{S}_k(t)$$
5. $t = s\langle r\rangle$: By IH $s\langle r\rangle \to^*_{\mathbf{m}} \mathtt{S}_k(s)\langle \mathtt{S}_k(r)\rangle = \mathtt{S}_k(s\langle r\rangle) = \mathtt{S}_k(t)$.
6. $\mathtt{L} = \Box$: Immediate, as $\Box \to^*_{\mathbf{m}} \Box = \mathtt{S}_k(\Box)$ in zero steps.
7. $\mathtt{L} = \mathtt{L}'\langle t\rangle$: By IH $\mathtt{L}'\langle t\rangle \to^*_{\mathbf{m}} \mathtt{S}_k(\mathtt{L}')\langle \mathtt{S}_k(t)\rangle = \mathtt{S}_k(\mathtt{L}'\langle t\rangle)$.

□

Lemma A.1.3 (Substitution of terms of lower type does not create abstractions)**.** If $\mathsf{h}(\mathsf{type}(t)) > \mathsf{h}(\mathsf{type}(s))$ and t not an **m**-abstraction, then $t[s/x]$ is not an **m**-abstraction.

Proof. By induction on t:

1. $t = y$: We claim that $y \neq x$. Indeed, note that the type of y is $\mathsf{type}(t)$ but the type of x is $\mathsf{type}(s)$. By contradiction, suppose that $x = y$. Then $\mathsf{type}(t) = \mathsf{type}(s)$ and in particular $\mathsf{h}(\mathsf{type}(t)) > \mathsf{h}(\mathsf{type}(s)) = \mathsf{h}(\mathsf{type}(t))$, which is impossible. Then we have that $y \neq x$, so $t[s/x] = y[s/x] = y$, which is not an **m**-abstraction.

2. $t = \lambda x.t'$: Impossible, since t is not an **m**-abstraction by hypothesis.

3. $t = t_1\, t_2$: Then $t[s/x] = t_1[s/x]\, t_2[s/x]$ is trivially not an abstraction.

4. $t = t_1\langle t_2\rangle$: Note that t_1 is not an **m**-abstraction by Rem. 2.3.2. Furthermore, note that $\mathsf{type}(t_1) = \mathsf{type}(t)$ so in particular $\mathsf{h}(\mathsf{type}(t_1)) = \mathsf{h}(\mathsf{type}(t)) > \mathsf{h}(\mathsf{type}(s))$. We are under the conditions to apply the IH on t_1, hence $t_1[s/x]$ is not an **m**-abstraction. To conclude, note that $t[s/x] = t_1[s/x]\langle t_2[s/x]\rangle$ cannot be an **m**-abstraction by Rem. 2.3.2.

□

Lemma A.1.4 (Simplification does not create abstractions)**.** If $\mathsf{h}(\mathsf{type}(t)) \geq k$ and $\delta_{\mathsf{max}}(t) \leq k$ and t is not an **m**-abstraction, then $\mathtt{S}_k(t)$ is not an **m**-abstraction.

Proof. By induction on t:

1. $t = x$: Then $\mathtt{S}_k(x) = x$ is not an **m**-abstraction.

2. $t = \lambda x.s$: Impossible, since t is not an **m**-abstraction by hypothesis.

3. $t = (\lambda x.s)\mathtt{L}\, r$ where $(\lambda x.s)\mathtt{L}$ is an **m**-abstraction of degree k: We claim that this case is impossible. Writing the types explicitly, we have that $\mathsf{type}(t) = B$ with $\mathsf{h}(B) = \mathsf{h}(\mathsf{type}(t)) \geq k$ by hypothesis. Then the type of the function must be of the form $\mathsf{type}(\lambda x.s) = A \to B$. But note that $\mathsf{h}(\mathsf{type}(\lambda x.s)) = \mathsf{h}(A \to B) > \mathsf{h}(B) \geq k$. This means that $(\lambda x.s)\mathtt{L}$ cannot be of degree k, contradicting the hypothesis of this case.

4. $t = s\, r$ where s is not an **m**-abstraction of degree k: Then $\mathtt{S}_k(t) = s\, r = \mathtt{S}_k(s)\, \mathtt{S}_k(r)$ is not an **m**-abstraction.

5. $t = s\langle r\rangle$: Note that $\mathsf{type}(s) = \mathsf{type}(t)$, so in particular $\mathsf{h}(\mathsf{type}(s)) = \mathsf{h}(\mathsf{type}(t)) \geq k$. Moreover, $\delta_{\mathsf{max}}(s) \leq \delta_{\mathsf{max}}(t) \leq k$ and s is not an **m**-abstraction by Rem. 2.3.2. We are under the conditions to apply the IH on s, hence $\mathtt{S}_k(s)$ is not an **m**-abstraction. To conclude, note that $\mathtt{S}_k(t) = \mathtt{S}_k(s)\langle \mathtt{S}_k(r)\rangle$ cannot be an **m**-abstraction by Rem. 2.3.2.

□

Lemma A.1.5 (Properties of the max-degree)**.**

1. $\delta_{\mathsf{max}}(t\mathtt{L}) = \max(\delta_{\mathsf{max}}(t), \delta_{\mathsf{max}}(\mathtt{L}))$

2. If $\delta_{\mathsf{max}}(t) < k$ and $\delta_{\mathsf{max}}(s) < k$ and $\mathsf{h}(\mathsf{type}(s)) < k$ then $\delta_{\mathsf{max}}(t[s/x]) < k$.

Proof. Item 1 is straightforward by induction on $\mathtt{L}$, since $\delta_{\mathsf{max}}(t\langle s\rangle) = \max(\delta_{\mathsf{max}}(t), \delta_{\mathsf{max}}(s))$. For item 2, we proceed by induction on t:

1. $t = y$: We consider two subcases, depending on whether $y = x$ or not. If $y = x$, then $\delta_{\mathsf{max}}(x[s/x]) = \delta_{\mathsf{max}}(s) < k$. If $y \neq x$, then $\delta_{\mathsf{max}}(y[s/x]) = \delta_{\mathsf{max}}(y) = \delta_{\mathsf{max}}(t) < k$.

2. $t = \lambda y.t'$: By α-conversion we assume that $x \neq y$. Note that $\delta_{\mathsf{max}}(t') = \delta_{\mathsf{max}}(\lambda y.t') = \delta_{\mathsf{max}}(t) < k$ by hypothesis. Then $\delta_{\mathsf{max}}((\lambda y.t')[s/x]) = \delta_{\mathsf{max}}(\lambda y.t'[s/x]) = \delta_{\mathsf{max}}(t'[s/x]) < k$ by IH.

3. $t = t_1\, t_2$: Note that $\delta_{\mathsf{max}}(t_1) \leq \delta_{\mathsf{max}}(t_1\, t_2) = \delta_{\mathsf{max}}(t) < k$ and, similarly, $\delta_{\mathsf{max}}(t_2) < k$. This means that we can apply the IH to obtain that $\delta_{\mathsf{max}}(t_1[s/x]) < k$ and $\delta_{\mathsf{max}}(t_2[s/x]) < k$. We proceed by case analysis, depending on whether $\mathsf{h}(\mathsf{type}(t_1)) < k$ or $\mathsf{h}(\mathsf{type}(t_1)) \geq k$:

   3.1. If $\mathsf{h}(\mathsf{type}(t_1)) < k$ then, by the substitution lemma (Lemma 2.3.6), the terms $t_1[s/x]$ and t_1 have the same type. In particular, $t_1[s/x]\, t_2[s/x]$ cannot be a redex of degree k or greater, since $\mathsf{h}(\mathsf{type}(t_1[s/x])) < k$. As a consequence, if $t_1[s/x]\, t_2[s/x]$ is a redex, its degree is at most $k-1$. Hence $\delta_{\mathsf{max}}((t_1\, t_2)[s/x]) = \delta_{\mathsf{max}}(t_1[s/x]\, t_2[s/x]) \leq \max(k-1, \delta_{\mathsf{max}}(t_1[s/x]), \delta_{\mathsf{max}}(t_2[s/x])) < k$.

   3.2. If $\mathsf{h}(\mathsf{type}(t_1)) \geq k$, note that t_1 cannot be an **m**-abstraction, because then $t = t_1\, t_2$ would be a redex of degree k or greater, but by hypothesis we know that $\delta_{\mathsf{max}}(t) < k$. Note that we are under the conditions of Lemma A.1.3, so we know that $t_1[s/x]$ is not an **m**-abstraction. In particular, $t_1[s/x]\, t_2[s/x]$ cannot be a redex. Hence $\delta_{\mathsf{max}}((t_1\, t_2)[s/x]) = \delta_{\mathsf{max}}(t_1[s/x]\, t_2[s/x]) = \max(\delta_{\mathsf{max}}(t_1[s/x]), \delta_{\mathsf{max}}(t_2[s/x])) < k$.

4. $t = t_1\langle t_2\rangle$: Note that $\delta_{\mathsf{max}}(t_1) \leq \delta_{\mathsf{max}}(t_1\langle t_2\rangle) = \delta_{\mathsf{max}}(t) < k$ and, similarly, $\delta_{\mathsf{max}}(t_2) < k$. This means that we can apply the IH to obtain that $\delta_{\mathsf{max}}(t_1[s/x]) < k$ and $\delta_{\mathsf{max}}(t_2[s/x]) < k$. Hence $\delta_{\mathsf{max}}((t_1\langle t_2\rangle)[s/x]) = \delta_{\mathsf{max}}(t_1[s/x]\langle t_2[s/x]\rangle) = \max(\delta_{\mathsf{max}}(t_1[s/x]), \delta_{\mathsf{max}}(t_2[s/x])) < k$.

□

Lemma A.1.6 (Simplification decreases the max-degree)**.** Suppose that $k \geq 1$. If $\delta_{\mathsf{max}}(t) \leq k$ then $\delta_{\mathsf{max}}(\mathtt{S}_k(t)) < k$.

Proof. Let $k \geq 1$ be such that $\delta_{\mathsf{max}}(t) \leq k$. We argue that $\delta_{\mathsf{max}}(\mathtt{S}_k(t)) < k$, that is, all the redexes in t have degree less than k. To prove it by induction, we generalize the statement to memories, proving also that $\delta_{\mathsf{max}}(\mathtt{S}_k(\mathtt{L})) < k$. We prove the statement simultaneously by induction on t and $\mathtt{L}$:

1. $t = x$: Then $\mathtt{S}_k(x) = x$ has no redexes, so $\delta_{\mathsf{max}}(x) = 0 < 1 \leq k$.

2. $t = \lambda x.s$: Note that $\delta_{\mathsf{max}}(s) = \delta_{\mathsf{max}}(\lambda x.s) \leq k$ so by IH $\delta_{\mathsf{max}}(\mathtt{S}_k(s)) < k$. Moreover, $\delta_{\mathsf{max}}(\mathtt{S}_k(\lambda x.s)) = \delta_{\mathsf{max}}(\lambda x.\mathtt{S}_k(s)) = \delta_{\mathsf{max}}(\mathtt{S}_k(s)) < k$.

3. $t = (\lambda x.s)\mathtt{L}\, r$ where $(\lambda x.s)\mathtt{L}$ is an **m**-abstraction of degree k: Note that $\delta_{\mathsf{max}}(s) \leq \delta_{\mathsf{max}}(t)$ because any redex in the subterm s is also a redex in the whole term t, so in particular $\delta_{\mathsf{max}}(s) \leq k$ and we may apply the IH on s to conclude that $\delta_{\mathsf{max}}(\mathtt{S}_k(s)) < k$. Similarly, by IH, we have that $\delta_{\mathsf{max}}(\mathtt{S}_k(\mathtt{L})) < k$ and $\delta_{\mathsf{max}}(\mathtt{S}_k(r)) < k$.

   Since t is typable, its type is of the form $\mathsf{type}(t) = B$ with $\mathsf{type}((\lambda x.s)\mathtt{L}) = A \to B$ and $\mathsf{type}(r) = A$. Note that $\mathsf{h}(\mathsf{type}(r)) = \mathsf{h}(A) < \mathsf{h}(A \to B) = k$ since $(\lambda x.s)\mathtt{L}$ is of degree k by hypothesis of this case.

To conclude this case, note that:

$$\begin{array}{rl} & \delta_{\mathsf{max}}(\mathtt{S}_k(t)) \\ = & \delta_{\mathsf{max}}(\mathtt{S}_k(s)[\mathtt{S}_k(r)/x]\langle \mathtt{S}_k(r)\rangle \mathtt{S}_k(\mathtt{L})) \\ & \quad \text{by definition} \\ \leq & \max(\delta_{\mathsf{max}}(\mathtt{S}_k(s)[\mathtt{S}_k(r)/x]), \delta_{\mathsf{max}}(\mathtt{S}_k(r)), \delta_{\mathsf{max}}(\mathtt{S}_k(\mathtt{L}))) \\ & \quad \text{by Lemma A.1.5 (1)} \\ < & \max(k,k,k) \\ & \quad \text{by Lemma A.1.5 (2) and the IH} \\ = & k \end{array}$$

For the last inequality, we use the fact that $\mathsf{h}(\mathsf{type}(r)) < k$.

4. $t = s\,r$ where s is not an $\mathbf{m}$-abstraction of degree k: Note that $\delta_{\mathsf{max}}(s) \leq \delta_{\mathsf{max}}(t)$ because any redex in the subterm s is also a redex in the whole term t, so in particular $\delta_{\mathsf{max}}(s) \leq k$ and we may apply the IH on s to conclude that $\delta_{\mathsf{max}}(\mathtt{S}_k(s)) < k$. Similarly, by IH, we have that $\delta_{\mathsf{max}}(\mathtt{S}_k(r)) < k$.

   We proceed by case analysis, depending on whether $\mathsf{h}(\mathsf{type}(s)) < k$ or $\mathsf{h}(\mathsf{type}(s)) \geq k$:

   4.1. If $\mathsf{h}(\mathsf{type}(s)) < k$, then by Lemma A.1.2 we know that $s \to_{\mathbf{m}}^* \mathtt{S}_k(s)$ and by subject reduction (Proposition 2.3.7) we have that $\mathsf{type}(s) = \mathsf{type}(\mathtt{S}_k(s))$. In particular, $\mathtt{S}_k(s)\,\mathtt{S}_k(r)$ cannot be a redex of degree k or greater, because $\mathsf{h}(\mathsf{type}(\mathtt{S}_k(s))) = \mathsf{h}(\mathsf{type}(s)) < k$. That is, if $\mathtt{S}_k(s)\,\mathtt{S}_k(r)$ is a redex, its degree is at most $k-1$. Hence we have that $\delta_{\mathsf{max}}(\mathtt{S}_k(t)) = \delta_{\mathsf{max}}(\mathtt{S}_k(s)\,\mathtt{S}_k(r)) \leq \max(k-1, \delta_{\mathsf{max}}(\mathtt{S}_k(s)), \delta_{\mathsf{max}}(\mathtt{S}_k(r))) < k$.

   4.2. If $\mathsf{h}(\mathsf{type}(s)) \geq k$, note that s cannot be an $\mathbf{m}$-abstraction. Indeed, we know by hypothesis of this case that s is not an abstraction of degree k. Furthermore, s cannot be an abstraction of degree $k' > k$, because then $t = s\,r$ would be a redex of degree $k' > k$, but then we would have that $k < k' \leq \delta_{\mathsf{max}}(t) \leq k$, which is a contradiction. Since s is not an $\mathbf{m}$-abstraction, $\delta_{\mathsf{max}}(s) \leq k$, and $\mathsf{h}(\mathsf{type}(s)) \geq k$, we are under the conditions to apply Lemma A.1.4 to conclude that $\mathtt{S}_k(s)$ is not an $\mathbf{m}$-abstraction. This means that $\mathtt{S}_k(s)\,\mathtt{S}_k(r)$ cannot be a redex. Hence we have that $\delta_{\mathsf{max}}(\mathtt{S}_k(t)) = \delta_{\mathsf{max}}(\mathtt{S}_k(s)\,\mathtt{S}_k(r)) = \max(\delta_{\mathsf{max}}(\mathtt{S}_k(s)), \delta_{\mathsf{max}}(\mathtt{S}_k(r))) < k$.

5. $t = s\langle r\rangle$: Note that $\delta_{\mathsf{max}}(s) \leq \delta_{\mathsf{max}}(t)$, so in particular $\delta_{\mathsf{max}}(s) \leq k$ and we may apply the IH on s to conclude that $\delta_{\mathsf{max}}(\mathtt{S}_k(s)) < k$. Similarly, by IH, we have that $\delta_{\mathsf{max}}(\mathtt{S}_k(r)) < k$. Hence we have that $\delta_{\mathsf{max}}(\mathtt{S}_k(t)) = \delta_{\mathsf{max}}(\mathtt{S}_k(s)\langle \mathtt{S}_k(r)\rangle) = \max(\delta_{\mathsf{max}}(\mathtt{S}_k(s)), \delta_{\mathsf{max}}(\mathtt{S}_k(r))) < k$.

6. $\mathtt{L} = \Box$: Immediate, as $\delta_{\mathsf{max}}(\Box) = 0 < 1 \leq k$.

7. $\mathtt{L} = \mathtt{L}'\langle t\rangle$: Similar to case 5 of this lemma.

□

Proposition A.1.7 (Simplification is normalization)**.** $t \to_{\mathbf{m}}^* \mathtt{S}_*(t)$ and $\mathtt{S}_*(t)$ is a $\to_{\mathbf{m}}$-normal form.

Proof. Let k be the max-degree of t. For each $0 \leq i \leq k$ we define $\mathtt{S}_{>i}(t)$ as follows, by induction on $k - i$:

$$\begin{array}{rcl} \mathtt{S}_{>k}(t) & := & t \\ \mathtt{S}_{>i}(t) & := & \mathtt{S}_{i+1}(\mathtt{S}_{>i+1}(t)) \quad \text{for each } 0 \leq i < k \end{array}$$

That is, $\mathtt{S}_{>i}(t) := \mathtt{S}_{i+1}(\ldots \mathtt{S}_{k-1}(\mathtt{S}_k(t)))$. Note that $\mathtt{S}_{>k}(t) = t$ and $\mathtt{S}_{>0}(t) = \mathtt{S}_*(t)$. Let us prove each of the two parts of the statement:

1. To show that $t \to_{\mathbf{m}}^* \mathtt{S}_*(t)$, note that for each $1 \leq i \leq k$ we have that $\mathtt{S}_{>i}(t) \to_{\mathbf{m}}^* \mathtt{S}_i(\mathtt{S}_{>i}(t)) = \mathtt{S}_{>i-1}(t)$ by Lemma A.1.2. Hence:
$$t = \mathtt{S}_{>k}(t) \to_{\mathbf{m}}^* \mathtt{S}_{>k-1}(t) \dots \to_{\mathbf{m}}^* \mathtt{S}_{>i}(t) \to_{\mathbf{m}}^* \mathtt{S}_{>i-1}(t) \dots \to_{\mathbf{m}}^* \mathtt{S}_{>0}(t) = \mathtt{S}_*(t)$$
2. To show that $\mathtt{S}_*(t)$ is a $\to_{\mathbf{m}}$-normal form, we claim that for each $0 \leq i \leq k$ we have that $\delta_{\mathsf{max}}(\mathtt{S}_{>i}(t)) \leq i$. We proceed by induction on $k - i$. In the base case, we have that $i = k$, so $\delta_{\mathsf{max}}(\mathtt{S}_{>k}(t)) = \delta_{\mathsf{max}}(t) = k$ since k is the max-degree of t. For the induction step, let $k - i > 0$, so $0 \leq i < k$. By IH we have that $\delta_{\mathsf{max}}(\mathtt{S}_{>i+1}(t)) \leq i + 1$. Then $\delta_{\mathsf{max}}(\mathtt{S}_{>i}(t)) = \delta_{\mathsf{max}}(\mathtt{S}_{i+1}(\mathtt{S}_{>i+1}(t))) < i + 1$ by Lemma A.1.6. This means that $\delta_{\mathsf{max}}(\mathtt{S}_{>i}(t)) \leq i$, as required.

□

A.2 Proofs of Section 2.5

A.2.1 Reduction by degrees

In this section we give detailed proofs of the results about reduction by degrees stated in Section 2.5.1.

Remark A.2.1. $t\mathtt{L}$ is in $\xrightarrow{d}_{\mathbf{m}}$-normal form if and only if t and $\mathtt{L}$ are in $\xrightarrow{d}_{\mathbf{m}}$-normal form.

Definition A.2.2 (Steps and reduction sequences)**.** A *step of degree* d —or just *step* if clear from the context— is formally a 5-uple $R = (\mathtt{C}, x^A, t, \mathtt{L}, s)$ where $\mathtt{C}$ is an arbitrary context and $\lambda x^A.t$ is an abstraction of degree d. The *source* of R is $R^{\mathsf{src}} := \mathtt{C}[(\lambda x.t)\mathtt{L}\, s]$ and its *target* is $R^{\mathsf{tgt}} := \mathtt{C}[t[s/x]\langle s\rangle\mathtt{L}]$. We write $R : t \xrightarrow{d}_{\mathbf{m}} s$ to mean that R is a step of degree d with source t and target s.

A *forgetful step* —or just *step* if clear from the context— is formally a triple $R = (\mathtt{C}, t, s)$ where $\mathtt{C}$ is an arbitrary context and t, s are terms. The *source* of R is $R^{\mathsf{src}} := \mathtt{C}[t\langle s\rangle]$ and its *target* is $R^{\mathsf{tgt}} := \mathtt{C}[t]$. We write $R : t \rhd s$ to mean that R is a forgetful step of degree with source t and target s.

Steps of degree d are generalized to *reduction sequences of degree* d (and, respectively, *forgetful reduction sequences*), which are sequences of composable steps of the corresponding kind. Formally, a reduction sequence is a pair $\rho = ((t_0, \dots, t_n), (R_1, \dots, R_n))$ where $(t_0, \dots, t_n)$ is a sequence of $n+1$ terms and $(R_1, \dots, R_n)$ is a sequence of n steps $R_i^{\mathsf{src}} = t_{i-1}$ and $R_i^{\mathsf{tgt}} = t_i$ for all $i \in 1..n$. The notions of source and target are extended to reduction sequences by declaring $\rho^{\mathsf{src}} = t_0$ and $\rho^{\mathsf{tgt}} = t_n$. We write $\rho : t \xrightarrow{d}_{\mathbf{m}}{}^* s$ to mean that ρ is a reduction sequence of degree d with source t and target s. Similarly, we write $\rho : t \rhd^* s$ to mean that ρ is a forgetful reduction sequence with source t and target s

A step R can be implicitly treated as the one-step reduction sequence $((R^{\mathsf{src}}, R^{\mathsf{tgt}}), R)$. If $\rho^{\mathsf{tgt}} = \sigma^{\mathsf{src}}$, we write $\rho\,\sigma$ for their composition, defined as expected.

Definition A.2.3 (Simultaneous reduction of degree d)**.** We define a relation $t \overset{d}{\Longrightarrow}_{\mathbf{m}} t'$, meaning that there is a *multi-step* of degree d from t to t', inductively by the following rules:

$$\frac{}{x \overset{d}{\Longrightarrow}_{\mathbf{m}} x}\,\mathtt{p\text{-}var} \qquad \frac{t \overset{d}{\Longrightarrow}_{\mathbf{m}} t'}{\lambda x.t \overset{d}{\Longrightarrow}_{\mathbf{m}} \lambda x.t'}\,\mathtt{p\text{-}abs} \qquad \frac{t \overset{d}{\Longrightarrow}_{\mathbf{m}} t' \quad s \overset{d}{\Longrightarrow}_{\mathbf{m}} s'}{t\, s \overset{d}{\Longrightarrow}_{\mathbf{m}} t'\, s'}\,\mathtt{p\text{-}app_1}$$

$$\frac{t \overset{d}{\Longrightarrow}_{\mathbf{m}} t' \quad \mathtt{L} \overset{d}{\Longrightarrow}_{\mathbf{m}} \mathtt{L}' \quad s \overset{d}{\Longrightarrow}_{\mathbf{m}} s' \quad \lambda x.t \text{ is of degree } d}{(\lambda x.t)\mathtt{L}\, s \overset{d}{\Longrightarrow}_{\mathbf{m}} t'[s'/x]\langle s'\rangle\mathtt{L}'}\,\mathtt{p\text{-}app_2} \qquad \frac{t \overset{d}{\Longrightarrow}_{\mathbf{m}} t' \quad s \overset{d}{\Longrightarrow}_{\mathbf{m}} s'}{t\langle s\rangle \overset{d}{\Longrightarrow}_{\mathbf{m}} t'\langle s'\rangle}\,\mathtt{p\text{-}wrap}$$

$$\dfrac{}{\Box \overset{d}{\Longrightarrow}_{\mathbf{m}} \Box}\,\texttt{p-ctx-hole} \qquad \dfrac{\mathtt{L} \overset{d}{\Longrightarrow}_{\mathbf{m}} \mathtt{L}' \quad t \overset{d}{\Longrightarrow}_{\mathbf{m}} t'}{\mathtt{L}\langle t\rangle \overset{d}{\Longrightarrow}_{\mathbf{m}} \mathtt{L}'\langle t'\rangle}\,\texttt{p-ctx-wrap}$$

If $\mathbf{R}$ is the derivation witnessing a multi-step $t \overset{d}{\Longrightarrow}^{+}_{\mathbf{m}} t'$ We say that $\mathbf{R}$ is empty if it does not use the rule $\texttt{p-app}_2$. We write $\mathbf{R} : t \overset{d}{\Longrightarrow}^{+}_{\mathbf{m}} t'$ if $\mathbf{R}$ uses the rule $\texttt{p-app}_2$ at least once.

Remark A.2.4 (Simultaneous reduction of terms with memory).

1. $t\mathtt{L} \overset{d}{\Longrightarrow}_{\mathbf{m}} s$ if and only if s is of the form $t'\mathtt{L}'$ where $t \overset{d}{\Longrightarrow}_{\mathbf{m}} t'$ and $\mathtt{L} \overset{d}{\Longrightarrow}_{\mathbf{m}} \mathtt{L}'$.
2. Furthermore, the set of derivations $\mathbf{R} : t\mathtt{L} \overset{d}{\Longrightarrow}_{\mathbf{m}} t'\mathtt{L}'$ is in bijective correspondence with the set of pairs of derivations $\mathbf{R}_1 : t \overset{d}{\Longrightarrow}_{\mathbf{m}} t'$ and $\mathbf{R}_2 : \mathtt{L} \overset{d}{\Longrightarrow}_{\mathbf{m}} \mathtt{L}'$.

Lemma A.2.5 (Properties of simultaneous reduction by degrees)**.**

1. For each step $R : t \overset{d}{\to}_{\mathbf{m}} t'$ there is a multi-step $\texttt{sim}(R) : t \overset{d}{\Longrightarrow}_{\mathbf{m}} t'$.
2. For each multi-step $\mathbf{R} : t \overset{d}{\Longrightarrow}_{\mathbf{m}} t'$ there is a reduction sequence $\texttt{red}(\mathbf{R}) : t \overset{d}{\to}^{*}_{\mathbf{m}} t'$. Moreover, if $\mathbf{R}$ is non-empty, then $\texttt{red}(\mathbf{R})$ contains at least one step.
3. Reflexivity: $t \overset{d}{\Longrightarrow}_{\mathbf{m}} t$.
4. Substitution: If $t \overset{d}{\Longrightarrow}_{\mathbf{m}} t'$ and $s \overset{d}{\Longrightarrow}_{\mathbf{m}} s'$ then $t[s/x] \overset{d}{\Longrightarrow}_{\mathbf{m}} t'[s'/x]$.

Proof. All items are straightforward by induction. □

Lemma A.2.6 (Commutation of simultaneous reduction by degrees)**.** Let $d, D \in \mathbb{N}_0$. Given a step $R : t_1 \overset{d}{\to}_{\mathbf{m}} t_2$ and a multi-step $\mathbf{S} : t_1 \overset{D}{\Longrightarrow}_{\mathbf{m}} t_3$, there exists a term t_4, a multi-step $\mathbf{S}/R : t_2 \overset{D}{\Longrightarrow}_{\mathbf{m}} t_4$ and a multi-step $R/\mathbf{S} : t_3 \overset{d}{\Longrightarrow}_{\mathbf{m}} t_4$. Graphically:

$$\begin{array}{ccc} t_1 & \xrightarrow{d} & t_2 \\ {\scriptstyle D}\Downarrow & & \Downarrow{\scriptstyle D} \\ t_3 & \underset{d}{\Longrightarrow} & t_4 \end{array}$$

Furthermore:

1. If $d \neq D$ then $R/\mathbf{S}$ is non-empty, *i.e.*, $R/\mathbf{S} : t_3 \overset{d}{\Longrightarrow}^{+}_{\mathbf{m}} t_4$.
2. If $d \neq D$, the first step of $\texttt{red}(R/\mathbf{S})$ determines the step R. More precisely, suppose that $\texttt{red}(R_1/\mathbf{S})$ and $\texttt{red}(R_2/\mathbf{S})$ start with the same step. Then $R_1 = R_2$.

Proof. We prove a more general version of the statement including memories, *i.e.*, we prove that for $R : \mathtt{X}_1 \overset{d}{\to}_{\mathbf{m}} \mathtt{X}_2$ and $\mathbf{S} : \mathtt{X}_1 \overset{D}{\Longrightarrow}_{\mathbf{m}} \mathtt{X}_3$, there exist $\mathtt{X}_4$ and $\mathbf{S}/R : \mathtt{X}_2 \overset{D}{\Longrightarrow}_{\mathbf{m}} \mathtt{X}_4$ and $R/\mathbf{S} : \mathtt{X}_3 \overset{d}{\Longrightarrow}_{\mathbf{m}} \mathtt{X}_4$, where $\mathtt{X}_1, \mathtt{X}_2, \mathtt{X}_3, \mathtt{X}_4$ stand for either terms or memories. We proceed by induction on $\mathtt{X}_1$:

1. $t_1 = x$: Impossible, as there are no reduction steps $x \overset{d}{\to}_{\mathbf{m}} t_2$.

2. $t_1 = \lambda x.s_1$: Then $R : t_1 = \lambda x.s_1 \xrightarrow{d}_{\mathbf{m}} \lambda x.s_2 = t_2$ with $s_1 \xrightarrow{d}_{\mathbf{m}} s_2$ and S must be derived from the `p-abs` rule, so $S : t_1 = \lambda x.s_1 \xRightarrow{d}_{\mathbf{m}} \lambda x.s_3 = t_3$ with $s_1 \xRightarrow{d}_{\mathbf{m}} s_3$. By IH we have the diagram on the left, and we can construct the one on the right:

$$\begin{array}{ccc} s_1 & \xrightarrow{d} & s_2 \\ {\scriptstyle D}\Downarrow & & \Downarrow{\scriptstyle D} \\ s_3 & \underset{d}{\Longrightarrow} & s_4 \end{array} \qquad \begin{array}{ccc} \lambda x.s_1 & \xrightarrow{d} & \lambda x.s_2 \\ {\scriptstyle D}\Downarrow & & \Downarrow{\scriptstyle D} \\ \lambda x.s_3 & \underset{d}{\Longrightarrow} & \lambda x.s_4 \end{array}$$

Furthermore, if $d \neq D$, using the IH it is easy to show that $R/\mathbf{S}$ is non-empty and that $R/\mathbf{S}$ determines R.

3. $t_1 = s_1\, u_1$: We consider three subcases, depending on whether R is at the root, internal to s_1, or internal to u_1:

   3.1. If R is at the root: Then s_1 is an $\mathbf{m}$-abstraction of degree d, *i.e.*, of the form $s_1 = (\lambda x.r_1)\mathtt{L}_1$, and $R : t_1 = (\lambda x.r_1)\mathtt{L}_1\, u_1 \xrightarrow{d}_{\mathbf{m}} r_1[u_1/x]\langle u_1\rangle\mathtt{L}_1 = t_2$. We consider two further subcases, depending on whether S is derived using the $\mathtt{p\text{-}app}_1$ or the $\mathtt{p\text{-}app}_2$ rule:

      3.1.1. If S is derived using the $\mathtt{p\text{-}app}_1$ rule: Then by Rem. A.2.4 we have that $S : t_1 = (\lambda x.r_1)\mathtt{L}_1\, u_1 \xRightarrow{D}_{\mathbf{m}} (\lambda x.r_3)\mathtt{L}_3\, u_3 = t_3$ where $r_1 \xRightarrow{D}_{\mathbf{m}} r_3$ and $\mathtt{L}_1 \xRightarrow{D}_{\mathbf{m}} \mathtt{L}_3$ and $u_1 \xRightarrow{D}_{\mathbf{m}} u_3$. By Lemma A.2.5 we can construct the following diagram, using reflexivity and $\mathtt{p\text{-}app}_2$ on the bottom:

$$\begin{array}{ccc} (\lambda x.r_1)\mathtt{L}_1\, u_1 & \xrightarrow{d} & r_1[u_1/x]\langle u_1\rangle\mathtt{L}_1 \\ {\scriptstyle D}\Downarrow & & \Downarrow{\scriptstyle D} \\ (\lambda x.r_3)\mathtt{L}_3\, u_3 & \underset{d}{\Longrightarrow} & r_3[u_3/x]\langle u_3\rangle\mathtt{L}_3 \end{array}$$

      3.1.2. If S is derived using the $\mathtt{p\text{-}app}_2$ rule: Then note that $d = D$ and we have that $S : t_1 = (\lambda x.r_1)\mathtt{L}_1\, u_1 \xRightarrow{D}_{\mathbf{m}} r_3[u_3/x]\langle u_3\rangle\mathtt{L}_3 = t_3$ where $r_1 \xRightarrow{D}_{\mathbf{m}} r_3$ and $\mathtt{L}_1 \xRightarrow{D}_{\mathbf{m}} \mathtt{L}_3$ and $u_1 \xRightarrow{D}_{\mathbf{m}} u_3$. By Lemma A.2.5 we can construct the following diagram, using reflexivity on the bottom:

$$\begin{array}{ccc} (\lambda x.r_1)\mathtt{L}_1\, u_1 & \xrightarrow{d} & r_1[u_1/x]\langle u_1\rangle\mathtt{L}_1 \\ {\scriptstyle D}\Downarrow & & \Downarrow{\scriptstyle D} \\ r_3[u_3/x]\langle u_3\rangle\mathtt{L}_3 & \underset{d}{\Longrightarrow} & r_3[u_3/x]\langle u_3\rangle\mathtt{L}_3 \end{array}$$

   3.2. If R is internal to s_1: Then $R : t_1 = s_1\, u_1 \xrightarrow{d}_{\mathbf{m}} s_2\, u_1$ with $s_1 \xrightarrow{d}_{\mathbf{m}} s_2$. We consider two further subcases, depending on whether S is derived using the $\mathtt{p\text{-}app}_1$ or the $\mathtt{p\text{-}app}_2$ rule:

      3.2.1. If S is derived using the $\mathtt{p\text{-}app}_1$ rule: Then $S : t_1 = s_1\, u_1 \xRightarrow{d}_{\mathbf{m}} s_3\, u_3 = t_3$ with $s_1 \xRightarrow{d}_{\mathbf{m}} s_3$ and $u_1 \xRightarrow{d}_{\mathbf{m}} u_3$. By IH we have the diagram on the left, and we can construct the one on the right:

$$\begin{array}{ccc} s_1 & \xrightarrow{d} & s_2 \\ {\scriptstyle D}\Downarrow & & \Downarrow{\scriptstyle D} \\ s_3 & \underset{d}{\Longrightarrow} & s_4 \end{array} \qquad \begin{array}{ccc} s_1\, u_1 & \xrightarrow{d} & s_2\, u_1 \\ {\scriptstyle D}\Downarrow & & \Downarrow{\scriptstyle D} \\ s_3\, u_3 & \underset{d}{\Longrightarrow} & s_4\, u_3 \end{array}$$

3.2.2. If S is derived using the $\texttt{p-app}_2$ rule: Then s_1 is an $\mathbf{m}$-abstraction of degree D, *i.e.*, of the form $s_1 = (\lambda x.r_1)\mathtt{L}_1$, and by Rem. A.2.4 we have that $S : t_1 = (\lambda x.r_1)\mathtt{L}_1\, u_1 \overset{D}{\Longrightarrow}_{\mathbf{m}} r_3[u_3/x]\langle u_3\rangle\mathtt{L}_3$ where $r_1 \overset{D}{\Longrightarrow}_{\mathbf{m}} r_3$ and $\mathtt{L}_1 \overset{D}{\Longrightarrow}_{\mathbf{m}} \mathtt{L}_3$ and $u_1 \overset{D}{\Longrightarrow}_{\mathbf{m}} u_3$. Moreover, since we know $s_1 = (\lambda x.r_1)\mathtt{L}_1 \overset{d}{\to}_{\mathbf{m}} s_2$ we consider two further subcases, depending on whether the step $s_1 \overset{d}{\to}_{\mathbf{m}} s_2$ is internal to r_1 or internal to $\mathtt{L}_1$. These subcases are similar; we only give the proof for the case in which the step is internal to r_1. In such case $s_1 = (\lambda x.r_1)\mathtt{L}_1 \overset{d}{\to}_{\mathbf{m}} (\lambda x.r_2)\mathtt{L}_1$ with $r_1 \overset{d}{\to}_{\mathbf{m}} r_2$. By IH we have the diagram on the left, and we can construct the one on the right, using Lemma A.2.5. On the right of the diagram, use $\texttt{p-app}_2$. On the bottom of the diagram, note that $u_3 \overset{d}{\Longrightarrow}_{\mathbf{m}} u_3$ by reflexivity so $r_3[u_3/x] \overset{d}{\Longrightarrow}_{\mathbf{m}} r_4[u_3/x]$:

$$\begin{array}{ccc} r_1 & \xrightarrow{d} & r_2 \\ {\scriptstyle D}\Downarrow & & \Downarrow{\scriptstyle D} \\ r_3 & \underset{d}{\Longrightarrow} & r_4 \end{array} \qquad \begin{array}{ccc} (\lambda x.r_1)\mathtt{L}_1\, u_1 & \xrightarrow{d} & (\lambda x.r_2)\mathtt{L}_1\, u_1 \\ {\scriptstyle D}\Downarrow & & \Downarrow{\scriptstyle D} \\ r_3[u_3/x]\langle u_3\rangle\mathtt{L}_3 & \underset{d}{\Longrightarrow} & r_4[u_3/x]\langle u_3\rangle\mathtt{L}_3 \end{array}$$

3.3. If R is internal to u_1: Then $R : t_1 = s_1\, u_1 \overset{d}{\to}_{\mathbf{m}} s_1\, u_2$ with $u_1 \overset{d}{\to}_{\mathbf{m}} u_2$. We consider two further subcases, depending on whether S is derived using the $\texttt{p-app}_1$ or the $\texttt{p-app}_2$ rule:

3.3.1. If S is derived using the $\texttt{p-app}_1$ rule: Then $S : t_1 = s_1\, u_1 \overset{d}{\Longrightarrow}_{\mathbf{m}} s_3\, u_3 = t_3$ with $s_1 \overset{d}{\Longrightarrow}_{\mathbf{m}} s_3$ and $u_1 \overset{d}{\Longrightarrow}_{\mathbf{m}} u_3$. By IH we have the diagram on the left, and we can construct the one on the right:

$$\begin{array}{ccc} u_1 & \xrightarrow{d} & u_2 \\ {\scriptstyle D}\Downarrow & & \Downarrow{\scriptstyle D} \\ u_3 & \underset{d}{\Longrightarrow} & u_4 \end{array} \qquad \begin{array}{ccc} s_1\, u_1 & \xrightarrow{d} & s_1\, u_2 \\ {\scriptstyle D}\Downarrow & & \Downarrow{\scriptstyle D} \\ s_3\, u_3 & \underset{d}{\Longrightarrow} & s_3\, u_4 \end{array}$$

3.3.2. If S is derived using the $\texttt{p-app}_2$ rule: Then s_1 is an $\mathbf{m}$-abstraction of degree D, *i.e.*, of the form $s_1 = (\lambda x.r_1)\mathtt{L}_1$, and by Rem. A.2.4 we have that $S : t_1 = (\lambda x.r_1)\mathtt{L}_1\, u_1 \overset{D}{\Longrightarrow}_{\mathbf{m}} r_3[u_3/x]\langle u_3\rangle\mathtt{L}_3$ where $r_1 \overset{D}{\Longrightarrow}_{\mathbf{m}} r_3$ and $\mathtt{L}_1 \overset{D}{\Longrightarrow}_{\mathbf{m}} \mathtt{L}_3$ and $u_1 \overset{D}{\Longrightarrow}_{\mathbf{m}} u_3$. By IH we have the diagram on the left, and we can construct the one on the right. On the right of the diagram, use $\texttt{p-app}_2$. On the bottom of the diagram, note that $r_3 \overset{d}{\Longrightarrow}_{\mathbf{m}} r_3$ by reflexivity so $r_3[u_3/x] \overset{d}{\Longrightarrow}_{\mathbf{m}} r_3[u_4/x]$:

$$\begin{array}{ccc} u_1 & \xrightarrow{d} & u_2 \\ {\scriptstyle D}\Downarrow & & \Downarrow{\scriptstyle D} \\ u_3 & \underset{d}{\Longrightarrow} & u_4 \end{array} \qquad \begin{array}{ccc} (\lambda x.r_1)\mathtt{L}_1\, u_1 & \xrightarrow{d} & (\lambda x.r_1)\mathtt{L}_1\, u_2 \\ {\scriptstyle D}\Downarrow & & \Downarrow{\scriptstyle D} \\ r_3[u_3/x]\langle u_3\rangle\mathtt{L}_3 & \underset{d}{\Longrightarrow} & r_3[u_4/x]\langle u_4\rangle\mathtt{L}_3 \end{array}$$

Furthermore, note that if $d \neq D$, then the multi-step $R/\mathbf{S} : t_3 \overset{d}{\Longrightarrow}_{\mathbf{m}} t_4$ at the bottom of the diagram must be non-empty. Indeed, case 3.1.1, uses exactly one occurrence of the $\texttt{p-app}_2$ rule to construct $R/\mathbf{S}$. Case 3.1.2 is impossible, because in such case $d = D$. In the remaining cases, the bottom of the diagram is constructed by resorting to the IH, which means that $R/\mathbf{S}$ is non-empty. An important observation is that in case 3.3.2 the argument is not erased, because it is always kept as a memorized term.

Furthermore, if $d \neq D$, to see that the first step of $\mathtt{red}(R/\mathbf{S})$ determines the step R, consider the first step T of $\mathtt{red}(R/\mathbf{S})$ and note that it its λ-abstraction can be uniquely traced back to the λ-abstraction of R (*i.e.*, it has a unique ancestor). Indeed, in case 3.1.1 the step at the bottom has R as its unique ancestor. Case 3.1.2 is impossible, because in such case $d = D$. In the remaining cases, it suffices to resort to the IH.

4. $t_1 = s_1\langle u_1\rangle$: We consider two subcases, depending on whether R is internal to s_1 or internal to u_1:

   4.1. If R is internal to s_1: Then $R : t_1 = s_1\langle u_1\rangle \xrightarrow{d}_{\mathbf{m}} s_2\langle u_1\rangle = t_2$. Note that S must be derived using the `p-wrap` rule, so $S : t_1 = s_1\langle u_1\rangle \overset{d}{\Longrightarrow}_{\mathbf{m}} s_3\langle u_3\rangle = t_3$. By IH we have the diagram on the left, and we can construct the one on the right:

$$\begin{array}{ccc} s_1 & \xrightarrow{d} & s_2 \\ {\scriptstyle D}\Downarrow & & \Downarrow{\scriptstyle D} \\ s_3 & \overset{d}{\Longrightarrow} & s_4 \end{array} \qquad \begin{array}{ccc} s_1\langle u_1\rangle & \xrightarrow{d} & s_2\langle u_1\rangle \\ {\scriptstyle D}\Downarrow & & \Downarrow{\scriptstyle D} \\ s_3\langle u_3\rangle & \overset{d}{\Longrightarrow} & s_4\langle u_3\rangle \end{array}$$

   4.2. If R is internal to u_1: Similar to the previous case.

   Furthermore, if $d \neq D$, using the IH it is easy to show that $R/\mathbf{S}$ is non-empty and that $R/\mathbf{S}$ determines R.

5. $\mathtt{L}_1 = \Box$: Impossible, as there are no steps $\Box \xrightarrow{d}_{\mathbf{m}} \mathtt{L}_2$.

6. $\mathtt{L}_1 = \mathtt{L}_1'\langle t_1\rangle$: Similar to case 4.

□

Proposition A.2.7 (Commutation of reduction by degrees)**.** Let $d, D \in \mathbb{N}_0$. Then $\xrightarrow{d}_{\mathbf{m}}$ and $\xrightarrow{D}_{\mathbf{m}}$ commute. More precisely, given reduction sequences $\rho : t_1 \xrightarrow{d}{}^*_{\mathbf{m}}\, t_2$ and $\sigma : t_1 \xrightarrow{D}{}^*_{\mathbf{m}}\, t_3$, there exists a term t_4 and reduction sequences $\sigma/\rho : t_2 \xrightarrow{D}{}^*_{\mathbf{m}}\, t_4$ and $\rho/\sigma : t_3 \xrightarrow{d}{}^*_{\mathbf{m}}\, t_4$. Graphically:

$$\begin{array}{ccc} t_1 & \overset{d}{\twoheadrightarrow} & t_2 \\ {\scriptstyle D}\downarrow & & \vdots{\scriptstyle D} \\ t_3 & \overset{d}{\cdots\!\!\twoheadrightarrow} & t_4 \end{array}$$

The reduction sequence ρ/σ is called the *projection of ρ after σ* and symmetrically for σ/ρ. Furthermore:

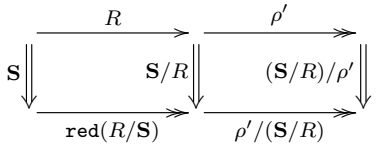
1. If $d \neq D$, then ρ/σ contains at least as many steps as ρ.
2. If $d \neq D$, then ρ/σ determines ρ. More precisely, if $\rho_1/\sigma = \rho_2/\sigma$ then $\rho_1 = \rho_2$.

Proof. Recall that $\xrightarrow{d}_{\mathbf{m}} \subseteq \overset{d}{\Longrightarrow}_{\mathbf{m}} \subseteq \xrightarrow{d}{}^*_{\mathbf{m}}$. by Lemma A.2.5. We prove this in two stages.

First, given a reduction sequence $\rho : t_1 \xrightarrow{d}{}^*_{\mathbf{m}}\, t_2$ and a multi-step $\mathbf{S} : t_1 \overset{D}{\Longrightarrow}_{\mathbf{m}} t_3$, we claim that there exists a term t_4 and constructing $\rho/\mathbf{S} : t_3 \xrightarrow{d}{}^*_{\mathbf{m}}\, t_4$ and $\mathbf{S}/\rho : t_2 \overset{D}{\Longrightarrow}_{\mathbf{m}} t_4$ as follows, by induction on ρ, resorting to Lemma A.2.6 for the constructions of $R/\mathbf{S}$ and $\mathbf{S}/R$.

$$\begin{array}{rcl} \epsilon/\mathbf{S} & := & \epsilon \\ (R\,\rho')/\mathbf{S} & := & \mathtt{red}(R/\mathbf{S})(\rho'/(\mathbf{S}/R)) \end{array} \qquad \begin{array}{rcl} \mathbf{S}/\epsilon & := & \mathbf{S} \\ \mathbf{S}/(R\,\rho') & := & (\mathbf{S}/R)/\rho' \end{array}$$

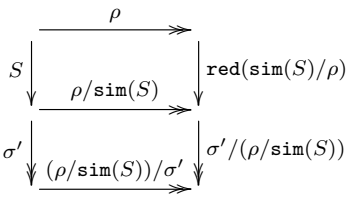
Recall from Lemma A.2.5 that if $\mathbf{R} : u \xRightarrow{d}_{\mathbf{m}} u'$ is a multi-step, then $\mathtt{red}(\mathbf{R}) : u \xrightarrow{d}{}^{*}_{\mathbf{m}} u'$ denotes a reduction sequence. The inductive cases correspond to the following diagram:



For the general case, we proceed by induction on σ resorting to the previous construction for the constructions of $\mathtt{sim}(S)/\rho$ and $\rho/\mathtt{sim}(S)$:

$$\begin{array}{rcl@{\qquad}rcl} \rho/\epsilon & := & \rho & \epsilon/\rho & := & \epsilon \\ \rho/(S\,\sigma') & := & (\rho/\mathtt{sim}(S))/\sigma' & (S\,\sigma')/\rho & := & \mathtt{red}(\mathtt{sim}(S)/\rho)\,(\sigma'/(\rho/\mathtt{sim}(S))) \end{array}$$

Recall from Lemma A.2.5 that if $R : u \xrightarrow{d}{}_{\mathbf{m}} u'$ is a step, then $\mathtt{sim}(R) : u \xRightarrow{d}_{\mathbf{m}} u'$ denotes a multi-step. The inductive cases correspond to the following diagram:



Furthermore, if $d \neq D$, note that $R/\mathbf{S} : t \xRightarrow{d}{}^{+}_{\mathbf{m}} s$ by Lemma A.2.6, so $\mathtt{red}(R/\mathbf{S}) : t \xrightarrow{d}{}^{+}_{\mathbf{m}} s$ by Lemma A.2.5. Then by induction on ρ, we can show that $\rho/\mathbf{S}$ contains at least as many steps as ρ. Finally, by induction on σ, we can show that ρ/σ contains at least as many steps as ρ.

Furthermore, to see that ρ/σ determines ρ, we proceed in stages:

1. First, if $\mathtt{red}(R_1/\mathbf{S})$ and $\mathtt{red}(R_2/\mathbf{S})$ start with the same step, then $R_1 = R_2$ by Lemma A.2.6.

2. Second, we can see that if $\rho_1/\mathbf{S} = \rho_2/\mathbf{S}$ then $\rho_1 = \rho_2$ by induction on ρ_1. Note that if $\rho_1/\mathbf{S} = \rho_2/\mathbf{S}$ then ρ_1 and ρ_2 are either both empty or both non-empty, because if $\rho_1 = R_1\,\rho_1'$ then by definition $\rho_1/\mathbf{S} = \mathtt{red}(R_1/\mathbf{S})\,(\rho_1'/(\mathbf{S}/R_1))$ and Lemma A.2.6 ensures that $R_1/\mathbf{S}$ is non-empty whenever $d \neq D$, so $\rho_2/\mathbf{S}$ is non-empty, and hence ρ_2 is non-empty. The base case is immediate. For the induction step, when $\rho_1 = R_1\,\rho_1'$ and $\rho_2 = R_2\,\rho_2'$ we have that $\rho_1/\mathbf{S} = \rho_2/\mathbf{S}$, so by definition $\mathtt{red}(R_1/\mathbf{S})\,(\rho_1'/(\mathbf{S}/R_1)) = \mathtt{red}(R_2/\mathbf{S})\,(\rho_2'/(\mathbf{S}/R_2))$. As before Lemma A.2.6 ensures that $\mathtt{red}(R_1/\mathbf{S})$ and $\mathtt{red}(R_2/\mathbf{S})$ are non-empty, so they must start with the same step. Hence by Lemma A.2.6 we have that $R_1 = R_2$. This in turn implies that $\rho_1'/(\mathbf{S}/R_1) = \rho_2'/(\mathbf{S}/R_2)$, so by IH $\rho_1' = \rho_2'$.

3. Finally, by induction on σ we can see that if $\rho_1/\sigma = \rho_2/\sigma$ then $\rho_1 = \rho_2$, resorting to the previous item.

□

Lemma A.2.8 (A term reduces to its simplification, by degrees)**.** For every term t and for all $d \geq 1$ we have that $t \xrightarrow{d}{}^{*}_{\mathbf{m}} \mathtt{S}_d(t)$.

Proof. The proof is essentially the same proof as that of Lemma A.1.2, noting that whenever a redex is contracted, its degree is exactly d. □

Lemma A.2.9 (Substitution of $\xrightarrow{d}_{\mathbf{m}}$-normal forms)**.**

1. If t and s are not $\mathbf{m}$-abstractions of degree d, then $t[s/x]$ is not an $\mathbf{m}$-abstraction of degree d.
2. Let t and s be terms in $\xrightarrow{d}_{\mathbf{m}}$-normal form such that s is not an abstraction of degree d. Then $t[s/x]$ is in $\xrightarrow{d}_{\mathbf{m}}$-normal form.

Proof. We prove the two items separately:

1. By induction on t:
   - 1.1. $t = x$: Then $t[s/x] = s$, which is not an $\mathbf{m}$-abstraction of degree d by hypothesis.
   - 1.2. $t = y \neq x$: Then $t[s/x] = y$ is not an $\mathbf{m}$-abstraction.
   - 1.3. $t = \lambda y.t'$: By α-conversion we may assume that $y \notin \{x\} \cup \mathsf{fv}(s)$. Note that t is an $\mathbf{m}$-abstraction but, by hypothesis, it cannot be of degree d. By the substitution lemma (Lemma 2.3.6) we have that $\mathsf{type}(t[s/x]) = \mathsf{type}(\lambda y.t'[s/x]) = \mathsf{type}(\lambda y.t')$, so $t[s/x] = \lambda y.t'[s/x]$ is an $\mathbf{m}$-abstraction, but it is not of degree d.
   - 1.4. $t = t_1\, t_2$: Then $t[s/x] = t_1[s/x]\, t_2[s/x]$ is an application, hence not an $\mathbf{m}$-abstraction.
   - 1.5. $t = t_1\langle t_2\rangle$: Since t is not an $\mathbf{m}$-abstraction of degree d, we have that t_1 is also not an $\mathbf{m}$-abstraction of degree d. By IH, $t_1[s/x]$ is not an $\mathbf{m}$-abstraction of degree d, so $t[s/x] = t_1[s/x]\langle t_2[s/x]\rangle$ is not an $\mathbf{m}$-abstraction of degree d.
2. By induction t:
   - 2.1. $t = x$: Then $t[s/x] = s$ is in $\xrightarrow{d}_{\mathbf{m}}$-normal form.
   - 2.2. $t = y \neq x$: Then $t[s/x] = y$ is in $\xrightarrow{d}_{\mathbf{m}}$-normal form.
   - 2.3. $t = \lambda y.t'$: By α-conversion we may assume that $y \notin \{x\} \cup \mathsf{fv}(s)$. By IH, $t'[s/x]$ is in $\xrightarrow{d}_{\mathbf{m}}$-normal form, so $t[s/x] = \lambda y.t'[s/x]$ is also in $\xrightarrow{d}_{\mathbf{m}}$-normal form.
   - 2.4. $t = t_1\, t_2$: By IH, $t_1[s/x]$ and $t_2[s/x]$ are in $\xrightarrow{d}_{\mathbf{m}}$-normal form. To show that the whole term $t[s/x] = t_1[s/x]\, t_2[s/x]$ is a normal form, we are only left to show that the term does not have a $\xrightarrow{d}_{\mathbf{m}}$-redex at the root, *i.e.*, that $t_1[s/x]$ is not an $\mathbf{m}$-abstraction of degree d. Note that t_1 cannot be an $\mathbf{m}$-abstraction of degree d, for otherwise $t = t_1\, t_2$ would be a redex of degree d, but we know by hypothesis that t is in $\xrightarrow{d}_{\mathbf{m}}$-normal form. Hence by item 1 of this lemma, $t_1[s/x]$ is not an $\mathbf{m}$-abstraction of degree d, as required.
   - 2.5. $t = t_1\langle t_2\rangle$: By IH, $t_1[s/x]$ and $t_2[s/x]$ are in $\xrightarrow{d}_{\mathbf{m}}$-normal form, so $t[s/x] = t_1[s/x]\langle t_2[s/x]\rangle$ is also in $\xrightarrow{d}_{\mathbf{m}}$-normal form.

□

Lemma A.2.10 (Simplification does not create abstractions, by degrees)**.** If t is not an $\mathbf{m}$-abstraction of degree d, then $\mathtt{S}_d(t)$ is not an $\mathbf{m}$-abstraction of degree d.

Proof. By induction on t:

1. $t = x$: Then $\mathtt{S}_d(t) = x$ is not an $\mathbf{m}$-abstraction of degree d.

2. $t = \lambda y.s$: Note that t is an abstraction but, by hypothesis, it cannot be of degree d. By the fact that a term reduces to its simplification (Lemma A.1.2) and by the substitution lemma (Lemma 2.3.6) we know that $\mathsf{type}(\mathtt{S}_d(t))$ is not of degree d, so in particular it cannot be an $\mathbf{m}$-abstraction of degree d.

3. $t = (\lambda y.s)\mathtt{L}\,u$, where $(\lambda y.s)\mathtt{L}$ is an $\mathbf{m}$-abstraction of degree d: Then

$$\mathtt{S}_d(t) = \mathtt{S}_d(s)[\mathtt{S}_d(u)/x]\langle \mathtt{S}_d(u)\rangle \mathtt{S}_d(\mathtt{L})$$

   To show that this term is not an $\mathbf{m}$-abstraction of degree d, it suffices to show that the term $\mathtt{S}_d(s)[\mathtt{S}_d(u)/x]$ is not an $\mathbf{m}$-abstraction of degree d. Note that the abstraction $\lambda y.s$ is of type $A \to B$ where $\mathsf{type}(s) = B$ and $\mathsf{type}(u) = A$. In particular, since the abstraction $\lambda y.s$ is of degree d, we have that $\mathsf{h}(A \to B) = d$. Furthermore, by the fact that a term reduces to its simplification (Lemma A.1.2) and by subject reduction (Proposition 2.3.7), we know that $\mathsf{h}(\mathsf{type}(\mathtt{S}_d(s))) = \mathsf{h}(\mathsf{type}(s)) = \mathsf{h}(B) < d$ and $\mathsf{h}(\mathsf{type}(\mathtt{S}_d(u))) = \mathsf{h}(\mathsf{type}(u)) = \mathsf{h}(A) < d$. In particular, $\mathtt{S}_d(s)$ and $\mathtt{S}_d(u)$ cannot be $\mathbf{m}$-abstractions of degree d. Finally by Lemma A.2.9(1) this means that $\mathtt{S}_d(s)[\mathtt{S}_d(u)/x]$ is not an $\mathbf{m}$-abstraction of degree d, as required.

4. $t = s\,u$, where s is not an $\mathbf{m}$-abstraction of degree d: Then $\mathtt{S}_d(t) = \mathtt{S}_d(s)\,\mathtt{S}_d(u)$ is an application, hence not an $\mathbf{m}$-abstraction of degree d.

5. $t = s\langle u\rangle$: Since t is not an $\mathbf{m}$-abstraction of degree d, we know that s is also not an $\mathbf{m}$-abstraction of degree d. By IH, $\mathtt{S}_d(s)$ is not an $\mathbf{m}$-abstraction of degree d, so $\mathtt{S}_d(t) = \mathtt{S}_d(s)\langle \mathtt{S}_d(u)\rangle$ is not an $\mathbf{m}$-abstraction of degree d.

□

Lemma A.2.11 (The simplification of a term is normal, by degrees)**.** $\mathtt{S}_d(t)$ is in $\xrightarrow{d}_{\mathbf{m}}$-normal form.

Proof. By induction on t, generalizing the statement also for memories, *i.e.*, showing that $\mathtt{S}_d(\mathtt{L})$ is in $\xrightarrow{d}_{\mathbf{m}}$-normal form:

1. $t = x$: Then $\mathtt{S}_d(t) = x$ is in $\xrightarrow{d}_{\mathbf{m}}$-normal form.

2. $t = \lambda x.s$: Then $\mathtt{S}_d(t) = \lambda x.\mathtt{S}_d(s)$ is in $\xrightarrow{d}_{\mathbf{m}}$-normal form because $\mathtt{S}_d(s)$ is in $\xrightarrow{d}_{\mathbf{m}}$-normal form by IH.

3. $t = (\lambda x.s)\mathtt{L}\,u$, where $(\lambda x.s)\mathtt{L}$ is an $\mathbf{m}$-abstraction of degree d: Then

$$\mathtt{S}_d(t) = \mathtt{S}_d(s)[\mathtt{S}_d(u)/x]\langle \mathtt{S}_d(u)\rangle \mathtt{S}_d(\mathtt{L})$$

   Note that, by IH, $\mathtt{S}_d(s)$, $\mathtt{S}_d(u)$, and $\mathtt{S}_d(\mathtt{L})$ are in $\xrightarrow{d}_{\mathbf{m}}$-normal form.

   Since $(\lambda x.s)\mathtt{L}$ is an abstraction of degree d, we know that $\mathsf{h}(\mathsf{type}(u)) < d$. Hence by the fact that a term reduces to its simplification (Lemma A.1.2) and by subject reduction (Proposition 2.3.7) we know that $\mathsf{h}(\mathsf{type}(\mathtt{S}_d(u))) < d$. In particular, $\mathtt{S}_d(u)$ is not an abstraction of degree d. This allows us to apply Lemma A.2.9(2) to conclude that $\mathtt{S}_d(s)[\mathtt{S}_d(u)/x]$ is in $\xrightarrow{d}_{\mathbf{m}}$-normal form. This lets us conclude that $\mathtt{S}_d(s)[\mathtt{S}_d(u)/x]\langle \mathtt{S}_d(u)\rangle \mathtt{S}_d(\mathtt{L})$ is in $\xrightarrow{d}_{\mathbf{m}}$-normal form.

4. $t = s\,u$, where s is not an $\mathbf{m}$-abstraction of degree d. Then $\mathtt{S}_d(t) = \mathtt{S}_d(s)\,\mathtt{S}_d(u)$, where by IH we have that $\mathtt{S}_d(s)$ and $\mathtt{S}_d(u)$ are in $\xrightarrow{d}_{\mathbf{m}}$-normal form, and by Lemma A.2.10 we have that $\mathtt{S}_d(s)$ is not an $\mathbf{m}$-abstraction of degree d. Hence $\mathtt{S}_d(t)$ is in $\xrightarrow{d}_{\mathbf{m}}$-normal form.

5. $t = s\langle u\rangle$: Then $\mathtt{S}_d(t) = \mathtt{S}_d(s)\langle \mathtt{S}_d(u)\rangle$ and we conclude by IH.

6. $\mathtt{L} = \Box$: Immediate, as $\mathtt{S}_d(\Box) = \Box$ is in $\xrightarrow{d}_{\mathbf{m}}$-normal form.

7. $\mathtt{L} = \mathtt{L}\langle t\rangle$: Then $\mathtt{S}_d(\mathtt{L}) = \mathtt{S}_d(\mathtt{L})\langle \mathtt{S}_d(t)\rangle$ and we conclude by IH.

□

Lemma A.2.12 (Reduction does not create redexes of higher degree)**.** Let $d \leq D$ and suppose that $t \xrightarrow{d}_{\mathbf{m}} s$.

1. If t is not an $\mathbf{m}$-abstraction of degree D, then s is not an $\mathbf{m}$-abstraction of degree D.

2. If t is in $\xrightarrow{D}_{\mathbf{m}}$-normal form, then s is also in $\xrightarrow{D}_{\mathbf{m}}$-normal form.

Proof. We prove the two items independently:

1. By induction on t:

   1.1. $t = x$: This case is impossible, as there are no reduction steps $t \xrightarrow{d}_{\mathbf{m}} s$.

   1.2. $t = \lambda x.t'$: Note that t is an $\mathbf{m}$-abstraction, so by IH it cannot be of degree D, that is, $\mathsf{h}(\mathsf{type}(t)) \neq D$. By subject reduction (Proposition 2.3.7) we have that $\mathsf{h}(\mathsf{type}(s)) = \mathsf{h}(\mathsf{type}(t)) \neq D$, so s cannot be an $\mathbf{m}$-abstraction of degree D.

   1.3. $t = t_1\, t_2$: We consider three subcases, depending on whether the reduction is at the root, internal to t_1, or internal to t_2:

      1.3.1. If the reduction is at the root: Then $t_1 = (\lambda x.t_1')\mathtt{L}$ is an abstraction of degree d, and the step is of the form $t = (\lambda x.t_1')\mathtt{L}\, t_2 \xrightarrow{d}_{\mathbf{m}} t_1'[t_2/x]\langle t_2\rangle\mathtt{L} = t'$. Note that $\lambda x.t_1'$ is an abstraction of degree d, so its type is of the form $A \to B$ with $\mathsf{h}(A \to B) = d$. The type of the body of the abstraction is $\mathsf{type}(t_1') = B$, so $\mathsf{h}(\mathsf{type}(t_1')) = \mathsf{h}(B) < d \leq D$, and the type of the argument of the abstraction is $\mathsf{type}(t_2) = A$, so $\mathsf{h}(\mathsf{type}(t_2)) = \mathsf{h}(A) < d \leq D$. This means that t_1' and t_2 cannot be $\mathbf{m}$-abstractions of degree D. Hence by Lemma A.2.9(1) we have that $t_1'[t_2/x]$ is not an $\mathbf{m}$-abstractions of degree D. From this we conclude that $t' = t_1'[t_2/x]\langle t_2\rangle\mathtt{L}$ is not an $\mathbf{m}$-abstractions of degree D.

      1.3.2. If the reduction is internal to t_1: Then the step is of the form $t = t_1\, t_2 \xrightarrow{d}_{\mathbf{m}} s_1\, t_2 = s$ with $t_1 \xrightarrow{d}_{\mathbf{m}} s_1$. Note that s is an application, and hence not an $\mathbf{m}$-abstraction of degree D.

      1.3.3. If the reduction is internal to t_2: Then the step is of the form $t = t_1\, t_2 \xrightarrow{d}_{\mathbf{m}} t_1\, s_2 = s$ with $t_2 \xrightarrow{d}_{\mathbf{m}} s_2$. Note that s is an application, and hence not an $\mathbf{m}$-abstraction of degree D.

   1.4. $t = t_1\langle t_2\rangle$: We consider two subcases, depending on whether the reduction is internal to t_1 or internal to t_2:

      1.4.1. If the reduction is internal to t_1: Then the step is of the form $t = t_1\langle t_2\rangle \xrightarrow{d}_{\mathbf{m}} s_1\langle t_2\rangle = s$ with $t_1 \xrightarrow{d}_{\mathbf{m}} s_1$. By hypothesis t is not an $\mathbf{m}$-abstraction of degree D, so t_1 is also not an $\mathbf{m}$-abstraction of degree D. By IH s_1 is not an $\mathbf{m}$-abstraction of degree D, so we conclude that $s = s_1\langle t_2\rangle$ is not an $\mathbf{m}$-abstraction of degree D.

1.4.2. If the reduction is internal to t_2: Then the step is of the form $t = t_1\langle t_2\rangle \xrightarrow{d}_{\mathbf{m}} t_1\langle s_2\rangle = s$ with $t_2 \xrightarrow{d}_{\mathbf{m}} s_2$. By hypothesis t is not an $\mathbf{m}$-abstraction of degree D, so t_1 is also not an $\mathbf{m}$-abstraction of degree D. Hence $s = t_1\langle s_2\rangle$ is not an $\mathbf{m}$-abstraction of degree D.

2. By induction on t:

2.1. $t = x$: This case is impossible, as there are no reduction steps $t \xrightarrow{d}_{\mathbf{m}} s$.

2.2. $t = \lambda x.t'$: Straightforward resorting to the IH.

2.3. $t = t_1\, t_2$: We consider three subcases, depending on whether the reduction is at the root, internal to t_1, internal to t_2:

2.3.1. If the reduction is at the root: Then $t_1 = (\lambda x.t_1')\mathtt{L}$ is an abstraction of degree d, and the step is of the form $t = (\lambda x.t_1')\mathtt{L}\, t_2 \xrightarrow{d}_{\mathbf{m}} t_1'[t_2/x]\langle t_2\rangle\mathtt{L} = s$. Note that by hypothesis, $t = (\lambda x.t_1')\mathtt{L}\, t_2$ is in $\xrightarrow{D}_{\mathbf{m}}$-normal form, which means in particular that t_1', $\mathtt{L}$ and t_2 are in $\xrightarrow{D}_{\mathbf{m}}$-normal form. Moreover, since $\lambda x.t_1'$ is an abstraction of degree d, its type is of the form $A \to B$ with $\mathsf{h}(A \to B) = d$. Moreover, its argument t_2 is such that $\mathsf{type}(t_2) = A$, so $\mathsf{h}(\mathsf{type}(t_2)) = \mathsf{h}(A) < d \leq D$. In particular, t_2 cannot be an abstraction of degree D. By Lemma A.2.9 this implies that $t_1'[t_2/x]$ is in $\xrightarrow{D}_{\mathbf{m}}$-normal form. Finally, this means that $t_1'[t_2/x]\langle t_2\rangle\mathtt{L}$ must also be in $\xrightarrow{D}_{\mathbf{m}}$-normal form.

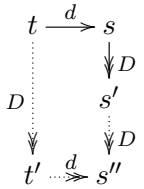
2.3.2. If the reduction is internal to t_1: Then the step is of the form $t = t_1\, t_2 \xrightarrow{d}_{\mathbf{m}} s_1\, t_2 = s$. By hypothesis $t = t_1\, t_2$ is in $\xrightarrow{D}_{\mathbf{m}}$-normal form, so we kwow that t_1 and t_2 must be in $\xrightarrow{D}_{\mathbf{m}}$-normal form and, moreover, that t_1 is not an $\mathbf{m}$-abstraction of degree D. By IH, we have that s_1 is a $\xrightarrow{D}_{\mathbf{m}}$-normal form. Moreover, by item 1 of this lemma, we have that s_1 is not an $\mathbf{m}$-abstraction of degree D. Hence we conclude that $s = s_1\, t_2$ is in $\xrightarrow{D}_{\mathbf{m}}$-normal form.

2.3.3. If the reduction is internal to t_2: Then the step is of the form $t = t_1\, t_2 \xrightarrow{d}_{\mathbf{m}} t_1\, s_2 = s$. By hypothesis $t = t_1\, t_2$ is in $\xrightarrow{D}_{\mathbf{m}}$-normal form, so we kwow that t_1 and t_2 must be in $\xrightarrow{D}_{\mathbf{m}}$-normal form and, moreover, that t_1 is not an $\mathbf{m}$-abstraction of degree D. By IH, we have that s_2 is a $\xrightarrow{D}_{\mathbf{m}}$-normal form. Hence we conclude that $s = s_1\, t_2$ is in $\xrightarrow{D}_{\mathbf{m}}$-normal form.

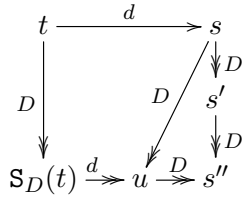
2.4. $t = t_1\langle t_2\rangle$: Straightforward resorting to the IH.

□

Proposition A.2.13 (Lifting property for lower steps)**.** Let $d < D$ and suppose that $t \xrightarrow{d}_{\mathbf{m}} s \xrightarrow{D}_{\mathbf{m}}^{*} s'$. Then there exist a term t' and a term s'' such that $t \xrightarrow{D}_{\mathbf{m}}^{*} t'$ and $s' \xrightarrow{D}_{\mathbf{m}}^{*} s''$ and $t' \xrightarrow{d}_{\mathbf{m}}^{+} s''$ in at least one step. Graphically:



Proof. Take $t' := \mathtt{S}_D(t)$. By the fact that a term reduces to its simplification (Lemma A.2.8) we have that $t \xrightarrow{D}{}^*_{\mathbf{m}} \mathtt{S}_D(t)$. Applying commutation (Proposition A.2.7) on the reduction sequences $t \xrightarrow{d}_{\mathbf{m}} s$ and $t \xrightarrow{d}{}^*_{\mathbf{m}} \mathtt{S}_D(t)$, we have that there exists a term u such that $s \xrightarrow{D}{}^*_{\mathbf{m}} u$ and $\mathtt{S}_D(t) \xrightarrow{d}{}^+_{\mathbf{m}} u$ in at least one step. Applying the commutation theorem again, this time on the reduction sequences $s \xrightarrow{D}{}^*_{\mathbf{m}} u$ and $s \xrightarrow{D}{}^*_{\mathbf{m}} s'$ we have that there exists a term s'' such that $u \xrightarrow{D}{}^*_{\mathbf{m}} s''$ and $s' \xrightarrow{D}{}^*_{\mathbf{m}} s''$. The situation is:



By Lemma A.2.11 we know that $\mathtt{S}_D(t)$ is in $\xrightarrow{D}_{\mathbf{m}}$-normal form, and since $\mathtt{S}_D(t) \xrightarrow{d}{}^*_{\mathbf{m}} u$ with $d < D$, by Lemma A.2.12 (2) we have that u is in $\xrightarrow{D}_{\mathbf{m}}$-normal form, so $u = s''$, which concludes the proof. □

Lemma A.2.14 (Local postponement of forgetful reduction)**.** If $R : t \rhd s$ is a forgetful step and $S : s \xrightarrow{d}_{\mathbf{m}} s'$ is a reduction step of degree d, there exists a term t', a forgetful reduction $R^{\frown}S : t' \rhd^* s'$ and a step $S^{\curvearrowleft}R : t \xrightarrow{d}_{\mathbf{m}} t'$. Graphically:

$$\begin{array}{ccc} t & \rhd & s \\ {\scriptstyle d}\vdots & & \downarrow{\scriptstyle d} \\ t' & \rhd^* & s' \end{array}$$

Furthermore, the step $S^{\curvearrowleft}R$ determines the step S. More precisely, if $S^{\curvearrowleft}R = T^{\curvearrowleft}R$ then $S = T$.

Proof. By induction on t:

1. $t = x$: Impossible, as there are no reduction steps $x \rhd s$.

2. $t = \lambda x.t_1$: The steps must be of the form $R : t = \lambda x.t_1 \rhd \lambda x.s_1 = s$ with $t_1 \rhd s_1$, and $S : s = \lambda x.s_1 \xrightarrow{d}_{\mathbf{m}} \lambda x.s_1' = s'$ with $s_1 \xrightarrow{d}_{\mathbf{m}} s_1'$. By IH we have the diagram on the left, so we can construct the one on the right:

$$\begin{array}{ccc} t_1 & \rhd & s_1 \\ {\scriptstyle d}\vdots & & \downarrow{\scriptstyle d} \\ t_1' & \rhd^* & s_1' \end{array} \qquad\qquad \begin{array}{ccc} \lambda x.t_1 & \rhd & \lambda x.s_1 \\ {\scriptstyle d}\vdots & & \downarrow{\scriptstyle d} \\ \lambda x.t_1' & \rhd^* & \lambda x.s_1' \end{array}$$

   By IH, the step $t_1 \xrightarrow{d}_{\mathbf{m}} t_1'$ determines the step $s_1 \xrightarrow{d}_{\mathbf{m}} s_1'$, which implies that the step $\lambda x.t_1 \xrightarrow{d}_{\mathbf{m}} \lambda x.t_1'$ determines the step $\lambda x.s_1 \xrightarrow{d}_{\mathbf{m}} \lambda x.s_1'$.

3. If $t = t_1\, t_2$: We consider two subcases, depending on whether the step $R : t \rhd s$ is internal to t_1 or internal to t_2.

   3.1. If R is internal to t_1, then $R : t = t_1\, t_2 \rhd s_1\, t_2 = s$ where $t_1 \rhd s_1$. We consider three further subcases, depending on whether the step $S : s = s_1\, t_2 \xrightarrow{d}_{\mathbf{m}} s'$ is at the root, internal to s_1, or internal to t_2:

3.1.1. If the S is at the root of $s = s_1\, t_2$: Then s_1 is an $\mathbf{m}$-abstraction of degree d, *i.e.*, of the form $s_1 = (\lambda x.s_{11})\mathtt{L}$, and S is of the form $S : (\lambda x.s_{11})\mathtt{L}\, t_2 \xrightarrow{d}_{\mathbf{m}} s_{11}[t_2/x]\langle t_2\rangle\mathtt{L}$. We consider three subcases, depending on the form of the step $R_1 : t_1 \rhd (\lambda x.s_{11})\mathtt{L}$:

3.1.1.1. If R_1 is of the form $t_1 = (\lambda x.t_{11})\mathtt{L} \rhd (\lambda x.s_{11})\mathtt{L} = s_1$ where $t_{11} \rhd s_{11}$, we can choose $t' := t_{11}[t_2/x]\langle t_2\rangle\mathtt{L}$, according to the diagram:

$$\begin{array}{ccc} (\lambda x.t_{11})\mathtt{L}\, t_2 & \rhd & (\lambda x.s_{11})\mathtt{L}\, t_2 \\ {\scriptstyle d}\vdots & & \downarrow{\scriptstyle d} \\ t_{11}[t_2/x]\langle t_2\rangle\mathtt{L} & \rhd & s_{11}[t_2/x]\langle t_2\rangle\mathtt{L} \end{array}$$

Here we use the fact that $t_{11} \rhd s_{11}$ implies $t_{11}[t_2/x] \rhd s_{11}[t_2/x]$, as stated in Lemma 2.4.13.

3.1.1.2. If R_1 is of the form $t_1 = (\lambda x.s_{11})\mathtt{L}_1\langle t_3\rangle\mathtt{L}_2 \rhd (\lambda x.s_{11})\mathtt{L}_1\langle s_3\rangle\mathtt{L}_2 = s_1$ with $t_3 \rhd s_3$, we can choose $t' := t_{11}[t_2/x]\langle t_2\rangle\mathtt{L}_1\langle t_3\rangle\mathtt{L}_2$, according to the diagram:

$$\begin{array}{ccc} (\lambda x.t_{11})\mathtt{L}_1\langle t_3\rangle\mathtt{L}_2\, t_2 & \rhd & (\lambda x.t_{11})\mathtt{L}_1\langle s_3\rangle\mathtt{L}_2\, t_2 \\ {\scriptstyle d}\vdots & & \downarrow{\scriptstyle d} \\ t_{11}[t_2/x]\langle t_2\rangle\mathtt{L}_1\langle t_3\rangle\mathtt{L}_2 & \rhd & t_{11}[t_2/x]\langle t_2\rangle\mathtt{L}_1\langle s_3\rangle\mathtt{L}_2 \end{array}$$

3.1.1.3. If R_1 is of the form $t_1 = (\lambda x.s_{11})\mathtt{L}_1\langle t_3\rangle\mathtt{L}_2 \rhd (\lambda x.s_{11})\mathtt{L}_1\mathtt{L}_2 = s_1$, we can choose $t' := t_{11}[t_2/x]\langle t_2\rangle\mathtt{L}_1\langle t_3\rangle\mathtt{L}_2$, according to the diagram:

$$\begin{array}{ccc} (\lambda x.t_{11})\mathtt{L}_1\langle t_3\rangle\mathtt{L}_2\, t_2 & \rhd & (\lambda x.t_{11})\mathtt{L}_1\mathtt{L}_2\, t_2 \\ {\scriptstyle d}\vdots & & \downarrow{\scriptstyle d} \\ t_{11}[t_2/x]\langle t_2\rangle\mathtt{L}_1\langle t_3\rangle\mathtt{L}_2 & \rhd & t_{11}[t_2/x]\langle t_2\rangle\mathtt{L}_1\mathtt{L}_2 \end{array}$$

3.1.2. If S is internal to s_1: Then S must be of the form $S : s = s_1\, t_2 \xrightarrow{d}_{\mathbf{m}} s_1'\, t_2$ with $s_1 \xrightarrow{d}_{\mathbf{m}} s_1'$. By IH we have the diagram on the left, so we can construct the one on the right:

$$\begin{array}{ccc} t_1 & \rhd & s_1 \\ {\scriptstyle d}\vdots & & \downarrow{\scriptstyle d} \\ t_1' & \rhd^* & s_1' \end{array} \qquad \begin{array}{ccc} t_1\, t_2 & \rhd & s_1\, t_2 \\ {\scriptstyle d}\vdots & & \downarrow{\scriptstyle d} \\ t_1'\, t_2 & \rhd^* & s_1'\, t_2 \end{array}$$

3.1.3. If S is internal to t_2: Then S must be of the form $S : s = s_1\, t_2 \xrightarrow{d}_{\mathbf{m}} s_1\, t_2'$ with $t_2 \xrightarrow{d}_{\mathbf{m}} t_2'$. Then we can choose $t' := t_1\, t_2'$, according to the diagram:

$$\begin{array}{ccc} t_1\, t_2 & \rhd & s_1\, t_2 \\ {\scriptstyle d}\vdots & & \downarrow{\scriptstyle d} \\ t_1\, t_2' & \rhd & s_1\, t_2' \end{array}$$

3.2. If R is internal to t_2, then $R : t = t_1\, t_2 \rhd t_1\, s_2 = s$ where $t_2 \rhd s_2$. We consider three further subcases, depending on whether the step S is at the root, internal to t_1 or internal to s_2:

3.2.1. If S is at the root of $t_1\, s_2$: Then t_1 is an $\mathbf{m}$-abstraction of degree d, *i.e.*, of the form $t_1 = (\lambda x.t_{11})\mathtt{L}$, and the step is of the form $S : s = (\lambda x.t_{11})\mathtt{L}\, s_2 \xrightarrow{d}_{\mathbf{m}} t_{11}[s_2/x]\langle s_2\rangle\mathtt{L} =$

s'. Then we can choose $t' := t_{11}[t_2/x]\langle t_2\rangle\mathtt{L}$, according to the diagram:

$$\begin{array}{ccc} (\lambda x.t_{11})\mathtt{L}\,t_2 & \rhd & (\lambda x.t_{11})\mathtt{L}\,s_2 \\ {\scriptstyle d}\vdots\!\downarrow & & \downarrow\!{\scriptstyle d} \\ t_{11}[t_2/x]\langle t_2\rangle\mathtt{L} & \rhd^* & t_{11}[s_2/x]\langle s_2\rangle\mathtt{L} \end{array}$$

Here we use the fact that $t_2 \rhd s_2$ implies $t_{11}[t_2/x] \rhd^* t_{11}[s_2/x]$, as stated in Lemma 2.4.13.

3.2.2. If S is internal to t_1: Then $S : s = t_1\,s_2 \xrightarrow{d}_{\mathbf{m}} t_1'\,s_2 = s'$ with $t_1 \xrightarrow{d}_{\mathbf{m}} t_1'$, and we can choose $t' := t_1'\,t_2$, according to the diagram:

$$\begin{array}{ccc} t_1\,t_2 & \rhd & t_1\,s_2 \\ {\scriptstyle d}\vdots\!\downarrow & & \downarrow\!{\scriptstyle d} \\ t_1'\,t_2 & \rhd & t_1'\,s_2 \end{array}$$

3.2.3. If S is internal to s_2: Then $S : t_1\,s_2 \xrightarrow{d}_{\mathbf{m}} t_1\,s_2'$ with $s_2 \xrightarrow{d}_{\mathbf{m}} s_2'$. By IH we have the diagram on the left, so we can construct the one on the right:

$$\begin{array}{ccc} t_2 & \rhd & s_2 \\ {\scriptstyle d}\vdots\!\downarrow & & \downarrow\!{\scriptstyle d} \\ t_2' & \rhd^* & s_2' \end{array} \qquad \begin{array}{ccc} t_1\,t_2 & \rhd & t_1\,s_2 \\ {\scriptstyle d}\vdots\!\downarrow & & \downarrow\!{\scriptstyle d} \\ t_1\,t_2' & \rhd^* & t_1\,s_2' \end{array}$$

Furthermore, to see that the step $S^\frown R : t \xrightarrow{d}_{\mathbf{m}} t'$ determines the step $S : t \xrightarrow{d}_{\mathbf{m}} t'$, it suffices to note that there are no overlappings between the diagrams, *i.e.*, if the step S and the step $S^\frown R$ are fixed, then no more than one of the cases above applies.

4. If $t = t_1\langle t_2\rangle$: We consider three subcases, depending on whether the step $R : t \rhd s$ is at the root, internal to t_1 or internal to t_2:

4.1. If the step R is at the root: Then R is of the form $R : t = t_1\langle t_2\rangle \rhd t_1 = s$ and S is of the form $S : s \xrightarrow{d}_{\mathbf{m}} s'$. Then we can choose $t' := s'\langle t_2\rangle$, according to the diagram:

$$\begin{array}{ccc} s\,t_2 & \rhd & s \\ {\scriptstyle d}\vdots\!\downarrow & & \downarrow\!{\scriptstyle d} \\ s'\langle t_2\rangle & \rhd & s'\,t_2 \end{array}$$

4.2. If the step R is internal to t_1: Then R is of the form $R : t = t_1\langle t_2\rangle \rhd s_1\langle t_2\rangle = s$ with $t_1 \rhd s_1$. We consider two subcases, depending on whether the step S is internal to s_1 or internal to t_2:

4.2.1. If R is internal to s_1: Then $R : s = s_1\langle t_2\rangle \xrightarrow{d}_{\mathbf{m}} s_1'\langle t_2\rangle = s'$ with $s_1 \xrightarrow{d}_{\mathbf{m}} s_1'$. By IH we have the diagram on the left, so we can construct the one on the right:

$$\begin{array}{ccc} t_1 & \rhd & s_1 \\ {\scriptstyle d}\vdots\!\downarrow & & \downarrow\!{\scriptstyle d} \\ t_1' & \rhd^* & s_1' \end{array} \qquad \begin{array}{ccc} t_1\langle t_2\rangle & \rhd & t_1\langle s_2\rangle \\ {\scriptstyle d}\vdots\!\downarrow & & \downarrow\!{\scriptstyle d} \\ t_1\langle t_2'\rangle & \rhd^* & t_1\langle s_2'\rangle \end{array}$$

4.2.2. If R is internal to t_2: Then $R : s = s_1\langle t_2\rangle \xrightarrow{d}_{\mathbf{m}} s_1\langle t_2'\rangle = s'$ with $t_2 \xrightarrow{d}_{\mathbf{m}} t_2'$ and we can choose $t' = t_1\langle t_2'\rangle$, according to the diagram:

$$\begin{array}{ccc} t_1\langle t_2\rangle & \rhd & s_1\langle t_2\rangle \\ {\scriptstyle d}\vdots\downarrow & & \downarrow{\scriptstyle d} \\ t_1\langle t_2'\rangle & \rhd & s_1\langle t_2'\rangle \end{array}$$

4.3. If the step R is internal to t_2: Symmetric to the previous case.

Furthermore, to see that the step $S^\frown R : t \xrightarrow{d}_{\mathbf{m}} t'$ determines the step $S : t \xrightarrow{d}_{\mathbf{m}} t'$, it suffices to note that there are no overlappings between the diagrams, *i.e.*, if the step S and the step $S^\frown R$ are fixed, then no more than one of the cases above applies.

□

Proposition A.2.15 (Postponement of forgetful reduction)**.** Let $\rho : t \rhd^* t'$ be a forgetful reduction sequence and let $\sigma : t' \xrightarrow{d}_{\mathbf{m}}{}^* s'$ be a reduction sequence of degree d. Then there exist a term s and reduction sequences $\rho^\curvearrowright\sigma : s \rhd^* s'$ and $\sigma^\curvearrowleft\rho : t \xrightarrow{d}_{\mathbf{m}}{}^* s$. Graphically:

$$\begin{array}{ccc} t & \rhd^* & t' \\ {\scriptstyle d}\vdots\downarrow & & \downarrow{\scriptstyle d} \\ s & \rhd^* & s' \end{array}$$

Furthermore, $\sigma^\curvearrowleft\rho$ determines σ, that is, More precisely, $\sigma_1^\curvearrowleft\rho = \sigma_2^\curvearrowleft\rho$ then $\sigma_1 = \sigma_2$.

Proof. First, if $\rho : t \rhd^* t'$ is a forgetful reduction sequence and and $S : t' \xrightarrow{d}_{\mathbf{m}} s'$ is a single step of degree d, we can construct a forgetful reduction sequence $\rho^\curvearrowright S$ and a step $S^\curvearrowleft\rho$ of degree d by induction on ρ as follows, resorting to Lemma A.2.14 for the constructions of $R^\curvearrowright(S^\curvearrowleft\rho')$ and $(S^\curvearrowleft\rho')^\curvearrowleft R$:

$$\begin{array}{rclcrcl} \epsilon^\curvearrowright S & := & \epsilon & & S^\curvearrowleft\epsilon & := & S \\ (R\,\rho')^\curvearrowright S & := & (R^\curvearrowright(S^\curvearrowleft\rho'))(\rho'^\curvearrowright S) & & S^\curvearrowleft(R\,\rho') & := & (S^\curvearrowleft\rho')^\curvearrowleft R \end{array}$$

The inductive cases correspond to the following diagram:

$$\begin{array}{ccc} & \xrightarrow{\quad R\quad} & \xrightarrow{\quad\rho'\quad} \\ {\scriptstyle (S^\curvearrowleft\rho')^\curvearrowleft R}\downarrow & {\scriptstyle S^\curvearrowleft\rho'}\downarrow & {\scriptstyle S}\downarrow \\ & \xrightarrow[R^\curvearrowright(S^\curvearrowleft\rho')]{} & \xrightarrow[\rho'^\curvearrowright S]{} \end{array}$$

For the general case, we proceed by induction on σ, resorting to the previous construction for the constructions of $\rho^\curvearrowright S$ and $S^\curvearrowleft\rho$:

$$\begin{array}{rclcrcl} \rho^\curvearrowright\epsilon & := & \rho & & \epsilon^\curvearrowleft\rho & := & \epsilon \\ \rho^\curvearrowright(S\,\sigma') & := & (\rho^\curvearrowright S)^\curvearrowright\sigma' & & (S\,\sigma')^\curvearrowleft\rho & := & (S^\curvearrowleft\rho)(\sigma'^\curvearrowleft(\rho^\curvearrowright S)) \end{array}$$

The inductive cases correspond to the following diagram:

$$\begin{array}{ccc} & \xrightarrow{\quad\rho\quad} & \\ {\scriptstyle S^\curvearrowleft\rho}\downarrow & {\scriptstyle \rho^\curvearrowright S}\downarrow & {\scriptstyle S}\downarrow \\ {\scriptstyle \sigma'^\curvearrowleft(\rho^\curvearrowright S)}\downarrow & {\scriptstyle (\rho^\curvearrowright S)^\curvearrowright\sigma'}\downarrow & {\scriptstyle \sigma'}\downarrow \\ & \xrightarrow[]{} & \end{array}$$

Furthermore, to see that $\sigma^\curvearrowleft\rho$ determines σ, we proceed in three stages:

1. First, if $S_1 \frown R = S_2 \frown R$ then $S_1 = S_2$ by Lemma A.2.14.

2. Second, by induction on ρ, it is easy to see that if $S_1 \frown \rho = S_2 \frown \rho$ then $S_1 = S_2$.

3. Finally, we can see that if $\sigma_1 \frown \rho = \sigma_2 \frown \rho$ then $\sigma_1 = \sigma_2$ by induction on σ_1. Note that $\sigma_1 \frown \rho = \sigma_2 \frown \rho$ then σ_1 and σ_2 are either both empty or both non-empty. The base case is immediate. For the induction step, we have that $\sigma_1 = S_1\,\sigma_1'$ and $\sigma_2 = S_2\,\sigma_2'$; then note that if $(S_1\,\sigma_1') \frown \rho = (S_2\,\sigma_2') \frown \rho$ then by definition $(S_1 \frown \rho)\,(\sigma_1' \frown (\rho \frown S)) = (S_2 \frown \rho)\,(\sigma_2' \frown (\rho \frown S))$ so we have that $S_1 \frown \rho = S_2 \frown \rho$ which, resorting to the previous item, means that $S_1 = S_2$, and we also have that $\sigma_1' \frown (\rho \frown S) = \sigma_2' \frown (\rho \frown S)$ which by IH implies $\sigma_1' = \sigma_2'$.

□

A.2.2 $\mathcal{T}^{\mathfrak{m}}$ measure

In this section we give detailed proofs of the results about reduction by degrees stated in Section 2.5.2.

Lemma A.2.16 (Properties of the pointwise multiset order)**.**

1. If $\mathfrak{m}_1 :\succ: \mathfrak{n}_1$ and $\mathfrak{m}_2 :\succ: \mathfrak{n}_2$ then $\mathfrak{m}_1 + \mathfrak{m}_2 :\succ: \mathfrak{n}_1 + \mathfrak{n}_2$.

2. If $\mathfrak{m} :\succ: \mathfrak{n}$ then for all $k \in \mathbb{N}_0$ we have that $\mathfrak{m} \succeq k \otimes \mathfrak{n}$. In particular, taking $k = 1$, $\mathfrak{m} \succeq \mathfrak{n}$.

3. If $\mathfrak{m} :\succ: \mathfrak{n}$ and $\mathfrak{m}$ is non-empty then $\mathfrak{m} \succ \mathfrak{n}$.

Proof. The first item is straightforward. For the second item, suppose that $\mathfrak{m} :\succ: \mathfrak{n}$ and proceed by induction on the cardinality of $\mathfrak{m}$. If $\mathfrak{m}$ is empty, then $\mathfrak{m} = [\,] = \mathfrak{n}$, so $\mathfrak{m} = [\,] \succeq [\,] = k \otimes [\,] = k \otimes \mathfrak{n}$. If $\mathfrak{m}$ is non-empty, then we can write $\mathfrak{m} = [x] + \mathfrak{m}'$ and $\mathfrak{n} = [y] + \mathfrak{n}'$ in such a way that $x > y$ and $\mathfrak{m}' :\succ: \mathfrak{n}'$. By IH we have that $\mathfrak{m}' \succeq k \otimes \mathfrak{n}'$, so $\mathfrak{m} = [x] + \mathfrak{m}' \succ k \otimes [y] + \mathfrak{m}' \succeq k \otimes [y] + k \otimes \mathfrak{n}' = k \otimes ([y] + \mathfrak{n}') = k \otimes \mathfrak{n}$. The third item is similar to the second. □

Lemma A.2.17 (Higher substitution lemma)**.** Let t, s be typable terms and let x be a variable. Then $\mathcal{T}_d^{\mathbf{m}}(t_0, t) \preceq \mathcal{T}_d^{\mathbf{m}}(t_0, t[s/x])$.

Proof. We generalize the lemma for the case in which t may also be a memory. That is, we prove that if $\mathtt{X}$ is a term or a memory, s is a term, and x is a variable then $\mathcal{T}_d^{\mathbf{m}}(t_0, \mathtt{X}) \preceq \mathcal{T}_d^{\mathbf{m}}(t_0, \mathtt{X}[s/x])$. We proceed by induction on $\mathtt{X}$:

1. $t = x$: Then $\mathcal{T}_d^{\mathbf{m}}(t_0, x) = [\,] \preceq \mathcal{T}_d^{\mathbf{m}}(t_0, s) = \mathcal{T}_d^{\mathbf{m}}(t_0, x[s/x])$.

2. $t = y \neq x$: Then $\mathcal{T}_d^{\mathbf{m}}(t_0, y) \preceq \mathcal{T}_d^{\mathbf{m}}(t_0, y) = \mathcal{T}_d^{\mathbf{m}}(t_0, y[s/x])$.

3. $t = \lambda y.t'$: By α-conversion, we may assume that $y \notin \{x\} \cup \mathsf{fv}(s)$. Then $\mathcal{T}_d^{\mathbf{m}}(t_0, \lambda y.t') = \mathcal{T}_d^{\mathbf{m}}(t_0, t') \preceq \mathcal{T}_d^{\mathbf{m}}(t_0, t'[s/x]) = \mathcal{T}_d^{\mathbf{m}}(t_0, (\lambda y.t')[s/x])$ by IH.

4. If $t = (\lambda x.t_1)\mathtt{L}\,t_2$ is a redex of degree d: Then $\mathcal{T}_d^{\mathbf{m}}(t_0, (\lambda x.t_1)\mathtt{L}\,t_2) = \mathcal{T}_d^{\mathbf{m}}(t_0, t_1) + \mathcal{T}_d^{\mathbf{m}}(t_0, \mathtt{L}) + \mathcal{T}_d^{\mathbf{m}}(t_0, t_2) + [(d, \mathcal{R}_d^{\mathbf{m}}(t_0))] \preceq \mathcal{T}_d^{\mathbf{m}}(t_0, t_1[s/x]) + \mathcal{T}_d^{\mathbf{m}}(t_0, \mathtt{L}[s/x]) + \mathcal{T}_d^{\mathbf{m}}(t_0, t_2[s/x]) + [(d, \mathcal{R}_d^{\mathbf{m}}(t_0))] = \mathcal{T}_d^{\mathbf{m}}(t_0, ((\lambda y.t_1)\mathtt{L}\,t_2)[s/x])$ by IH.

5. If $t = t_1\,t_2$ is not a redex of degree d: Then $\mathcal{T}_d^{\mathbf{m}}(t_0, t_1\,t_2) = \mathcal{T}_d^{\mathbf{m}}(t_0, t_1) + \mathcal{T}_d^{\mathbf{m}}(t_0, t_2) \preceq \mathcal{T}_d^{\mathbf{m}}(t_0, t_1[s/x]) + \mathcal{T}_d^{\mathbf{m}}(t_0, t_2[s/x]) = \mathcal{T}_d^{\mathbf{m}}(t_0, (t_1\,t_2)[s/x])$ by IH.

6. $t = t_1\langle t_2 \rangle$: Similar to case 5.

7. $\mathtt{L} = \Box$: Then $\mathcal{T}^{\mathbf{m}}_d(t_0, \Box) \preceq \mathcal{T}^{\mathbf{m}}_d(t_0, \Box) = \mathcal{T}^{\mathbf{m}}_d(t_0, \Box[s/x])$.

8. $\mathtt{L} = \mathtt{L}_1\langle t\rangle$: Similar to case 5.

□

Proposition A.2.18 (High/increase)**.** Let $D \in \mathbb{N}_0$. Then the following hold:

1. If $1 \leq d < D$ and $t \xrightarrow{D}_{\mathbf{m}} t'$ then $\mathcal{R}^{\mathbf{m}}_d(t) \preceq \mathcal{R}^{\mathbf{m}}_d(t')$.
2. If $0 \leq d < D$ and $t_0 \xrightarrow{D}_{\mathbf{m}} t'_0$ then $\mathcal{T}^{\mathbf{m}}_d(t_0, t) \preceq \mathcal{T}^{\mathbf{m}}_d(t'_0, t)$.
3. If $0 \leq d < D$ and $t_0 \xrightarrow{D}_{\mathbf{m}} t'_0$ and $t \xrightarrow{D}_{\mathbf{m}} t'$ then $\mathcal{T}^{\mathbf{m}}_d(t_0, t) \preceq \mathcal{T}^{\mathbf{m}}_d(t'_0, t')$.
4. If $0 \leq d < D$ and $t \xrightarrow{D}_{\mathbf{m}} t'$ then $\mathcal{T}^{\mathbf{m}}_{\leq d}(t) \preceq \mathcal{T}^{\mathbf{m}}_{\leq d}(t')$.

Proof. We prove a more general version of the statement: in items 2 and 3 we allow t to be either a term or a memory. For example, the statement of item 2 is generalized as follows: if $1 \leq d < D$ and $t_0 \xrightarrow{d}_{\mathbf{m}} t'_0$ then $\mathcal{T}^{\mathbf{m}}_d(t_0, \mathtt{X}) \preceq \mathcal{T}^{\mathbf{m}}_d(t'_0, \mathtt{X})$, where $\mathtt{X}$ is either a term or a memory.

We prove all items simultaneously by induction on d. Note that: item 1. resorts to the IH; item 2. resorts to item 1. (without decreasing d); item 3. resorts to items 1. and 2. (without decreasing d); item 4. resorts to item 3. (without necessarily decreasing d).

1. Let $1 \leq d < D$ and $t \xrightarrow{D}_{\mathbf{m}} t'$. We argue that $\mathcal{R}^{\mathbf{m}}_d(t) \preceq \mathcal{R}^{\mathbf{m}}_d(t')$. Let X and Y be the sets of reduction sequences $X = \{\rho \mid (\exists s)\ \rho : t \xrightarrow{d}{}^*_{\mathbf{m}} s\}$ and $Y = \{\sigma \mid (\exists s')\ \sigma : t' \xrightarrow{d}{}^*_{\mathbf{m}} s'\}$. Note that, by definition, $\mathcal{R}^{\mathbf{m}}_d(t) = [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathsf{tgt}}) \mid\mid \rho \in X]$ and $\mathcal{R}^{\mathbf{m}}_d(t') = [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\sigma^{\mathsf{tgt}}) \mid\mid \sigma \in Y]$. We construct a function $\varphi : X \to Y$ as follows. Consider a reduction step $R : t \xrightarrow{D}_{\mathbf{m}} t'$; note that there may be more than one such step, but we know by hypothesis that there is at least one. By commutation (Proposition A.2.7), given a reduction sequence $\rho \in X$, *i.e.*, $\rho : t \xrightarrow{d}{}^*_{\mathbf{m}} s$ there exists a term s'_ρ and reduction sequences $\rho/R : t' \xrightarrow{d}{}^*_{\mathbf{m}} s'_\rho$ and $R/\rho : s \xrightarrow{D}{}^*_{\mathbf{m}} s'_\rho$. In particular, $\rho/R \in Y$, and we can define $\varphi(\rho) := \rho/R$. Moreover, φ is injective, because if $\rho_1, \rho_2 \in X$ are such that $\rho_1/R = \rho_2/R$ then by commutation (Proposition A.2.7) we have that $\rho_1 = \rho_2$, given that $d < D$.

   First, we claim that $\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathsf{tgt}}) \preceq \mathcal{T}^{\mathbf{m}}_{\leq d-1}(\varphi(\rho)^{\mathsf{tgt}})$ for every $\rho \in X$. If $d = 1$, this is immediate since $\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathsf{tgt}}) = \mathcal{T}^{\mathbf{m}}_{\leq 0}(\rho^{\mathsf{tgt}}) = [\,] = \mathcal{T}^{\mathbf{m}}_{\leq 0}(\varphi(\rho)^{\mathsf{tgt}}) = \mathcal{T}^{\mathbf{m}}_{\leq d-1}(\varphi(\rho)^{\mathsf{tgt}})$. Assume now that $d > 1$. Then we have that:

$$\begin{array}{lcll} \mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathsf{tgt}}) & = & \mathcal{T}^{\mathbf{m}}_{\leq d-1}(s) & \\ & \preceq & \mathcal{T}^{\mathbf{m}}_{\leq d-1}(s'_\rho) & \text{by item 4 of the IH} \\ & = & \mathcal{T}^{\mathbf{m}}_{\leq d-1}((\rho/R)^{\mathsf{tgt}}) & \\ & = & \mathcal{T}^{\mathbf{m}}_{\leq d-1}(\varphi(\rho)) & \end{array}$$

   To be able to apply item 4 of the IH, observe that $1 \leq d - 1 < D$ holds because $1 \leq d < D$. We resort to the IH as many times as the length of the reduction $s \xrightarrow{D}{}^*_{\mathbf{m}} s'_\rho$. To conclude the

proof, let $Z = Y \setminus \varphi(X)$, so that $Y = \varphi(X) \uplus Z$, and note that:

$$\begin{array}{rcl}
\mathcal{R}^{\mathbf{m}}_d(t) & = & [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in X] \\
& \preceq & [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\varphi(\rho)^{\mathtt{tgt}}) \mid\mid \rho \in X] \quad (\star) \\
& = & [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\sigma^{\mathtt{tgt}}) \mid\mid \sigma \in \varphi(X)] \quad (\star\star) \\
& \preceq & [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\sigma^{\mathtt{tgt}}) \mid\mid \sigma \in \varphi(X)] + [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\sigma^{\mathtt{tgt}}) \mid\mid \sigma \in Z] \\
& = & [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\sigma^{\mathtt{tgt}}) \mid\mid \sigma \in \varphi(X) \uplus Z] \\
& = & [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\sigma^{\mathtt{tgt}}) \mid\mid \sigma \in Y] \\
& = & \mathcal{R}^{\mathbf{m}}_d(t')
\end{array}$$

To justify the step marked with $(\star)$, note that $[\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in X] = \sum_{\rho \in X}[\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathtt{tgt}})] \preceq \sum_{\rho \in X}[\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\varphi(\rho)^{\mathtt{tgt}})] = [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\varphi(\rho)^{\mathtt{tgt}}) \mid\mid \rho \in X]$ because $\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathtt{tgt}}) \preceq \mathcal{T}^{\mathbf{m}}_{\leq d-1}(\varphi(\rho)^{\mathtt{tgt}})$, as we have already claimed. To justify the step marked with $(\star\star)$, note that φ is injective, so X and $\varphi(X)$ have the same cardinality.

2. Let $0 \leq d < D$ and $t_0 \xrightarrow{D}_{\mathbf{m}} t'_0$. We argue that $\mathcal{T}^{\mathbf{m}}_d(t_0, \mathtt{X}) \preceq \mathcal{T}^{\mathbf{m}}_d(t'_0, \mathtt{X})$, where $\mathtt{X}$ is either a term ($\mathtt{X} = t$) or a memory ($\mathtt{X} = \mathtt{L}$). We proceed by induction on $\mathtt{X}$:

   2.1. $t = x$: Then $\mathcal{T}^{\mathbf{m}}_d(t_0, x) = [\,] \preceq [\,] = \mathcal{T}^{\mathbf{m}}_d(t'_0, x)$.

   2.2. $t = \lambda x.s$: Then $\mathcal{T}^{\mathbf{m}}_d(t_0, \lambda x.s) = \mathcal{T}^{\mathbf{m}}_d(t_0, s) \preceq \mathcal{T}^{\mathbf{m}}_d(t'_0, s) = \mathcal{T}^{\mathbf{m}}_d(t'_0, \lambda x.s)$ by the internal IH.

   2.3. If $t = (\lambda x.s)\mathtt{L}\, u$ is a redex of degree d: Then:

$$\begin{array}{rcll}
\mathcal{T}^{\mathbf{m}}_d(t_0, t) & = & \mathcal{T}^{\mathbf{m}}_d(t_0, (\lambda x.s)\mathtt{L}\, u) & \\
& = & \mathcal{T}^{\mathbf{m}}_d(t_0, s) + \mathcal{T}^{\mathbf{m}}_d(t_0, \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t_0, u) + [(d, \mathcal{R}^{\mathbf{m}}_d(t_0))] & \\
& \preceq & \mathcal{T}^{\mathbf{m}}_d(t'_0, s) + \mathcal{T}^{\mathbf{m}}_d(t'_0, \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t'_0, u) + [(d, \mathcal{R}^{\mathbf{m}}_d(t_0))] & \text{by the internal IH} \\
& \preceq & \mathcal{T}^{\mathbf{m}}_d(t'_0, s) + \mathcal{T}^{\mathbf{m}}_d(t'_0, \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t'_0, u) + [(d, \mathcal{R}^{\mathbf{m}}_d(t'_0))] & \text{by item 1} \\
& = & \mathcal{T}^{\mathbf{m}}_d(t'_0, (\lambda x.s)\mathtt{L}\, u) & \\
& = & \mathcal{T}^{\mathbf{m}}_d(t'_0, t) &
\end{array}$$

   2.4. If $t = s\, u$ is not a redex of degree d: Then:

$$\begin{array}{rcll}
\mathcal{T}^{\mathbf{m}}_d(t_0, t) & = & \mathcal{T}^{\mathbf{m}}_d(t_0, s\, u) & \\
& = & \mathcal{T}^{\mathbf{m}}_d(t_0, s) + \mathcal{T}^{\mathbf{m}}_d(t_0, u) & \\
& \preceq & \mathcal{T}^{\mathbf{m}}_d(t'_0, s) + \mathcal{T}^{\mathbf{m}}_d(t'_0, u) & \text{by the internal IH} \\
& = & \mathcal{T}^{\mathbf{m}}_d(t'_0, s\, u) &
\end{array}$$

   2.5. $t = s\langle u \rangle$: Similar to case 2.4.

   2.6. $\mathtt{L} = \Box$: Then $\mathcal{T}^{\mathbf{m}}_d(t_0, \Box) = [\,] \preceq [\,] = \mathcal{T}^{\mathbf{m}}_d(t'_0, \Box)$.

   2.7. $\mathtt{L} = \mathtt{L}_1\langle t \rangle$: Similar to case 2.4.

3. Let $0 \leq d < D$ and $t_0 \xrightarrow{D}_{\mathbf{m}} t'_0$ and let $\mathtt{X}, \mathtt{X}'$ be and $\mathtt{X} \xrightarrow{D}_{\mathbf{m}} \mathtt{X}'$ where $\mathtt{X}, \mathtt{X}'$ are either terms ($\mathtt{X} = t$ and $\mathtt{X} = t'$) or memories ($\mathtt{X} = \mathtt{L}$ and $\mathtt{X} = \mathtt{L}'$). We argue that $\mathcal{T}^{\mathbf{m}}_d(t_0, \mathtt{X}) \preceq \mathcal{T}^{\mathbf{m}}_d(t'_0, \mathtt{X}')$. We proceed by induction on $\mathtt{X}$:

   3.1. $t = x$: Impossible, as there are no steps $x \xrightarrow{D}_{\mathbf{m}} t'$.

   3.2. $t = \lambda x.s$: Then the step is of the form $t = \lambda x.s \xrightarrow{D}_{\mathbf{m}} \lambda x.s' = t'$ with $s \xrightarrow{D}_{\mathbf{m}} s'$, so $\mathcal{T}^{\mathbf{m}}_d(t_0, \lambda x.s) = \mathcal{T}^{\mathbf{m}}_d(t_0, s) \preceq \mathcal{T}^{\mathbf{m}}_d(t'_0, s') = \mathcal{T}^{\mathbf{m}}_d(t'_0, \lambda x.s')$ by the internal IH.

3.3. If $t = (\lambda x.s)\mathtt{L}\,u$ is the redex of degree D contracted by the step $t \xrightarrow{D}_{\mathbf{m}} t'$: Then the step is of the form $t = (\lambda x.s)\mathtt{L}\,u \xrightarrow{D}_{\mathbf{m}} s[u/x]\langle u\rangle\mathtt{L} = t'$. Note that t is not a redex of degree d because $d < D$, so:

$$\begin{array}{rcll}
\mathcal{T}^{\mathbf{m}}_d(t_0,t) & = & \mathcal{T}^{\mathbf{m}}_d(t_0,(\lambda x.s)\mathtt{L}\,u) & \\
& = & \mathcal{T}^{\mathbf{m}}_d(t_0,s) + \mathcal{T}^{\mathbf{m}}_d(t_0,\mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t_0,u) & \\
& \preceq & \mathcal{T}^{\mathbf{m}}_d(t'_0,s) + \mathcal{T}^{\mathbf{m}}_d(t'_0,\mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t'_0,u) & \text{by item 2} \\
& = & \mathcal{T}^{\mathbf{m}}_d(t'_0,s) + \mathcal{T}^{\mathbf{m}}_d(t'_0,u) + \mathcal{T}^{\mathbf{m}}_d(t'_0,\mathtt{L}) & \\
& \preceq & \mathcal{T}^{\mathbf{m}}_d(t'_0,s[u/x]) + \mathcal{T}^{\mathbf{m}}_d(t'_0,u) + \mathcal{T}^{\mathbf{m}}_d(t'_0,\mathtt{L}) & \text{by Lemma A.2.17} \\
& = & \mathcal{T}^{\mathbf{m}}_d(t'_0,s[u/x]\langle u\rangle\mathtt{L}) & \\
& = & \mathcal{T}^{\mathbf{m}}_d(t'_0,t') &
\end{array}$$

3.4. If $t = (\lambda x.s)\mathtt{L}\,u$ is a redex of degree d: Note that t is not a redex of degree D because $d < D$. We consider three subcases, depending on whether the step $t \xrightarrow{D}_{\mathbf{m}} t'$ is internal to s, internal to $\mathtt{L}$, or internal to u. All these subcases are similar; we only give the proof for the case in which the step is internal to s. Then:

$$\begin{array}{rcll}
\mathcal{T}^{\mathbf{m}}_d(t_0,t) & = & \mathcal{T}^{\mathbf{m}}_d(t_0,(\lambda x.s)\mathtt{L}\,u) & \\
& = & \mathcal{T}^{\mathbf{m}}_d(t_0,s) + \mathcal{T}^{\mathbf{m}}_d(t_0,\mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t_0,u) + [(d,\mathcal{R}^{\mathbf{m}}_d(t_0))] & \\
& \preceq & \mathcal{T}^{\mathbf{m}}_d(t'_0,s') + \mathcal{T}^{\mathbf{m}}_d(t_0,\mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t_0,u) + [(d,\mathcal{R}^{\mathbf{m}}_d(t_0))] & \text{by the internal IH} \\
& \preceq & \mathcal{T}^{\mathbf{m}}_d(t'_0,s') + \mathcal{T}^{\mathbf{m}}_d(t'_0,\mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t'_0,u) + [(d,\mathcal{R}^{\mathbf{m}}_d(t_0))] & \text{by item 2} \\
& \preceq & \mathcal{T}^{\mathbf{m}}_d(t'_0,s') + \mathcal{T}^{\mathbf{m}}_d(t'_0,\mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t'_0,u) + [(d,\mathcal{R}^{\mathbf{m}}_d(t'_0))] & \text{by item 1} \\
& = & \mathcal{T}^{\mathbf{m}}_d(t'_0,(\lambda x.s')\mathtt{L}\,u) & \\
& = & \mathcal{T}^{\mathbf{m}}_d(t'_0,t') &
\end{array}$$

3.5. If $t = s\,u$ is not the redex contracted by the step $t \xrightarrow{D}_{\mathbf{m}} t'$ nor a redex of degree d: We consider two subcases, depending on whether the step $t \xrightarrow{D}_{\mathbf{m}} t'$ is internal to s or internal to u:

3.5.1. If the step is internal to s, then the step is of the form $t = s\,u \xrightarrow{D}_{\mathbf{m}} s'\,u = t'$ with $s \xrightarrow{D}_{\mathbf{m}} s'$. We know that s is not an $\mathbf{m}$-abstraction of degree d, but $d < D$, so it may be the case that s' is an $\mathbf{m}$-abstraction of degree d, *i.e.*, reduction at degree D may create an abstraction of degree $d < D$. We consider two further subcases, depending on whether s' is an $\mathbf{m}$-abstraction of degree d or not:

3.5.1.1. If $s' = (\lambda x.s'')\mathtt{L}$ is an $\mathbf{m}$-abstraction of degree d, then:

$$\begin{array}{rcll}
\mathcal{T}^{\mathbf{m}}_d(t_0,t) & = & \mathcal{T}^{\mathbf{m}}_d(t_0,s\,u) & \\
& = & \mathcal{T}^{\mathbf{m}}_d(t_0,s) + \mathcal{T}^{\mathbf{m}}_d(t_0,u) & \\
& \preceq & \mathcal{T}^{\mathbf{m}}_d(t'_0,(\lambda x.s'')\mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t_0,u) & \text{by the internal IH} \\
& \preceq & \mathcal{T}^{\mathbf{m}}_d(t'_0,(\lambda x.s'')\mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t'_0,u) & \text{by item 2} \\
& \preceq & \mathcal{T}^{\mathbf{m}}_d(t'_0,(\lambda x.s'')\mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t'_0,u) + [(d,\mathcal{R}^{\mathbf{m}}_d(t'_0))] & \\
& = & \mathcal{T}^{\mathbf{m}}_d(t'_0,(\lambda x.s'')\mathtt{L}\,u) & \\
& = & \mathcal{T}^{\mathbf{m}}_d(t'_0,t') &
\end{array}$$

3.5.1.2. If s' is not an $\mathbf{m}$-abstraction of degree d, then:

$$\begin{array}{rcll}
\mathcal{T}^{\mathbf{m}}_d(t_0,t) & = & \mathcal{T}^{\mathbf{m}}_d(t_0,s\,u) & \\
& = & \mathcal{T}^{\mathbf{m}}_d(t_0,s) + \mathcal{T}^{\mathbf{m}}_d(t_0,u) & \\
& \preceq & \mathcal{T}^{\mathbf{m}}_d(t'_0,s') + \mathcal{T}^{\mathbf{m}}_d(t_0,u) & \text{by the internal IH} \\
& \preceq & \mathcal{T}^{\mathbf{m}}_d(t'_0,s') + \mathcal{T}^{\mathbf{m}}_d(t'_0,u) & \text{by item 2} \\
& = & \mathcal{T}^{\mathbf{m}}_d(t'_0,s'\,u) & \\
& = & \mathcal{T}^{\mathbf{m}}_d(t'_0,t') &
\end{array}$$

3.5.2. If the step is internal to u: Similar to case 3.5.1.2.

3.6. $t = s\langle u\rangle$: Similar to case 3.5.

3.7. $\mathtt{L} = \Box$: Impossible, as there are no reduction steps $\Box \xrightarrow{D}_{\mathbf{m}} \mathtt{L}'$.

3.8. $\mathtt{L} = \mathtt{L}_1\langle t\rangle$: Similar to case 3.5.

4. Let $0 \leq d < D$ and $t \xrightarrow{D}_{\mathbf{m}} t'$. We argue that $\mathcal{T}^{\mathbf{m}}_{\leq d}(t) \preceq \mathcal{T}^{\mathbf{m}}_{\leq d}(t')$. Indeed:

$$\begin{array}{rcll} \mathcal{T}^{\mathbf{m}}_{\leq d}(t) & = & \sum_{i=1}^{d} \mathcal{T}^{\mathbf{m}}_{i}(t,t) & \\ & \preceq & \sum_{i=1}^{d} \mathcal{T}^{\mathbf{m}}_{i}(t',t') & \text{by item 3, resorting to the IH when } i < d \\ & = & \mathcal{T}^{\mathbf{m}}_{\leq d}(t') & \end{array}$$

Note that for the value $i = d$, we resort directly to item 3 and not to the IH.

□

Lemma A.2.19 (Substitution of degree d does not create abstractions)**.** If t and s are not **m**-abstractions of degree d, then $t[s/x]$ is not an **m**-abstraction of degree d.

Proof. By induction on t:

1. $t = x$: Then $t[s/x] = s$ is not an **m**-abstraction of degree d.
2. $t = y \neq x$: Then $t[s/x] = y$ is not an **m**-abstraction of degree d.
3. $t = \lambda y.t'$: Then t is an **m**-abstraction but, by hypothesis, we know that it cannot be of degree d. Hence $t[s/x] = \lambda y.t'[s/x]$. By the substitution lemma (Lemma 2.3.6) t and $t[s/x]$ have the same type, so $\lambda y.t'[s/x]$ is an **m**-abstraction but it is not of degree d.
4. $t = t_1\, t_2$: Then $t[s/x] = t_1[s/x]\, t_2[s/x]$ is an application, hence not an **m**-abstraction of degree d.
5. $t = t_1\langle t_2\rangle$: Since t is not an **m**-abstraction of degree d, we have that t_1 is also not an **m**-abstraction of degree d. By IH, $t_1[s/x]$ is not an **m**-abstraction of degree d. So $t[s/x] = t_1[s/x]\langle t_2[s/x]\rangle$ is not an **m**-abstraction of degree d.

□

Lemma A.2.20 (Lower substitution lemma)**.** Let t, s be typable terms and let x be a variable such that s is not an **m**-abstraction of degree d. Then there exists $k \in \mathbb{N}_0$ such that $\mathcal{T}^{\mathbf{m}}_{d}(t_0, t[s/x]) = \mathcal{T}^{\mathbf{m}}_{d}(t_0, t) + k \otimes \mathcal{T}^{\mathbf{m}}_{d}(t_0, s)$.

Proof. We generalize the lemma for the case in which t may also be a memory. That is, we prove that if $\mathtt{X}$ is a term or a memory, s is a term, and x is a variable such that s is not an **m**-abstraction of degree d, then there exists $k \in \mathbb{N}_0$ such that $\mathcal{T}^{\mathbf{m}}_{d}(t_0, \mathtt{X}[s/x]) = \mathcal{T}^{\mathbf{m}}_{d}(t_0, \mathtt{X}) + k \otimes \mathcal{T}^{\mathbf{m}}_{d}(t_0, s)$. We proceed by induction on $\mathtt{X}$:

1. $t = x$: Take $k := 1$. Then $\mathcal{T}^{\mathbf{m}}_{d}(t_0, t[s/x]) = \mathcal{T}^{\mathbf{m}}_{d}(t_0, x[s/x]) = \mathcal{T}^{\mathbf{m}}_{d}(t_0, s) = [\,] + 1 \otimes \mathcal{T}^{\mathbf{m}}_{d}(t_0, s) = \mathcal{T}^{\mathbf{m}}_{d}(t_0, x) + 1 \otimes \mathcal{T}^{\mathbf{m}}_{d}(t_0, s) = \mathcal{T}^{\mathbf{m}}_{d}(t_0, t) + 1 \otimes \mathcal{T}^{\mathbf{m}}_{d}(t_0, s)$.
2. $t = y \neq x$: Take $k := 0$. Then $\mathcal{T}^{\mathbf{m}}_{d}(t_0, t[s/x]) = \mathcal{T}^{\mathbf{m}}_{d}(t_0, y[s/x]) = \mathcal{T}^{\mathbf{m}}_{d}(t_0, y) = \mathcal{T}^{\mathbf{m}}_{d}(t_0, y) + 0 \otimes \mathcal{T}^{\mathbf{m}}_{d}(t_0, s) = \mathcal{T}^{\mathbf{m}}_{d}(t_0, t) + 0 \otimes \mathcal{T}^{\mathbf{m}}_{d}(t_0, s)$.

3. $t = \lambda y.t'$: By α-conversion we may assume that $y \notin \{x\} \cup \mathsf{fv}(s)$. Resorting to the IH, we have $\mathcal{T}_d^{\mathbf{m}}(t_0, t[s/x]) = \mathcal{T}_d^{\mathbf{m}}(t_0, \lambda y.t'[s/x]) = \mathcal{T}_d^{\mathbf{m}}(t_0, t'[s/x]) = \mathcal{T}_d^{\mathbf{m}}(t_0, t') + k \otimes \mathcal{T}_d^{\mathbf{m}}(t_0, s) = \mathcal{T}_d^{\mathbf{m}}(t_0, \lambda y.t') + k \otimes \mathcal{T}_d^{\mathbf{m}}(t_0, s) = \mathcal{T}_d^{\mathbf{m}}(t_0, t) + k \otimes \mathcal{T}_d^{\mathbf{m}}(t_0, s)$.

4. $t = (\lambda x.t_1)\mathtt{L}\, t_2$ where $(\lambda x.t_1)\mathtt{L}$ is an $\mathbf{m}$-abstraction of degree d. Note that, by the substitution lemma (Lemma 2.3.6) we have that $(\lambda x.t_1[s/x])(\mathtt{L}[s/x])$ is also an abstraction of degree d. Then by IH there exist $k_1, k_2, k_3 \in \mathbb{N}_0$ such that:

$$\begin{array}{rl} & \mathcal{T}_d^{\mathbf{m}}(t_0, t[s/x]) \\ = & \mathcal{T}_d^{\mathbf{m}}(t_0, (\lambda x.t_1[s/x])(\mathtt{L}[s/x])(t_2[s/x])) \\ = & \mathcal{T}_d^{\mathbf{m}}(t_0, \lambda x.t_1[s/x]) + \mathcal{T}_d^{\mathbf{m}}(t_0, \mathtt{L}[s/x]) + \mathcal{T}_d^{\mathbf{m}}(t_0, t_2[s/x]) + [(d, \mathcal{R}_d^{\mathbf{m}}(t_0))] \\ = & (\mathcal{T}_d^{\mathbf{m}}(t_0, \lambda x.t_1) + k_1 \otimes \mathcal{T}_d^{\mathbf{m}}(t_0, s)) + (\mathcal{T}_d^{\mathbf{m}}(t_0, \mathtt{L}) + k_2 \otimes \mathcal{T}_d^{\mathbf{m}}(t_0, s)) \\ & + (\mathcal{T}_d^{\mathbf{m}}(t_0, t_2) + k_3 \otimes \mathcal{T}_d^{\mathbf{m}}(t_0, s)) + [(d, \mathcal{R}_d^{\mathbf{m}}(t_0))] \\ = & \mathcal{T}_d^{\mathbf{m}}(t_0, \lambda x.t_1) + \mathcal{T}_d^{\mathbf{m}}(t_0, \mathtt{L}) + \mathcal{T}_d^{\mathbf{m}}(t_0, t_2) + [(d, \mathcal{R}_d^{\mathbf{m}}(t_0))] + (k_1 + k_2 + k_3) \otimes \mathcal{T}_d^{\mathbf{m}}(t_0, s) \\ = & \mathcal{T}_d^{\mathbf{m}}(t_0, (\lambda x.t_1)\mathtt{L}\, t_2) + (k_1 + k_2 + k_3) \otimes \mathcal{T}_d^{\mathbf{m}}(t_0, s) \\ = & \mathcal{T}_d^{\mathbf{m}}(t_0, t) + (k_1 + k_2 + k_3) \otimes \mathcal{T}_d^{\mathbf{m}}(t_0, s) \end{array}$$

   So taking $k := k_1 + k_2 + k_3$ we are done.

5. $t = t_1\, t_2$ where t_1 is not an $\mathbf{m}$-abstraction of degree d: Note by Lemma A.2.19 that $t_1[t_2/x]$ is not an $\mathbf{m}$-abstraction of degree d. Then by IH there exist $k_1, k_2 \in \mathbb{N}_0$ such that:

$$\begin{array}{rcl} \mathcal{T}_d^{\mathbf{m}}(t_0, t[s/x]) & = & \mathcal{T}_d^{\mathbf{m}}(t_0, t_1[s/x]\, t_2[s/x]) \\ & & \mathcal{T}_d^{\mathbf{m}}(t_0, t_1[s/x]) + \mathcal{T}_d^{\mathbf{m}}(t_0, t_2[s/x]) \\ & = & (\mathcal{T}_d^{\mathbf{m}}(t_0, t_1) + k_1 \otimes \mathcal{T}_d^{\mathbf{m}}(t_0, s)) + (\mathcal{T}_d^{\mathbf{m}}(t_0, t_2) + k_2 \otimes \mathcal{T}_d^{\mathbf{m}}(t_0, s)) \\ & = & \mathcal{T}_d^{\mathbf{m}}(t_0, t_1) + \mathcal{T}_d^{\mathbf{m}}(t_0, t_2) + (k_1 + k_2) \otimes \mathcal{T}_d^{\mathbf{m}}(t_0, s)) \\ & = & \mathcal{T}_d^{\mathbf{m}}(t_0, t_1\, t_2) + (k_1 + k_2) \otimes \mathcal{T}_d^{\mathbf{m}}(t_0, s)) \end{array}$$

   So taking $k := k_1 + k_2$ we are done.

6. $t = t_1\langle t_2 \rangle$: Similar to the previous case.

7. $\mathtt{L} = \Box$: Take $k := 0$. Then $\mathcal{T}_d^{\mathbf{m}}(t_0, \Box[s/x]) = \mathcal{T}_d^{\mathbf{m}}(t_0, \Box) = \mathcal{T}_d^{\mathbf{m}}(t_0, \Box) + 0 \otimes \mathcal{T}_d^{\mathbf{m}}(t_0, s)$.

8. $\mathtt{L} = \mathtt{L}'\langle t \rangle$: Similar to case 5.

□

Proposition A.2.21 (Low/decrease)**.** Let $D \in \mathbb{N}_0$. Then the following hold:

1. If $1 \leq d \leq j \leq D$ and $t \xrightarrow{d}_{\mathbf{m}} t'$ then $\mathcal{R}_j^{\mathbf{m}}(t) \succ \mathcal{R}_j^{\mathbf{m}}(t')$.

2. If $1 \leq d \leq j \leq D$ and $t_0 \xrightarrow{d}_{\mathbf{m}} t_0'$ then $\mathcal{T}_j^{\mathbf{m}}(t_0, t) \mathbin{:\succ:} \mathcal{T}_j^{\mathbf{m}}(t_0', t)$.

3. If $1 \leq d \leq D$ and $t_0 \xrightarrow{d}_{\mathbf{m}} t_0'$ and $t \xrightarrow{d}_{\mathbf{m}} t'$, then for all $\mathfrak{m} \in \mathbb{T}_{d-1}$ we have $\mathcal{T}_d^{\mathbf{m}}(t_0, t) \succ \mathcal{T}_d^{\mathbf{m}}(t_0', t') + \mathfrak{m}$.

4. If $1 \leq d < j \leq D$ and $t_0 \xrightarrow{d}_{\mathbf{m}} t_0'$ and $t \xrightarrow{d}_{\mathbf{m}} t'$ then $\mathcal{T}_j^{\mathbf{m}}(t_0, t) \succeq \mathcal{T}_j^{\mathbf{m}}(t_0', t')$.

5. If $1 \leq d \leq D$ and $t \xrightarrow{d}_{\mathbf{m}} t'$ then $\mathcal{T}_{\leq D}^{\mathbf{m}}(t) \succ \mathcal{T}_{\leq D}^{\mathbf{m}}(t')$.

Proof. We prove a more general version of the statement: in items 2, 3, and 4 we allow t to be either a term or a memory. For example, the statement of item 2 is generalized as follows: if $1 \leq d \leq j \leq D$ and $t_0 \xrightarrow{d}_{\mathbf{m}} t_0'$ then $\mathcal{T}_j^{\mathbf{m}}(t_0, \mathtt{X}) \mathrel{:\!\succ:} \mathcal{T}_j^{\mathbf{m}}(t_0', \mathtt{X})$, where $\mathtt{X}$ is either a term or a memory.

We prove all items simultaneously by induction on D. Note that: item 1. resorts to the IH; item 2. resorts to item 1. (without decreasing D); items 3. and 4. resort to items 1. and 2. (without decreasing D); item 5. resorts to items 3. and 4. (without decreasing D).

1. Let $1 \leq d \leq j \leq D$ and $t \xrightarrow{d}_{\mathbf{m}} t'$. We argue that $\mathcal{R}_j^{\mathbf{m}}(t) \succ \mathcal{R}_j^{\mathbf{m}}(t')$. Let X and Y be the sets of reduction sequences $X := \{\rho \mid (\exists s)\ \rho : t \xrightarrow{j}_{\mathbf{m}}^* s\}$ and $Y := \{\sigma \mid (\exists s')\ \sigma : t' \xrightarrow{j}_{\mathbf{m}}^* s'\}$. Note that, by definition, $\mathcal{R}_j^{\mathbf{m}}(t) = [\mathcal{T}_{\leq j-1}^{\mathbf{m}}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in X]$ and $\mathcal{R}_j^{\mathbf{m}}(t') = [\mathcal{T}_{\leq j-1}^{\mathbf{m}}(\sigma^{\mathtt{tgt}}) \mid\mid \sigma \in Y]$. We consider two subcases, depending on whether $d = j$ or $d < j$:

   1.1. If $d = j$, let R be the step $R : t \xrightarrow{d}_{\mathbf{m}} t'$. We construct a function $\varphi : Y \to X$ given by $\varphi(\sigma) = R\,\sigma$. Observe that φ is injective and that if $\sigma : t' \xrightarrow{d}_{\mathbf{m}}^* s'$ then $R\,\sigma : t \xrightarrow{d}_{\mathbf{m}}^* s'$, and in particular σ and $\varphi(\sigma)$ have the same target. Let $Z = X \setminus \varphi(Y)$, so that $X = \varphi(Y) \uplus Z$. Note that:

$$\begin{array}{rcll}
\mathcal{R}_j^{\mathbf{m}}(t) & = & [\mathcal{T}_{\leq j-1}^{\mathbf{m}}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in X] & \\
& = & [\mathcal{T}_{\leq j-1}^{\mathbf{m}}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in \varphi(Y) \uplus Z] & \\
& = & [\mathcal{T}_{\leq j-1}^{\mathbf{m}}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in \varphi(Y)] + [\mathcal{T}_{\leq j-1}^{\mathbf{m}}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in Z] & \\
& = & [\mathcal{T}_{\leq j-1}^{\mathbf{m}}(R\sigma^{\mathtt{tgt}}) \mid\mid \sigma \in Y] + [\mathcal{T}_{\leq j-1}^{\mathbf{m}}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in Z] & \text{since } \varphi \text{ is injective} \\
& = & [\mathcal{T}_{\leq j-1}^{\mathbf{m}}(\sigma^{\mathtt{tgt}}) \mid\mid \sigma \in Y] + [\mathcal{T}_{\leq j-1}^{\mathbf{m}}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in Z] & \\
& = & \mathcal{R}_j^{\mathbf{m}}(t') + [\mathcal{T}_{\leq j-1}^{\mathbf{m}}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in Z] &
\end{array}$$

   By this chain of equations, in order to conclude that $\mathcal{R}_j^{\mathbf{m}}(t) \succ \mathcal{R}_j^{\mathbf{m}}(t')$, it suffices to show that Z is non-empty. Indeed, let $\epsilon : t \xrightarrow{d}_{\mathbf{m}}^* t$ be the empty reduction sequence. Then $\epsilon \in X \setminus \varphi(Y)$, so $\epsilon \in Z$.

   1.2. If $d < j$, we construct a function $\varphi : Y \to X$ using Proposition A.2.13. More precisely, since $d < j$ and $t \xrightarrow{d}_{\mathbf{m}} t'$, for each reduction sequence $\sigma : t' \xrightarrow{j}_{\mathbf{m}}^* s'$ by Proposition A.2.13 there exist a term s_σ and a term u_σ, such that there is a reduction sequence $\varphi(\sigma) : t \xrightarrow{j}_{\mathbf{m}}^* s_\sigma$ and such that $s' \xrightarrow{j}_{\mathbf{m}}^* u_\sigma$, and $s_\sigma \xrightarrow{d}_{\mathbf{m}}^+ u_\sigma$ in at least one step.
   First, we claim that $\mathcal{T}_{\leq j-1}^{\mathbf{m}}(\varphi(\sigma)^{\mathtt{tgt}}) \succ \mathcal{T}_{\leq j-1}^{\mathbf{m}}(\sigma^{\mathtt{tgt}})$ for every $\sigma \in Y$. Indeed:

$$\begin{array}{rcll}
\mathcal{T}_{\leq j-1}^{\mathbf{m}}(\varphi(\sigma)^{\mathtt{tgt}}) & = & \mathcal{T}_{\leq j-1}^{\mathbf{m}}(s_\sigma) & \\
& \succ & \mathcal{T}_{\leq j-1}^{\mathbf{m}}(u_\sigma) & \text{by item 5 of the IH} \\
& \succeq & \mathcal{T}_{\leq j-1}^{\mathbf{m}}(s') & \text{by high/increase (Proposition A.2.18(4))} \\
& = & \mathcal{T}_{\leq j-1}^{\mathbf{m}}(\sigma^{\mathtt{tgt}}) &
\end{array}$$

   To be able to apply item 5 of the IH, observe that we have that $1 \leq d \leq j-1 < D$ holds because we know $d < j \leq D$. We resort to the IH as many times as the length of the reduction $s_\sigma \xrightarrow{d}_{\mathbf{m}}^+ u_\sigma$. The inequality is strict because this reduction contains at least one step. To be able to apply the high/increase property, observe that $0 \leq j-1 < j$. We resort to this lemma as many times as the length of the reduction $s' \xrightarrow{j}_{\mathbf{m}}^* u_\sigma$, which may be empty. Hence, every element $\mathcal{T}_{\leq j-1}^{\mathbf{m}}(\sigma^{\mathtt{tgt}})$ in $\mathcal{R}_j^{\mathbf{m}}(t')$ is strictly smaller than some element $\mathcal{T}_{\leq j-1}^{\mathbf{m}}(\varphi(\sigma)^{\mathtt{tgt}})$ in $\mathcal{R}_j^{\mathbf{m}}(t)$. Then:

$$\begin{aligned}\mathcal{R}^{\mathbf{m}}_j(t) &= [\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\rho_1^{\mathtt{tgt}})] + ... + [\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\rho_n^{\mathtt{tgt}})]\\ &\succ^1 \mathfrak{n}_1 + ... + [\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\rho_n^{\mathtt{tgt}})]\\ &\succ^1 ...\\ &\succ^1 \mathfrak{n}_1 + ... + \mathfrak{n}_n\\ &= \mathcal{R}^{\mathbf{m}}_j(t')\end{aligned}$$

where $\mathfrak{n}_i$ is the multiset of elements $\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\sigma^{\mathtt{tgt}})$ from $\mathcal{R}^{\mathbf{m}}_j(t')$ such that $\varphi(\sigma) = \rho_i$, which entails $\mathcal{T}^{\mathbf{m}}_{\leq j-1}(\rho_i^{\mathtt{tgt}}) \succ \mathcal{T}^{\mathbf{m}}_{\leq j-1}(\sigma^{\mathtt{tgt}})$. We conclude the proof by noting that the sequence cannot be empty: Y contains the empty reduction $\epsilon : t' \xrightarrow{j}{}^*_{\mathbf{m}} t'$, therefore $\mathcal{R}^{\mathbf{m}}_j(t')$ is non-empty, and so is $\mathcal{R}^{\mathbf{m}}_j(t)$.

2. Let $1 \leq d \leq j \leq D$ and $t_0 \xrightarrow{d}_{\mathbf{m}} t_0'$. We argue that $\mathcal{T}^{\mathbf{m}}_j(t_0, \mathtt{X}) \mathbin{:\!\succ:} \mathcal{T}^{\mathbf{m}}_j(t_0', \mathtt{X})$, where $\mathtt{X}$ is either a term ($\mathtt{X} = t$) or a memory ($\mathtt{X} = \mathtt{L}$). We proceed by induction on $\mathtt{X}$:

   2.1. $t = x$: Then $\mathcal{T}^{\mathbf{m}}_j(t_0, x) = [\,] \mathbin{:\!\succ:} [\,] = \mathcal{T}^{\mathbf{m}}_j(t_0', x)$.

   2.2. $t = \lambda x.s$: Then $\mathcal{T}^{\mathbf{m}}_j(t_0, \lambda x.s) = \mathcal{T}^{\mathbf{m}}_j(t_0, s) = \mathcal{T}^{\mathbf{m}}_j(t_0', s) = \mathcal{T}^{\mathbf{m}}_j(t_0', \lambda x.s)$ by the internal IH.

   2.3. If $t = (\lambda x.s)\mathtt{L}\,u$ is a redex of degree j: Then:

$$\begin{aligned}& \mathcal{T}^{\mathbf{m}}_j(t_0, (\lambda x.s)\mathtt{L}\,u)\\ = \;& \mathcal{T}^{\mathbf{m}}_j(t_0, s) + \mathcal{T}^{\mathbf{m}}_j(t_0, \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_j(t_0, u) + [(j, \mathcal{R}^{\mathbf{m}}_j(t_0))]\\ \mathbin{:\!\succ:} \;& \mathcal{T}^{\mathbf{m}}_j(t_0', s) + \mathcal{T}^{\mathbf{m}}_j(t_0', \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_j(t_0', u) + [(j, \mathcal{R}^{\mathbf{m}}_j(t_0))]\\ & \quad \text{by the internal IH}\\ \mathbin{:\!\succ:} \;& \mathcal{T}^{\mathbf{m}}_j(t_0', s) + \mathcal{T}^{\mathbf{m}}_j(t_0', \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_j(t_0', u) + [(j, \mathcal{R}^{\mathbf{m}}_j(t_0'))]\\ & \quad \text{since } \mathcal{R}^{\mathbf{m}}_j(t_0) \succ \mathcal{R}^{\mathbf{m}}_j(t_0') \text{ by item 1}\\ = \;& \mathcal{T}^{\mathbf{m}}_j(t_0', (\lambda x.s)\mathtt{L}\,u)\end{aligned}$$

   Note that we can resort to item 1, because $1 \leq d \leq j \leq D$ and $t_0 \xrightarrow{d}_{\mathbf{m}} t_0'$.

   2.4. If $t = s\,u$ is not a redex of degree j: Then $\mathcal{T}^{\mathbf{m}}_j(t_0, s\,u) = \mathcal{T}^{\mathbf{m}}_j(t_0, s) + \mathcal{T}^{\mathbf{m}}_j(t_0, u) \mathbin{:\!\succ:} \mathcal{T}^{\mathbf{m}}_j(t_0', s) + \mathcal{T}^{\mathbf{m}}_j(t_0', u) = \mathcal{T}^{\mathbf{m}}_j(t_0', s\,u)$ by the internal IH.

   2.5. $t = s\langle u\rangle$: Then $\mathcal{T}^{\mathbf{m}}_j(t_0, s\langle u\rangle) = \mathcal{T}^{\mathbf{m}}_j(t_0, s) + \mathcal{T}^{\mathbf{m}}_j(t_0, u) \mathbin{:\!\succ:} \mathcal{T}^{\mathbf{m}}_j(t_0', s) + \mathcal{T}^{\mathbf{m}}_j(t_0', u) = \mathcal{T}^{\mathbf{m}}_j(t_0', s\langle u\rangle)$ by the internal IH.

   2.6. $\mathtt{L} = \Box$: Then $\mathcal{T}^{\mathbf{m}}_j(t_0, \Box) = [\,] \mathbin{:\!\succ:} [\,] = \mathcal{T}^{\mathbf{m}}_j(t_0', \Box)$.

   2.7. $\mathtt{L} = \mathtt{L}_1\langle t\rangle$: $\mathcal{T}^{\mathbf{m}}_j(t_0, \mathtt{L}_1\langle t\rangle) = \mathcal{T}^{\mathbf{m}}_j(t_0, \mathtt{L}_1) + \mathcal{T}^{\mathbf{m}}_j(t_0, t) \mathbin{:\!\succ:} \mathcal{T}^{\mathbf{m}}_j(t_0', \mathtt{L}_1) + \mathcal{T}^{\mathbf{m}}_j(t_0', t)$ by IH.

3. Let $1 \leq d \leq D$ and $t_0 \xrightarrow{d}_{\mathbf{m}} t_0'$ and $\mathtt{X} \xrightarrow{d}_{\mathbf{m}} \mathtt{X}'$, where $\mathtt{X}, \mathtt{X}'$ are either terms ($\mathtt{X} = t$ and $\mathtt{X} = t'$) or memories ($\mathtt{X} = \mathtt{L}$ and $\mathtt{X} = \mathtt{L}'$). We argue that for all $\mathfrak{m} \in \mathbb{T}_{d-1}$ we have $\mathcal{T}^{\mathbf{m}}_d(t_0, \mathtt{X}) \succ \mathcal{T}^{\mathbf{m}}_d(t_0', \mathtt{X}') + \mathfrak{m}$. We proceed by induction on $\mathtt{X}$:

   3.1. $t = x$: Impossible, as there are no reduction steps $x \xrightarrow{d}_{\mathbf{m}} t'$.

   3.2. $t = \lambda x.s$: Then the step is of the form $t = \lambda x.s \xrightarrow{d}_{\mathbf{m}} \lambda x.s' = t'$ with $s \xrightarrow{d}_{\mathbf{m}} s'$. Let $\mathfrak{m} \in \mathbb{T}_{d-1}$. Then $\mathcal{T}^{\mathbf{m}}_d(t_0, t) = \mathcal{T}^{\mathbf{m}}_d(t_0, \lambda x.s) = \mathcal{T}^{\mathbf{m}}_d(t_0, s) \succ \mathcal{T}^{\mathbf{m}}_d(t_0', s') + \mathfrak{m} = \mathcal{T}^{\mathbf{m}}_d(t_0', \lambda x.s') + \mathfrak{m} = \mathcal{T}^{\mathbf{m}}_d(t_0', t') + \mathfrak{m}$ by the internal IH.

3.3. If $t = (\lambda x.s)\mathtt{L}\,u$ is the redex of degree d contracted by the step $t \xrightarrow{d}_{\mathbf{m}} t'$: If the reduction step is at the root, then $(\lambda x.s)\mathtt{L}$ is an $\mathbf{m}$-abstraction of degree d, and the step is of the form $t = (\lambda x.s)\mathtt{L}\,u \xrightarrow{d}_{\mathbf{m}} s[u/x]\langle u\rangle\mathtt{L} = t'$. Given that $(\lambda x.s)\mathtt{L}$ is an $\mathbf{m}$-abstraction of degree d, the type of its argument u is of height strictly less than d, that is, $\mathsf{h}(\mathsf{type}(u)) < d$. In particular, u is not an abstraction of degree d so we may apply the lower substitution lemma (Lemma A.2.20) which ensures that there exists $k \in \mathbb{N}_0$ such that $\mathcal{T}^{\mathbf{m}}_d(t'_0, s[u/x]) = \mathcal{T}^{\mathbf{m}}_d(t'_0, s) + k \otimes \mathcal{T}^{\mathbf{m}}_d(t'_0, u)$. Furthermore, observe that $\mathcal{T}^{\mathbf{m}}_d(t_0, u) \succeq (1+k) \otimes \mathcal{T}^{\mathbf{m}}_d(t'_0, u)$. Indeed, by item 2 we have that $\mathcal{T}^{\mathbf{m}}_d(t_0, u) \mathrel{:\!\succ\!:} \mathcal{T}^{\mathbf{m}}_d(t'_0, u)$, so by Lemma A.2.16 $\mathcal{T}^{\mathbf{m}}_d(t_0, u) \succeq (1 + k) \otimes \mathcal{T}^{\mathbf{m}}_d(t'_0, u)$. To conclude this case:

$$
\begin{array}{lll}
 & \mathcal{T}^{\mathbf{m}}_d(t_0, t) & \\
= & \mathcal{T}^{\mathbf{m}}_d(t_0, (\lambda x.s)\mathtt{L}\,u) & \\
= & \mathcal{T}^{\mathbf{m}}_d(t_0, s) + \mathcal{T}^{\mathbf{m}}_d(t_0, \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t_0, u) + [(d, \mathcal{R}^{\mathbf{m}}_d(t_0))] & \\
\succeq & \mathcal{T}^{\mathbf{m}}_d(t'_0, s) + \mathcal{T}^{\mathbf{m}}_d(t'_0, \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t_0, u) + [(d, \mathcal{R}^{\mathbf{m}}_d(t_0))] & \\
 & \quad \text{by item 2 and Lemma A.2.16} & \\
\succeq & \mathcal{T}^{\mathbf{m}}_d(t'_0, s) + \mathcal{T}^{\mathbf{m}}_d(t'_0, \mathtt{L}) + (1 + k) \otimes \mathcal{T}^{\mathbf{m}}_d(t'_0, u) + [(d, \mathcal{R}^{\mathbf{m}}_d(t_0))] & \\
 & \quad \text{since } \mathcal{T}^{\mathbf{m}}_d(t_0, u) \succeq (1 + k) \otimes \mathcal{T}^{\mathbf{m}}_d(t'_0, u) & \\
\succ & \mathcal{T}^{\mathbf{m}}_d(t'_0, s) + \mathcal{T}^{\mathbf{m}}_d(t'_0, \mathtt{L}) + (1 + k) \otimes \mathcal{T}^{\mathbf{m}}_d(t'_0, u) + \mathfrak{m} & \\
 & \quad \text{since } \mathfrak{m} \in \mathbb{T}_{d-1}, \text{ so } [(d, \mathcal{R}^{\mathbf{m}}_d(t_0))] \succ \mathfrak{m} & \\
= & \mathcal{T}^{\mathbf{m}}_d(t'_0, s) + k \otimes \mathcal{T}^{\mathbf{m}}_d(t'_0, u) + \mathcal{T}^{\mathbf{m}}_d(t'_0, u) + \mathcal{T}^{\mathbf{m}}_d(t'_0, \mathtt{L}) + \mathfrak{m} & \\
= & \mathcal{T}^{\mathbf{m}}_d(t'_0, s[u/x]) + \mathcal{T}^{\mathbf{m}}_d(t'_0, u) + \mathcal{T}^{\mathbf{m}}_d(t'_0, \mathtt{L}) + \mathfrak{m} & \\
= & \mathcal{T}^{\mathbf{m}}_d(t'_0, s[u/x]\langle u\rangle\mathtt{L}) + \mathfrak{m} & \\
= & \mathcal{T}^{\mathbf{m}}_d(t'_0, t') + \mathfrak{m} &
\end{array}
$$

3.4. If $t = (\lambda x.s)\mathtt{L}\,u$ is a redex of degree d, but not the redex contracted by the step $t \xrightarrow{d}_{\mathbf{m}} t'$: There are three subcases, depending on whether the step $t \xrightarrow{d}_{\mathbf{m}} t'$ is internal to s, internal to $\mathtt{L}$, or internal to u. All these subcases are similar; we only give the proof for the case in which the step is internal to s. Then the step is of the form $t = (\lambda x.s)\mathtt{L}\,u \xrightarrow{d}_{\mathbf{m}} (\lambda x.s')\mathtt{L}\,u = t'$ with $s \xrightarrow{d}_{\mathbf{m}} s'$, and we have:

$$
\begin{array}{lll}
 & \mathcal{T}^{\mathbf{m}}_d(t_0, t) & \\
= & \mathcal{T}^{\mathbf{m}}_d(t_0, (\lambda x.s)\mathtt{L}\,u) & \\
= & \mathcal{T}^{\mathbf{m}}_d(t_0, s) + \mathcal{T}^{\mathbf{m}}_d(t_0, \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t_0, u) + [(d, \mathcal{R}^{\mathbf{m}}_d(t_0))] & \\
\succ & \mathcal{T}^{\mathbf{m}}_d(t'_0, s') + \mathcal{T}^{\mathbf{m}}_d(t_0, \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t_0, u) + [(d, \mathcal{R}^{\mathbf{m}}_d(t_0))] + \mathfrak{m} & \text{by the internal IH} \\
\succeq & \mathcal{T}^{\mathbf{m}}_d(t'_0, s') + \mathcal{T}^{\mathbf{m}}_d(t'_0, \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t'_0, u) + [(d, \mathcal{R}^{\mathbf{m}}_d(t_0))] + \mathfrak{m} & \text{by item 2} \\
 & & \text{and Lemma A.2.16} \\
\succeq & \mathcal{T}^{\mathbf{m}}_d(t'_0, s') + \mathcal{T}^{\mathbf{m}}_d(t'_0, \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t'_0, u) + [(d, \mathcal{R}^{\mathbf{m}}_d(t'_0))] + \mathfrak{m} & \text{by item 1} \\
= & \mathcal{T}^{\mathbf{m}}_d(t'_0, (\lambda x.s')\mathtt{L}\,u) + \mathfrak{m} & \\
= & \mathcal{T}^{\mathbf{m}}_d(t'_0, t') + \mathfrak{m} &
\end{array}
$$

3.5. If $t = s\,u$ is not a redex of degree d: There are two subcases, depending on whether the step $t \xrightarrow{d}_{\mathbf{m}} t'$ is internal to s or internal to u:

3.5.1. If the step is internal to s, then the step is of the form $t = s\,u \xrightarrow{d}_{\mathbf{m}} s'\,u = t'$ with $s \xrightarrow{d}_{\mathbf{m}} s'$. Note that s is not an $\mathbf{m}$-abstraction of degree d (because $s\,u$ is not a redex

of degree d). Hence by Lemma A.2.12 s' is not an $\mathbf{m}$-abstraction of degree d. Then:

$$\begin{array}{rcll}
\mathcal{T}_d^{\mathbf{m}}(t_0, t) & = & \mathcal{T}_d^{\mathbf{m}}(t_0, s\,u) & \\
& = & \mathcal{T}_d^{\mathbf{m}}(t_0, s) + \mathcal{T}_d^{\mathbf{m}}(t_0, u) & \\
& \succ & \mathcal{T}_d^{\mathbf{m}}(t_0', s') + \mathcal{T}_d^{\mathbf{m}}(t_0, u) + \mathfrak{m} & \text{by the internal IH} \\
& \succeq & \mathcal{T}_d^{\mathbf{m}}(t_0', s') + \mathcal{T}_d^{\mathbf{m}}(t_0', u) + \mathfrak{m} & \text{by item 2 and Lemma A.2.16} \\
& = & \mathcal{T}_d^{\mathbf{m}}(t_0', s'\,u) + \mathfrak{m} & \\
& = & \mathcal{T}_d^{\mathbf{m}}(t_0', t') + \mathfrak{m} &
\end{array}$$

3.5.2. If the step is internal to u: Similar to the previous case.

3.6. $t = s\langle u\rangle$: Similar to case 3.5.

3.7. $\mathtt{L} = \Box$: Impossible, as there are no reduction steps $\Box \xrightarrow{d}_{\mathbf{m}} \mathtt{L}'$.

3.8. $\mathtt{L} = \mathtt{L}_1\langle t\rangle$: Similar to case 3.5

4. Let $1 \leq d < j \leq D$ and $t_0 \xrightarrow{d}_{\mathbf{m}} t_0'$ and $\mathtt{X} \xrightarrow{d}_{\mathbf{m}} \mathtt{X}'$, where $\mathtt{X}, \mathtt{X}'$ are either terms ($\mathtt{X} = t$ and $\mathtt{X} = t'$) or memories ($\mathtt{X} = \mathtt{L}$ and $\mathtt{X} = \mathtt{L}'$). We argue that $\mathcal{T}_j^{\mathbf{m}}(t_0, \mathtt{X}) \succeq \mathcal{T}_j^{\mathbf{m}}(t_0', \mathtt{X}')$. We proceed by induction on $\mathtt{X}$:

4.1. $t = x$: Impossible, as there are no reduction steps $x \xrightarrow{d}_{\mathbf{m}} t'$.

4.2. $t = \lambda x.s$: Then the step is of the form $t = \lambda x.s \xrightarrow{d}_{\mathbf{m}} \lambda x.s' = t'$ with $s \xrightarrow{d}_{\mathbf{m}} s'$. Then $\mathcal{T}_j^{\mathbf{m}}(t_0, t) = \mathcal{T}_j^{\mathbf{m}}(t_0, \lambda x.s) = \mathcal{T}_j^{\mathbf{m}}(t_0, s) \succeq \mathcal{T}_j^{\mathbf{m}}(t_0', s') = \mathcal{T}_j^{\mathbf{m}}(t_0', \lambda x.s') = \mathcal{T}_j^{\mathbf{m}}(t_0', t')$ by the internal IH.

4.3. If $t = (\lambda x.s)\mathtt{L}\,u$ is the redex of degree d contracted by the step $t \xrightarrow{d}_{\mathbf{m}} t'$: Then the step is of the form $t = (\lambda x.s)\mathtt{L}\,u \xrightarrow{d}_{\mathbf{m}} s[u/x]\langle u\rangle\mathtt{L} = t'$. Recall that by hypothesis $d < j$, and note that the abstraction $(\lambda x.s)\mathtt{L}$ is of degree d, so the type of the argument u must be of height less than d, that is, $\mathsf{h}(\mathsf{type}(u)) < d < j$. In particular, the argument u cannot be an $\mathbf{m}$-abstraction of degree j, so we may apply the lower substitution lemma (Lemma A.2.20), which ensures that there exists $k \in \mathbb{N}_0$ such that $\mathcal{T}_j^{\mathbf{m}}(t_0', s[u/x]) = \mathcal{T}_j^{\mathbf{m}}(t_0', s) + k \otimes \mathcal{T}_j^{\mathbf{m}}(t_0', u)$. Furthermore, observe that $\mathcal{T}_j^{\mathbf{m}}(t_0, u) \succeq (1+k) \otimes \mathcal{T}_j^{\mathbf{m}}(t_0', u)$. Indeed by item 2 $\mathcal{T}_j^{\mathbf{m}}(t_0, u) \mathbin{:\!\succ\!:} \mathcal{T}_j^{\mathbf{m}}(t_0', u)$ so by Lemma A.2.16 $\mathcal{T}_j^{\mathbf{m}}(t_0, u) \succeq (1+k) \otimes \mathcal{T}_j^{\mathbf{m}}(t_0', u)$. Moreover, note that t is not a redex of degree j, so:

$$\begin{array}{rcll}
\mathcal{T}_j^{\mathbf{m}}(t_0, t) & = & \mathcal{T}_j^{\mathbf{m}}(t_0, (\lambda x.s)\mathtt{L}\,u) & \\
& = & \mathcal{T}_j^{\mathbf{m}}(t_0, s) + \mathcal{T}_j^{\mathbf{m}}(t_0, \mathtt{L}) + \mathcal{T}_j^{\mathbf{m}}(t_0, u) & \\
& \succeq & \mathcal{T}_j^{\mathbf{m}}(t_0', s) + \mathcal{T}_j^{\mathbf{m}}(t_0', \mathtt{L}) + \mathcal{T}_j^{\mathbf{m}}(t_0, u) & \text{by item 2 and Lemma A.2.16} \\
& \succeq & \mathcal{T}_j^{\mathbf{m}}(t_0', s) + \mathcal{T}_j^{\mathbf{m}}(t_0', \mathtt{L}) + (1+k) \otimes \mathcal{T}_j^{\mathbf{m}}(t_0', u) & \\
& & \quad \text{since } \mathcal{T}_j^{\mathbf{m}}(t_0, u) \succeq (1+k) \otimes \mathcal{T}_j^{\mathbf{m}}(t_0', u) & \\
& = & \mathcal{T}_j^{\mathbf{m}}(t_0', s) + k \otimes \mathcal{T}_j^{\mathbf{m}}(t_0', u) + \mathcal{T}_j^{\mathbf{m}}(t_0', u) + \mathcal{T}_j^{\mathbf{m}}(t_0', \mathtt{L}) & \\
& = & \mathcal{T}_j^{\mathbf{m}}(t_0', s[u/x]) + \mathcal{T}_j^{\mathbf{m}}(t_0', u) + \mathcal{T}_j^{\mathbf{m}}(t_0', \mathtt{L}) & \\
& = & \mathcal{T}_j^{\mathbf{m}}(t_0', s[u/x]\langle u\rangle\mathtt{L}) & \\
& = & \mathcal{T}_j^{\mathbf{m}}(t_0, t') &
\end{array}$$

4.4. If $t = (\lambda x.s)\mathtt{L}\,u$ is a redex of degree j: Note that the step $t \xrightarrow{d}_{\mathbf{m}} t'$ cannot be at the root, because $d < j$, so the redex at the root is not of degree d. There are three subcases, depending on whether the step is internal to s, internal to $\mathtt{L}$, or internal to u. All these subcases are similar; we only give the proof for the case in which the step is internal to

s. Then the step is of the form $t = (\lambda x.s)\mathtt{L}\,u \xrightarrow{d}_{\mathbf{m}} (\lambda x.s')\mathtt{L}\,u = t'$ with $s \xrightarrow{d}_{\mathbf{m}} s'$, and we have:

$$\begin{array}{lll}
& \mathcal{T}_j^{\mathbf{m}}(t_0, t) & \\
= & \mathcal{T}_j^{\mathbf{m}}(t_0, (\lambda x.s)\mathtt{L}\,u) & \\
= & \mathcal{T}_j^{\mathbf{m}}(t_0, s) + \mathcal{T}_j^{\mathbf{m}}(t_0, \mathtt{L}) + \mathcal{T}_j^{\mathbf{m}}(t_0, u) + [(j, \mathcal{R}_j^{\mathbf{m}}(t_0))] & \\
\succeq & \mathcal{T}_j^{\mathbf{m}}(t_0', s') + \mathcal{T}_j^{\mathbf{m}}(t_0, \mathtt{L}) + \mathcal{T}_j^{\mathbf{m}}(t_0, u) + [(j, \mathcal{R}_j^{\mathbf{m}}(t_0))] & \text{by the internal IH} \\
\succeq & \mathcal{T}_j^{\mathbf{m}}(t_0', s') + \mathcal{T}_j^{\mathbf{m}}(t_0', \mathtt{L}) + \mathcal{T}_j^{\mathbf{m}}(t_0', u) + [(j, \mathcal{R}_j^{\mathbf{m}}(t_0))] & \text{by item 2 and Lemma A.2.16} \\
\succeq & \mathcal{T}_j^{\mathbf{m}}(t_0', s') + \mathcal{T}_j^{\mathbf{m}}(t_0', \mathtt{L}) + \mathcal{T}_j^{\mathbf{m}}(t_0', u) + [(j, \mathcal{R}_j^{\mathbf{m}}(t_0'))] & \text{by item 1} \\
= & \mathcal{T}_j^{\mathbf{m}}(t_0', (\lambda x.s')\mathtt{L}\,u) & \\
= & \mathcal{T}_j^{\mathbf{m}}(t_0', t') &
\end{array}$$

4.5. If $t = s\,u$ is not the redex contracted by the step $t \xrightarrow{d}_{\mathbf{m}} t'$ nor a redex of degree j: There are two subcases, depending on whether the step $t \xrightarrow{d}_{\mathbf{m}} t'$ is internal to s or internal to u:

4.5.1. If the step is internal to s, then the step is of the form $t = s\,u \xrightarrow{d}_{\mathbf{m}} s'\,u = t'$ with $s \xrightarrow{d}_{\mathbf{m}} s'$. Note that s is not an $\mathbf{m}$-abstraction of degree j, (because $s\,u$ is not a redex of degree j). Moreover $d < j$ so by Lemma A.2.12 we have that s' is not an $\mathbf{m}$-abstraction of degree j. Then:

$$\begin{array}{lll}
\mathcal{T}_j^{\mathbf{m}}(t_0, t) & = & \mathcal{T}_j^{\mathbf{m}}(t_0, s\,u) \\
& = & \mathcal{T}_j^{\mathbf{m}}(t_0, s) + \mathcal{T}_j^{\mathbf{m}}(t_0, u) \\
& \succeq & \mathcal{T}_j^{\mathbf{m}}(t_0', s') + \mathcal{T}_j^{\mathbf{m}}(t_0, u) \quad \text{by the internal IH} \\
& \succeq & \mathcal{T}_j^{\mathbf{m}}(t_0', s') + \mathcal{T}_j^{\mathbf{m}}(t_0', u) \quad \text{by item 2 and Lemma A.2.16} \\
& = & \mathcal{T}_j^{\mathbf{m}}(t_0', s'\,u) \\
& = & \mathcal{T}_j^{\mathbf{m}}(t_0', t')
\end{array}$$

4.5.2. If the step is internal to u: Similar to the previous case.

4.6. $t = s\langle u\rangle$: Similar to case 4.5.

4.7. $\mathtt{L} = \Box$: Impossible, as there are no reduction steps $\Box \xrightarrow{d}_{\mathbf{m}} \mathtt{L}'$.

4.8. $\mathtt{L} = \mathtt{L}_1\langle t\rangle$: Similar to case 4.5.

5. Let $1 \leq d \leq D$ and $t \xrightarrow{d}_{\mathbf{m}} t'$. We argue that $\mathcal{T}^{\mathbf{m}}_{\leq D}(t) \succ \mathcal{T}^{\mathbf{m}}_{\leq D}(t')$. Indeed:

$$\begin{array}{lll}
\mathcal{T}^{\mathbf{m}}_{\leq D}(t) & = & \sum_{i=1}^{D} \mathcal{T}_i^{\mathbf{m}}(t, t) \\
& = & \mathcal{T}^{\mathbf{m}}_{\leq d-1}(t) + \mathcal{T}_d^{\mathbf{m}}(t, t) + (\sum_{j=d+1}^{D} \mathcal{T}_j^{\mathbf{m}}(t, t)) \\
& \succeq & \mathcal{T}_d^{\mathbf{m}}(t, t) + (\sum_{j=d+1}^{D} \mathcal{T}_j^{\mathbf{m}}(t, t)) \\
& & \quad \text{removing the first term} \\
& \succ & \mathcal{T}^{\mathbf{m}}_{\leq d-1}(t') + \mathcal{T}_d^{\mathbf{m}}(t', t') + (\sum_{j=d+1}^{D} \mathcal{T}_j^{\mathbf{m}}(t, t)) \\
& & \quad \text{by item 3, taking } \mathfrak{m} := \mathcal{T}^{\mathbf{m}}_{\leq d-1}(t') \\
& \succeq & \mathcal{T}^{\mathbf{m}}_{\leq d-1}(t') + \mathcal{T}_d^{\mathbf{m}}(t', t') + (\sum_{j=d+1}^{D} \mathcal{T}_j^{\mathbf{m}}(t', t')) \\
& & \quad \text{by item 4} \\
& = & \mathcal{T}^{\mathbf{m}}_{\leq D}(t')
\end{array}$$

□

Proposition A.2.22 (Forget/decrease)**.** Let $d \in \mathbb{N}_0$. Then the following hold:

1. If $t \rhd t'$ then $\mathcal{R}^{\mathbf{m}}_d(t) \succeq \mathcal{R}^{\mathbf{m}}_d(t')$.
2. If $t_0 \rhd t'_0$ then $\mathcal{T}^{\mathbf{m}}_d(t_0, t) \succeq \mathcal{T}^{\mathbf{m}}_d(t'_0, t)$.
3. If $t_0 \rhd t'_0$ and $t \rhd t'$ then $\mathcal{T}^{\mathbf{m}}_d(t_0, t) \succeq \mathcal{T}^{\mathbf{m}}_d(t'_0, t')$.
4. If $t \rhd t'$ then $\mathcal{T}^{\mathbf{m}}_{\leq d}(t) \succeq \mathcal{T}^{\mathbf{m}}_{\leq d}(t')$.

Proof. We prove a more general version of the statement: in items 2 and 3 we allow t to be either a term or a memory. For example, the statement of item 2 is generalized as follows: if $t_0 \rhd t'_0$ then $\mathcal{T}^{\mathbf{m}}_j(t_0, \mathtt{X}) \succeq \mathcal{T}^{\mathbf{m}}_j(t'_0, \mathtt{X})$, where $\mathtt{X}$ is either a term or a memory.

We prove all items simultaneously by induction on d. Note that: item 1. resorts to the IH; item 2. resorts to item 1. (without decreasing d); item 3. resorts to items 1. and 2. (without decreasing d); item 4. resorts to item 3. (without necessarily decreasing d).

1. Let $t \rhd t'$. We argue that $\mathcal{R}^{\mathbf{m}}_d(t) \succeq \mathcal{R}^{\mathbf{m}}_d(t')$. Let X and Y be the sets of reduction sequences $X := \{\rho \mid (\exists s)\ \rho : t \xrightarrow{d}{}^*_{\mathbf{m}} s\}$ and $Y := \{\sigma \mid (\exists s')\ \sigma : t' \xrightarrow{d}{}^*_{\mathbf{m}} s'\}$. Note that, by definition, $\mathcal{R}^{\mathbf{m}}_j(t) = [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in X]$ and $\mathcal{R}^{\mathbf{m}}_j(t') = [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\sigma^{\mathtt{tgt}}) \mid\mid \sigma \in Y]$. We construct a function $\varphi : Y \to X$ as follows. Consider a forgetful step $R : t \rhd t'$; there may be more than one such step, but there is at least one by hypothesis. By postponement of forgetful reduction Proposition A.2.15, for each reduction sequence $\sigma : t' \xrightarrow{d}{}^*_{\mathbf{m}} s'$ there exists a term s_σ such that $s_\sigma \rhd^* s'$ and a reduction sequence $\sigma^\frown R : t \xrightarrow{d}{}^*_{\mathbf{m}} s_\sigma$. In particular, $(\sigma^\frown R) \in X$, and we can define $\varphi(\sigma) := \sigma^\frown R$. Moreover, φ is injective because if $\sigma_1, \sigma_2 \in Y$ are such that $\sigma_1^\frown R = \sigma_2^\frown R$ then by Proposition A.2.15 we have that $\sigma_1 = \sigma_2$.

   First, we claim that $\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\varphi(\sigma)^{\mathtt{tgt}}) \succeq \mathcal{T}^{\mathbf{m}}_{\leq d-1}(\sigma^{\mathtt{tgt}})$ for every $\sigma \in Y$. Indeed:

$$\begin{array}{rcll} \mathcal{T}^{\mathbf{m}}_{\leq d-1}(\varphi(\sigma)^{\mathtt{tgt}}) & = & \mathcal{T}^{\mathbf{m}}_{\leq d-1}(s_\sigma) & \\ & \succeq & \mathcal{T}^{\mathbf{m}}_{\leq d-1}(s') & \text{by item 4 of the IH} \\ & = & \mathcal{T}^{\mathbf{m}}_{\leq d-1}(\sigma^{\mathtt{tgt}}) & \end{array}$$

   To be able to apply item 4 of the IH, observe that we have $d - 1 < d$. We apply the IH as many times as the number of forgetful steps in $s_\sigma \rhd^* s'$.

   To conclude the proof, let $Z = X \setminus \varphi(Y)$, so that $X = \varphi(Y) \uplus Z$, and note that:

$$\begin{array}{rcll} \mathcal{R}^{\mathbf{m}}_d(t) & = & [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in X] & \\ & = & [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in \varphi(Y) \uplus Z] & \\ & = & [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in \varphi(Y)] + [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in Z] & \\ & \succeq & [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\rho^{\mathtt{tgt}}) \mid\mid \rho \in \varphi(Y)] & \\ & = & [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\varphi(\sigma)^{\mathtt{tgt}}) \mid\mid \sigma \in Y] & (\star) \\ & \succeq & [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\sigma^{\mathtt{tgt}}) \mid\mid \sigma \in Y] & (\star\star) \\ & = & \mathcal{R}^{\mathbf{m}}_d(t') & \end{array}$$

   To justify the step marked with $(\star)$, note that φ is injective, so Y and $\varphi(Y)$ have the same cardinality. To justify the step marked with $(\star\star)$, note that

$$\begin{aligned} [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\varphi(\sigma)^{\mathtt{tgt}}) \mid\mid \sigma \in Y] &= \sum_{\sigma \in Y} [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\varphi(\sigma)^{\mathtt{tgt}})] \\ &\succeq \sum_{\sigma \in Y} [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\sigma^{\mathtt{tgt}})] = [\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\sigma^{\mathtt{tgt}}) \mid\mid \sigma \in Y]. \end{aligned}$$

because $\mathcal{T}^{\mathbf{m}}_{\leq d-1}(\varphi(\sigma)^{\mathtt{tgt}}) \succeq \mathcal{T}^{\mathbf{m}}_{\leq d-1}(\sigma^{\mathtt{tgt}})$, as we have already justified.

2. Let $t_0 \rhd t_0'$ and let $\mathtt{X}$ be either a term or a memory. We argue that $\mathcal{T}^{\mathbf{m}}_d(t_0, \mathtt{X}) \succeq \mathcal{T}^{\mathbf{m}}_d(t_0', \mathtt{X})$. We proceed by induction on $\mathtt{X}$:

   2.1. $t = x$: Then $\mathcal{T}^{\mathbf{m}}_d(t_0, x) = [\,] \succeq [\,] = \mathcal{T}^{\mathbf{m}}_d(t_0', x)$.

   2.2. $t = \lambda x.s$: Then $\mathcal{T}^{\mathbf{m}}_d(t_0, \lambda x.s) = \mathcal{T}^{\mathbf{m}}_d(t_0, s) \succeq \mathcal{T}^{\mathbf{m}}_d(t_0', s) = \mathcal{T}^{\mathbf{m}}_d(t_0', \lambda x.s)$ by the internal IH.

   2.3. If $t = (\lambda x.s)\mathtt{L}\,u$ is a redex of degree d: Then:

$$\begin{array}{rcll}
\mathcal{T}^{\mathbf{m}}_d(t_0, t) & = & \mathcal{T}^{\mathbf{m}}_d(t_0, (\lambda x.s)\mathtt{L}\,u) & \\
& = & \mathcal{T}^{\mathbf{m}}_d(t_0, s) + \mathcal{T}^{\mathbf{m}}_d(t_0, \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t_0, u) + [(d, \mathcal{R}^{\mathbf{m}}_d(t_0))] & \\
& \succeq & \mathcal{T}^{\mathbf{m}}_d(t_0', s) + \mathcal{T}^{\mathbf{m}}_d(t_0', \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t_0', u) + [(d, \mathcal{R}^{\mathbf{m}}_d(t_0))] & \text{by the internal IH} \\
& \succeq & \mathcal{T}^{\mathbf{m}}_d(t_0', s) + \mathcal{T}^{\mathbf{m}}_d(t_0', \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t_0', u) + [(d, \mathcal{R}^{\mathbf{m}}_d(t_0'))] & \text{by item 1} \\
& = & \mathcal{T}^{\mathbf{m}}_d(t_0', (\lambda x.s)\mathtt{L}\,u) & \\
& = & \mathcal{T}^{\mathbf{m}}_d(t_0', t) &
\end{array}$$

   2.4. If $t = s\,u$ is not a redex of degree d: Then:

$$\begin{array}{rcll}
\mathcal{T}^{\mathbf{m}}_d(t_0, t) & = & \mathcal{T}^{\mathbf{m}}_d(t_0, s\,u) & \\
& = & \mathcal{T}^{\mathbf{m}}_d(t_0, s) + \mathcal{T}^{\mathbf{m}}_d(t_0, u) & \\
& \succeq & \mathcal{T}^{\mathbf{m}}_d(t_0', s) + \mathcal{T}^{\mathbf{m}}_d(t_0', u) & \text{by the internal IH} \\
& = & \mathcal{T}^{\mathbf{m}}_d(t_0', t) &
\end{array}$$

   2.5. $t = s\langle u \rangle$: Similar to case 2.4.

   2.6. $\mathtt{L} = \Box$: Then $\mathcal{T}^{\mathbf{m}}_d(t_0, \Box) = [\,] \succeq [\,] = \mathcal{T}^{\mathbf{m}}_d(t_0', \Box)$.

   2.7. $\mathtt{L} = \mathtt{L}_1\langle t \rangle$: Similar to case 2.4.

3. Let $t_0 \rhd t_0'$ and $\mathtt{X} \rhd \mathtt{X}'$, where $\mathtt{X}$ and $\mathtt{X}'$ are either terms ($\mathtt{X} = t$ and $\mathtt{X} = t'$) or memories ($\mathtt{X} = \mathtt{L}$ and $\mathtt{X} = \mathtt{L}'$). We argue that $\mathcal{T}^{\mathbf{m}}_d(t_0, \mathtt{X}) \succeq \mathcal{T}^{\mathbf{m}}_d(t_0', \mathtt{X}')$. We proceed by induction on $\mathtt{X}$:

   3.1. $t = x$: Impossible, as there are no forgetful steps $x \rhd t'$.

   3.2. $t = \lambda x.s$: Then $t = \lambda x.s \rhd \lambda x.s' = t'$ with $s \rhd s'$ and $\mathcal{T}^{\mathbf{m}}_d(t_0, t) = \mathcal{T}^{\mathbf{m}}_d(t_0, \lambda x.s) = \mathcal{T}^{\mathbf{m}}_d(t_0, s) \succeq \mathcal{T}^{\mathbf{m}}_d(t_0', s') \succeq \mathcal{T}^{\mathbf{m}}_d(t_0', \lambda x.s') \succeq \mathcal{T}^{\mathbf{m}}_d(t_0', t')$ by the internal IH.

   3.3. If $t = (\lambda x.s)\mathtt{L}\,u$ is a redex of degree d: There are three subcases, depending on whether the forgetful step $t \rhd t'$ is internal to s, internal to $\mathtt{L}$, or internal to u. All these subcases are similar; we only give the proof for the case in which the step is internal to s. Then $t = (\lambda x.s)\mathtt{L}\,u \rhd (\lambda x.s')\mathtt{L}\,u = t'$ with $s \rhd s'$ and:

$$\begin{array}{rcll}
\mathcal{T}^{\mathbf{m}}_d(t_0, t) & = & \mathcal{T}^{\mathbf{m}}_d(t_0, (\lambda x.s)\mathtt{L}\,u) & \\
& = & \mathcal{T}^{\mathbf{m}}_d(t_0, s) + \mathcal{T}^{\mathbf{m}}_d(t_0, \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t_0, u) + [(d, \mathcal{R}^{\mathbf{m}}_d(t_0))] & \\
& \succeq & \mathcal{T}^{\mathbf{m}}_d(t_0', s') + \mathcal{T}^{\mathbf{m}}_d(t_0, \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t_0, u) + [(d, \mathcal{R}^{\mathbf{m}}_d(t_0))] & \text{by the internal IH} \\
& \succeq & \mathcal{T}^{\mathbf{m}}_d(t_0', s') + \mathcal{T}^{\mathbf{m}}_d(t_0', \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t_0', u) + [(d, \mathcal{R}^{\mathbf{m}}_d(t_0))] & \text{by item 2} \\
& \succeq & \mathcal{T}^{\mathbf{m}}_d(t_0', s') + \mathcal{T}^{\mathbf{m}}_d(t_0', \mathtt{L}) + \mathcal{T}^{\mathbf{m}}_d(t_0', u) + [(d, \mathcal{R}^{\mathbf{m}}_d(t_0'))] & \text{by item 1} \\
& = & \mathcal{T}^{\mathbf{m}}_d(t_0', (\lambda x.s')\mathtt{L}\,u) & \\
& = & \mathcal{T}^{\mathbf{m}}_d(t_0', t') &
\end{array}$$

3.4. If $t = s\,u$ is not a redex of degree d: There are two subcases, depending on whether the forgetful step $t \rhd t'$ is internal to s or internal to u. These subcases are similar; we only give the proof for the case in which the step is internal to s. Then $t = s\,u \rhd s'\,u = t'$ with $s \rhd s'$ and:

$$\begin{array}{lcll} \mathcal{T}^{\mathbf{m}}_d(t_0, t) & = & \mathcal{T}^{\mathbf{m}}_d(t_0, s\,u) & \\ & = & \mathcal{T}^{\mathbf{m}}_d(t_0, s) + \mathcal{T}^{\mathbf{m}}_d(t_0, u) & \\ & \succeq & \mathcal{T}^{\mathbf{m}}_d(t'_0, s') + \mathcal{T}^{\mathbf{m}}_d(t_0, u) & \text{by the internal IH} \\ & \succeq & \mathcal{T}^{\mathbf{m}}_d(t'_0, s') + \mathcal{T}^{\mathbf{m}}_d(t'_0, u) & \text{by item 2} \\ & = & \mathcal{T}^{\mathbf{m}}_d(t'_0, s'\,u) & \\ & = & \mathcal{T}^{\mathbf{m}}_d(t'_0, t') & \end{array}$$

3.5. $t = s\langle u\rangle$: There are three subcases, depending on whether the forgetful step $t \rhd t'$ is at the root, internal to s, or internal to u:

3.5.1. If the step is at the root: Then the step is of the form $t = s\langle u\rangle \rhd s = t'$ and:

$$\begin{array}{lcll} \mathcal{T}^{\mathbf{m}}_d(t_0, t) & = & \mathcal{T}^{\mathbf{m}}_d(t_0, s\langle u\rangle) & \\ & = & \mathcal{T}^{\mathbf{m}}_d(t_0, s) + \mathcal{T}^{\mathbf{m}}_d(t_0, u) & \\ & \succeq & \mathcal{T}^{\mathbf{m}}_d(t_0, s) & \\ & \succeq & \mathcal{T}^{\mathbf{m}}_d(t'_0, s) & \text{by item 2} \\ & = & \mathcal{T}^{\mathbf{m}}_d(t'_0, t') & \end{array}$$

3.5.2. If the step is internal to s: Similar to case 3.4.

3.5.3. If the step is internal to u: Similar to case 3.4.

3.6. $\mathtt{L} = \Box$: Impossible, as there are no forgetful steps $\Box \rhd \mathtt{L}'$.

3.7. $\mathtt{L} = \mathtt{L}_1\langle t\rangle$: Similar to case 3.5.

4. Let $t \rhd t'$. We argue that $\mathcal{T}^{\mathbf{m}}_{\leq d}(t) \succeq \mathcal{T}^{\mathbf{m}}_{\leq d}(t')$. Indeed:

$$\begin{array}{lcll} \mathcal{T}^{\mathbf{m}}_{\leq d}(t) & = & \sum_{i=1}^{d} \mathcal{T}^{\mathbf{m}}_d(t, t) & \\ & \succeq & \sum_{i=1}^{d} \mathcal{T}^{\mathbf{m}}_d(t', t') & \text{by item 3, resorting to the IH when } i < d \\ & = & \mathcal{T}^{\mathbf{m}}_{\leq d}(t') & \end{array}$$

Note that for the value $i = d$, we resort directly to item 3 and not to the IH.

□

Appendix B

Proofs of Chapter 3

B.1 Proofs of Section 3.2

We prove that confluence holds for $\Lambda^{\mathsf{Ch}}_{\cap}$. We follow the parallel reduction approach.

Definition B.1.1.

1. The parallel reduction $\Rightarrow_{\mathsf{Ch}}$ is defined on terms and set-terms by the following rules, closed under contexts.

$$\begin{array}{llll}
x^A & \Rightarrow_{\mathsf{Ch}} & x^A & \\
\lambda x^{\vec{A}}.t & \Rightarrow_{\mathsf{Ch}} & \lambda x^{\vec{A}}.t' & \texttt{if} \quad t \Rightarrow_{\mathsf{Ch}} t' \\
t\vec{s} & \Rightarrow_{\mathsf{Ch}} & t'\vec{s}' & \texttt{if} \quad t \Rightarrow_{\mathsf{Ch}} t' \quad \texttt{and} \quad \vec{s} \Rightarrow_{\mathsf{Ch}} \vec{s}' \\
(\lambda x^{\vec{A}}.t)\vec{s} & \Rightarrow_{\mathsf{Ch}} & t'[\vec{s}'/x^{\vec{A}}] & \texttt{if} \quad t \Rightarrow_{\mathsf{Ch}} t' \quad \texttt{and} \quad \vec{s} \Rightarrow_{\mathsf{Ch}} \vec{s}' \\
\{t_1, \ldots, t_n\} & \Rightarrow_{\mathsf{Ch}} & \{t'_1, \ldots, t'_n\} & \texttt{if} \quad t_i \Rightarrow_{\mathsf{Ch}} t'_i (1 \leq i \leq n)
\end{array}$$

   Note that $\Rightarrow_{\mathsf{Ch}}$ is non-deterministic, since, when the term is a $\rightarrow_{\mathsf{Ch}}$ redex, either the third or the fourth rule can be applied. Roughly speaking, $\Rightarrow_{\mathsf{Ch}}$ corresponds to reduce simultaneously a subset of the visible redexes by the $\rightarrow_{\mathsf{Ch}}$ reduction.

2. The complete development of a term t is the following function, which denotes the result of the simultaneous reduction of all visible redexes in a term.

$$\begin{array}{rcl}
\mathtt{C}(x^A) & := & x^A \\
\mathtt{C}(\lambda x^{\vec{A}}.t) & := & \lambda x^{\vec{A}}.\mathtt{C}(t) \\
\mathtt{C}((\lambda x^{\vec{A}}.t)\,\vec{s}) & := & \mathtt{C}(t)[\mathtt{C}(\vec{s})/x^{\vec{A}}] \\
\mathtt{C}(t\,\vec{s}) & := & \mathtt{C}(t)\,\mathtt{C}(\vec{s}) \quad \text{if } t \text{ is not an abstraction} \\
\mathtt{C}(\{t_1, \ldots, t_n\}) & := & \{\mathtt{C}(t_1), \ldots, \mathtt{C}(t_n)\}
\end{array}$$

The $\Rightarrow_{\mathsf{Ch}}$ reduction trivially satisfies the following properties.

Lemma B.1.2 (Properties of $\Rightarrow_{\mathsf{Ch}}$)**.**

1. $t \rightarrow_{\mathsf{Ch}} t'$ implies $t \Rightarrow_{\mathsf{Ch}} t'$.
2. $t \Rightarrow_{\mathsf{Ch}} t'$ implies $t \rightarrow^*_{\mathsf{Ch}} t'$.
3. $\rightarrow^*_{\mathsf{Ch}}$ is the transitive closure of $\Rightarrow_{\mathsf{Ch}}$. □

Lemma B.1.3 (Substitution)**.** If $\Gamma, x : \vec{A}, y : \vec{B} \vdash_{\mathsf{Ch}} t : C$ and $\Gamma, y : \vec{B} \Vdash_{\mathsf{Ch}} \vec{s} : \vec{A}$ and $\Gamma \Vdash_{\mathsf{Ch}} \vec{r} : \vec{B}$ and $x \notin \mathsf{fv}(\vec{r})$ and $x \neq y$, then $t[\vec{s}/x^{\vec{A}}][\vec{r}/y^{\vec{B}}] = t[\vec{r}/y^{\vec{B}}][\vec{s}[\vec{r}/y^{\vec{B}}]/x^{\vec{A}}]$

Proof. By induction on t. The interesting cases are those of variables and set-terms. The variables case consists on three subcases, depending on the variable being equal to x, to y, or to neither.

1. $t = x^{A'}$ with $A' \in \vec{A}$: By hypothesis $\Gamma, y : \vec{B} \Vdash_{\mathsf{Ch}} \vec{s} : \vec{A}$ and Observation 3.2.14, we know that there exists a unique $s' \in \vec{s}$ such that $\Gamma \vdash_{\mathsf{Ch}} s' : A'$. Hence, by definition of substitution, it holds that $t[\vec{s}/x^{\vec{A}}][\vec{r}/y^{\vec{B}}] = x^{A'}[\vec{s}/x^{\vec{A}}][\vec{r}/y^{\vec{B}}] = s'[\vec{r}/y^{\vec{B}}]$. Again by definition of substitution, we conclude $x^{A'}[\vec{r}/y^{\vec{B}}][\vec{s}[\vec{r}/y^{\vec{B}}]/x^{\vec{A}}] = x^{A'}[\vec{s}[\vec{r}/y^{\vec{B}}]/x^{\vec{A}}] = s'[\vec{r}/y^{\vec{B}}]$.

2. $t = y^{B'}$ with $B' \in \vec{B}$: By hypothesis $\Gamma \Vdash_{\mathsf{Ch}} \vec{r} : \vec{B}$ and Observation 3.2.14, we know that there exists a unique $r' \in \vec{r}$ such that $\Gamma \vdash_{\mathsf{Ch}} r' : B'$. Hence, by definition of substitution, it holds that $t[\vec{s}/x^{\vec{A}}][\vec{r}/y^{\vec{B}}] = y^{B'}[\vec{s}/x^{\vec{A}}][\vec{r}/y^{\vec{B}}] = y^{B'}[\vec{r}/y^{\vec{B}}] = r'$. By hypothesis, $x^{\vec{A}}$ does not occur free in $\vec{r}$, thus neither in r'. Therefore $y^{B'}[\vec{r}/y^{\vec{B}}][\vec{s}[\vec{r}/y^{\vec{B}}]/x^{\vec{A}}] = r'[\vec{s}[\vec{r}/y^{\vec{B}}]/x^{\vec{A}}] = r'$.

3. $t = z^C$ with z distinct from x and y: By definition of substitution, we have $t[\vec{s}/x^{\vec{A}}][\vec{r}/y^{\vec{B}}] = z[\vec{s}/x^{\vec{A}}][\vec{r}/y^{\vec{B}}] = z[\vec{r}/y^{\vec{B}}] = z$ and $z[\vec{r}/y^{\vec{B}}][\vec{s}[\vec{r}/y^{\vec{B}}]/x^{\vec{A}}] = z[\vec{s}[\vec{r}/y^{\vec{B}}]/x^{\vec{A}}] = z$.

4. $t = \{t_1, \ldots, t_n\}$: By IH we have $t_i[\vec{s}/x^{\vec{A}}][\vec{r}/y^{\vec{B}}] = t_i[\vec{r}/y^{\vec{B}}][\vec{s}[\vec{r}/y^{\vec{B}}]/x^{\vec{A}}]$ for all $i \in \{1..n\}$. Hence, by definition of substitution, we conclude

$$\begin{aligned}\{t_1, \ldots, t_n\}[\vec{s}/x^{\vec{A}}][\vec{r}/y^{\vec{B}}] &= \{t_1[\vec{s}/x^{\vec{A}}][\vec{r}/y^{\vec{B}}], \ldots, t_n[\vec{s}/x^{\vec{A}}][\vec{r}/y^{\vec{B}}]\} \\ &= \{t_1[\vec{r}/y^{\vec{B}}][\vec{s}[\vec{r}/y^{\vec{B}}]/x^{\vec{A}}], \ldots, t_n[\vec{r}/y^{\vec{B}}][\vec{s}[\vec{r}/y^{\vec{B}}]/x^{\vec{A}}]\} \\ &= \{t_1, \ldots, t_n\}[\vec{r}/y^{\vec{B}}][\vec{s}[\vec{r}/y^{\vec{B}}]/x^{\vec{A}}]\end{aligned}$$

□

Lemma B.1.4 (Compatibility of substitution and parallel reduction)**.** Let $t \Rightarrow_{\mathsf{Ch}} t'$ and $\vec{s} \Rightarrow_{\mathsf{Ch}} \vec{s'}$. Then $t[\vec{s}/x^{\vec{A}}] \Rightarrow_{\mathsf{Ch}} t'[\vec{s'}/x^{\vec{A}}]$.

Proof. By induction on t, where the interesting case is that of $t = (\lambda y^{\vec{B}}.u)\vec{r}$. Then t' depends on whether the head redex is contracted or not. In both cases it holds that $u \Rightarrow_{\mathsf{Ch}} u'$, $\vec{r} \Rightarrow_{\mathsf{Ch}} \vec{r'}$.

1. If $t' = (\lambda y^{\vec{B}}.u')\vec{r'}$, then, by definition of substitution and IH, we have that

$$\begin{aligned}((\lambda y^{\vec{B}}.u)\vec{r})[\vec{s}/x^{\vec{A}}] &= (\lambda y^{\vec{B}}.u[\vec{s}/x^{\vec{A}}])\vec{r}[\vec{s}/x^{\vec{A}}] \\ &\Rightarrow_{\mathsf{Ch}} (\lambda y^{\vec{B}}.u'[\vec{s'}/x^{\vec{A}}])\vec{r'}[\vec{s'}/x^{\vec{A}}] \\ &= ((\lambda y^{\vec{B}}.u')\vec{r'})[\vec{s'}/x^{\vec{A}}]\end{aligned}$$

2. If $t' = u'[\vec{r'}/y^{\vec{B}}]$, then, by definition of substitution, IH, and Lemma B.1.3 (y is bound, so we can safely assume it does not occur in $\vec{s'}$), we have that

$$\begin{aligned}((\lambda y^{\vec{B}}.u)\vec{r})[\vec{s}/x^{\vec{A}}] &= (\lambda y^{\vec{B}}.u[\vec{s}/x^{\vec{A}}])\vec{r}[\vec{s}/x^{\vec{A}}] \\ &\Rightarrow_{\mathsf{Ch}} u'[\vec{s'}/x^{\vec{A}}][\vec{r'}[\vec{s'}/x^{\vec{A}}]/y^{\vec{B}}] \\ &= u'[\vec{r'}/y^{\vec{B}}][\vec{s'}/x^{\vec{A}}]\end{aligned}$$

□

Lemma B.1.5. $t \Rightarrow_{\mathsf{Ch}} t'$ implies $t' \Rightarrow_{\mathsf{Ch}} \mathsf{C}(t)$.

Proof. By induction on t. Let $t = (\lambda x^{\vec{A}}.s)\vec{r}$. So t' is either $(\lambda x^{\vec{A}}.s')\vec{r}'$ or $s'[\vec{r}'/x^{\vec{A}}]$, where $\vec{r} \Rightarrow_{\mathsf{Ch}} \vec{r}'$ and $s \Rightarrow_{\mathsf{Ch}} s'$. By IH, $s' \Rightarrow_{\mathsf{Ch}} \mathtt{C}(s)$ and $\vec{r}' \Rightarrow_{\mathsf{Ch}} \mathtt{C}(\vec{r})$. In both cases, we have that $t' \Rightarrow_{\mathsf{Ch}} \mathtt{C}(s)[\mathtt{C}(\vec{r})/x^{\vec{A}}] = \mathtt{C}(t)$; the former holds by definition of parallel reduction, the latter by Lemma B.1.4. The remaining cases are straightforward by IH. □

Lemma B.1.6 (Diamond)**.** If $t \Rightarrow_{\mathsf{Ch}} t_1$ and $t \Rightarrow_{\mathsf{Ch}} t_2$, then there is t_3 such that $t_1 \Rightarrow_{\mathsf{Ch}} t_3$ and $t_2 \Rightarrow_{\mathsf{Ch}} t_3$.

Proof. By Lemma B.1.5, $t_1 \Rightarrow_{\mathsf{Ch}} \mathtt{C}(t)$ and $t_2 \Rightarrow_{\mathsf{Ch}} \mathtt{C}(t)$, so $t_3 = \mathtt{C}(t)$. □

Proposition 3.2.19 (Confluence)**.** If $t \to^*_{\mathsf{Ch}} s$ and $t \to^*_{\mathsf{Ch}} r$ then there is u such that $s \to^*_{\mathsf{Ch}} u$ and $r \to^*_{\mathsf{Ch}} u$.

Proof. By Lemma B.1.2.3 and Lemma B.1.6. □

B.2 Proofs of Section 3.3

We prove confluence of $\Lambda^{\mathbf{m}}_{\cap}$. We extend the proof of $\Lambda^{\mathsf{Ch}}_{\cap}$ (Proposition 3.2.19) by adding the cases involving wrappers, and by modifying those involving Ch-redexes, following $\mathbf{m}$-redexes (*i.e.*, allowing a list of wrappers $\mathtt{L}$ between the $\mathbf{m}$-abstraction and the argument).

Definition B.2.1.

1. The parallel reduction $\Rightarrow_{\mathbf{m}}$ is extended by modifying the β case to allow lists of wrappers, and by adding the congruence cases of wrappers and lists of wrappers:

$$\begin{array}{lll} (\lambda x^{\vec{A}}.t)\mathtt{L}\vec{s} & \Rightarrow_{\mathbf{m}} \; t'[\vec{s}'/x^{\vec{A}}]\langle\vec{s}'\rangle\mathtt{L}' & \text{if } \; t \Rightarrow_{\mathbf{m}} t',\ \vec{s} \Rightarrow_{\mathbf{m}} \vec{s}' \ \text{ and } \ \mathtt{L} \Rightarrow_{\mathbf{m}} \mathtt{L}' \\ t\langle\vec{s}\rangle & \Rightarrow_{\mathbf{m}} \; t'\langle\vec{s}'\rangle & \text{if } \; t \Rightarrow_{\mathbf{m}} t' \ \text{ and } \ \vec{s} \Rightarrow_{\mathbf{m}} \vec{s}' \\ \Box\langle\vec{t_1}\rangle \dots \langle\vec{t_n}\rangle & \Rightarrow_{\mathbf{m}} \; \Box\langle\vec{t'_1}\rangle \dots \langle\vec{t'_n}\rangle & \text{if } \; \vec{t_i} \Rightarrow_{\mathbf{m}} \vec{t'_i} \ \text{ for all } \ i \in 1..n \end{array}$$

2. The complete development of a term t is extended by modifying the β case to allow lists of wrappers, and by adding the congruence cases of wrappers and lists of wrappers

$$\begin{array}{rcl} \mathtt{C}((\lambda x^{\vec{A}}.t)\mathtt{L}\vec{s}) & := & \mathtt{C}(t)[\mathtt{C}(\vec{s})/x^{\vec{A}}]\langle\mathtt{C}(\vec{s})\rangle\mathtt{C}(\mathtt{L}) \\ \mathtt{C}(t\langle\vec{s}\rangle) & := & \mathtt{C}(t)\langle\mathtt{C}(\vec{s})\rangle \\ \mathtt{C}(\Box\langle\vec{t_1}\rangle \dots \langle\vec{t_n}\rangle) & := & \Box\langle\mathtt{C}(\vec{t_1})\rangle \dots \langle\mathtt{C}(\vec{t_n})\rangle \end{array}$$

As well as $\Rightarrow_{\mathsf{Ch}}$ reduction, $\Rightarrow_{\mathbf{m}}$ reduction trivially satisfies the following properties:

Lemma B.2.2 (Properties of $\Rightarrow_{\mathbf{m}}$)**.**

1. $t \to_{\mathbf{m}} t'$ implies $t \Rightarrow_{\mathbf{m}} t'$.
2. $t \Rightarrow_{\mathbf{m}} t'$ implies $t \to^*_{\mathbf{m}} t'$.
3. $\to^*_{\mathbf{m}}$ is the transitive closure of $\Rightarrow_{\mathbf{m}}$. □

Lemma B.2.3 (Substitution)**.** If $\Gamma, x : \vec{A}, y : \vec{B} \vdash_{\mathbf{m}} t : C$ and $\Gamma, y : \vec{B} \Vdash_{\mathbf{m}} \vec{s} : \vec{A}$ and $\Gamma \Vdash_{\mathbf{m}} \vec{r} : \vec{B}$ and $x \notin \mathsf{fv}(\vec{r})$ and $x \neq y$, then $t[\vec{s}/x^{\vec{A}}][\vec{r}/y^{\vec{B}}] = t[\vec{r}/y^{\vec{B}}][\vec{s}[\vec{r}/y^{\vec{B}}]/x^{\vec{A}}]$

Proof. Analogous to the proof of Lemma B.1.3, adding the wrappers case. Let $t = t'\langle\vec{u}\rangle$. By IH and definition of substitution we have

$$\begin{aligned}(t'\langle\vec{u}\rangle)[\vec{s}/x^{\vec{A}}][\vec{r}/y^{\vec{B}}] &= t'[\vec{s}/x^{\vec{A}}][\vec{r}/y^{\vec{B}}]\langle\vec{u}[\vec{s}/x^{\vec{A}}][\vec{r}/y^{\vec{B}}]\rangle \\ &= t'[\vec{r}/y^{\vec{B}}][\vec{s}[\vec{r}/y^{\vec{B}}]/x^{\vec{A}}]\langle\vec{u}[\vec{r}/y^{\vec{B}}][\vec{s}[\vec{r}/y^{\vec{B}}]/x^{\vec{A}}]\rangle \\ &= (t'\langle\vec{u}\rangle)[\vec{r}/y^{\vec{B}}][\vec{s}[\vec{r}/y^{\vec{B}}]/x^{\vec{A}}]\end{aligned}$$

□

Lemma B.2.4 (Compatibility of substitution and parallel reduction)**.** Let $t \Rightarrow_{\mathbf{m}} t'$ and $\vec{s} \Rightarrow_{\mathbf{m}} \vec{s}'$. Then $t[\vec{s}/x^{\vec{A}}] \Rightarrow_{\mathbf{m}} t'[\vec{s}'/x^{\vec{A}}]$.

Proof. By induction on t, where the interesting case is that of $t = (\lambda y^{\vec{B}}.u)\mathtt{L}\vec{r}$. Then t' depends on whether the head $\mathbf{m}-$redex is contracted or not. In both cases it holds that $u \Rightarrow_{\mathbf{m}} u'$, $\vec{r} \Rightarrow_{\mathbf{m}} \vec{r}'$, and $\mathtt{L} \Rightarrow_{\mathbf{m}} \mathtt{L}'$.

1. If $t' = (\lambda y^{\vec{B}}.u')\mathtt{L}'\vec{r}'$, then, by definition of substitution and IH, we have that

$$\begin{array}{rcl}((\lambda y^{\vec{B}}.u)\mathtt{L}\vec{r})[\vec{s}/x^{\vec{A}}] & = & (\lambda y^{\vec{B}}.u[\vec{s}/x^{\vec{A}}])\mathtt{L}[\vec{s}/x^{\vec{A}}]\vec{r}[\vec{s}/x^{\vec{A}}] \\ & \Rightarrow_{\mathbf{m}} & (\lambda y^{\vec{B}}.u'[\vec{s}'/x^{\vec{A}}])\mathtt{L}'[\vec{s}'/x^{\vec{A}}]\vec{r}'[\vec{s}'/x^{\vec{A}}] \\ & = & ((\lambda y^{\vec{B}}.u')\mathtt{L}'\vec{r}')[\vec{s}'/x^{\vec{A}}]\end{array}$$

2. If $t' = u'[\vec{r}'/y^{\vec{B}}]\langle\vec{r}'\rangle\mathtt{L}'$, then, by definition of substitution, IH, and Lemma B.2.3, we have that

$$\begin{array}{rcl}((\lambda y^{\vec{B}}.u)\mathtt{L}\vec{r})[\vec{s}/x^{\vec{A}}] & = & (\lambda y^{\vec{B}}.u[\vec{s}/x^{\vec{A}}])\mathtt{L}[\vec{s}/x^{\vec{A}}]\vec{r}[\vec{s}/x^{\vec{A}}] \\ & \Rightarrow_{\mathbf{m}} & u'[\vec{s}'/x^{\vec{A}}][\vec{r}'[\vec{s}'/x^{\vec{A}}]/y^{\vec{B}}]\langle\vec{r}'[\vec{s}'/x^{\vec{A}}]\rangle\mathtt{L}'[\vec{s}'/x^{\vec{A}}] \\ & = & u'[\vec{r}'/y^{\vec{B}}][\vec{s}'/x^{\vec{A}}]\langle\vec{r}'[\vec{s}'/x^{\vec{A}}]\rangle\mathtt{L}'[\vec{s}'/x^{\vec{A}}] \\ & = & (u'[\vec{r}'/y^{\vec{B}}]\langle\vec{r}'\rangle\mathtt{L}')[\vec{s}'/x^{\vec{A}}]\end{array}$$

□

Lemma B.2.5 (Composition lemma for wrappers)**.** If $t \Rightarrow_{\mathbf{m}} t'$ and $\mathtt{L} \Rightarrow_{\mathbf{m}} \mathtt{L}'$, then $t\mathtt{L} \Rightarrow_{\mathbf{m}} t'\mathtt{L}'$.

Proof. Let $\mathtt{L} = \Box\langle\vec{t_1}\rangle \dots \langle\vec{t_n}\rangle$. We proceed by induction on n.

1. If $n = 0$, then $t\mathtt{L} = t\Box = t \Rightarrow_{\mathbf{m}} t' = t'\Box = t'\mathtt{L}'$.
2. If $n = k + 1$, then by IH we have that $t\Box\langle\vec{t_1}\rangle \dots \langle\vec{t_k}\rangle \Rightarrow_{\mathbf{m}} t'\Box\langle\vec{t'_1}\rangle \dots \langle\vec{t'_k}\rangle$, hence

$$\begin{array}{rcl}t\mathtt{L} & = & t\langle\vec{t_1}\rangle \dots \langle\vec{t_k}\rangle\langle\vec{t_{k+1}}\rangle \\ & = & t\langle\vec{t_1}\rangle \dots \langle\vec{t_k}\rangle\langle\vec{t_{k+1}}\rangle \\ & \Rightarrow_{\mathbf{m}} & t'\langle\vec{t'_1}\rangle \dots \langle\vec{t'_k}\rangle\langle\vec{t'_{k+1}}\rangle \\ & = & t'\Box\langle\vec{t'_1}\rangle \dots \langle\vec{t'_k}\rangle\langle\vec{t'_{k+1}}\rangle \\ & = & t'\mathtt{L}'\end{array}$$

□

Lemma B.2.6. $t \Rightarrow_{\mathbf{m}} t'$ implies $t' \Rightarrow_{\mathbf{m}} \mathtt{C}(t)$.

Proof. By induction on t. The interesting case is that of $t = (\lambda x^{\vec{A}}.u)\mathtt{L}\vec{s}$. Then t' depends on whether the head $\mathbf{m}$-redex is contracted or not. In both cases it holds that $u \Rightarrow_{\mathbf{m}} u'$, $\vec{s} \Rightarrow_{\mathbf{m}} \vec{s}'$, and $\mathtt{L} \Rightarrow_{\mathbf{m}} \mathtt{L}'$.

1. If $t' = (\lambda x^{\vec{A}}.u')\mathtt{L}'\vec{s}'$, then by IH we have that

$$\begin{array}{lll}(\lambda x^{\vec{A}}.u)\mathtt{L}\vec{s} & \Rightarrow_{\mathbf{m}} & (\lambda x^{\vec{A}}.u')\mathtt{L}'\vec{s}' \\ & \Rightarrow_{\mathbf{m}} & \mathtt{C}(u)[\mathtt{C}(\vec{s})/x^{\vec{A}}]\langle\mathtt{C}(\vec{s})\rangle\mathtt{C}(\mathtt{L}) \\ & = & \mathtt{C}((\lambda x^{\vec{A}}.u)\mathtt{L}\vec{s})\end{array}$$

2. If $t' = u'[\vec{s}'/x^{\vec{A}}]\langle\vec{s}'\rangle\mathtt{L}'$, then by IH and Lemma B.2.4 we have that

$$\begin{array}{lll}(\lambda x^{\vec{A}}.u)\mathtt{L}\vec{s} & \Rightarrow_{\mathbf{m}} & u'[\vec{s}'/x^{\vec{A}}]\langle\vec{s}'\rangle\mathtt{L}' \\ & \Rightarrow_{\mathbf{m}} & \mathtt{C}(u)[\mathtt{C}(\vec{s})/x^{\vec{A}}]\langle\mathtt{C}(\vec{s})\rangle\mathtt{C}(\mathtt{L}) \\ & = & \mathtt{C}((\lambda x^{\vec{A}}.u)\mathtt{L}\vec{s})\end{array}$$

The remaining cases are straightforward by IH. □

Lemma B.2.7 (Diamond)**.** If $t \Rightarrow_{\mathbf{m}} t_1$ and $t \Rightarrow_{\mathbf{m}} t_2$, then there is t_3 such that $t_1 \Rightarrow_{\mathbf{m}} t_3$ and $t_2 \Rightarrow_{\mathbf{m}} t_3$.

Proof. By Lemma B.2.6, we have $t_1 \Rightarrow_{\mathbf{m}} \mathtt{C}(t)$ and $t_2 \Rightarrow_{\mathbf{m}} \mathtt{C}(t)$, hence $t_3 = \mathtt{C}(t)$. □

Proposition 3.3.13 (Confluence)**.** If $t_1 \to_{\mathbf{m}}^* t_2$ and $t_1 \to_{\mathbf{m}}^* t_3$, there exists a term t_4 such that $t_2 \to_{\mathbf{m}}^* t_4$ and $t_3 \to_{\mathbf{m}}^* t_4$.

Proof. By Lemma B.2.2.3 and Lemma B.2.7. □

B.3 Proofs of Section 3.4

Lemma B.3.1 (Forgetful reduction commutes with reduction)**.** If $t_1 \to_{\mathbf{m}}^* t_2$ and $t_1 \rhd^+ t_3$ there exists t_4 such that $t_2 \rhd^+ t_4$ and $t_3 \to_{\mathbf{m}}^* t_4$.

Proof. It suffices to prove a local commutation result, namely that if $t_1 \to_{\mathbf{m}} t_2$ and $t_1 \rhd t_3$ there exists t_4 such that $t_2 \rhd^+ t_4$ and $t_3 \to_{\mathbf{m}}^{=} t_4$. Graphically:

$$\begin{array}{lll}t_1 & \to_{\mathbf{m}} & t_2 \\ \triangledown & & \triangledown^+ \\ t_3 & \to_{\mathbf{m}}^{=} & t_4\end{array}$$

We proceed by induction on t_1.

1. $t_1 = x^{\vec{A}}$: this case is impossible, since x does not reduce.

2. $t_1 = \lambda x^{\vec{A}}.t_1'$: then $t_2 = \lambda x^{\vec{A}}.t_2'$ and $t_3 = \lambda x^{\vec{A}}.t_3'$, with $t_1' \to_{\mathbf{m}} t_2'$ and $t_1' \rhd t_3'$. By IH there exists t_4' s.t. $t_3' \to_{\mathbf{m}}^{=} t_4'$ and $t_2' \rhd^+ t_4'$. By congruence, the following holds

$$\begin{array}{lll}\lambda x^{\vec{A}}.t_1' & \to_{\mathbf{m}} & \lambda x^{\vec{A}}.t_2' \\ \triangledown & & \triangledown^+ \\ \lambda x^{\vec{A}}.t_3' & \to_{\mathbf{m}}^{=} & \lambda x^{\vec{A}}.t_4'\end{array}$$

3. $t_1 = t_{11}\,\vec{t}_{12}$. We consider two subcases here, depending on t_{11} being a w-abstraction and on which redex is contracted.

   3.1. $t_1 = (\lambda x^{\vec{A}}.t_{11})\mathtt{L}\,\vec{t}_{12}$ and the contracted redex is at head position. Then

$$\begin{array}{ccc} (\lambda x^{\vec{A}}.t_{11})\mathtt{L}\,\vec{t}_{12} & \to_{\mathbf{m}} & t_{11}[\vec{t}_{12}/x^{\vec{A}}]\langle\vec{t}_{12}\rangle\mathtt{L} \\ \triangledown & & \\ (\lambda x^{\vec{A}}.t'_{11})\mathtt{L}'\,\vec{t}'_{12} & & \end{array}$$

   where since there is a single $\rhd$-step, only one of t'_{11}, $\mathtt{L}'$, or $\vec{t}'_{12}$, can be different from t_{11}, $\mathtt{L}$, or $\vec{t}_{12}$, respectively. If the $\rhd$-step occurs in t_{11} or $\mathtt{L}$, then the same step can be reproduced from t_2. Else, considering $\vec{t}_{12}$ can appear more than once in t_2, the $\rhd$-step may have to be reproduced several times, hence the $\rhd^+$. Then we have that

$$\begin{array}{ccc} (\lambda x^{\vec{A}}.t_{11})\mathtt{L}\,\vec{t}_{12} & \to_{\mathbf{m}} & t_{11}[\vec{t}_{12}/x^{\vec{A}}]\langle\vec{t}_{12}\rangle\mathtt{L} \\ \triangledown & & \triangledown^+ \\ (\lambda x^{\vec{A}}.t'_{11})\mathtt{L}'\,\vec{t}'_{12} & \to_{\mathbf{m}} & t'_{11}[\vec{t}'_{12}/x^{\vec{A}}]\langle\vec{t}'_{12}\rangle\mathtt{L}' \end{array}$$

   3.2. t_{11} is not a w-abstraction or the contracted redex is not the one at head position. We consider two subcases.

      3.2.1. Both steps occur in the same subterm. It suffices to resort to the IH:

$$\begin{array}{ccc} t_{11}\vec{t}_{12} & \to_{\mathbf{m}} & t'_{11}\vec{t}_{12} \\ \triangledown & & \triangledown^+ \\ t''_{11}\vec{t}_{12} & \to^{=}_{\mathbf{m}} & t'''_{11}\vec{t}_{12} \end{array} \qquad \begin{array}{ccc} t_{11}\vec{t}_{12} & \to_{\mathbf{m}} & t_{11}\vec{t}'_{12} \\ \triangledown & & \triangledown^+ \\ t_{11}\vec{t}''_{12} & \to^{=}_{\mathbf{m}} & t_{11}\vec{t}'''_{12} \end{array}$$

      3.2.2. The steps occur in different subterms. Then the diagram can be closed immediately:

$$\begin{array}{ccc} t_{11}\vec{t}_{12} & \to_{\mathbf{m}} & t'_{11}\vec{t}_{12} \\ \triangledown & & \triangledown \\ t_{11}\vec{t}'_{12} & \to_{\mathbf{m}} & t'_{11}\vec{t}'_{12} \end{array} \qquad \begin{array}{ccc} t_{11}\vec{t}_{12} & \to_{\mathbf{m}} & t_{11}\vec{t}'_{12} \\ \triangledown & & \triangledown \\ t'_{11}\vec{t}_{12} & \to_{\mathbf{m}} & t'_{11}\vec{t}'_{12} \end{array}$$

4. $t_1 = t_{11}\langle\vec{t}_{12}\rangle$. We consider three general subcases:

   4.1. Both steps occur in the same subterm. It suffices to resort to the IH:

$$\begin{array}{ccc} t_{11}\langle\vec{t}_{12}\rangle & \to_{\mathbf{m}} & t'_{11}\langle\vec{t}_{12}\rangle \\ \triangledown & & \triangledown^+ \\ t''_{11}\langle\vec{t}_{12}\rangle & \to^{=}_{\mathbf{m}} & t'''_{11}\langle\vec{t}_{12}\rangle \end{array} \qquad \begin{array}{ccc} t_{11}\langle\vec{t}_{12}\rangle & \to_{\mathbf{m}} & t_{11}\langle\vec{t}'_{12}\rangle \\ \triangledown & & \triangledown^+ \\ t_{11}\langle\vec{t}''_{12}\rangle & \to^{=}_{\mathbf{m}} & t_{11}\langle\vec{t}'''_{12}\rangle \end{array}$$

   4.2. The steps occur in different subterms. Then the diagram can be closed immediately:

$$\begin{array}{ccc} t_{11}\langle\vec{t}_{12}\rangle & \to_{\mathbf{m}} & t'_{11}\langle\vec{t}_{12}\rangle \\ \triangledown & & \triangledown \\ t_{11}\langle\vec{t}'_{12}\rangle & \to_{\mathbf{m}} & t'_{11}\langle\vec{t}'_{12}\rangle \end{array} \qquad \begin{array}{ccc} t_{11}\langle\vec{t}_{12}\rangle & \to_{\mathbf{m}} & t_{11}\langle\vec{t}'_{12}\rangle \\ \triangledown & & \triangledown \\ t'_{11}\langle\vec{t}_{12}\rangle & \to_{\mathbf{m}} & t'_{11}\langle\vec{t}'_{12}\rangle \end{array}$$

   4.3. The $\rhd$-step occurs at the root. We perform the $\to_{\mathbf{m}}$ step, or not.

$$\begin{array}{ccc} t_{11}\langle\vec{t}_{12}\rangle & \to_{\mathbf{m}} & t'_{11}\langle\vec{t}_{12}\rangle \\ \triangledown & & \triangledown \\ t_{11} & \to_{\mathbf{m}} & t'_{11} \end{array} \qquad \begin{array}{ccc} t_{11}\langle\vec{t}_{12}\rangle & \to_{\mathbf{m}} & t_{11}\langle\vec{t}'_{12}\rangle \\ \triangledown & & \triangledown \\ t_{11} & = & t_{11} \end{array}$$

5. $t_1 = \{s_1, \ldots, s_n\}$. We consider two subcases.

   5.1. Both steps occur in the same subterm. It suffices to resort to the IH:

$$\begin{array}{ccc} \{s_1, \ldots, s_i, \ldots, s_n\} & \to_{\mathbf{m}} & \{s_1, \ldots, s'_i, \ldots, s_n\} \\ \triangledown & & \triangledown^{+} \\ \{s_1, \ldots, s''_i, \ldots, s_n\} & \to^{=}_{\mathbf{m}} & \{s_1, \ldots, s'''_i, \ldots, s_n\} \end{array}$$

   5.2. The steps occur in different subterms. Then the diagram can be closed immediately. Indeed, let $i, j \in 1..n$:

$$\begin{array}{ccc} \{s_1, \ldots, s_i, \ldots, s_j, \ldots, s_n\} & \to_{\mathbf{m}} & \{s_1, \ldots, s'_i, \ldots, s_j, \ldots, s_n\} \\ \triangledown & & \triangledown \\ \{s_1, \ldots, s_i, \ldots, s'_j, \ldots, s_n\} & \to_{\mathbf{m}} & \{s_1, \ldots, s'_i, \ldots, s'_j, \ldots, s_n\} \end{array}$$

□

Appendix C

Proofs of Chapter 4

C.1 Proofs of Section 4.2

We prove that confluence holds for $\lambda^{\hat{\mathbf{m}}}$. We follow the parallel reduction approach.

Definition C.1.1.

1. The parallel reduction $\Rightarrow_{\hat{\mathbf{m}}}$ is defined on terms and memories by the following rules, closed under contexts.

$$
\begin{array}{llll}
x^A & \Rightarrow_{\hat{\mathbf{m}}} & x^A & \\
\lambda x^A.t & \Rightarrow_{\hat{\mathbf{m}}} & \lambda x^A.t' & \text{if} \quad t \Rightarrow_{\hat{\mathbf{m}}} t' \\
t\,s & \Rightarrow_{\hat{\mathbf{m}}} & t'\,s' & \text{if} \quad t \Rightarrow_{\hat{\mathbf{m}}} t' \quad \text{and} \quad s \Rightarrow_{\hat{\mathbf{m}}} s' \\
(\lambda x^A.t)\mathtt{L}\,s & \Rightarrow_{\hat{\mathbf{m}}} & t'\langle s'\rangle\mathtt{L}' & \text{if} \quad t \Rightarrow_{\hat{\mathbf{m}}} t',\ s \Rightarrow_{\hat{\mathbf{m}}} s',\ \mathtt{L} \Rightarrow_{\hat{\mathbf{m}}} \mathtt{L}' \quad \text{and} \quad x \notin \mathsf{fv}(t) \\
(\lambda x^A.t)\mathtt{L}\,s & \Rightarrow_{\hat{\mathbf{m}}} & t'[s'/x]\langle\rangle\mathtt{L}' & \text{if} \quad t \Rightarrow_{\hat{\mathbf{m}}} t',\ s \Rightarrow_{\hat{\mathbf{m}}} s',\ \mathtt{L} \Rightarrow_{\hat{\mathbf{m}}} \mathtt{L}' \quad \text{and} \quad x \in \mathsf{fv}(t) \\
t\langle s\rangle & \Rightarrow_{\hat{\mathbf{m}}} & t'\langle s'\rangle & \text{if} \quad t \Rightarrow_{\hat{\mathbf{m}}} t' \quad \text{and} \quad s \Rightarrow_{\hat{\mathbf{m}}} s' \\
t\langle\rangle & \Rightarrow_{\hat{\mathbf{m}}} & t'\langle\rangle & \text{if} \quad t \Rightarrow_{\hat{\mathbf{m}}} t' \\
\Box & \Rightarrow_{\hat{\mathbf{m}}} & \Box & \\
\mathtt{L}\langle\rangle & \Rightarrow_{\hat{\mathbf{m}}} & \mathtt{L}'\langle\rangle & \text{if} \quad \mathtt{L} \Rightarrow_{\hat{\mathbf{m}}} \mathtt{L}' \\
\mathtt{L}\langle u\rangle & \Rightarrow_{\hat{\mathbf{m}}} & \mathtt{L}'\langle u'\rangle & \text{if} \quad \mathtt{L} \Rightarrow_{\hat{\mathbf{m}}} \mathtt{L}' \quad \text{and} \quad u \Rightarrow_{\hat{\mathbf{m}}} u'
\end{array}
$$

   Note that $\Rightarrow_{\hat{\mathbf{m}}}$ is non-deterministic, since when the term is a $\rightarrow_{\hat{m}}$-redex the application rule may be applied instead of contracting the head redex.

2. The complete development of a term t is the following function, which denotes the result of the simultaneous reduction of all visible redexes in t:

$$
\begin{array}{rcll}
\mathtt{C}(x^A) & := & x^A & \\
\mathtt{C}(\lambda x^A.t) & := & \lambda x^A.\mathtt{C}(t) & \\
\mathtt{C}((\lambda x^A.t)\mathtt{L}\,s) & := & \mathtt{C}(t)\langle\mathtt{C}(s)\rangle\mathtt{C}(\mathtt{L}) & \text{if } x \notin \mathsf{fv}(t) \\
\mathtt{C}((\lambda x^A.t)\mathtt{L}\,s) & := & \mathtt{C}(t)[\mathtt{C}(s)/x]\langle\rangle\mathtt{C}(\mathtt{L}) & \text{if } x \in \mathsf{fv}(t) \\
\mathtt{C}(t\,s) & := & \mathtt{C}(t)\,\mathtt{C}(s) & \text{if } t \text{ is not an } \hat{\mathbf{m}}\text{-abstraction} \\
\mathtt{C}(t\langle s\rangle) & := & \mathtt{C}(t)\langle\mathtt{C}(s)\rangle & \\
\mathtt{C}(t\langle\rangle) & := & \mathtt{C}(t)\langle\rangle & \\
\mathtt{C}(\Box) & := & \Box & \\
\mathtt{C}(\mathtt{L}\langle\rangle) & := & \mathtt{C}(\mathtt{L})\langle\rangle & \\
\mathtt{C}(\mathtt{L}\langle u\rangle) & := & \mathtt{C}(\mathtt{L})\langle\mathtt{C}(u)\rangle &
\end{array}
$$

The $\Rightarrow_{\hat{\mathbf{m}}}$ reduction trivially satisfies the following properties.

Lemma C.1.2 (Properties of $\Rightarrow_{\hat{\mathbf{m}}}$)**.**

1. $t \to_{\hat{\mathbf{m}}} t'$ implies $t \Rightarrow_{\hat{\mathbf{m}}} t'$.
2. $t \Rightarrow_{\hat{\mathbf{m}}} t'$ implies $t \to^*_{\hat{\mathbf{m}}} t'$.
3. $\to^*_{\hat{\mathbf{m}}}$ is the transitive closure of $\Rightarrow_{\hat{\mathbf{m}}}$. □

Lemma C.1.3 (Compatibility of substitution and parallel reduction)**.** Let $t \Rightarrow_{\hat{\mathbf{m}}} t'$ and $s \Rightarrow_{\hat{\mathbf{m}}} s'$. Then $t[s/x] \Rightarrow_{\hat{\mathbf{m}}} t'[s'/x]$.

Proof. By induction on t. □

Lemma C.1.4 (Free variables through parallel reduction)**.** $t \Rightarrow_{\hat{\mathbf{m}}} t'$ implies $\mathsf{fv}(t) = \mathsf{fv}(t')$

Proof. By induction on $t \Rightarrow_{\hat{\mathbf{m}}} t'$. The interesting cases are those involving head redexes, where $t = (\lambda x^A.u)\mathtt{L}s$, with $u \Rightarrow_{\hat{\mathbf{m}}} u'$, $s \Rightarrow_{\hat{\mathbf{m}}} s'$, and $\mathtt{L} \Rightarrow_{\hat{\mathbf{m}}} \mathtt{L}'$. According to whether x occurs in u or not, t' may be $u'[s'/x]\langle\rangle\mathtt{L}'$ or $u'\langle s'\rangle\mathtt{L}'$. In both cases, it holds that $\mathsf{fv}(t) = \mathsf{fv}(u)\backslash\{x\} \cup \mathsf{fv}(\mathtt{L}) \cup \mathsf{fv}(s) =_{\text{IH}} \mathsf{fv}(u')\backslash\{x\} \cup \mathsf{fv}(\mathtt{L}') \cup \mathsf{fv}(s') = \mathsf{fv}(t')$, as occurrences of s' remains either in the substituted term or in a wrapper. □

Lemma C.1.5. $t \Rightarrow_{\hat{\mathbf{m}}} t'$ implies $t' \Rightarrow_{\hat{\mathbf{m}}} \mathtt{C}(t)$.

Proof. By induction on t. The interesting case is $t = (\lambda x.u)\mathtt{L}\,s$. There t' is either $(\lambda x.u')\mathtt{L}'\,s'$, $u'[s'/x]\langle\rangle\mathtt{L}'$ or $u'\langle s'\rangle\mathtt{L}'$, with $u \Rightarrow_{\hat{\mathbf{m}}} u'$, $s \Rightarrow_{\hat{\mathbf{m}}} s'$, and $\mathtt{L} \Rightarrow_{\hat{\mathbf{m}}} \mathtt{L}'$. We analyze each case, noticing that, by IH, we have $u' \Rightarrow_{\hat{\mathbf{m}}} \mathtt{C}(u)$, $s' \Rightarrow_{\hat{\mathbf{m}}} \mathtt{C}(s)$, and $\mathtt{L}' \Rightarrow_{\hat{\mathbf{m}}} \mathtt{C}(\mathtt{L})$.

1. $t' = (\lambda x.u')\mathtt{L}'\,s'$: We consider two subcases, according to x occurring free in u' or not:
   1.1. $x \notin \mathsf{fv}(u')$: Then $x \notin \mathsf{fv}(u)$ by Lemma C.1.4, so, by IH and Definition C.1.1, we conclude $(\lambda x.u')\mathtt{L}'\,s' \Rightarrow_{\hat{\mathbf{m}}} \mathtt{C}(u)\langle\mathtt{C}(s)\rangle\mathtt{C}(\mathtt{L}) = \mathtt{C}((\lambda x.u)\mathtt{L}\,s)$.
   1.2. $x \in \mathsf{fv}(u')$: Then $x \in \mathsf{fv}(u)$ by Lemma C.1.4, so, by IH and Definition C.1.1, we conclude $(\lambda x.u')\mathtt{L}'\,s' \Rightarrow_{\hat{\mathbf{m}}} \mathtt{C}(u)[\mathtt{C}(s)/x]\langle\rangle\mathtt{C}(\mathtt{L}) = \mathtt{C}((\lambda x.u)\mathtt{L}\,s)$.
2. $t' = u'\langle s'\rangle\mathtt{L}'$: Then $x \notin \mathsf{fv}(u)$, so by IH $u'\langle s'\rangle\mathtt{L}' \Rightarrow_{\hat{\mathbf{m}}} \mathtt{C}(u)\langle\mathtt{C}(s)\rangle\mathtt{C}(\mathtt{L}) = \mathtt{C}((\lambda x.u)\mathtt{L}\,s)$.
3. $t' = u'[s'/x]\langle\rangle\mathtt{L}'$: Then $x \in \mathsf{fv}(u)$, so, by IH and Lemma C.1.3, we have that $u'[s'/x] \Rightarrow_{\hat{\mathbf{m}}} \mathtt{C}(u)[\mathtt{C}(s)/x]$, hence $u'[s'/x]\langle\rangle\mathtt{L}' \Rightarrow_{\hat{\mathbf{m}}} \mathtt{C}(u)[\mathtt{C}(s)/x]\langle\rangle\mathtt{C}(\mathtt{L}) = \mathtt{C}((\lambda x.u)\mathtt{L}\,s)$.

The other cases follow directly from IH. □

Lemma C.1.6 (Diamond)**.** If $t \Rightarrow_{\hat{\mathbf{m}}} t_1$ and $t \Rightarrow_{\hat{\mathbf{m}}} t_2$, then there exists t_3 such that $t_1 \Rightarrow_{\hat{\mathbf{m}}} t_3$ and $t_2 \Rightarrow_{\hat{\mathbf{m}}} t_3$.

Proof. By Lemma C.1.5, take $t_3 = \mathtt{C}(t)$. □

Proposition 4.2.6 (Confluence)**.** If $t_1 \to^*_{\hat{\mathbf{m}}} t_2$ and $t_1 \to^*_{\hat{\mathbf{m}}} t_3$, there exists a term t_4 such that $t_2 \to^*_{\hat{\mathbf{m}}} t_4$ and $t_3 \to^*_{\hat{\mathbf{m}}} t_4$.

Proof. By Lemma C.1.2.3 and Lemma C.1.6. □